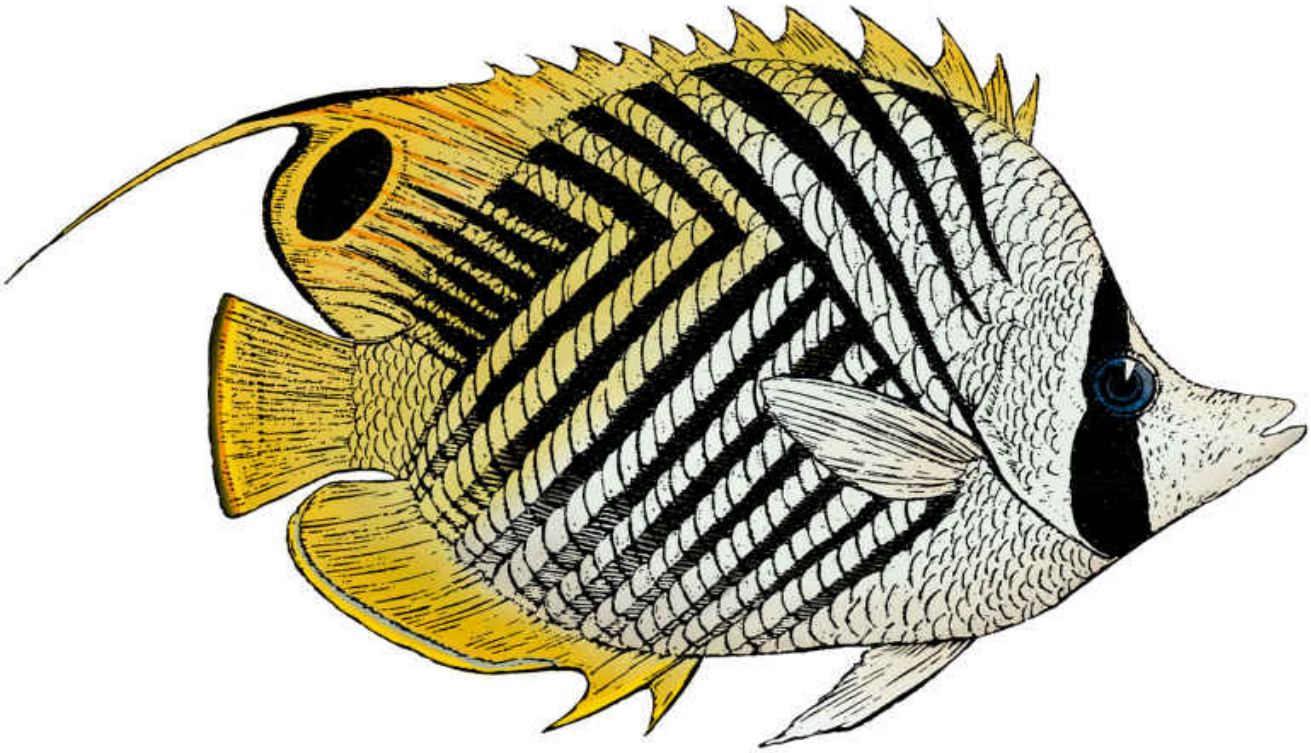
O'REILLY

Engenharia de Software para Cientistas de Dados

De Notebooks a Sistemas Escaláveis



novatec

Catherine Nelson

Elogios ao *Engenharia de Software para Cientistas de Dados*

Este livro é o elo perdido que cientistas de dados há muito procuravam, preenchendo com maestria a lacuna entre ciência de dados e engenharia de software. Além de ser um guia elucidativo e prático, preenche também a lacuna relacionada às habilidades imprescindíveis de engenharia de software que muitos cientistas têm dificuldades em aprender, elevando suas práticas de programação a um novo patamar. Sem dúvidas, esta é a obra que ansiosamente aguardávamos.

– Gabriela de Queiroz, Diretora de IA, Microsoft; Startup Advisor e Angel Investor

O livro de Catherine desmistifica como escalar seu trabalho até a capacidade de produção. Não importa se você é um cientista de dados, desenvolvedor ou executivo, ela faz com que os serviços de dados sejam acessíveis em grande escala. Desde os dados de startups até dados de grandes corporações, adotar suas práticas recomendadas garantirá o sucesso de seus projetos de dados.

– Carol Willing, Core Developer de Python; vencedora do Prêmio ACM Software System de 2017 por suas contribuições duradoras no desenvolvimento do Jupyter

Amo este livro! É obra obrigatória para todo cientista de dados. Durante anos, bootcamps, universidades e gerentes da área têm tentado fazer com que cientistas qualificados se comportem mais como engenheiros de software. Somente este livro foi capaz de preencher essa lacuna.

– Shawn Ling Ramirez, CEO da eloraHQ

Engenharia de Software para Cientistas de Dados é leitura obrigatória para quem deseja transformar suas habilidades de ciência de dados e ideias em sistemas completos e funcionais. É um guia notável para ajudá-lo nos aspectos de engenharia mais importantes da programação. Se eu tivesse tido acesso a este livro anos atrás, teria economizado incontáveis horas!

Recomendo sem hesitação.

– Laurence Moroney, Líder de Apoio à IA, na Google

Desde os primórdios, a ciência de dados tem atraído profissionais de diversas áreas do conhecimento e com experiências distintas. Em muitos aspectos, embora isso tenha sido ponto positivo da área, não raro os cientistas de dados não têm as habilidades de engenharia de software para trabalhar em estreita colaboração com colegas que têm experiências mais tradicionais de desenvolvimento de software. Nesta obra, Catherine Nelson estabelece uma ponte muito necessária entre as duas disciplinas, fornecendo aos cientistas de dados o conhecimento para aperfeiçoar o próprio trabalho e impacto.

– Chris Albon, diretor de machine learning na Fundação Wikimedia

Engenharia de Software para Cientistas de Dados

De Notebooks a Sistemas Escaláveis

Catherine Nelson

O'REILLY
Novatec

Authorized Portuguese translation of the English edition of Software Engineering for Data Scientists ISBN 9781098136208 © 2024 Catherine Nelson. This translation is published and sold by permission of O'Reilly Media, Inc., the owner of all rights to publish and sell the same.

Tradução em português autorizada da edição em inglês da obra Software Engineering for Data Scientists ISBN 9781098136208 © 2024 Catherine Nelson. Esta tradução é publicada e vendida com a permissão da O'Reilly Media, Inc., detentora de todos os direitos para publicação e venda desta obra.

© Novatec Editora Ltda. [2024].

Editor: Rubens Prates GRA20240809
Tradução: Cibelle Ravaglia
Revisão gramatical: Tássia Carvalho

ISBN impresso: : 978-85-7522-917-0

ISBN ebook: 978-85-7522-918-7

Histórico de impressões:

Agosto/2024 Primeira edição

Novatec Editora Ltda.
Rua Luís Antônio dos Santos 110
02460-000 – São Paulo, SP – Brasil
Tel.: +55 11 2959-6529
Email: *novatec@novatec.com.br*
Site: *https://novatec.com.br*
Twitter: *twitter.com/novateceditora*
Facebook: *facebook.com/novatec*
LinkedIn: *linkedin.com/in/novatec*
GRA20240809

Sumário

Prefácio

Capítulo 1 O que é um bom código?

Por que um bom código é importante
Adaptando-se aos requisitos em constante mudança
Simplicidade
  Princípio DRY
  Evite código verboso
Modularidade
Legibilidade
  Padrões e convenções
  Nomes
  Limpando o código
  Documentação
Desempenho
Robustez
  Erros e registro em logs
  Testando o código
Aspectos fundamentais

Capítulo 2 Analisando o desempenho do código

Métodos para melhorar o desempenho
Cronometrando a execução do código
Criando o perfilamento do código
  cProfile
  line_profiler
  Perfilamento de memória com Memray
Complexidade de tempo
  Como estimar a complexidade de tempo
  Notação Big O
Aspectos fundamentais

Capítulo 3 Usando estruturas de dados de modo eficaz

Estruturas nativas de dados do Python
- Listas
- Tuplas
- Dicionários
- Conjuntos

Arrays do NumPy
- Funcionalidades dos arrays do NumPy
- Considerações sobre o desempenho dos arrays do NumPy
- Operações de arrays com a biblioteca Dask
- Arrays em machine learning

DataFrames do pandas
- Funcionalidades dos DataFrames do pandas
- Considerações sobre o desempenho dos DataFrames do pandas

Aspectos fundamentais

Capítulo 4 Programação orientada a objetos e programação funcional

Programação orientada a objetos
- Classes, métodos e atributos
- Definindo as próprias classes
- Princípios de POO

Programação funcional
- Funções lambda e map()
- Aplicando funções a DataFrames

Qual paradigma devo usar?

Aspectos fundamentais

Capítulo 5 Erros, registro em logs e depuração

Erros em Python
- Interpretando mensagens de erro do Python
- Tratamento de erros
- Gerando erros com raise

Registro em logs
- O que registrar em logs
- Configuração do módulo logging

Como registrar em logs
Depuração
Estratégias para depuração
Ferramentas para depuração
Aspectos fundamentais

Capítulo 6 Formatação de código, linting e verificação de tipo

Formatação de código e guias de estilo
PEP8
Formatação de importação
Formatação automática de código com o Black
Linting
Ferramentas de linting
Linting em seu IDE
Verificação de tipo
Anotações de tipo
Verificação de tipo com o mypy
Aspectos fundamentais

Capítulo 7 Testando seu código

Por que devemos escrever testes
Quando testar
Como escrever e executar testes
Um teste básico
Testando entradas inesperadas
Executando testes automatizados com o Pytest
Tipos de testes
Testes unitários
Testes de integração
Validação de dados
Exemplos de validação de dados
Usando o Pandera para validação de dados
Validação de dados com o Pydantic
Testes para machine learning
Testando o treinamento de modelo
Testando a inferência do modelo

Aspectos fundamentais

Capítulo 8 Design e refatoração

Design e estrutura do projeto
  Considerações sobre o design do projeto
  Um exemplo de projeto de machine learning
Design de código
  Código modular
  Um framework para design de código
  Interfaces e contratos
  Acoplamento
De notebooks a scripts escaláveis
  Por que usar scripts em vez de notebooks?
  Criando scripts a partir de notebooks
Refatoração
  Estratégias para refatoração
  Um exemplo de fluxo de trabalho de refatoração
Aspectos fundamentais

Capítulo 9 Documentação

Documentação na base de código
  Nomes
  Comentários
  Docstrings
  Readmes, tutoriais e outros documentos mais extensos
Documentação em notebooks do Jupyter
Documentando experimentos de machine learning
Aspectos fundamentais

Capítulo 10 Compartilhando o código: controle de versão, dependências e empacotamento

Controle de versão usando o Git
  Como o Git funciona?
  Acompanhando mudanças e commit
  Remoto e Local
  Branches e pull requests
Dependências e ambientes virtuais

Ambientes virtuais
Gerenciando dependências com o pip
Gerenciando dependências com o Poetry
Empacotamento do Python
Conceitos básicos de empacotamento
pyproject.toml
Criando e fazendo upload de pacotes
Aspectos fundamentais

Capítulo 11 APIs

Chamando uma API
Métodos HTTP e códigos de status
Obtendo dados da API SDG
Criando sua API usando o FastAPI
Configurando a API
Adicionando funcionalidade à sua API
Criando requisições para sua API
Aspectos fundamentais

Capítulo 12 Automação e implantação

Implantação do código
Exemplos de automação
Hooks de pré-commit
GitHub Actions
Implantações em nuvem
Contêineres e Docker
Criando um contêiner do Docker
Implantando uma API no Google Cloud
Implantando uma API em outros provedores de nuvem
Aspectos fundamentais

Capítulo 13 Segurança

O que é segurança?
Riscos de segurança
Credenciais, segurança física e engenharia social
Pacotes de terceiros
O módulo pickle do Python

Riscos de controle de versão
Riscos de segurança de APIs
Práticas de segurança
Análises e políticas de segurança
Ferramentas para programação segura
Varredura simplificada de código
Segurança para machine learning
Ataques a sistemas de ML
Práticas de segurança para sistemas de ML
Aspectos fundamentais

Capítulo 14 Trabalhando em software

Princípios e práticas de desenvolvimento
O ciclo de vida de desenvolvimento de software
Modelo de desenvolvimento de software em cascata
Desenvolvimento ágil de software
Ciência ágil de dados
Funções na indústria de software
Engenheiro de software
Engenheiro de QA ou de testes
Engenheiro de dados
Analista de dados
Gerente de produto
Pesquisador de UX
Designer
Comunidade
Open-source
Palestrando em eventos
Comunidade Python
Aspectos fundamentais

Capítulo 15 Próximos passos

Futuro do código
Seu futuro no código
Obrigada

Prefácio

A ciência de dados se materializa no código. Independentemente de você estar desenvolvendo um sistema de machine learning [aprendizado de máquina], explorando dados pela primeira vez, visualizando a distribuição de dados ou executando uma análise estatística, tudo isso só é possível por causa de suas habilidades de programação e de computação. Caso esteja trabalhando com código de produção, essas habilidades são indispensáveis para escrever um código eficaz e de fácil manutenibilidade. Mesmo que não trabalhe em uma equipe de software de produção, será vantajoso escrever um código mais robusto e reproduzível, que outros cientistas de dados possam usar com facilidade. E, caso trabalhe sozinho, boas práticas farão você programar mais rápido, além de ajudá-lo a retomar o código de onde parou após um intervalo.

Nem sempre enxerguei o valor da engenharia de software. No início da minha carreira em ciência de dados, entrei para uma equipe na qual eu era a única cientista de dados. Meus colegas de equipe eram engenheiros de software e designers, e eu receava que seria difícil aprimorar minhas habilidades sem nenhum outro cientista de dados com quem aprender. Compartilhei minha preocupação com um colega desenvolvedor. Ele disse: “Mas aprender a escrever um código melhor possibilitará que você faça mais projetos de ciência de dados”. Esse comentário mexeu comigo, e, desde então, descobri que aprimorar minhas habilidades de engenharia de software tem sido extremamente benéfico para meu trabalho em ciência de dados. Isso me ajudou a escrever um código mais fácil de ser usado por meus colegas de trabalho e que ainda é fácil de mudar quando o retomo meses depois.

Meu intuito com este livro é orientá-lo em sua jornada para escrever um código melhor de ciência de dados. Abordarei em detalhes as melhores práticas para tarefas comuns, incluindo testes, tratamento de erros [error handling] e registro em logs [logging]. Explicarei como escrever um código mais fácil de manter e que permanecerá robusto à medida que seus projetos se expandem. Mostrarei como facilitar o uso de seu código para outras

pessoas e, ao final do livro, você será capaz de integrar seu código de ciência de dados a uma base de código [codebase] maior.

Talvez você acredite que as habilidades de engenharia de software são menos úteis na era da IA generativa. O ChatGPT não pode simplesmente escrever código para você? Eu diria que o conteúdo desta obra ainda é igualmente útil, mesmo quando podemos programar mais rápido com um assistente de IA. Como mostrarei no decorrer deste livro, existem muitas opções disponíveis para cada função que escrevemos, e ajuda bastante entendermos os princípios do porquê escolhemos uma linha de código em vez de outra. Será necessário avaliar a saída [output] de qualquer assistente de IA e verificar se ele fez uma boa escolha para o problema em que você está trabalhando. Este livro o ajudará a fazer isso.

A quem se destina este livro?

Este livro se destina a cientistas de dados, mas profissionais de áreas estritamente relacionadas, como analistas de dados, engenheiros de machine learning (ML) e engenheiros de dados, também encontrarão nele um aliado útil. Explicarei princípios consolidados de engenharia de software que serão úteis para qualquer pessoa que escreva código, porém os cientistas de dados estão mais familiarizados com os exemplos que usarei para ilustrar esses princípios.

Meu objetivo foi escrever um livro acessível a cientistas de dados que estão iniciando sua jornada na área. Talvez você tenha acabado de se formar em ciência de dados ou tenha conseguido seu primeiro emprego no setor. Este livro abordará as habilidades práticas de engenharia de software que nem sempre são elencadas nos cursos introdutórios de ciência de dados. Ou, talvez, você não tenha feito um curso formal de ciência de dados. Talvez você seja autodidata ou esteja migrando da matemática ou de outra área para a ciência de dados. Não importa qual caminho você esteja trilhando na ciência de dados, este livro é para você.

Cientistas de dados mais experientes também aprenderão bastante, e você perceberá que este livro tem grande serventia, caso tenha um trabalho em que costume interagir com desenvolvedores de software. Você aprenderá não apenas as habilidades que o ajudarão a trabalhar de forma eficaz em

uma base de código maior, mas também a escrever código Python que funcionará de modo eficiente em produção.

Suponho que já conheça os fundamentos da ciência de dados, incluindo exploração de dados, visualização de dados, data wrangling[1], os princípios básicos de ML e as habilidades matemáticas que os acompanham. Suponho também que já saiba as noções básicas de como programar em Python: como escrever funções e instruções de fluxo de controle, e como usar módulos, incluindo NumPy, Matplotlib, pandas e scikit-learn. Se tudo isso for novidade para você, recomendo os seguintes livros:

- *Python Data Science Handbook* [Handbook de Ciência de Dados em Python, em tradução livre], de Jake VanderPlas (O'Reilly, 2023)
- *Data Science do Zero: Noções Fundamentais com Python*, de Joel Grus (Novatec, 2021)
- *Learning Data Science* [Aprendendo Ciência de Dados, em tradução livre], de Sam Lau, Joseph Gonzalez e Deborah Nolan (O'Reilly, 2023).

Este livro não se destina a desenvolvedores de software que procuram aprender ciência de dados e habilidades de machine learning. Se for o seu caso, recomendo a leitura da obra *AI and Machine Learning for Coders* [IA e Machine Learning para Programadores, em tradução livre], de Laurence Moroney (O'Reilly, 2020).

Engenharia de software versus ciência de dados

Logo de início, é útil definir o que considero ser a distinção entre as mentalidades da ciência de dados e da engenharia de software. Geralmente, os cientistas de dados têm uma bagagem e experiências que enfatizam processos científicos de exploração, descoberta e teste de hipóteses. Ninguém sabe qual será o resultado final do projeto no início dele. Em contrapartida, a engenharia de software é um processo cujo foco é planejar o que será desenvolvido, projetar a melhor forma de desenvolver e, em seguida, escrever o código para desenvolver o que foi planejado. Todo mundo sabe o resultado esperado do projeto desde o início dele. As práticas de engenharia de software enfatizam a padronização e a automação. Os cientistas de dados podem recorrer aos aspectos da mentalidade de engenharia a fim de aperfeiçoar a qualidade de seu código, assunto que abordarei detalhadamente no Capítulo 1.

Por que Python?

Neste livro, todos os exemplos de código são em Python e, em muitos capítulos, abordo ferramentas específicas do Python. Nos últimos anos, o

Python se tornou a linguagem de programação mais popular para ciência de dados. A citação a seguir é de uma pesquisa de 2021 (https://oreil.ly/kmmBp) com mais de 3.000 cientistas de dados realizada pela Anaconda:

> "Sessenta e três por cento dos entrevistados afirmaram que sempre ou frequentemente usam Python, o que a torna a linguagem mais popular incluída na pesquisa deste ano. Além do mais, 71% dos educadores estão ensinando Python e 88% dos alunos afirmaram estar aprendendo Python para entrar na área de ciência de dados/ML."

O Python tem um conjunto extremamente robusto de bibliotecas open-source para ciência de dados, com bom suporte e uma comunidade participativa de mantenedores. Empresas de grande porte que ditam tendências escolheram o Python para seus principais frameworks de ML, incluindo TensorFlow (Google) e PyTorch (Meta). Por isso, o Python é bastante popular entre os cientistas de dados que trabalham com código de produção de machine learning, área em que boas habilidades de programação são imprescindíveis.

Em minha experiência, a comunidade Python tem sido amigável e receptiva, com muitos eventos excelentes que me ajudaram a aprimorar minhas habilidades. Como é minha linguagem de programação preferida, foi uma escolha fácil para este livro.

O que este livro não aborda

Como mencionei em "A quem se destina este livro?", na página 12, esta obra não é uma introdução à ciência de dados ou à programação. Além disso, nenhum dos seguintes tópicos é abordado neste livro:

- *Instalação do Python*: presumo que você já tenha instalado uma versão recente do Python (3.9 ou posterior) e que tenha alguma IDE (ambiente de desenvolvimento integrado) em que possa escrever código, como VS Code ou PyCharm. Não explicarei como instalar o Python, mas detalharei como configurar um ambiente virtual no Capítulo 10.
- *Outras linguagens de programação*: este livro aborda somente Python, pelas razões apresentadas em "Por que Python?", na página 13. Não incluí exemplos em R, Julia, SQL, MATLAB ou em qualquer outra linguagem.

- *Script de linha de comando:* o script de linha de comando ou shell é uma maneira extremamente eficaz de trabalhar com arquivos e texto. Não o incluo aqui porque outras fontes o abordam em detalhes, como o livro *Data Science at the Command Line* [Ciência de Dados na Linha de Comando, em tradução livre], de Jeroen Janssens (O'Reilly, 2021).
- *Python avançado:* os exemplos deste livro contêm código relativamente simples. Para uma abordagem de programação Python mais avançada, recomendo o livro *Robust Python [Python Robusto, em tradução livre]*, de Patrick Viafore (O'Reilly, 2021).

Guia para este livro

Neste livro, começo apresentando as boas práticas a nível de escrita de funções individuais e entro em detalhes sobre como você pode melhorar suas habilidades de programação. Nos próximos capítulos, abordarei como você pode facilitar o uso do código para outras pessoas, e explicarei algumas técnicas comuns de implantação [deployment] e melhores práticas para trabalhar com software.

Esta obra está dividida em 15 capítulos. Vejamos um panorama geral do conteúdo de cada um deles:

No Capítulo 1, "O que é um bom código?", apresento as noções básicas de como escrever um código simples, modular, legível, eficiente e robusto.

No Capítulo 2, "Analisando o desempenho do código", abordo como mensurar o desempenho de seu código e analiso algumas opções para tornar seu código de ciência de dados mais eficiente.

No Capítulo 3, "Usando estruturas de dados de modo eficaz", analiso os prós e contras relativos à escolha das estruturas de dados com as quais você trabalha. A estrutura de dados escolhida pode fazer grande diferença na eficiência de seu código.

No Capítulo 4, "Programação orientada a objetos e programação funcional", abordo os conceitos básicos desses estilos de programação. Quando devidamente usados, podem ajudá-lo a escrever um código bem estruturado e eficiente.

No Capítulo 5, "Erros, registro em logs e depuração", explico o que fazer

quando o código quebra, como gerar erros úteis e estratégias para identificar qual é a origem desses erros.

No Capítulo 6, “Formatação de código, linting e verificação de tipo”, abordo como padronizar seu código com ferramentas que podem automatizar esse processo.

No Capítulo 7, “Testando seu código”, apresento como tornar seu código robusto às mudanças nas entradas por meio de testes. Trata-se de uma etapa vital para escrever um código com fácil manutenibilidade.

No Capítulo 8, “Design e refatoração”, analiso como estruturar seus projetos de forma padronizada e consistente e como passar de um notebook para um script.

No Capítulo 9, “Documentação”, apresento como fazer com que seu código seja legível para outras pessoas, incluindo as melhores práticas para nomear e comentar o código.

No Capítulo 10, “Compartilhando o código: controle de versão, dependências e empacotamento”, elenco as noções básicas de controle de versão usando o Git e como gerenciar as dependências do seu projeto em ambientes virtuais. Além disso, abordo as etapas envolvidas na transformação de um script em um pacote Python.

No Capítulo 11, “APIs”, apresento o conceito de APIs, mostro como você pode usá-las e incluo um exemplo básico usando o FastAPI.

No Capítulo 12, “Automação e implantação”, detalho os conceitos básicos de implantação de código, como automatizar implantações de código usando CI/CD (Integração contínua/implantação ou entrega contínua) e o GitHub Actions, e como implantar código em um ambiente de nuvem, em um contêiner do Docker.

No Capítulo 13, “Segurança”, analiso os riscos comuns de segurança, como podem ser mitigados e algumas das ameaças exclusivas de segurança do machine learning.

No Capítulo 14, “Trabalhando em software”, apresento práticas usuais em equipes de desenvolvimento de software, incluindo formas de trabalhar com a metodologia ágil, detalho os papéis comuns desempenhados em equipes de software e apresento a comunidade em geral.

No Capítulo 15, “Próximos passos”, finalizo a obra com algumas reflexões sobre como a programação pode mudar no futuro e algumas sugestões sobre seus próximos passos.

Ordem de leitura

Você não precisa necessariamente ler os capítulos deste livro na sequência, mas recomendo que comece pelo Capítulo 1. Neste capítulo, explicarei os conceitos básicos de como escrever um bom código e apresentarei tópicos que abordarei com mais detalhes no restante da obra. Além do mais, apresentarei diversos dos exemplos de código que usarei ao longo do livro.

Após o Capítulo 1, muitos dos capítulos podem ser lidos na sequência que os leitores preferirem, exceto:

- É necessário ler o Capítulo 2 antes de ler o Capítulo 3.
- É necessário ler os capítulos 6, 7, 10 e 11 antes de ler o Capítulo 12.

Alguns capítulos têm uma seção que se aprofunda em um tópico sobre machine learning. As seções sempre são apresentadas com o nome de ML e, caso não trabalhe com ML, pode desconsiderá-las, pois isso não impactará no entendimento do restante do capítulo.

Convenções usadas neste livro

Neste livro, as seguintes convenções tipográficas são usadas:

Itálico

Indica termos novos, URLs, endereços de e-mail, nomes de arquivos e extensões de arquivos.

Fonte monoespaçada

Usado para listagens de programas, bem como dentro de parágrafos, para referenciar elementos do programa, como nomes de variáveis ou funções, bancos de dados, tipos de dados, variáveis de ambiente, declarações e palavras-chave.

Fonte monoespaçada em negrito

Mostra comandos ou outro texto que deve ser digitado pelo usuário.

Fonte monoespaçada em itálico

Indica o texto que deve ser substituído por valores fornecidos pelo usuário ou por valores determinados pelo contexto.

Este elemento representa uma dica ou sugestão.

Este elemento representa uma nota geral.

Este elemento indica um aviso ou cuidado.

Uso de exemplos de código de acordo com a política da O'Reilly

O material complementar deste livro (exemplos de código, exercícios etc.) está disponível para download em *https://github.com/catherinenelson1/SEforDS*.

O intuito desta obra é ajudá-lo a realizar suas conquistas. De modo geral, você pode usar os códigos apresentados neste livro em seus programas e documentações. Não é necessário entrar em contato com a editora para pedir permissão de uso, a não ser que reproduza uma parte importante do código. Por exemplo, escrever um programa que usa diversos blocos de código desta obra não exige permissão. Vender ou distribuir exemplos dos livros da O'Reilly em parceria com a Novatec exigirá permissão. Citar esta obra ou mencionar um exemplo de código dela para responder a uma pergunta não exige permissão. No entanto, inserir um volume expressivo de exemplos de código, referentes a este livro, na documentação de seu produto exige permissão.

Agradecemos, mas normalmente não exigimos, citações ou reconhecimentos. Em geral, referências englobam o título, o autor, a editora e o ISBN. No Brasil: NELSON, Catherine. *Engenharia de Software para Cientistas de Dados*. São Paulo: Editora Novatec, 2024.

Se você acredita que o uso de exemplos de código foge ao uso justo ou à

permissão concedia acima, sinta-se à vontade para entrar em contato conosco via *permissions@oreilly.com*.

Como entrar em contato conosco

Envie comentários e dúvidas sobre este livro para: *novatec@novatec.com.br*.

Temos uma página da web para este livro, na qual incluímos a lista de erratas, exemplos e qualquer outra informação adicional.

- Página da edição em português

  http://www.novatec.com.br/livros/engenharia-software-para-cientistas-dados

- Página da edição original, em inglês

  https://oreil.ly/software-engineering-data-scientists

Para obter mais informações sobre livros da Novatec, acesse nosso site em *http://www.novatec.com.br*.

Agradecimentos

Quero agradecer imensamente a todos que me ajudaram com este livro! Os comentários, feedback, discussões e apoio foram inestimáveis.

Foi uma satisfação imensa trabalhar com a equipe da O'Reilly. Obrigada a Virginia Wilson por ser uma editora excelente e prestativa. Gostei muito de trabalhar com você. Obrigada a Nicole Butterfield pela orientação geral valiosa e ajuda com o processo de proposta de livro. Obrigada a Jeff Bleiel pelas revisões minuciosas de diversos capítulos e a Chris Faucher por fazer com que o processo de produção transcorresse sem problemas.

Quero agradecer aos meus revisores técnicos William Jamir Silva, Ganesh Harke, Jo Stichbury, Antony Milne, Jess Males e Swetha Kommuri. O feedback de todos vocês foi superconstrutivo e melhorou muito a versão final do livro. Gostei muito da atenção aos detalhes e das sugestões úteis. Quero agradecer a Rob Masson pelo excelente feedback sobre a versão final e discussões ponderadas durante todo o processo de escrita.

Obrigada a Carol Willing, Ricardo Martín Brualla, Chris Trudeau, Michelle

Liu, Maryam Ehsani, Shivani Patel, John Sweet, Andy Ross e Abigail Mesrenyame Dogbe pelas discussões técnicas valiosas e conversas elucidativas. Fui também incrivelmente beneficiada por fazer parte das comunidades Python e PyLadies; obrigada a todos os voluntários que dedicarem seu tempo.

Por último, obrigada aos meus amigos e familiares incríveis por todo o apoio. Rob, mamãe, Richard, Lina, Salomé, Ricardo, Chris, Kiana e Katie – imensa gratidão a todos.

---

1 N.T.: Data wrangling se refere ao processo de transformação e mapeamento de dados brutos em formatos utilizáveis, podendo incluir a coleta de dados, data cleaning [limpeza de dados], organização, transformação e enriquecimento.

CAPÍTULO 1

O que é um bom código?

O objetivo deste livro é ajudá-lo a escrever um código melhor. Mas, afinal, o que torna um código “bom”? Podemos responder a essa pergunta de diversas maneiras: o melhor código pode ser aquele que executa mais rápido. Ou pode ser aquele mais fácil de ler. Outra definição possível de um bom código é aquele com fácil manutenibilidade. Ou seja, se o projeto passar por mudanças, deve ser fácil voltar ao código e adaptá-lo aos requisitos novos. Os requisitos do código estão sujeitos a mudanças frequentes, seja por atualizações no problema de negócio que estamos tentando abordar, novas orientações de pesquisa ou atualizações em outras partes da base de código.

Além disso, seu código não deve ser complexo e não deve quebrar caso receba uma entrada inesperada. Adicionar uma simples feature nova ao código deve ser fácil; se for difícil, isso sugere que seu código não está bem escrito. Neste capítulo, apresentarei aspectos de um bom código e mostrarei exemplos para cada um deles. Dividirei esses aspectos em cinco categorias: simplicidade, modularidade, legibilidade, desempenho e robustez.

Por que um bom código é importante

Um bom código é extremamente importante quando seu código de ciência de dados é integrado a um sistema maior. Ele pode ser incorporado a um modelo de machine learning em produção, escrever pacotes para ampla utilização ou criar ferramentas para outros cientistas de dados. Pode ser mais útil para bases de código maiores que serão executadas repetidas vezes. À medida que seu projeto aumenta em tamanho e em complexidade, o valor de um bom código também aumenta.

Não raro, o código que você escreve será pontual, um protótipo que precisa ser improvisado hoje para uma demonstração amanhã. E, se você

efetivamente for executar o código apenas uma vez, não perca tempo tentando deixá-lo elegante e aprimorado: basta escrever o código para fazer as tarefas necessárias. No entanto, em minha experiência, mesmo o código que você escreve exclusivamente para uma demonstração quase sempre é executado de novo ou reutilizado para outra finalidade. Após abordar as urgências, incentivo os leitores a dedicar um tempo para revisitar o código e aplicar tidyings[1] para uso futuro.

Práticas de código: craftsmanship

Muitos engenheiros de software enxergam o código como algo que vale a pena escrever bem pelo simples prazer de escrevê-lo. Um trecho de código eficiente e elegante tem valor inerente. Eles se orgulham de um trabalho bem feito, assim como um marceneiro se orgulha de um belo armário de madeira cujas portas abrem sem esforço e as gavetas se encaixam perfeitamente. Eles encontram satisfação profissional em criar programas com alta durabilidade.

Isso não significa que você deva passar horas intermináveis aprimorando os detalhes do código, mas, sim, que terá de tomar uma infinidade de decisões cotidianas sempre que se sentar em frente a um teclado. Sabendo o que procurar, você pode optar por escrever um código melhor. É uma sensação gratificante praticar a arte (craftmanship) de escrever software e criar algo de que você se orgulhe.

Um bom código também é mais fácil de manter. Existe um fenômeno conhecido como degradação de dados [bit-rot]: a necessidade de atualizar o código que não é usado há algum tempo. Isso ocorre porque os elementos dos quais seu código depende também mudam (por exemplo, bibliotecas de terceiros ou até mesmo o sistema operacional usado). Caso revisite o código que não usa há algum tempo, provavelmente precisará fazer alguns ajustes para modernizá-lo. Isso fica mais fácil se o seu código estiver bem estruturado e bem documentado.

Dívida técnica (às vezes abreviada como dívida tech) é um termo comumente usado para trabalho adiado, resultante de quando o código é escrito rapidamente em vez de adequadamente. A dívida tech pode assumir a forma de documentação ausente, código mal estruturado, variáveis nomeadas de forma indevida ou quaisquer outros atalhos. Tudo isso dificulta a manutenibilidade ou refatoração do código e, possivelmente, você dedicará mais tempo futuro a fim de corrigir bugs do que dedicaria caso escrevesse melhor um código. Dito isso, a dívida tech muitas vezes é necessária devido aos prazos e orçamentos de negócios. Nem sempre temos tempo para aperfeiçoar o código.

Adaptando-se aos requisitos em constante mudança

Escrever código não é como construir uma ponte cujo projeto é meticulosamente elaborado, os planos são definitivos e a construção se materializa. A única constante ao escrever código, para um projeto de ciência de dados ou qualquer outro, é esperar mudanças à medida que trabalhamos no projeto. As mudanças podem resultar de suas descobertas durante o processo de pesquisa, de requisitos de negócios em constante mudanças ou de inovações que queremos incluir no projeto. Um bom código pode ser adaptado com facilidade para operar eficientemente com essas mudanças.

A adaptabilidade se torna mais importante à medida que sua base de código cresce. Com um único script conciso, fazer mudanças é simples. Contudo, à medida que o projeto aumenta e é dividido em diversos scripts ou em notebooks que dependem uns dos outros, pode ficar mais complexo e difícil fazer mudanças. Usar boas práticas desde o início facilitará a modificação do código em um projeto maior.

Por mais que a ciência de dados ainda seja uma área relativamente nova, as equipes já começam a se deparar com situações em que trabalham na mesma base de código por vários anos, em que muitas pessoas trabalham no código, algumas das quais podem ter saído da empresa. Nesse cenário, em que um projeto é passado de uma pessoa para outra, a qualidade do código se torna imprescindível. Fica mais fácil entender o trabalho de outra pessoa se estiver adequadamente documentado e for de fácil leitura.

A engenharia de software, como disciplina, lida com requisitos em constante mudança e complexidade crescente há décadas. A disciplina desenvolveu diversas estratégias úteis que você, como cientista de dados, pode emprestar e aproveitar. Caso explore esse universo, pode se deparar com referências como “código limpo”, do livro homônimo de Robert C. Martin, ou siglas como SOLID (*https://oreil.ly/16t90*).

Como mencionei, neste capítulo optei por dividir esses princípios em cinco aspectos de um bom código: simplicidade, modularidade, legibilidade, desempenho e robustez. Explicarei cada um deles em detalhes no decorrer

do capítulo.

Simplicidade

> Simple is better than complex [Simples é melhor do que complexo].
>
> – TIM PETERS, *O ZEN DO PYTHON*

Caso esteja trabalhando em um projeto pequeno, talvez um notebook de visualização de dados ou um breve script de data wrangling possa ajudá-lo a não se esquecer de todos os detalhes. No entanto, à medida que seu projeto aumenta e torna-se mais complexo, isso fica inviável. É possível se lembrar das etapas de treinamento do seu modelo de machine learning, mas é impossível se lembrar do pipeline de dados de entrada ou do processo de implantação do modelo.

A complexidade dificulta a modificação do código quando seus requisitos mudam e pode ser definida da seguinte forma:

> Complexidade é qualquer coisa relacionada à estrutura de um sistema que dificulta a compreensão e modificação de um sistema.
>
> – JOHN K. OUSTERHOUT, *A PHILOSOPHY OF SOFTWARE DESIGN*

Não é uma definição precisa, mas, com a experiência, você terá uma noção de quando um sistema se torna mais complexo. Podemos pensar sobre isso da seguinte forma: quando fazemos uma mudança, algo não relacionado pode quebrar de modo inesperado. Por exemplo, você pode treinar um modelo de machine learning em dados sobre avaliações de clientes, a fim de extrair o item que o cliente comprou com técnicas de processamento de linguagem natural (PLN). Você tem uma etapa separada de pré-processamento que trunca a avaliação em 512 caracteres. Mas, ao implantar o modelo, você se esquece de adicionar a etapa de pré-processamento ao código de inferência. De repente, seu modelo acusa erros porque os dados de entrada são maiores que 512 caracteres. Esse sistema está começando a ficar difícil de entender e está se tornando complexo.

Ao abordar a complexidade no código, geralmente me refiro àquela imprevista, que difere da complexidade inerente de um projeto. Talvez seu projeto de machine learning seja complexo porque você quer testar diversos tipos diferentes de modelos e combinações de features para encontrar qual funciona melhor. Sua análise pode ser complexa porque os dados que você

está usando têm muitos parâmetros interdependentes. Nenhuma delas pode ser simplificada; a complexidade faz parte do projeto. Por outro lado, a complexidade imprevista (também chamada de complexidade acidental) é quando não temos certeza de qual função dentro do código precisa ser alterada para conseguirmos realizar determinada ação.

No entanto, podemos usar ferramentas para nos ajudar a reduzir a complexidade do código. Simplificar cada pequeno detalhe ao longo do processo traz enormes benefícios quando o projeto se torna grande. Na próxima seção, explicarei como manter seu código simples, evitando repetições. Em seguida, abordarei como manter seu código conciso. Você também pode evitar que seu código se torne complexo ao dividi-los em partes reutilizáveis, como explicarei em "Modularidade", na página 26.

Princípio DRY

Um dos princípios mais importantes para escrever um bom código é que a informação não deve ser repetida. Para isso, usamos o acrônimo DRY, do inglês Don't Repeat Yourself [Não Repita a Si Mesmo]. No código, todo conhecimento deve ter uma única representação. Se as informações forem repetidas em múltiplos locais e precisarem ser atualizadas devido a mudanças de requisitos, uma mudança implicará muitas atualizações. Seria necessário se lembrar de todos os locais em que essas informações precisam ser atualizadas. É difícil fazer isso e aumenta a complexidade do código. Além do mais, a duplicação aumenta as chances de bugs, e um código mais extenso exige mais tempo para ser lido e compreendido.

O esforço mental também aumenta quando nos deparamos com dois trechos de código que são muito parecidos, mas não exatamente duplicados. Fica difícil afirmar se os dois trechos estão executando a mesma ação ou uma diferente.

Vejamos um exemplo simples: imagine que quer abrir três arquivos CSV, lê-los em um DataFrame do pandas, processá-los um pouco e retornar cada DataFrame. Os dados deste exemplo foram obtidos a partir dos Objetivos de Desenvolvimento Sustentável (ODS) da ONU (*https://oreil.ly/2MeuI*), e usarei os dados desse site ao longo do livro. Para mais detalhes, confira a seção "Dados neste livro", na página 32. O código e os arquivos CSV estão disponíveis no repositório do GitHub deste livro (*https://oreil.ly/SEforDS*).

Vejamos uma primeira abordagem:

```
import pandas as pd

df = pd.read_csv("sdg_literacy_rate.csv")
df = df.drop(["Series Name", "Series Code", "Country Code"], axis=1)
df = df.set_index("Country Name").transpose()

df2 = pd.read_csv("sdg_electricity_data.csv")
df2 = df2.drop(["Series Name", "Series Code", "Country Code"], axis=1)
df2 = df2.set_index("Country Name").transpose()

df3 = pd.read_csv("sdg_urban_population.csv")
df3 = df3.drop(["Series Name", "Series Code", "Country Code"], axis=1)
df3 = df3.set_index("Country Name").transpose()
```

O problema desse código é ser desnecessariamente prolixo e repetitivo. Para obter o mesmo resultado, seria melhor inserir o código repetido dentro de um loop `for`. Caso seja algo que use repetidas vezes, é possível inserir o código dentro de uma função, assim:

```
def process_sdg_data(csv_file, columns_to_drop):
    df = pd.read_csv(csv_file)
    df = df.drop(columns_to_drop, axis=1)
    df = df.set_index("Country Name").transpose()
    return df
```

Outros casos, mais sutis, podem levar à duplicação de código. Vamos conferir alguns exemplos:

- Talvez você se dê conta de que está usando um código muito semelhante em múltiplos projetos sem perceber, por exemplo, código de processamento de dados. Segmentar o código de processamento, de forma que ele possa aceitar dados levemente variados em vez de apenas um tipo exato, pode ajudá-lo a evitar essa duplicação.
- Muitas pessoas trabalhando em projetos semelhantes podem escrever códigos parecidos, principalmente se não se comunicarem em que estão trabalhando. Facilitar o uso do código para outras pessoas e fornecer uma boa documentação ajudará a reduzir esse tipo de duplicação.

- Comentários e documentação também podem ser uma forma de duplicação. O mesmo conhecimento é representado no código e na documentação que o descreve. Não faça comentários que descrevem exatamente o que o código está fazendo; do contrário, use comentários a fim de adicionar informações relevantes. No Capítulo 9, analisaremos esse tópico em mais detalhes.

O princípio DRY é imprescindível ao escrever um bom código. Pode parecer irrelevante, mas, para evitar repetição, o código precisa ser modular e legível. Mais adiante, analisaremos esses conceitos.

Evite código verboso

Às vezes, podemos simplificar o código para ter menos linhas. Isso significa menos chances de bugs e menos código para outra pessoa ler e entender. No entanto, outras vezes, temos de analisar os prós e contras entre um código menor e um código menos legível. Na seção "Legibilidade", da página 28, discutiremos como garantir um código legível.

Recomendo que procure tornar seu código conciso, mas que ainda seja legível. Para isso, evite coisas que tornam o código desnecessariamente prolixo, como escrever as próprias funções em vez de usar funções built-in ou variáveis temporárias desnecessárias. Devem-se também evitar repetições, conforme descrito na seção anterior.

Vejamos um exemplo de uma variável temporária desnecessária:

```
i = float(i)
image_vector.append(i/255.0)
```

Podemos simplificá-la da seguinte forma:

```
image_vector.append(float(i)/255)
```

Claro que existem desvantagens em condensar o código em menos linhas. Se muita coisa estiver acontecendo em uma única linha, qualquer outra pessoa pode ter muita dificuldade em entender isso. Ou seja, fica mais difícil para outra pessoa trabalhar em seu código, e isso pode resultar em mais bugs. Em caso de dúvida, recomendo que mantenha o código legível, mesmo que isso signifique usar algumas linhas extras.

Modularidade

Escrever código modular é a arte de dividir um grande sistema em componentes menores. O código modular tem diversas vantagens importantes: facilita a leitura e a localização da origem de um problema e simplifica a reutilização dele em outro projeto. Fica também mais fácil testar o código que está dividido em componentes menores, como veremos no Capítulo 7.

Mas como lidar com uma tarefa grande? É possível escrever um script extenso para fazer tudo, e isso pode funcionar no início de um projeto pequeno. No entanto, grandes projetos precisam ser divididos em partes menores. Para tal, é necessário pensar a longo prazo no futuro do projeto o máximo possível e tentar prever o funcionamento do sistema como um todo e quais abordagens razoáveis para dividi-lo. No Capítulo 8, analisaremos esse processo de planejamento em mais detalhes.

Escrever código modular é um processo contínuo, não conseguimos escrever com excelência desde o início, ainda que tenhamos as melhores das intenções. Espera-se que o código passe por mudanças à medida que o projeto evolui. Na página 160, “Refatoração”, abordarei técnicas que ajudarão os leitores a melhorar o código.

É possível dividir um grande projeto de ciência de dados em uma série de etapas, ao considerá-lo como um fluxograma, conforme mostrado na Figura 1.1. Primeiro, extraímos alguns dados, em seguida os exploramos, os limpamos e os visualizamos.

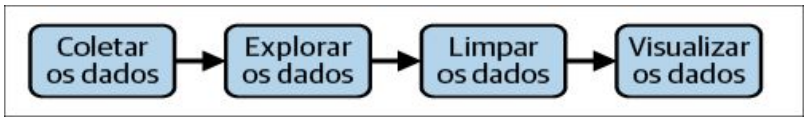


Figura 1.1: Dividindo um grande projeto de ciência de dados em etapas discretas.

A princípio, pode ser uma série de notebooks Jupyter. No final de cada um, é possível salvar os dados em um arquivo e carregá-los novamente no próximo notebook. À medida que o projeto amadurece, talvez você queira executar uma análise semelhante repetidas vezes. Assim sendo, você pode definir qual deve ser a estrutura básica do sistema: talvez uma função que

extraia os dados e, em seguida, passe-os para uma função que limpa os dados. O exemplo a seguir usa a instrução `pass` para criar uma função vazia. Isso garante que, quando você chamar essa função, ela não causará um erro antes de ser escrita.

Por exemplo, vejamos o que poderia ser a estrutura básica de um sistema que carrega alguns dados, limpa-os ao ajustá-los [cropping] em um determinado tamanho e os plota com alguns parâmetros de representação gráfica.

```
def load_data(csv_file):
    pass


def clean_data(input_data, max_length):
    pass


def plot_data(clean_data, x_axis_limit, line_width):
    pass
```

Ao criar essa estrutura, dividimos o sistema em componentes individuais e sabemos o que cada um desses componentes deve aceitar como entrada. Podemos fazer a mesma coisa em nível de um arquivo Python. Usar um paradigma de programação, como programação orientada a objetos ou programação funcional, pode ajudá-lo a descobrir como dividir seu código em funções e classes (no Capítulo 4, falaremos mais a respeito). Independentemente de como você divida seu sistema, cada um dos componentes deve ser o mais independente e autônomo possível, a fim de que a mudança de um componente não mude outro. No Capítulo 8, analisaremos o código modular em mais detalhes.

Legibilidade

> ...o código é lido com muito mais frequência do que escrito...
>
> – PEP8

Ao escrever código, é importante que outras pessoas também possam utilizá-lo. Você pode migrar para um projeto diferente ou até mesmo mudar de emprego. Caso saia de um projeto por um tempo e depois retorne, após um mês, seis meses ou seis anos, você ainda conseguirá entender o que

estava fazendo na época em que escreveu o código? Você escreveu determinado código por um motivo, para uma tarefa que era importante, e torná-lo legível garante sua vida útil.

Os métodos para tornar o código mais legível incluem adotar padrões e convenções da linguagem de programação em que está escrito, escolher bons nomes, remover código não utilizado e elaborar documentação. É tentador tratar esses aspectos como segundo plano e focar nossa energia nas funcionalidades; porém, quando nos preocupamos com a legibilidade assim que escrevemos um código, criamos um código menos complexo e mais fácil de manter. Nesta seção, apresentarei esses métodos e os explorarei detalhadamente nos capítulos 6 e 9.

Padrões e convenções

Apesar de parecerem os tópicos menos interessantes que abordarei neste livro, os padrões de programação e de formatação são extremamente importantes. Podemos expressar o mesmo código de diversas maneiras, até mesmo em pequenos detalhes, como o espaçamento ao redor do sinal de `+` quando somamos dois números inteiros. Os padrões de programação foram desenvolvidos para incentivar a consistência entre todos que escrevem código em Python, e o objetivo é fazer com que o código pareça familiar, mesmo quando outra pessoa o escreveu. Isso ajuda a reduzir o esforço necessário para ler e editar um código que não escrevemos. No Capítulo 6, abordarei esse tópico em mais detalhes.

O Python é inerentemente mais legível em comparação com muitas linguagens de programação; adotar um padrão de programação tornará a leitura do código ainda mais fácil. O principal padrão de programação para o Python é o PEP8 (Python Enhancement Proposal 8 [Proposta de Melhoria para o Python, em tradução livre], *https://oreil.ly/UwHsO*), estabelecido em 2001. O exemplo a seguir demonstra um trecho do PEP8, e você pode observar que existem convenções até para os detalhes mais ínfimos do seu código. Guias de estilo, como o Python Style Guide [Guia de Estilo Python] do Google (*https://oreil.ly/Q3eZL*), complementam o PEP8 com orientações e informações adicionais.

Vejamos um exemplo dos detalhes que o PEP8 especifica, mostrando as formas corretas e incorretas de formatar espaços entre colchetes:

```
# Correto:
spam(ham[1], {eggs: 2})

# Incorreto:
spam( ham[ 1 ], { eggs: 2 } )
```

Felizmente, existem muitas formas automatizadas de verificar se seu código está em conformidade com os padrões de programação, poupando-lhe o trabalho tedioso de verificar se cada sinal de `+` tem um único espaço ao seu redor. Linters como Flake8 e Pylint destacam os locais em que o código não está em conformidade com o PEP8. Formatadores automáticos, como o Black, atualizarão o código automaticamente para estar em conformidade com os padrões de programação. No Capítulo 6, abordaremos como usá-los.

Nomes

Ao escrever código para ciência de dados, você precisará escolher nomes em diversos momentos: nomes de funções, variáveis, projetos e até mesmo nomes de ferramentas inteiras. Sua escolha de nomes afeta a facilidade de trabalhar em seu código. Caso escolha nomes que não sejam descritivos ou precisos, precisará lembrar o que verdadeiramente significam, e isso aumentará o esforço mental para entender seu código. Por exemplo, é possível importar a biblioteca pandas como `p` e nomear suas variáveis `x` e `f`, como mostrado aqui:

```
import pandas as p

x = p.read_csv(f, index_col=0)
```

Esse código executou corretamente, sem erros. Mas vejamos um exemplo de código mais fácil de ler, com nomes de variáveis mais informativos, que adotam as convenções padrão de nomenclatura:

```
import pandas as pd

df = pd.read_csv(input_file, index_col=0)
```

No Capítulo 9, veremos mais detalhes sobre como escrever bons nomes.

Limpando o código

Outra forma de tornar seu código mais legível é limpá-lo, após terminar de

criar uma função. Depois de testá-lo e ter certeza de que está funcionando, você deve remover o código que foi comentado e as chamadas desnecessárias à função print(), que você pode ter usado como uma forma simples de depuração. É muito confuso ler as seções comentadas no código de outra pessoa.

Quando vemos seções bagunçadas, temos a impressão de que a qualidade insatisfatória do código é aceitável em um projeto. Isso significa que há menos incentivo para outros colaboradores escreverem um bom código. O código desorganizado também pode ser copiado e adaptado em outras partes do projeto. Isso é conhecido como a Teoria das Janelas Quebradas (*https://oreil.ly/3sa7c*). Estabelecer altos padrões em um projeto incentiva todos que trabalham nele a escrever um bom código.

Se quiser melhorar a qualidade do seu código, pode optar por refatorá-lo. Refatorar significa mudar o código sem alterar seu comportamento geral. Talvez você esteja pensando em maneiras de tornar seu código mais eficiente ou em como melhor estruturá-lo para que seus colegas de equipe possam usar partes dele em outro projeto. Nesse processo, os testes são essenciais, pois verificarão se o novo código ainda tem o mesmo comportamento geral. Na seção “Refatoração”, da página 160, abordarei a refatoração.

Documentação

A documentação também ajuda outras pessoas a ler seu código. O código pode ser documentado em múltiplos níveis de detalhe: com comentários simples em linha, com docstrings que explicam uma função inteira, com uma página README, exibida em algum repositório GitHub, e até com tutoriais para ensinar os usuários a usar um pacote. Todos esses aspectos da documentação ajudam a explicar a outras pessoas como usar seu código. Podem até explicar seu código para você mesmo no futuro (um usuário muito importante!). Se quiser que outras pessoas usem seu código, você deve facilitar isso ao escrever uma boa documentação.

Escrever uma excelente documentação é importante, mas também é preciso mantê-la e atualizá-la regularmente. Uma documentação que se refere a uma versão desatualizada do código é pior do que nenhuma documentação. Isso causará uma confusão que levará mais tempo para ser resolvida. No

Capítulo 9, analisaremos detalhadamente todas as modalidades de documentação.

Desempenho

Um bom código precisa ser eficiente. E o mensuramos tanto pelo tempo de execução [running time] quanto pelo uso de memória. Ao tomar decisões sobre como escrever seu código, é útil saber quais estruturas de dados e algoritmos são mais eficientes. Ajuda muito saber quando estamos fazendo coisas que podem comprometer significativamente nosso código, sobretudo quando temos alternativas disponíveis. Devemos também estar cientes de quais partes do código estão demorando para ser executadas.

O desempenho é imprescindível quando estamos escrevendo um código de produção que será chamado sempre que um usuário realizar uma ação específica. Caso sua base de usuários cresça ou seu projeto seja um sucesso, seu código pode ser chamado milhares de vezes todos os dias. Nesse caso, até as singelas melhorias no código podem economizar tempo para os usuários. Você não quer que seu código seja o ponto de lentidão em uma aplicação importante. No Capítulo 2, explicarei como podemos medir o desempenho do nosso código e, no Capítulo 3, mostrarei como escolher as melhores estruturas de dados a fim de otimizar o desempenho de nosso código.

Robustez

Um bom código também deve ser robusto. Significa que ele deve ser reproduzível: você deve ser capaz de executar seu código do início ao fim sem que ele falhe. Seu código também deve ser capaz de responder com elegância se as entradas do sistema mudarem inesperadamente. Em vez de gerar um erro inesperado que pode causar a falha de um sistema maior, seu código deve ser projetado para responder às mudanças. Seu código pode ser mais robusto com o tratamento adequado de erros, o registro em logs do que aconteceu e a escrita de bons testes.

Erros e registro em logs

Um código robusto não deve se comportar de maneira inesperada ao receber uma entrada incorreta. Precisamos escolher se queremos que código falhe em uma entrada inesperada ou trate esse erro e faça algo a respeito. Por exemplo, se faltar metade das linhas esperadas de dados em um arquivo CSV, você quer que seu código retorne um erro ou continue processando somente metade dos dados? Você deve fazer uma escolha explícita a fim de gerar um alerta de que algo não está funcionando como deveria, tratar o erro ou falhar sem acusar erros [fail silently]. No Capítulo 5, analisaremos os erros em mais detalhes.

Se o erro for tratado, ainda pode ser importante registrar sua ocorrência para que não falhe sem acusar erros, caso essa não seja a sua intenção. Trata-se de um caso de uso para o registro em logs; no Capítulo 5, explorarei outros usos do registro em logs.

Testando o código

Testes são fundamentais para escrever um código robusto. A engenharia de software usa duas abordagens principais de teste: teste de usuário, em que uma pessoa usa um software para validar se ele funciona adequadamente, e teste automatizado. Um método comum de testes automatizados é enviar uma entrada de exemplo para um trecho de código e validar se a saída é o que você esperava. Neste livro, abordarei apenas os testes automatizados.

Testes são necessários porque, mesmo que o código funcione perfeitamente em sua máquina, não significa que funcionará na máquina de outra pessoa ou mesmo na sua máquina futuramente. Dados mudam, bibliotecas são atualizadas e máquinas distintas executam versões diferentes do Python. Se outra pessoa quiser usar seu código na máquina dela, poderá executar testes para validar se ele funciona.

Podemos executar inúmeros tipos diferentes de testes. Testes unitários testam uma única função, testes de ponta a ponta (ESE) testam um projeto inteiro e testes de integração testam um pedaço de código que contém muitas funções, embora seja menor do que um projeto inteiro. No Capítulo 7, exploraremos a fundo as estratégias de teste e as bibliotecas. No entanto, caso tenha acesso a uma grande base de código que não tenha sido testada, uma boa estratégia para começar é escrever um teste quando o código quebra a fim de garantir que isso não aconteça novamente.

Dados neste livro

No decorrer desta obra, usarei dados sobre os Objetivos de Desenvolvimento Sustentável (ODS) das Nações Unidas (*https://sdgs.un.org/goals*). Os ODS são um conjunto de 17 metas que fazem parte da Agenda de 2030 para o Desenvolvimento Sustentável, adotada pelos países-membros da ONU em 2015. Os objetivos incluem acabar com a pobreza, erradicar a fome, acesso à educação, igualdade de gênero e muito mais. Os ODS são divididos em metas secundárias e monitorados com um conjunto de mais de 200 indicadores estatísticos. Os indicadores medem quantitativamente o progresso no sentido de alcançar esses objetivos.

Por exemplo, a meta do ODS 1 é "Acabar com a pobreza em todas as suas formas, em todos os lugares"; A meta 1.1 é, "Até 2030, erradicar a pobreza extrema para todas as pessoas em todos os lugares, atualmente medida como pessoas vivendo com menos de US$1,25 por dia". O indicador 1.1.1 é "Percentual da população abaixo da linha internacional de pobreza extrema, por sexo, idade, status de ocupação e localização geográfica (urbano/rural)".

Os dados para esses indicadores estão disponíveis em um banco de dados online (*https://oreil.ly/X3rDE*) e por meio de uma API (*https://oreil.ly/rBEv-*). Neste livro, usarei os dados dos indicadores nos códigos de exemplo.

Aspectos fundamentais

Escrever um bom código pode ajudá-lo de diversas formas: será mais fácil para outras pessoas usarem seu código; ajudará você a entender o que estava fazendo quando retomar o trabalho seis meses depois; e ajudará seu código a aumentar a escala [scale-up] e a interagir com um sistema maior. Um bom código também facilitará bastante a sua vida, caso precise adicionar features a ele que não estavam previamente planejadas pelo projeto.

Caso tenha interesse em ler mais sobre os princípios para escrever um bom código, recomendo os seguintes livros:

- *The Pragmatic Programmer*, 20th Anniversary Edition [O Programador Pragmático, edição do 20º aniversário, em tradução livre], de David Thomas e Andrew Hunt (Addison-Wesley Professional)
- *A Philosophy of Software Architecture* [Uma Filosofia de Arquitetura de Software, em tradução livre], de John Ousterhout (Yaknyam Press)

Em suma, vejamos algumas considerações de como escrever um bom código:

Simplicidade

Você deve evitar escrever um código com repetição, complexidade

desnecessária e linhas redundantes de código.

Modularidade

Você deve dividir seu código em funções lógicas, com entradas e saídas bem definidas.

Legibilidade

Quando escrever código, você deve adotar os padrões de formatação do PEP8, escolher bem os nomes e documentá-lo adequadamente.

Desempenho

Seu código não deve demorar desnecessariamente para ser executado ou usar mais recursos do que os disponíveis.

Robustez

Você deve escrever um código que seja reproduzível, que gere mensagens úteis de erro e que lide com entradas inesperadas sem falhar.

No próximo capítulo, analisarei com mais detalhes um dos aspectos de um bom código: o desempenho.

---

1 N.T.: O termo tidy ou tidyings se refere à filosofia criada por Kent Beck para manter o código limpo e organizado. É como se fossem minirrefatorações que contribuem para a legibilidade e manutenibilidade do código. Para mais informações, sugiro a leitura do livro Tidy First? Minirrefatorações para um melhor design de software,publicado pela Novatec.

CAPÍTULO 2

Analisando o desempenho do código

No Capítulo 1, expliquei alguns aspectos da escrita de um bom código e, neste capítulo, me aprofundarei em um desses aspectos: o desempenho. Se você está escrevendo um código que faz parte de um sistema maior, é crucial que ele seja eficiente. Ele deve retornar saídas em tempo hábil e não deve consumir mais do que os recursos computacionais disponíveis.

No entanto, primeiro, o mais importante é que seu código funcione. Antes de aplicar qualquer uma das técnicas deste capítulo ou do próximo, garanta que seu código resolva o problema que precisa resolver e retorne as saídas esperadas. Qualquer otimização ou aceleração deve ocorrer depois que o código já estiver sendo executado corretamente.

Segundo, pergunte se há um requisito para melhorar o desempenho. Será necessário conhecer as expectativas do sistema maior com o qual seu código está interagindo. Seu código precisa retornar um resultado dentro de um período de tempo específico? Seu usuário está esperando para agir ou tomar alguma decisão com base nos resultados do seu código? Em caso afirmativo, quanto mais rápido seu código funcionar, melhor será a experiência do usuário.

Por exemplo, seu modelo de machine learning pode precisar retornar uma predição dentro de 100 ms, de modo que possa ser mostrada ao usuário. Caso seu código já retorne o resultado dentro do tempo permitido, talvez não haja necessidade de mobilar tempo otimizando seu desempenho.

No entanto, é uma boa ideia conhecer os princípios deste capítulo. É útil saber quando estamos escrevendo um código ineficiente (por exemplo, caso precise implementar um solução improvisada às pressas) para podermos retomá-lo mais tarde e otimizá-lo. No Capítulo 3, abordarei como escolher a estrutura de dados mais eficiente para o seu código.

Como cientista de dados, você estará familiarizado com a ideia de que

medições são importantes. A primeira etapa para melhorar o desempenho do seu código é identificar quais partes dele estão mais lentas ou ocupam mais memória. Após coletar esses dados, você saberá se essas partes lentas ou que consomem muita memória podem ser corrigidas. A lentidão pode resultar de ineficiências em seu código ou ele já pode ser o mais eficiente possível, mas você precisará de dados para saber qual é o seu caso. Na seção “Cronometrando a execução do código”, na página 38, e na “Criando o perfilamento do código”, na página 41, abordarei como medir o desempenho do seu código.

Otimização prematura

> ... a otimização prematura é a raiz de todo o mal (ou, pelo menos, a maior parte dele) na programação.
>
> – DONALD KNUTH, *THE ART OF COMPUTER PROGRAMMING [A ARTE DA PROGRAMAÇÃO DE COMPUTADORES, EM TRADUÇÃO LIVRE]*

Esta citação de Donald Knuth é um clássico amplamente conhecido no mundo da engenharia de software. E, apesar de muitas outras coisas poderem ser consideradas como a raiz de todo o mal no mundo da ciência de dados, essa citação ainda é um conselho útil. Interpreto ela como uma recomendação para não otimizar o código antes de conhecer os requisitos de tempo ou de memória deste mesmo código ou antes de saber onde estão os gargalos.

Métodos para melhorar o desempenho

Nesta seção, definirei bom desempenho como a minimização dos recursos computacionais que o código usa. Não entrarei em detalhes sobre arquitetura computacional por se tratar de um tópico extremamente abrangente. Mas é possível mensurar o efeito de diferentes códigos quanto ao tempo que levam para concluir uma tarefa ou quanto à quantidade necessária de memória do computador. Portanto, código com alto desempenho significa código mais rápido ou que ocupa menos espaço na memória, mesmo que, não raro, precisemos ponderar os prós e contras de que um código mais rápido consome mais memória.

A escolha de métodos para melhorar o desempenho do seu código depende, entre outros fatores, da linguagem de programação que você está usando. No Python, as principais opções disponíveis são as seguintes:

A escolha do algoritmo

As escolhas que você faz no código que escreve podem fazer uma grande

diferença em seu desempenho. Por exemplo, evite loops aninhados sempre que possível para não iterar uma lista duas vezes quando precisar iterá-la somente uma vez.

A escolha da estrutura de dados

Dependendo da tarefa que você precisa realizar, diferentes estruturas de dados podem significar diferentes prós e contras. Por exemplo, é muito mais rápido procurar um valor em um dicionário do que o procurar em uma lista. No Capítulo 3, abordaremos esse tópico com mais detalhes.

Usando funções built-in

Se existe uma função bult-in para resolver seu problema, quase sempre é mais eficiente usá-la do que escrever a própria função. Muitas dessas funções built-in são implementadas em C, o que geralmente é mais rápido do que recriar a mesma função em Python.

Compilando Python

É possível executar seu código de forma mais rápida ao compilar o Python para uma linguagem de baixo nível, usando ferramentas como Cython (*https://cython.org*), Numba (*https://numba.pydata.org*) e PyPy (*https://www.pypy.org*). A escolha do que você usará dependerá do seu caso de uso. O Numba contém um subconjunto do Python; o Cython é um superconjunto do Python com opções adicionais em C; e o PyPy é uma reimplementação do Python com compilação just-in-time (JIT).

Código assíncrono

É possível acelerar seu código fazendo com que ele realize uma tarefa enquanto aguarda o resultado de outra. Por exemplo, seu código pode estar aguardando uma API retornar uma resposta em uma rede lenta.

Computação paralela e computação distribuída

Computação paralela significa executar seu código em mais de um processador dentro de um computador. Com o módulo multiprocessing, é possível executar seu código em diversas CPUs ao mesmo tempo. Computação distribuída significa executar seu código em diversas máquinas diferentes ao mesmo tempo.

Neste livro, foco o que os leitores podem fazer para melhorar o código que

escrevem rotineiramente, sem precisar dedicar tempo para aprender muitas ferramentas e técnicas novas. Ou seja, não abordarei todas as opções listadas anteriormente. Neste capítulo, me concentrarei em como medir o desempenho e em como escolher um algoritmo. Saber como medir o desempenho do seu código possibilitará que você saiba quando as funções built-in podem lhe fornecer uma melhoria. No Capítulo 3, também abordarei como escolher a melhor estrutura de dados para o seu caso de uso. E mencionarei brevemente a computação paralela e a computação distribuída na seção "Operações de arrays com a biblioteca Dask", da página 65.

Caso queira se aprofundar no tópico sobre desempenho do código, recomendo o livro *High Performance Python: Practical Performant Programming for Humans* [Python Com Alto Desempenho: Programação Prática Com Alto Desempenho para Humanos, em tradução livre], de Micha Gorelick e Ian Osvald (O'Reilly, 2020), como próximo passo. Esse livro também inclui detalhes abrangentes sobre diferentes compiladores para Python. Além disso, você pode ler minha introdução favorita ao código assíncrono na documentação da FastAPI (*https://oreil.ly/JtZED*).

Cronometrando a execução do código

O modo mais simples de descobrir quais partes do seu código estão sendo executadas lentamente é medir o tempo que demora para executar uma função ou uma linha de código, fazer uma mudança e, em seguida, medir o tempo novamente. É boa prática fazer apenas uma mudança de cada vez, caso contrário, você não saberá o que causou o aumento ou a redução do desempenho.

Neste capítulo, uso exemplos de algumas funções estatísticas simples. Não recomendo que use esse código ou implemente essas funções sozinho (elas estão disponíveis no pacote statistics ou no NumPy). Eu as uso somente para exemplificar minhas observações.

O exemplo a seguir mostra um trecho de código simples que calcula uma medida estatística, conhecida como moda, de uma lista de números. Vejamos como você pode medir o tempo de execução do seu código usando este exemplo:

```
def slow_way_to_calculate_mode(list_of_numbers):
    result_dict = {}
    for i in list_of_numbers:
        if i not in result_dict:
            result_dict[i] = 1
        else:
            result_dict[i] += 1

    mode_vals = []
    max_frequency = max(result_dict.values())
    for key, value in result_dict.items():
        if value == max_frequency:
            mode_vals.append(key)

    return mode_vals
```

Agora, geraremos uma lista de 100.000 números inteiros aleatórios para inserir nesta função:

```
import numpy as np

random_integers = np.random.randint(1, 1_000_000, 1_000_000)
```

A forma mais simples de medir quanto tempo essa função leva para ser executada é recorrer ao módulo `time`. É possível cronometrar o tempo no relógio do computador antes e depois de executar a função e, em seguida, exibir a diferença:

```
import time

start = time.time()
slow_way_to_calculate_mode(random_integers)
end = time.time()

print(end-start)
```

No entanto, o tempo de execução de uma função pode variar, por isso é bom medi-lo múltiplas vezes a fim de obter um resultado robusto. Podemos fazer essa medição com o módulo `timeit`. Caso esteja usando o Jupyter Notebook, é possível realizar elegante e magicamente a medição da célula

com o comando %%timeit:

```
>>> %%timeit
... slow_way_to_calculate_mode(random_integers)

267 ms ± 23.4 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

Nesse caso, o timeit percorreu o código 10 vezes e repetiu o conjunto de dez loops 7 vezes e, em seguida, retornou as estatísticas resumidas dessas execuções. O padrão do timeit é o número de execuções e de loops que caberão em uma janela de 2 segundos, mas podemos especificar o número de execuções com a flag -r e o número de loops com a flag -n.

Caso esteja usando um script independente, é possível usar o timeit, conforme mostrado neste exemplo:

```
import numpy as np
import timeit

random_integers = np.random.randint(1, 100_000, 1000)

def slow_way_to_calculate_mode(list_of_numbers):
    result_dict = {}
    for i in list_of_numbers:
        if i not in result_dict:
            result_dict[i] = 1
        else:
            result_dict[i] += 1

    mode_vals = []
    max_frequency = max(result_dict.values())
    for key, value in result_dict.items():
        if value == max_frequency:
            mode_vals.append(key)

    return mode_vals

mode_timer = timeit.Timer(stmt="slow_way_to_calculate_mode(random_integers)",
                          setup="from __main__ import"\
```

```
                    "slow_way_to_calculate_mode,random_integers")

time_taken = mode_timer.timeit(number=10)

print(f"Execution time: {time_taken} seconds")
```

Mas esse código é bom? Duzentos e sessenta e sete ms é rápido ou lento? Como esse é apenas um exemplo sem requisitos de velocidade do mundo cotidiano, vou compará-lo com outra forma de calcular a moda e verificar se o desempenho melhora. Vejamos outra forma de realizar o mesmo cálculo:

```
from collections import Counter

def mode_using_counter(list_of_numbers):
    c = Counter(list_of_numbers)
    return c.most_common(1)[0][0]
```

E usaremos mais uma vez o timeit para medir quanto tempo leva para ser executado:

```
>>> %%timeit
... mode_using_counter(random_integers)

23.2 ms ± 737 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

Como podemos observar, essa versão do código demora menos tempo para ser executada. Em vez de uma média de cerca de 267 ms, leva aproximadamente 23 ms. O desvio-padrão também é menor, o que ajuda a garantir um limite superior para o tempo de execução.

Você pode extrair duas observações a partir deste exemplo: a primeira é que códigos diferentes podem executar exatamente a mesma função, porém uma versão pode ser bem mais rápida do que outra. A segunda é que o uso de funções built-in (neste caso, a Counter do módulo collections) geralmente é mais rápido do que escrevermos nossa própria função. Os módulos collections e itertools têm muitas funções built-in que podem ajudá-lo com o alto desempenho de funções comuns.

Criando o perfilamento do código

Embora cronometrar o código com o %% timeit seja ótimo para linhas individuais de código, se tivermos uma função maior ou um script inteiro, será cansativo dividir cada linha em sua própria célula de notebook e cronometrar tudo separadamente. É nesse momento que o profiler entra em cena. Profilers conseguem indicar qual parte de uma função leva mais tempo para executar e fornecer níveis extras de detalhes, facilitando a localização de gargalos em seu código. Nesta seção, também incluirei detalhes sobre como criar um profile do uso de memória do seu código.

cProfile

O cProfile é um profiler built-in do Python. Podemos usá-lo para obter uma visão geral e básica dos locais de gargalos em um script mais extenso. No exemplo a seguir, inseri o gerador de números aleatórios dentro da função mode da seção anterior, a fim de que o profiler mostrasse mais informações:

```
import numpy as np
from collections import Counter

def mode_using_counter(n_integers):
    random_integers = np.random.randint(1, 100_000, n_integers)
    c = Counter(random_integers)
    return c.most_common(1)[0][0]
```

Para executar o profiler, use o comando:

```
%%prun
mode_using_counter(10_000_000)
```

Você deve receber uma saída mais ou menos assim:

```
    25 function calls in 2.585 seconds

Ordered by: internal time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
     1    2.467    2.467    2.467    2.467 {built-in method
                                   _collections._count_elements}
     1    0.104    0.104    0.104    0.104 {method 'randint' of
                                   'numpy.random.mtrand.RandomState'
                                   objects}
```

```
1   0.007   0.007   2.584   2.584 <string>:1(<module>)
1   0.006   0.006   0.006   0.006 {built-in method builtins.max}
1   0.000   0.000   2.467   2.467 __init__.py:649(update)
1   0.000   0.000   2.585   2.585 {built-in method builtins.exec}
1   0.000   0.000   0.000   0.000 {method 'reduce' of
                          'numpy.ufunc' objects}
1   0.000   0.000   2.577   2.577 3744758285.py:
                          (mode_using_counter)
1   0.000   0.000   2.467   2.467 __init__.py:581(__init__)
1   0.000   0.000   0.000   0.000 fromnumeric.py:69(_wrapreduction)
1   0.000   0.000   0.006   0.006 heapq.py:521(nlargest)
1   0.000   0.000   0.006   0.006 __init__.py:600(most_common)
1   0.000   0.000   0.000   0.000 <__array_function__ internals>:177
                          (prod)
1   0.000   0.000   0.000   0.000 {built-in method
                          _abc._abc_instancecheck}
1   0.000   0.000   0.000   0.000 fromnumeric.py:2927(prod)
1   0.000   0.000   0.000   0.000 abc.py:117(__instancecheck__)
1   0.000   0.000   0.000   0.000 {built-in method
                          builtins.isinstance}
1   0.000   0.000   0.000   0.000 {built-in method numpy.core.
                          _multiarray_umath.
                          implement_array_function}
1   0.000   0.000   0.000   0.000 {built-in method builtins.getattr}
1   0.000   0.000   0.000   0.000 fromnumeric.py:70(<dictcomp>)
2   0.000   0.000   0.000   0.000 {method 'items' of 'dict' objects}
1   0.000   0.000   0.000   0.000 {built-in method builtins.iter}
1   0.000   0.000   0.000   0.000 {method 'disable' of
                          '_lsprof.Profiler' objects}
1   0.000   0.000   0.000   0.000 fromnumeric.py:2922
                          (_prod_dispatcher)
```

Nessa saída, a coluna tottime mostra em que local o computador mobilizou a maior parte do tempo ao executar esse código: na função built-in method_collections._count_elements, que é a função Counter. A segunda parte mais demorada foi method 'randint' of 'numpy.random.mtrand.RandomState' objects, o passo [step] que criou a lista de números aleatórios. Todos os outros passos

demoraram pouco tempo. Usar cProfile tem uma desvantagem: é necessário mapear cada uma dessas chamadas de função de volta às linhas do seu código.

Podemos também usar o pacote SnakeViz para obter uma visualização gráfica dos resultados do cProfile.

É possível instalar o SnakeViz com o seguinte comando:

```
$ pip install snakeviz
```

Em seguida, caso esteja usando o Jupyter Notebook, poderá usar a extensão do SnakeViz. É possível carregá-la com o seguinte comando:

```
>>> %load_ext snakeviz
```

Em seguida, é possível executar o SnakeViz assim:

```
>>> %%snakeviz
>>> mode_using_counter(10_000_000)
```

Isso gera o resultado mostrado na Figura 2.1.

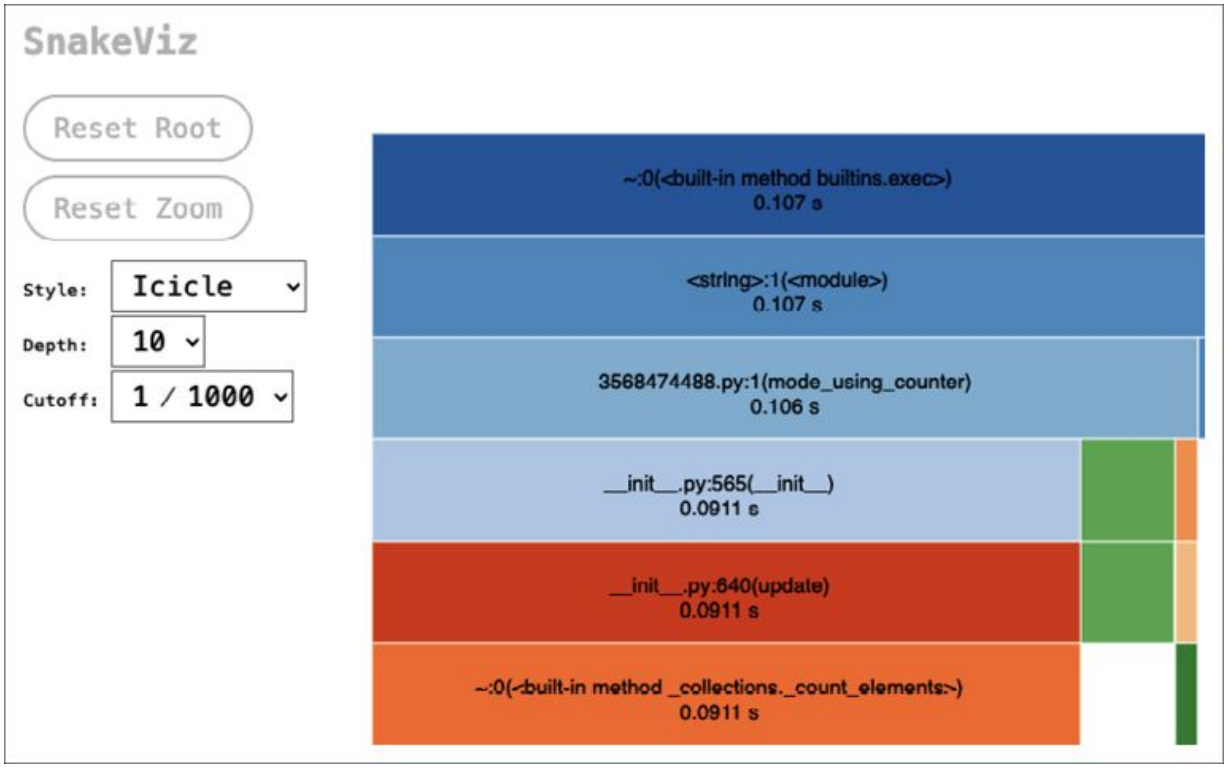


Figura 2.1: Visualizando a saída do cProfile *com o SnakeViz.*

Isso pode ser mais fácil de interpretar do que a saída padrão do cProfile.

line_profiler

O cProfile fornece uma análise detalhada de como o computador aloca seu tempo, incluindo detalhes do funcionamento interno do Python. No entanto, sua saída não é nada fácil de ler e pode se aprofundar demais nos métodos built-in do Python chamados pelo seu código. O pacote line_profiler (*https://oreil.ly/KcGKg*) fornece um detalhamento mais legível do seu código.

O line_profiler não vem instalado por padrão. Ou seja, é possível instalá-lo com:

```
$ pip install line_profiler
```

Em seguida, é necessário carregá-lo como uma extensão separada no Jupyter Notebook:

```
>>> %load_ext line_profiler
```

É necessário informar explicitamente ao line_profiler qual função deve ser analisada com a flag -f, e é possível executar a função com o seguinte comando:

```
>>> %lprun -f mode_using_counter mode_using_counter(10_000_000)
```

Vejamos a saída para a função mode_using_counter:

```
Timer unit: 1e-09 s

Total time: 2.60904 s
File: .../3744758285.py
Function: mode_using_counter at line 1

Line # Hits         Time      Per Hit  % Time  Line Contents
==============================================================
    1                                          def mode_using_counter
                                                (n_integers):
    2    1   124198000.0  124198000.0     4.8  some_list = np.random.randint
                                                (1, 100000, n_integers)
    3    1  2479064000.0 2479064000.0    95.0  c = Counter(some_list)
    4    1     5780000.0    5780000.0     0.2  return c.most_common(1)[0][0]
```

É muito mais fácil ler essa saída do que a saída do cProfile. Fica claro que, nessa função, aproximadamente 5% do tempo é gasto gerando a lista de

números aleatórios e 95% do tempo é alocado contando esses números com a função built-in Counter. Isso indica que, se quisermos melhorar o desempenho dessa função, devemos focar a linha que contém a Counter. Ser capaz de identificar a linha que demora mais tempo é imprescindível quando temos funções e scripts mais extensos.

Além do mais, podemos usar o line_profiler com um script que usa decorators [decoradores] para identificar as funções que queremos perfilar, e é possível encontrar mais detalhes sobre como fazer isso na documentação do line_profiler (*https://oreil.ly/xwGkQ*).

Perfilamento de memória com Memray

É possível perfilar o uso de memória do seu código, assim como o tempo que demora para ser executado. Talvez você também precise otimizar o uso de memória, dependendo dos requisitos do seu código. É importante considerar isso à medida que o tamanho dos seus dados aumenta, porque você pode atingir os limites superiores do hardware que está usando. Além disso, a CPU tem de trabalhar duro para gerenciar a memória. Isso pode aumentar o tempo de execução do código se a CPU estiver gastando tempo demasiado para gerenciar a memória em vez de executar o código.

O Memray (*https://oreil.ly/n-HqF*) é uma ferramenta de perfilamento de memória, desenvolvida pela Bloomberg, a qual pode fornecer relatórios diferentes sobre o uso de memória do seu código. É possível instalar o Memray com o seguinte comando:

```
$ pip install memray
```

Vejamos como usar o Memray com um script independente em Python, contendo a função mode_using_counter. Aqui está o script completo:

```
import numpy as np
from collections import Counter

def mode_using_counter(n_integers):
    random_integers = np.random.randint(1, 100_000, n_integers)
    c = Counter(random_integers)
    return c.most_common(1)[0][0]

if __name__ == '__main__':
```

```
print(mode_using_counter(10_000_000))
```

Para coletar os dados sobre o uso de memória do seu script, será necessário executar o Memray com o seguinte comando:

```
$ memray run mode_using_counter.py
```

Você deve receber esta saída:

```
Writing profile results into memray-mode_using_counter.py.17881.bin
26008
[memray] Successfully generated profile results.


You can now generate reports from the stored allocation records.
```

O Memray gerou um arquivo binário (*.bin*) com os resultados. Agora, podemos executar outros comandos para gerar uma variedade de relatórios a partir desse arquivo binário. O comando `flamegraph` é o mais útil:

```
$ memray flamegraph memray-mode_using_counter.py.17881.bin
```

Será necessário substituir *memray-mode_using_counter.py.17881.bin* pelo nome de arquivo que o Memray gerou para você. Isso gerará um arquivo HTML e, se abri-lo, verá algo como a Figura 2.2.

É possível analisar o uso de memória de cada linha de código. Nesse exemplo, o maior uso foi pela linha `random_integers = np.random.randint (1, 100_000, n_integers)`, totalizando um uso de memória de 76,3 MB. Trata-se de um exemplo simples; no entanto, com um script mais extenso, essa ferramenta se torna extremamente útil para rastrear as linhas de código que consomem mais memória. Isso possibilita saber onde concentrar nossos esforços, caso precisemos otimizar o uso de memória do script.

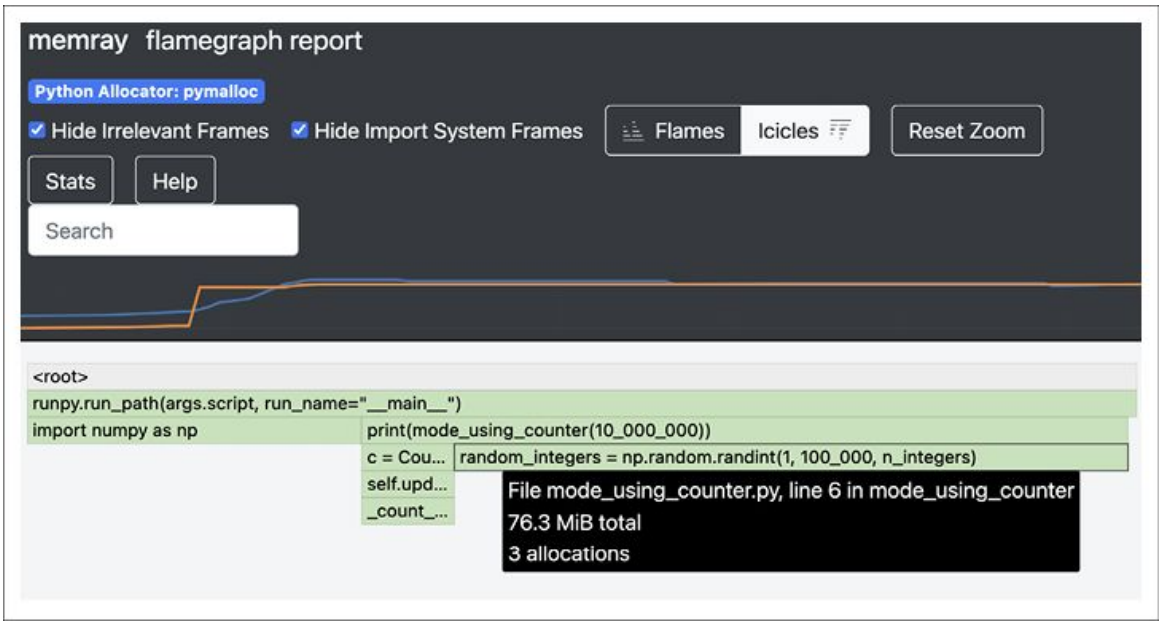


Figura 2.2: Relatório gerado pelo comando flamegraph do Memray.

Podemos também usar o Memray em um notebook do Jupyter com a extensão memray. É possível saber detalhes a respeito na documentação do Memray (*https://oreil.ly/i-E-J*).

Seja qual for o método usado para medir o desempenho do seu código, é importante medi-lo antes de otimizar o código, recorrendo aos métodos descritos em "Métodos para melhorar o desempenho", na página 36. Desse modo, você saberá onde estão os gargalos e poderá focar adequadamente seus esforços.

Complexidade de tempo

Na seção anterior, apresentei como encontrar gargalos em seu código. Isso sinaliza o desempenho do seu código em relação à quantidade de dados que você tem atualmente, mas o que acontece quando esperamos que a quantidade de dados aumente em um futuro próximo? Ou e se partes do seu código demorarem tanto para serem executadas que não seja viável medi-las? Nessas situações, o conceito de complexidade de tempo pode ajudá-lo.

A complexidade de tempo detalha como o tempo de execução de um algoritmo aumenta à medida que o tamanho da entrada aumenta. Isso significa uma tendência geral, não específica, para qualquer hardware. Para concretizar essa explicação, considere a seguinte pergunta: caso execute seu

código em uma lista de números e, em seguida, dobre o comprimento da lista, o que acontece com o tempo de execução do código? Permanece o mesmo, dobra ou acontece algo diferente?

Como estimar a complexidade de tempo

Vejamos um exemplo. A função a seguir calcula uma média ponderada, recebendo como entrada uma lista de números e uma lista de pesos a serem aplicados a esses números:

```
def weighted_mean(list_of_numbers, weights):
    running_total = 0
    for i in range(len(list_of_numbers)):
        running_total += (list_of_numbers[i] * weights[i])
    return (running_total/sum(weights))
```

Se o comprimento de `num_list` dobrar, o número de iterações no loop `for` também dobra. As outras linhas dessa função (inicializando o total que está sendo executado e dividindo-o pelo comprimento da lista) não aumentam o tempo de execução. Podemos considerar isso como uma equação do tempo de execução do código em termos de número de passos dessa função: cada passo é alguma operação realizada pelo código. Inicializar o total que está sendo executado é um passo; cada vez que esse mesmo total é incrementado é outro passo; retornar o resultado é mais um passo. A equação seria:

`number_of_steps = n + 2`, em que `n` é o comprimento das listas de entrada.

Assim, à medida que o tamanho de `n` aumenta, o número de passos aumenta linearmente, mas o 2 permanece constante.

A função a seguir aceita duas listas como entradas e calcula a covariância (uma medida da força da correlação entre duas variáveis aleatórias) delas. Novamente, não é a melhor forma de fazer esse cálculo; usei a função somente para ilustrar minha observação:

```
def covariance(X, Y):
    cov_sum = 0
    for i in range(len(X)):
        for j in range(len(Y)):
            cov_sum += 0.5 * (X[i] - X[j]) * (Y[i] - Y[j])
    return cov_sum / (len(X) ** 2)
```

Nessa função, como existem dois loops for aninhados (um dentro do outro), se os tamanhos de X e Y dobrarem, o número de passos para calcular o resultado aumentará quatro vezes. Nessa função, X e Y devem ter o mesmo comprimento. O aumento do tamanho do conjunto de dados faz com que o tempo de execução do código aumente proporcionalmente ao quadrado.

Isso sugere uma forma de melhorar seu código: e se for possível usar um método mais rápido para realizar o mesmo cálculo? A função a seguir também calcula a covariância, mas usa apenas um loop for:

```
def covariance_fast(X, Y):
    avg_X = sum(X) / len(X)
    avg_Y = sum(Y) / len(Y)

    result = 0
    for i in range(len(X)):
        result += (X[i] - avg_X) * (Y[i] - avg_Y)

    return result / len(X)
```

Quero fazer mais uma observação: os métodos de cronometragem e de perfilamento, apresentados nas seções “Cronometrando a execução do código”, da página 38, e “Criando o perfilamento do código”, da página 41, também indicam que os loops for aninhados foram a parte mais demorada da função. Ou seja, essa é uma ótima oportunidade para procurar algoritmos mais eficientes.

Notação Big O

> A notação Big O mostra como o código se torna mais lento à medida que os dados aumentam.
>
> – Ned Batchelder, *PyCon de 2018*

A notação Big O é um modo mais formal de detalhar os padrões de como o tempo de execução do código aumenta à medida que o tamanho dos dados também aumenta, conforme apresentei na seção anterior. Aqui, a letra O se refere à ordem da função, outra forma de se referir à taxa de crescimento da função. É uma maneira proveitosa de descrever diferentes classes de algoritmos, dependendo de como se comportam à medida que os conjuntos de dados aumentam. Além disso, independe do hardware que você está usando. Em uma máquina com alta capacidade de processamento, uma

função pode ser executada bem mais rápido; já em uma máquina com baixa capacidade de processamento, a mesma função pode ser executada mais lentamente. Essa análise possibilita predizer o desempenho do seu código, independentemente da máquina em que está sendo executado.

A notação Big O é semelhante à equação para o número de passos do exemplo da média ponderada da seção anterior, mas os coeficientes são omitidos. Na notação big O, o tempo de execução é representado como uma função do tamanho da entrada, n, e é escrito como $O(f(n))$. $f(n)$ é um limite superior relacionado ao número de operações realizadas pelo algoritmo como uma função de n. Assim, o exemplo da média ponderada seria classificado como $O(n)$.

Talvez você esteja se perguntando por que o exemplo da média ponderada é $O(n)$, e não $O(n + 2)$, visto que mostrei que a equação para ele é number of steps = n + 2. A notação Big O considera o limite dessa equação conforme n se torna grande e, nesse ponto, os dois passos extras são insignificantes. É uma aproximação que o ajuda a comparar diferentes abordagens, não uma medida exata de quanto tempo seu código leva para ser executado. Da mesma forma, se o código executar $n^2 + n$ passos, isso será aproximado para $O(n^2)$.

Algumas classes comuns na notação Big O incluem:

O(1)

Conhecido também como “tempo constante”, significando que o tempo de execução é independente do tamanho do conjunto de dados. Um exemplo é procurar um determinado elemento em uma lista. Retornar o último elemento de uma lista demora o mesmo tempo, ainda que a lista seja muito grande.

O(n)

Conhecido também como “tempo linear”, significando que o tempo de execução aumenta linearmente com o tamanho do conjunto de dados, conforme mostrado no exemplo da média ponderada.

$O(n^2)$

Conhecido também como “tempo quadrático”, significando que o tempo de execução aumenta em proporção ao quadrado do tamanho do conjunto

de dados, conforme mostrado no exemplo do cálculo lento da covariância.

Essas três classes são representadas na Figura 2.3, com o tempo exponencial $O(2^n)$, o tempo logarítmico $O(log\ n)$ e $O(n\ log\ n)$. O tempo exponencial significa que o tempo de execução aumenta proporcionalmente a 2 elevado ao expoente do tamanho do conjunto de dados e, em geral, os algoritmos recursivos fornecem uma complexidade de tempo de $O(2^n)$. O tempo logarítmico significa que o tempo de execução aumenta em proporção ao log do tamanho do conjunto de dados, e muitos algoritmos de pesquisa têm essa complexidade de tempo.

Como podemos ver no gráfico, $O(n^2)$ e $O(2^n)$ apresentam desempenho lento de forma muito rápida. Portanto, caso precise que seu código seja mais eficiente, tente evitá-los.

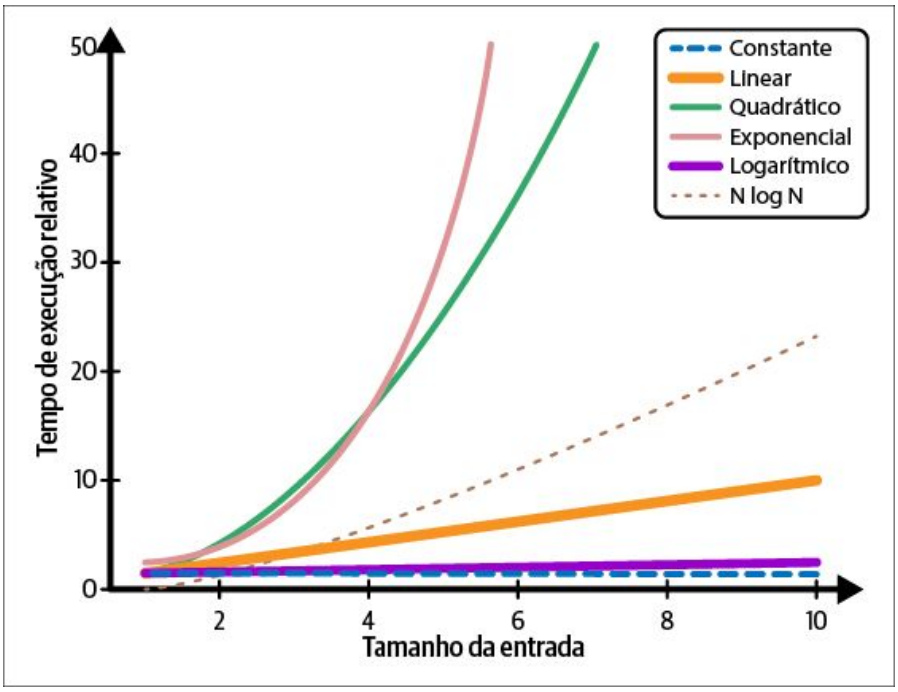


Figura 2.3: Tempo de execução relativo em comparação com o tamanho da entrada.

Deve-se também considerar um intervalo realista de *n* para o seu caso de uso a fim de evitar a otimização prematura. Por exemplo, se *n* nunca for maior que 5, não importa muito se o tempo de execução do seu código aumenta linear ou exponencialmente.

No próximo capítulo, veremos um exemplo de como a notação Big O pode

ser de grande utilidade quando analisamos a escolha de diferentes estruturas de dados.

Aspectos fundamentais

Ao considerar o desempenho do código, primeiro é necessário saber quando é adequado otimizar. Caso a quantidade de dados seja pequena ou seu código já atenda a todos os requisitos, não perca tempo otimizando-o.

Mas, se precisa melhorar o desempenho do seu código, a primeira etapa deve ser medi-lo, para descobrir onde estão os gargalos. A forma mais simples de fazer isso é cronometrar quanto tempo a função ou a linha de código leva para ser executada. Se precisar de mais detalhes ou quiser obter os pormenores de tempo de um script mais extenso, use um profiler.

Além do mais, pode ser útil considerar o número de passos que seu código executará e como isso será escalado à medida que a quantidade de dados aumentar. Não raro, você verá a notação Big O sendo usada para detalhar isso. No geral, lembre-se de que existem muitas opções para escrever o mesmo trecho de código, e algumas lhe fornecerão desempenho melhor do que outras!

CAPÍTULO 3

Usando estruturas de dados de modo eficaz

No capítulo anterior, vimos como medir o desempenho do nosso código. Neste capítulo, mostrarei como sua escolha da estrutura de dados pode afetar o desempenho do código e analisarei como escolher a melhor estrutura de dados para o problema em que você está trabalhando.

Como cientista de dados, ao escrever código, você precisará usar uma variedade de estruturas de dados a fim de armazenar seus dados. Você terá muitas opções de qual estrutura de dados usar, e é provável que algumas sejam adequadas para o problema em que está trabalhando e outras sejam escolhas menos adequadas.

> ...grande parte da programação de alto desempenho é saber quais perguntas você está tentando fazer aos seus dados e escolher uma estrutura de dados que possa responder de forma rápida a essas perguntas.
>
> – MICHA GORELICK E IAN OSVALD, *HIGH PERFORMANCE PYTHON*

É importante utilizar a estrutura de dados correta para o problema em que você está trabalhando por dois motivos principais: primeiro, a estrutura de dados é otimizada para esse caso de uso e, segundo, métodos úteis estão associados a ela. Desse modo, caso escolha a estrutura de dados correta, seu código terá um desempenho melhor e também será mais fácil de usar. Isso também significa que seu código é mais previsível e mais fácil de entender.

Neste capítulo, abordarei algumas das estruturas de dados mais comuns que você usará ao escrever código de ciência de dados: listas, tuplas, dicionários e conjuntos do Python; arrays do NumPy; e DataFrames do pandas. Explicarei as vantagens e desvantagens de cada uma e analisarei quais oferecem melhor desempenho em termos de tempo e uso de memória. Analisarei também como usar melhor as diferentes estruturas de dados para obter um desempenho eficiente.

Em muitos livros e artigos, nos deparamos com muitas estruturas de dados que não abordarei aqui, por exemplo, listas vinculadas, heaps [montes],

filas [queues], árvores binárias de busca e muito mais. Elas tendem a ser mais importantes na engenharia de software do que na ciência de dados, sobretudo em linguagens de programação diferentes do Python. Caso queira saber mais sobre estruturas de dados, recomendo o livro *A Common-Sense Guide to Data Structures and Algorithms in Python* [Guia Racional para Estruturas de Dados e Algoritmos em Python, em tradução livre], de Jay Wengrow (Pragmatic Bookshelf).

Estruturas nativas de dados do Python

Nesta seção, quero analisar quatro estruturas de dados fundamentais em Python: listas, tuplas, dicionários e conjuntos. É provável que você já as esteja usando em seu código. Quero fornecer algumas informações sobre como essas estruturas de dados são implementadas e alguns detalhes da complexidade de tempo para operações comuns. Para isso, usarei a notação Big O que descrevi na seção "Complexidade de tempo", da página 47. É inestimável ter intuição sobre o que as estruturas de dados podem fazer rapidamente e em que sentido seu desempenho é lento. Isso o ajudará a escolher a melhor estrutura de dados para seu caso de uso e fornecerá algumas ideias sobre em que local concentrar seus esforços, caso precise otimizar seu código.

Listas

Listas são uma estrutura essencial de dados em Python e você se deparará frequentemente com elas. Listas em Python são um tipo de array: uma estrutura de dados que tem alguma ordem. Isso significa que podemos procurar o primeiro, o terceiro ou qualquer outro elemento de uma lista.

Uma lista em Python é um array dinâmico, ou seja, pode ser redimensionada se mais elementos forem adicionados. Ela também pode armazenar elementos de diferentes tipos; logo, por exemplo, uma lista pode conter strings e números inteiros.

O Python aloca um fragmento contínuo de memória conforme o tamanho da lista: cada elemento da lista fica próximo ao local de memória adjacente do próximo elemento. Isso gera implicações importantes: é muito fácil procurar um elemento em uma lista. O interpretador do Python sabe a

posição de memória do início da lista. Assim, se estiver procurando o quinto elemento da lista, ele simplesmente recupera o elemento que está na quinta posição de memória a partir do início.

É possível medir como o tempo necessário para procurar um elemento em uma lista muda à medida que a lista aumenta. Primeiro, crie uma lista com 10 elementos:

```
small_list = list(range(10))
```

Em seguida, você pode usar o %%timeit (que apresentei na seção "Cronometrando a execução do código", na página 38) para medir o tempo necessário, a fim de procurar o último elemento da lista:

```
>>> %%timeit
>>> last_element = small_list[-1]

... 19.6 ns ± 0.13 ns per loop (mean ± std. dev. of 7 runs,
10,000,000 loops each)
```

Depois, crie uma lista de 10.000 elementos para executar o mesmo experimento:

```
large_list = list(range(10_000))
```

Por último, meça o tempo necessário para procurar o último elemento:

```
>>> %%timeit
>>> last_element = large_list[-1]

... 18 ns ± 0.0944 ns per loop (mean ± std. dev. of 7 runs,
100,000,000 loops each)
```

O tempo gasto é aproximadamente o mesmo para ambas as listas, embora a segunda seja 1.000 vezes maior que a primeira. As pesquisas em listas são representadas como *O(1)* (tempo constante). Essa é a notação Big O, que expliquei no Capítulo 2. Por maior que seja sua lista, ainda demora aproximadamente o mesmo tempo para procurar um elemento nela.

Sempre que adicionamos um elemento a uma lista, ele ocupa espaço extra na memória. O Python aloca um pouco de espaço extra além do comprimento da lista original, mas, quando esse espaço estiver cheio, toda a lista precisa ser copiada para um local novo da memória com mais espaço contíguo. Anexar ao final de uma lista é expresso como *O(1)*, mas gera um

pouco de sobrecarga devido à cópia da lista quando o espaço extra fica cheio.

Se inserir um elemento no meio de uma lista, tudo depois desse novo elemento precisa ser movido para um local novo da memória. Portanto, a operação de insert é representada como *O(n)*. Excluir um elemento do meio de uma lista também é representado como *O(n)*. Inserir e excluir são ambos expressos como *O(n)*, pois a lista deve sempre estar em um fragmento contínuo da memória.

Caso já saiba que adicionará elementos a uma lista com um tamanho específico, .append() pode não ser a forma mais eficiente de fazer isso. Talvez seja melhor usar uma list comprehension (*https://oreil.ly/ZqTu0*) ou criar uma lista de zeros com o comprimento correto e, em seguida, atualizar os valores nela. Caso queira anexar itens ao início e ao final de uma lista, é possível usar a estrutura de dados deque (Fila duplamente *encadeada*) (*https://oreil.ly/7a7CU*) do módulo collections.

Se você precisar pesquisar um elemento em uma lista, é necessário comparar o elemento que está pesquisando com todos os itens da lista. À medida que a lista aumenta, é possível medir como isso muda recorrendo às listas dos exemplos anteriores de código.

Primeiro, podemos medir quanto tempo leva para pesquisar na lista que contém 10 elementos:

```
>>> %%timeit
>>> 4200 in small_list

... 75.8 ns ± 0.298 ns per loop (mean ± std. dev. of 7
runs, 10,000,000 loops each)
```

Em seguida, podemos medir o mesmo para a lista com 10.000 elementos:

```
>>> %%timeit
>>> 4200 in large_list

... 23.8 µs ± 1.9 µs per loop (mean ± std. dev. of 7 runs,
10,000 loops each)
```

Demora aproximadamente 1.000 vezes mais tempo. Pesquisar em uma lista em Python dessa forma é representado como *O(n)*. Podemos usar formas

mais eficientes de pesquisar em uma lista, incluindo a pesquisa binária, expressa como *O(log n)*. Mas, caso precise pesquisar com frequência se algum elemento está presente, provavelmente é melhor usar um dicionário ou um conjunto, como explicarei a seguir.

Tuplas

No Python, as tuplas também são um array, mas têm tamanho estático. Elas são imutáveis — após criar uma tupla, você não consegue alterá-la. Por exemplo, não podemos anexar elementos extras ao final de uma tupla.

Tuplas são úteis quando temos apenas alguns itens que queremos armazenar em uma estrutura de dados, e eles não serão alterados. Vale ressaltar que uma tupla é armazenada em cache no tempo de execução do Python, em vez de ser armazenada na memória, por isso é ainda mais rápido procurar um elemento em uma tupla do que em uma lista. Procurar um elemento em uma tupla é representado como *O(1)*.

Dicionários

Dicionários são outra estrutura essencial de dados em Python e baseiam-se em pares de chave-valor. Isso significa que existem pares de elementos de dados que têm algum vínculo entre eles, por exemplo, o nome de uma pessoa e seu endereço. Dicionários são melhores para dados que não têm ordenação inerente.

Nos bastidores, os dicionários em Python são implementados por tabelas hash. Elas usam uma função hash para transformar uma chave no índice de uma lista. A Figura 3.1 mostra como isso funciona: aqui, a chave `'Heather Hancock'` é transformada pela função hash no índice `3`, e isso possibilita recuperar o endereço associado ao nome dessa pessoa.

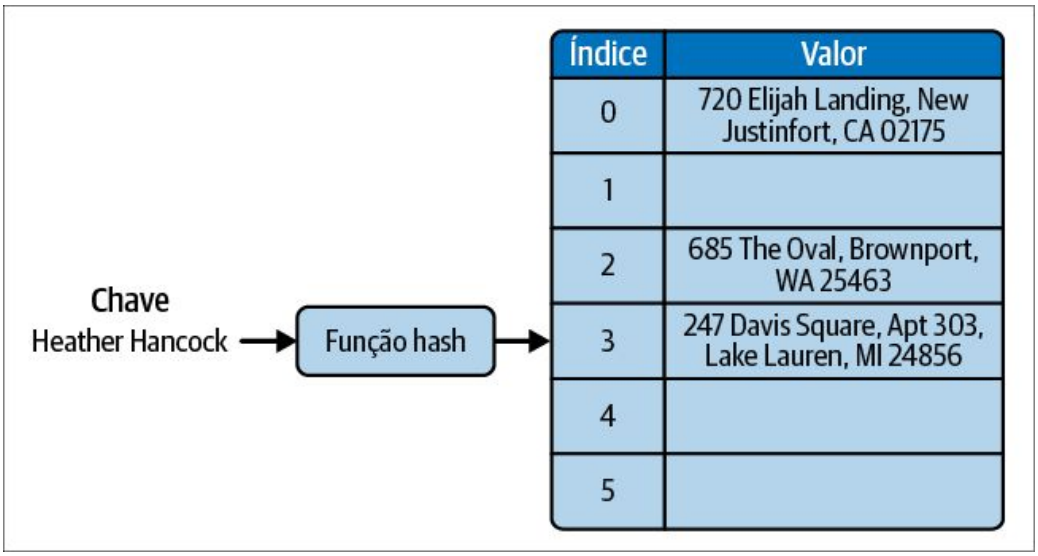


Figura 3.1: Pesquisa de dicionário usando uma tabela hash.

Uma função hash mapeia strings ou números inteiros, seja lá o que estiver na chave do dicionário, para um número inteiro, que é o índice relevante na lista de valores. Existem muitos tipos diferentes de funções hash. Obter o módulo de um número inteiro é um exemplo simples. Em um dicionário Python, a função hash é muito mais complexa.

A função hash deve retornar o mesmo número inteiro sempre que for aplicada à mesma chave. No dicionário, as chaves também devem ser exclusivas para que seja possível retornar o valor correto. Uma chave de dicionário precisa ser um tipo hashable, como uma string, um número inteiro ou um float. Listas em Python não são hashables devido à sua mutabilidade.

A tabela hash fornece uma propriedade útil: procurar o valor correspondente a uma chave específica em um dicionário é representado como *O(1)*. Você obtém o mesmo desempenho que procurar um elemento em uma lista, mas não precisa saber a ordem da lista. Inserir, atualizar e excluir um par de chave-valor são expressos como *O(1)*. Na tabela hash, os itens podem ser inseridos em qualquer ordem.

Podemos executar um experimento a fim de verificar se as pesquisas no dicionário permanecem constantes. A biblioteca Faker (*https://oreil.ly/4JRuL*) ajuda a fazer isso: ela cria dados falsos [fake data] que parecem realistas, incluindo dados pessoais como nomes, endereços, endereços de e-mail e assim por diante.

É possível inicializar um gerador Faker a fim de gerar dados falsos com o seguinte comando:

```
from faker import Faker

fake = Faker()
```

Em seguida, use-o para popular um dicionário a fim de executar experimentos:

```
small_dict = {}

for i in range(10):
    small_dict[fake.name()] = fake.address()
```

Você pode cronometrar quanto tempo as pesquisas levam neste pequeno dicionário com 10 pares de chave-valor.

```
>>> %%timeit
>>> small_dict['Erin Hogan']

... 21.7 ns ± 0.125 ns per loop (mean ± std. dev. of 7 runs,
10,000,000 loops each)
```

Em seguida, podemos criar um dicionário com 10.000 pares de chave-valor:

```
large_dict = {}

for i in range(10_000):
    large_dict[fake.name()] = fake.address()
```

E podemos medir o tempo necessário para pesquisar um valor a partir de uma chave neste dicionário:

```
>>> %%timeit
>>> large_dict['Nicole Morgan']

... 23.9 ns ± 0.449 ns per loop (mean ± std. dev. of 7 runs,
10,000,000 loops each)
```

Demora aproximadamente a mesma quantidade de tempo, embora esse dicionário seja 1.000 vezes maior do que o anterior. É extremamente eficiente procurar elementos em um dicionário. Em geral, a desvantagem é que eles consomem muita memória.

Conjuntos

Um conjunto é outra estrutura valiosa de dados para dados sem ordem inerente. No Python, os conjuntos são implementados com uma tabela hash, semelhante aos dicionários. No entanto, em vez de pares de chave-valor, eles têm somente um conjunto de chaves exclusivas. Ou seja, todos os elementos de um conjunto devem ser exclusivos.

Conjuntos compartilham a propriedade com dicionários em que operações como inserir, excluir e atualizar itens são todas expressas como *O(1)*. Procurar um elemento em um conjunto também é representado como *O(1)*. Converter uma lista em um conjunto e, em seguida, procurar o comprimento é uma maneira eficiente de contar o número de elementos exclusivos em uma lista.

Visto que as pesquisas em conjuntos são representadas como *O(1)*, mas as pesquisas em listas são expressas como *O(n)*, seria mais eficiente converter uma lista em um conjunto e depois realizar a pesquisa? Faremos um experimento para descobrir.

Usando a enorme lista anterior com 10.000 elementos, podemos medir o tempo necessário para consultar em uma lista:

```
>>> %%timeit
>>> 4200 in large_list

... 23.8 µs ± 1.9 µs per loop (mean ± std. dev. of 7 runs,
10,000 loops each)
```

Em seguida, podemos medir o tempo necessário para converter essa lista em um conjunto e depois realizar a consulta:

```
>>> %%timeit
>>> large_set = set(large_list)
>>> 4200 in large_set

... 76.7 µs ± 1.28 µs per loop (mean ± std. dev. of 7 runs,
10,000 loops each)
```

Converter uma lista em um conjunto e depois realizar a consulta demora três vezes mais tempo que a consulta de lista devido ao tempo necessário para converter a lista em conjunto. Contudo, depois de converter a lista em

um conjunto, as consultas subsequentes são rápidas:

```
>>> %%timeit
>>> 4592 in large_set

... 22 ns ± 0.188 ns per loop (mean ± std. dev. of 7 runs,
10,000,000 loops each)
```

A pesquisa de conjunto é 1.000 vezes mais rápida do que a pesquisa de lista. Converter em um conjunto é uma ótima opção quando você quer verificar repetidamente se os itens estão presentes em uma lista, mas não vale a pena, caso queira fazer isso pouquíssimas vezes. Como em tudo neste capítulo, vale a pena experimentar e medir o que é mais rápido para o problema específico em que você está trabalhando.

Caso queira saber mais sobre o funcionamento interno de dicionários e de conjuntos, recomendo o seguinte post do blog do Luciano Ramalho (*https://oreil.ly/MFOuT*).

Existem algumas outras estruturas nativas de dados do Python que não abordei nesta obra, mas que os leitores podem achar úteis, dependendo do caso de uso. Tuplas nomeadas (*https://oreil.ly/TxRJc*) são tuplas que possibilitam pesquisar elementos por um nome atribuído, em vez de apenas uma posição. Dataclasses (*https://oreil.ly/qtVog*) são um tipo de classe (que abordarei no Capítulo 4) otimizada para armazenar dados.

Arrays do NumPy

Nesta seção, quero passar das estruturas nativas de dados do Python para a análise dos arrays do NumPy. NumPy é uma das bibliotecas do Python mais comumente usadas para ciência de dados, e isso se deve à sua estrutura principal de dados: o `ndarray` ou array *n*-dimensional.

Não abordarei os detalhes de como criar e usar arrays do NumPy ou todas as ações que podemos realizar com eles. Para isso, recomendo o livro *Python Para Análise de Dados: Tratamento de Dados com Pandas, NumPy & Jupyter*, de Wes McKinney (Novatec, 2023), que tem um guia de referência completo sobre como usar o NumPy.

Apresentarei aos leitores uma visão geral de alguns dos motivos pelos quais

podemos optar por usar arrays do NumPy e por que são tão populares em ciência de dados. Além disso, apresentarei um panorama geral do que podemos fazer com os arrays do NumPy, e analisaremos brevemente algumas considerações de desempenho.

Funcionalidades dos arrays do NumPy

Os arrays do NumPy são arrays n-dimensionais, e esse fato fornece uma excelente pista sobre o melhor momento para usar um: quando seus dados são multidimensionais — com os quais você se deparará com frequência. Em machine learning, diversos cálculos são realizados em dados que estão em uma matriz ou em uma estrutura de array bidimensional. Quando comparados às listas nativas do Python, os arrays do NumPy facilitam bastante a manipulação de dados, e diversas bibliotecas de ML funcionam perfeitamente com os arrays do NumPy.

É possível criar uma estrutura de dados multidimensionais a partir de listas aninhadas do Python; porém, mais do que depressa, fica difícil efetuar cálculos nelas. Ilustrarei isso com um exemplo: considerando uma matriz bidimensional, como podemos procurar valores na primeira coluna?

Para fazer isso com listas aninhadas, seria necessário escrever uma linha de código para percorrer todas as linhas e extrair o primeiro valor. Vejamos uma forma de fazer isso com uma list comprehension:

```
python_2d_list =[[1, 3, 5], [2, 4, 6], [7, 9, 11]]

first_column = [python_2d_list[i][0] for i in range(len(python_2d_list))]
```

Mas, se você recorrer a um array do NumPy, poderá simplesmente procurar os valores na primeira coluna com a sintaxe de fatiamento [slicing] do array do NumPy:

```
import numpy as np

np_2d_array = np.array([[1, 3, 5], [2, 4, 6], [7, 9, 11]])

first_column = np_2d_array[:, 0]
```

Caso procure elementos em uma lista e atribua um novo nome de variável à seleção, isso criará uma cópia nova desses elementos. Mas, se fizer o mesmo com um array do NumPy, terá uma visualização do array original. Ou seja, se mudarmos os valores no objeto np_2d_array do exemplo anterior, isso também mudará os valores correspondentes no objeto first_column. Criar uma visualização é mais rápido e mais eficiente em termos de memória do que criar uma cópia, e essa é outra maneira pela qual os arrays do NumPy oferecem melhor desempenho do que as listas.

Na mesma linha, muitas outras operações em dados multidimensionais são mais fáceis de realizar com arrays do NumPy do que com listas aninhadas. Isso inclui multiplicação de matrizes, concatenação de arrays, transposição de arrays e remodelagem de arrays.

No entanto, vale ressaltar uma consideração sobre os arrays do NumPy (conforme entrarei em detalhes na próxima seção): eles podem armazenar dados de apenas um tipo. Assim sendo, eles não são uma escolha válida se seus dados contiverem tipos mistos (por exemplo, caso precise que sua estrutura de dados contenha strings e números inteiros).

Considerações sobre o desempenho dos arrays do NumPy

Como mencionado na seção anterior, os arrays do NumPy permitem dados de apenas um único tipo. Talvez isso pareça uma limitação, mas, na verdade, possibilita enormes vantagens quanto ao desempenho. É o contrário do que ocorre com as listas do Python, em que o seguinte código é uma opção válida:

```
mixed_type_list = ["one", 2, 3.14]
```

Uma lista do Python pode armazenar strings, inteiros e floats na mesma estrutura de dados. É possível tentar criar um array do NumPy para armazenar os mesmos dados, assim:

```
mixed_type_array = np.array(["one", 2, 3.14])
```

E essa linha de código é executada sem erros. No entanto, se analisarmos este array, podemos verificar que todos os elementos foram convertidos em strings:

```
>>> print(mixed_type_array)
... ['one' '2' '3.14']
```

O NumPy converteu todos os elementos para que fossem do mesmo tipo.

O tipo dos elementos em um array do NumPy é armazenado com ele e pode ser pesquisado com o atributo .dtype:

```
>>> integer_array = np.array([1, 2, 3])


>>> print(integer_array.dtype)
... dtype('int64')
```

O tipo é int64 em vez do padrão int do Python. O NumPy usa tipos diferentes dos tipos padrão do Python, o que também pode resultar em alguns ganhos de desempenho. Mais adiante nesta seção, abordarei esse tópico.

Saber que cada elemento em um array do NumPy é do mesmo tipo resulta em grandes ganhos de desempenho, sobretudo para o que é conhecido como cálculos vetorizados. Em um loop for regular do Python, o tipo de cada elemento precisa ser verificado para que o interpretador do Python saiba qual função aplicar a ele. Contudo, o NumPy pode ignorar essa etapa e acessar diretamente o código C otimizado para muitas operações matemáticas. Ele opera em todos os elementos de um array de uma só vez, ao contrário de iterar em cada elemento. Esse processo se chama vetorização. Caso queira saber mais sobre vetorização, recomendo o artigo de Itamar Turner-Trauring (*https://oreil.ly/DumcS*).

Vejamos um exemplo dos ganhos de desempenho que você pode obter usando arrays do NumPy em comparação às listas do Python. Primeiro, gerarei um array de 100.000 números inteiros e aleatórios com o NumPy e, em seguida, o converterei em uma lista normal do Python:

```
random_int_array = np.random.randint(1, 100_000, 100_000)
random_int_list = list(random_int_array)
```

Depois, você pode somar a lista com o sum() do Python, e usar o %%timeit apresentado na seção “Cronometrando a execução do código”, na página 38, a fim de medir quanto tempo leva:

```
>>> %%timeit -r 7 -n 100
>>> sum(random_int_list)


... 1.96 ms ± 463 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

Em seguida, podemos executar a mesma operação com o np.sum(), uma operação vetorizada, e medir novamente quanto tempo leva:

```
>>> %%timeit -r 7 -n 100
>>> np.sum(random_int_array)

... 13.9 µs ± 3.63 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

É aproximadamente 100 vezes mais rápido! Trata-se de um enorme aumento de desempenho. Se a operação que quer executar estiver disponível como um método vetorizado do array do NumPy, você definitivamente deve usá-lo em vez de usar um método nativo do Python ou escrever o próprio código. É possível consultar a documentação do NumPy (*https://oreil.ly/11vRd*) para descobrir se a operação está disponível.

Ao usar arrays do NumPy, você também precisa considerar se precisará adicionar mais elementos a um array posteriormente. Ao contrário de uma lista normal do Python, quando aloca espaço para um array, o NumPy não permite nenhum espaço extra. Portanto, se você anexar mais elementos a um array do NumPy, todo o array sempre precisará ser movido para um local novo da memória. Isso significa que anexar a um array do NumPy é representado como *O(n)*. Definitivamente, vale a pena inicializar seu array com a quantidade correta de espaço, e uma maneira fácil de fazer isso é usar `onp.zeros`, deste modo:

```
array_to_fill = np.zeros(1000)
```

Podemos então substituir os zeros pelos elementos novos em vez de anexá-los ao array.

Além disso, é possível economizar muito espaço de memória com os arrays do NumPy aproveitando os diferentes tipos do NumPy. Como os arrays do NumPy são carregados na memória, reduzir tamanho deles pode ser útil quando você estiver lidando com arrays grandes.

É possível gerar um array de números inteiros e aleatórios como antes:

```
random_int_array = np.random.randint(1, 100_000, 100_000)
```

Podemos procurar o número de bytes que isso ocupa inspecionando o atributo `.nbytes`:

```
>>> random_int_array.nbytes
... 800000
```

Esse array ocupa 800.000 bytes de espaço.

Você pode descobrir o tipo a partir do atributo `.dtype`:

```
>>> random_int_array.dtype
... dtype('int64')
```

Nesse caso, é int64. Referindo-se à documentação NumPy (*https://oreil.ly/beqRz*), esse é um número inteiro de 64 bits com um intervalo permitido de -9,223,372,036,854_775,808 a 9,223,372,036,854_775,808.

Caso sua expectativa seja não precisar desse intervalo completo de números inteiros, pode converter seu array em um tipo de número inteiro de 32 bits, que tem um intervalo permitido de -2,147,483,648 a 2,147,483,648. Você também pode usar um número inteiro de 16 bits se o intervalo de dados for pequeno o suficiente.

É possível converter seu array com o método .astype:

```
random_int_array_32 = random_int_array.astype(np.int32)
```

Se inspecionarmos o número de bytes, veremos que ocupa metade do espaço:

```
>>> random_int_array_32.nbytes
... 400000
```

É possível também fazer isso imediatamente, especificando o dtype ao inicializar o array:

```
small_array = np.array([1, 3, 5], dtype=np.int16)
```

A documentação do NumPy (*https://oreil.ly/uHITq*) disponibiliza uma lista completa de todos os tipos disponíveis.

Operações de arrays com a biblioteca Dask

Caso tenha tentado empregar as estratégias da seção anterior para melhorar o desempenho do seu código usando arrays do NumPy, mas o código ainda precisar ser otimizado, a biblioteca Dask (*https://www.dask.org*) é uma ótima opção. Ela possibilita realizar operações de array em paralelo, para processamento computacional mais rápido e para dados que não cabem na memória do seu computador. A Dask fornece uma interface muito semelhante aos arrays padrão do NumPy, mas adiciona um pouco mais de complexidade, por isso vale a pena usá-la somente se você precisar do aumento de desempenho. Com essa biblioteca, é possível executar cálculos em múltiplos núcleos ao mesmo tempo em seu computador e em sistemas

distribuídos (clusters).

A Dask funciona dividindo um array em trechos, executando cálculos em um ou em diversos trechos ao mesmo tempo e, em seguida, combinando os resultados. Por exemplo, caso queira encontrar o valor máximo de um array muito grande, é possível dividi-lo em uma série de trechos, encontrar o máximo de cada um e, em seguida, encontrar o máximo de todos os resultados de cada trecho combinado. Nem todas as operações podem ser paralelizadas assim, mas, se isso se aplicar ao problema em que você está trabalhando, pode tornar seu código mais eficiente.

A Dask também possibilita executar cálculos em dados maiores que a memória do seu sistema. Como nem todos os trechos são carregados e avaliados ao mesmo tempo, não é necessário carregar todo o array na memória e cada trecho pode ser avaliado sequencialmente.

É possível instalar a Dask com o seguinte comando:

```
$ python -m pip install "dask[complete]"
```

É possível executar a mesma operação com o NumPy e a Dask e comparar o tempo que leva.

Um experimento que podemos fazer é encontrar o valor máximo de um array grande. É possível criar um array do NumPy preenchido com números inteiros e aleatórios usando `np.random.randint()`. O código a seguir cria um array com 1 bilhão de números inteiros:

```
large_np_array = np.random.randint(1, 100000, 1000000000)
```

Podemos medir o tempo necessário para realizar esse cálculo em um array padrão do NumPy:

```
>>> %%timeit -r 1 -n 7
>>> np.max(large_np_array)
... 30.7 s ± 0 ns per loop (mean ± std. dev. of 1 run, 7 loops each)
```

O array da biblioteca Dask é uma estrutura de dados diferente de um array do NumPy. Muitos métodos do NumPy são replicados na Dask, de modo que possamos criar um array da Dask com números inteiros e aleatórios com o seguinte código:

```
import dask.array as da


large_dask_array = da.random.randint(1, 100_000, 1_000_000_000)
```

É possível também criar um array da Dask a partir de um array do NumPy, assim:

```
large_dask_array = da.from_array(large_np_array)
```

Em comparação com o NumPy, precisamos seguir uma etapa extra com a Dask. Primeiro, é necessário inicializar a operação, nesse caso com o método .max(). Em seguida, é necessário calcular a operação usando o método .compute(). É possível medir o tempo dessa etapa, a fim de compará-la com o array do NumPy:

```
>>> %%timeit -r 1 -n 7
>>> array_max = large_dask_array.max()
>>> array_max.compute()
... 1.51 s ± 0 ns per loop (mean ± std. dev. of 1 run, 7 loops each)
```

Encontrar o máximo em um array da Dask é aproximadamente 20 vezes mais rápido do que em um array do NumPy!

O cálculo em cada trecho também pode ser feito por um núcleo ou máquina diferente. A Dask Distributed (*https://oreil.ly/yTAkI*) agenda as tarefas para você. É necessário criar um objeto Client para usá-la:

```
from dask.distributed import Client


client = Client(n_workers=4)
client
```

A Figura 3.2 mostra um cliente local que está preparado para usar todos os núcleos em um computador.

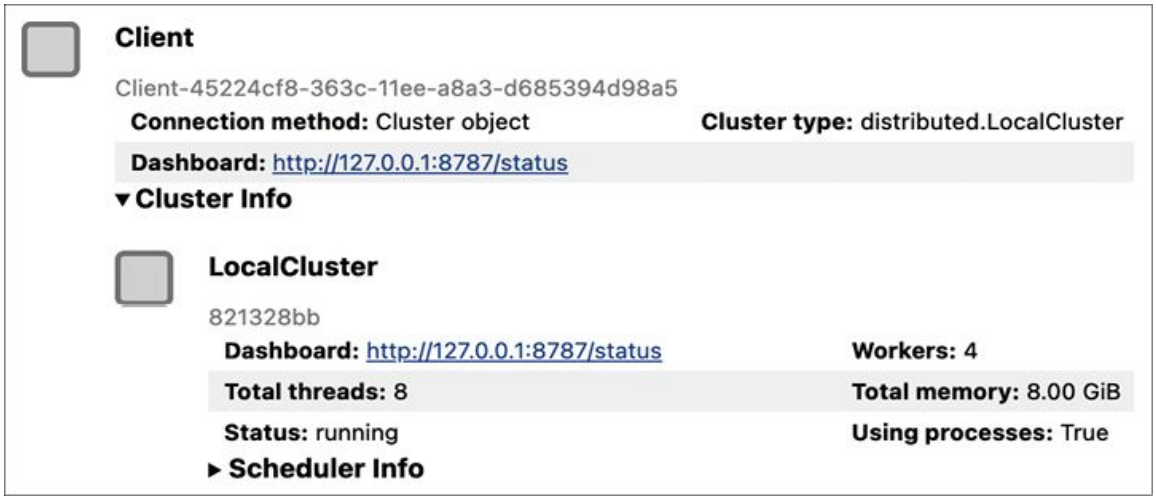


Figura 3.2: Dashboard do cluster local da Dask.

Assim que esse cliente estiver pronto, você pode usar os arrays da Dask como acima, e os cálculos serão distribuídos pelo número de workers que você especificar.

Caso queira saber mais sobre a Dask, consulte a documentação oficial (*https://oreil.ly/ES1I9*) para obter alguns ótimos exemplos.

Arrays em machine learning

A multiplicação de arrays é grande parte de muitos algoritmos de machine learning. Geralmente, em ML, os dados são armazenados em matrizes (arrays bidimensionais) ou tensores (arrays com múltiplas dimensões), sejam dados categóricos, de imagem ou de texto. À medida que os modelos de ML ficavam cada vez maiores, houve grande empenho para tornar as operações de array mais eficientes, a fim de que os modelos treinassem mais rápido.

Os dois frameworks de treinamento mais populares para modelos de ML, TensorFlow e PyTorch, disponibilizam estruturas otimizadas de dados para ML que aproveitam os ganhos de velocidade obtidos pelo treinamento em GPUs. É possível converter facilmente arrays do NumPy em qualquer um desses frameworks.

Crie um array do NumPy como antes:

```
np_tensor = np.random.rand(4,4)
```

Em seguida, converta-o para o formato `tensor` do TensorFlow, assim:

```
import tensorflow as tf


tf_tensor = tf.convert_to_tensor(tensor_numpy)
```

Ou para o formato `tensor` do PyTorch:

```
import torch


pytorch_tensor = torch.from_numpy(np_tensor)
```

As estruturas de dados de ML são otimizadas para GPUs, pois esses processadores podem executar muitas operações tensoriais em paralelo. Muitos algoritmos de ML são muito fáceis de paralelizar, incluindo o algoritmo de retropropagação [backpropagation] em redes neurais.

Quero mencionar também as matrizes esparsas: estruturas de dados com uso eficiente de memória para armazenar matrizes, compostas principalmente de zeros. Um exemplo comum da utilidade delas é a contagem da frequência de diferentes palavras em dados de texto. Para qualquer bloco de texto, se o vocabulário que você está usando for grande, a maioria dos features será zero. Na biblioteca scikit-learn, o `CountVectorizer` usa uma matriz esparsa da biblioteca SciPy. É possível converter isso de ou para um array do NumPy, porém a matriz esparsa é mais eficiente em termos de memória.

Otimizar o treinamento do seu modelo de ML é um tópico enorme, que foge ao escopo deste livro. Caso queira se aprofundar no assunto, tanto o PyTorch (*https://oreil.ly/Bz2H0*) quanto o TensorFlow (*https://oreil.ly/zhG6e*) disponibilizam um profiler para descobrir em que locais estão os gargalos em seu código de treinamento.

DataFrames do pandas

O pandas é uma das bibliotecas mais populares para realizar atividades relacionadas à ciência de dados em Python. É uma biblioteca fundamental para manipular e analisar dados. O pandas tem a reputação de ser lento e consumir muita memória, mas, nesta seção, demonstrarei diversas formas de melhorar seu desempenho.

Originalmente, o pandas foi desenvolvido com base na biblioteca NumPy, sendo importante se lembrar disso quando estamos trabalhando com ele. Por mais que os princípios que se aplicam aos arrays do NumPy também se apliquem aos DataFrames do pandas, a biblioteca pandas tem funcionalidades específicas. A versão 2.0 do pandas, disponibilizada em abril de 2023, adicionou a opção de usar as estruturas de dados do PyArrow (*https://oreil.ly/vVzYz*) como back-end.

Assim como os arrays do NumPy, não abordarei todas as funções associadas aos DataFrames do pandas e, novamente, recomendo o livro *Python para Análise de Dados*, de Wes McKinney (Novatec, 2023), para todos os detalhes completos.

Funcionalidades dos DataFrames do pandas

O pandas tem duas estruturas principais de dados: DataFrames e Series. Um DataFrame é composto de uma ou mais Series. Uma Series é semelhante a um array unidimensional do NumPy, mas conta com o fato extra de ter um índice. Isso possibilita que você procure um item em uma Series por seu índice, assim como por sua localização.

Podemos criar uma Series, da seguinte forma:

```
usa_data = pd.Series([13.33, 14.02, 14.02, 14.25],
                     index=["2000", "2001", "2002", "2003"])
```

Isso fornece a seguinte estrutura, em que o ano é o índice:

```
2000    13.33
2001    14.02
2002    14.02
2003    14.25
dtype: float64
```

Assim como um array do NumPy, uma Series é criada como um bloco contínuo de memória. Isso significa que algumas das mesmas considerações de desempenho se aplicam. Por exemplo, é lento anexar itens novos ao final de uma Series porque toda a estrutura deve ser movida para um local novo da memória. Na próxima seção, abordarei com mais detalhes as considerações de desempenho.

Um DataFrame do pandas é um arranjo bidimensional de estruturas Series do pandas, com um índice de coluna também. É possível criar um a partir da seguinte Series do pandas:

```
india_data = pd.Series([9.02, 9.01, 8.84, 8.84],
                       index=["2000", "2001", "2002", "2003"])


df = pd.DataFrame({'USA': usa_data, 'India': india_data})
```

Isso fornece o seguinte DataFrame:

```
      USA    India
2000  13.33  9.02
2001  14.02  9.01
2002  14.02  8.84
2003  14.25  8.84
```

Ao contrário dos arrays do NumPy, cada coluna em um DataFrame pode ser

de um tipo diferente. O pandas também oferece um tipo de coluna object que possibilita combinar dados de diferentes tipos em uma Series. O pandas também tem mais funções para lidar com dados ausentes do que o NumPy.

As estruturas de dados do pandas são extremamente úteis para dados tabulares bidimensionais com informações de linha e coluna. Elas são ótimas para dados no estilo de planilha. Além do mais, podem ser usadas de forma semelhante às tabelas de banco de dados, fornecendo-lhe a opção de fazer join ou consultar dados, porém funcionam melhor quando o projeto é muito pequeno para valer a pena configurar um banco de dados por completo. O pandas também disponibiliza muitas funções especializadas para trabalhar com séries temporais.

Considerações sobre o desempenho dos DataFrames do pandas

Assim como o NumPy, a biblioteca pandas tem muitas operações vetorizadas que aplicam cálculos a todos os elementos de um array de uma só vez. Muitos deles usam o NumPy nos bastidores. Se estiver fazendo algo em que uma operação vetorizada esteja disponível, quase sempre isso lhe fornecerá o melhor desempenho.

Por exemplo, operações aritméticas são vetorizadas:

```
>>> %%timeit
>>> df["India_fraction"] = df["India"] / 100


... 65.7 µs ± 1.01 µs per loop (mean ± std. dev. of 7 runs,
    10,000 loops each)
```

Além das operações vetorizadas disponíveis no NumPy, o pandas inclui operações de string vetorizadas (*https://oreil.ly/CwzmR*). Dentre elas, temos lower, strip, split e assim por diante. É mais rápido usar df['column_name'].str.lower() do que o método .lower() normal do Python.

Se o que está tentando fazer não estiver disponível como uma função built-in, é possível usar apply com qualquer função que você definir (no próximo capítulo, abordarei mais a apply). Como podemos observar, o cálculo passa a ter um pouco de sobrecarga:

```
>>> %%timeit
>>> df["India_fraction"] = df["India"].apply(lambda x: x / 100)
```

```
... 87.7 µs ± 302 ns per loop (mean ± std. dev. of 7 runs,
    10,000 loops each)
```

Se quiser executar alguma operação em cada linha do seu DataFrame, parece lógico iterar por todas as linhas. E o pandas fornece o método iterrows para isso. No entanto, como podemos observar a seguir, esse método é mais lento do que uma operação vetorizada ou uma função apply. Ou seja, devemos evitá-lo:

```
>>> %%timeit
>>> df["India_fraction"] = [row['India'] / 100 for index, row in df.iterrows()]


... 348 µs ± 4.14 µs per loop (mean ± std. dev. of 7 runs,
    1,000 loops each)
```

Por padrão, um DataFrame do pandas é carregado na memória. Ou seja, se o DataFrame for maior que a memória do seu computador, você terá um problema. É difícil também estimar quanta memória seu processamento de dados consumirá. Logo, mesmo que o DataFrame seja menor que a memória do seu computador, você ainda poderá ter um problema. No pandas, o gerenciamento de memória está melhorando de forma rápida, mas você pode fazer algumas coisas básicas, caso o DataFrame seja maior do que a memória do seu computador. Primeiro, pode simplesmente carregar apenas as colunas em que quer trabalhar. Por exemplo, é possível especificar as colunas que queremos ler em um arquivo CSV usando o argumento usecols em read_csv. Segundo, podemos usar o argumento chunksize para criar um iterador que possibilitará trabalhar em somente um subconjunto de linhas por vez. A recente introdução do PyArrow como back-end opcional do pandas também suporta tipos de dados com uso mais eficiente de memória (*https://oreil.ly/kCzMf*).

Outra opção para grandes quantidades de dados é usar a biblioteca Dask. Como vimos com o NumPy, se seus dados forem grandes demais para caber na memória, a Dask poderá dividi-los. É também uma ótima opção se o seu processamento de dados for lento e você quiser paralelizá-lo em múltiplos núcleos ou máquinas. A Dask tem o próprio DataFrame como estrutura de dados (*https://oreil.ly/yv913*), sendo possível criar um diretamente a partir de seus dados ou de um DataFrame existente do pandas.

É possível também considerar migrar para a biblioteca Polars (*https://www.pola.rs*) a fim de melhorar o desempenho com DataFrames. Apesar de ser um projeto menos maduro do que o pandas, é mais rápido e mais eficiente em termos de memória. Por mais que o código subjacente seja escrito em Rust e use o Apache Arrow, tem uma interface muito parecida à do pandas.

Aspectos fundamentais

Neste capítulo, você aprendeu a usar estruturas de dados de modo eficaz para uma variedade de casos diferentes de uso. Além de pensar sobre o desempenho esperado de cada uma dessas estruturas de dados, é importante medir o desempenho do código que você está escrevendo e experimentar diferentes opções a fim de verificar o que funcionará melhor para o problema em que está trabalhando.

Não raro, as estruturas de dados built-in do Python são tudo o que você precisa para resolver seu problema. Listas são uma escolha eficiente, caso precise procurar itens por seu índice, mas pesquisar itens em uma lista é relativamente lento. Dicionários são uma boa escolha para pesquisas frequentes com chaves, e conjuntos podem ser uma maneira eficiente de verificar se itens estão presentes.

Arrays do NumPy são perfeitos para dados multidimensionais. Como contêm apenas um tipo de dados, o NumPy pode ir diretamente para o código C, compilado para operações vetorizadas. Isso proporciona enorme ganho de velocidade. Ou seja, se um método vetorizado estiver disponível para o cálculo que deseja realizar, basta usá-lo.

Além disso, é possível aproveitar os tipos do NumPy a fim de reduzir o consumo de memória e usar a biblioteca Dask para arrays de memória maiores ou para cálculos paralelizados. Em projetos de ML, talvez você acredite ser vantajoso usar estruturas de dados otimizadas para GPUs, e matrizes esparsas também são úteis.

DataFrames do pandas são perfeitos para dados tabulares e para realizar análises básicas de dados. Eles têm considerações de desempenho semelhantes aos arrays do NumPy e devemos sempre usar operações vetorizadas, se estiverem disponíveis. É muito ineficiente iterar por meio

das linhas em um DataFrame. Se precisar de um desempenho mais eficiente do que o padrão fornecido pelo pandas, pode recorrer à biblioteca Dask ou à biblioteca Polars.

Caso queira se aprofundar nos conceitos deste capítulo, recomendo a leitura das seguintes obras:

- *Python in a Nutshell* [Python em Poucas Palavras, em tradução livre], de Alex Martelli, Anna Martelli Ravenscroft, Steve Holden e Paul McGuire (O'Reilly, 2023), para uma boa visão geral das estruturas de dados do Python.
- *Python para Análise de Dados*, de Wes McKinney (Novatec, 2023), para mais detalhes sobre arrays do NumPy e DataFrames do pandas.
- *High Performance Python*, de Micha Gorelick e Ian Osvald (O'Reilly, 2020), para saber detalhes minuciosos sobre a otimização do desempenho do Python.
- *Scaling Python with Dask* [Escalando o Python com a Biblioteca Dask, em tradução livre], de Holden Karau e Mika Kimmins (O'Reilly, 2023), para mais informações sobre como usar a biblioteca Dask para ciência de dados.
- Artigos sobre ciência de dados no site do Python Speed (*https://oreil.ly/ptdyp*).

CAPÍTULO 4

Programação orientada a objetos e programação funcional

Neste capítulo, quero apresentar dois estilos de programação que os leitores provavelmente verão ao longo de suas carreiras em ciência de dados: programação orientada a objetos (POO) e programação funcional (PF). É extremamente útil conhecer ambas. Mesmo que nunca escreva código em nenhum desses estilos, você se deparará com pacotes que usam amplamente uma ou outra. Isso inclui pacotes padrão de ciência de dados do Python, como das bibliotecas pandas e Matplotlib. Agora, gostaria de muni-lo com conhecimentos sobre POO e PF a fim de que possa utilizar quaisquer códigos que não escreveu de modo eficaz.

POO e PF são paradigmas de programação baseados em princípios básicos da ciência da computação. Algumas linguagens de programação suportam apenas um desses paradigmas ou privilegiam enfaticamente um em detrimento de outro. Por exemplo, o Java é uma linguagem orientada a objetos. O Python suporta tanto a POO quanto a PF. Em Python, a POO é mais popular como estilo geral, mas você também pode se deparar com o uso ocasional da PF.

Ambos os estilos também fornecem um referencial para dividir seu código. Ao escrever código, podemos simplesmente escrever tudo o que queremos como um único script extenso. E, apesar de isso executar sem problemas, seria difícil manter e depurar o código. Conforme analisado no Capítulo 1, é importante dividir o código em partes menores, e tanto a POO quanto o PF podem sugerir boas maneiras de fazer isso.

Em meu código, não sigo rigorosamente os princípios de programação funcional ou os princípios de programação orientada a objetos. Às vezes, defino minhas próprias classes seguindo os princípios de POO e, ocasionalmente, escrevo funções que estão em conformidade com os princípios de PF. A maioria dos programas modernos em Python segue um

meio-termo que combina ambos os paradigmas. Neste capítulo, apresentarei uma visão geral de ambos os estilos para que você entenda os conceitos básicos dos dois.

Programação orientada a objetos

A programação orientada a objetos é muito comum em Python. Mas o que é um “objeto” nesse contexto? Podemos considerar um objeto como uma “entidade” que pode ser descrita por um substantivo. No código de ciência de dados, alguns objetos comuns podem ser um DataFrame do pandas, um array do NumPy, uma figura da Matplotlib ou um estimador do scikit-learn.

Um objeto pode armazenar dados, comporta algumas ações associadas a ele e pode interagir com outros objetos. Por exemplo, um objeto DataFrame do pandas contém uma lista de nomes de colunas. Uma ação associada a um objeto DataFrame é renomear as colunas. O DataFrame pode interagir com um objeto Series do pandas adicionando essa série como uma coluna nova.

Podemos também considerar um objeto como uma estrutura personalizada de dados. Nós o projetamos a fim de armazenar os dados desejados, de modo que possamos usá-los posteriormente. Vejamos novamente um DataFrame do pandas como exemplo: os designers do pandas criaram uma estrutura que poderia armazenar dados em um formato tabular. Assim, podemos acessar os dados em linhas e colunas e manipulá-los nesses formatos.

Na próxima seção, apresentarei a terminologia principal de POO e ilustrarei alguns exemplos de como você já pode as estar usando.

Classes, métodos e atributos

Em POO, você verá termos importantes como classes, métodos e atributos. A seguir, elenco uma visão geral de cada um:

- Uma classe define um objeto, e podemos considerá-la como um modelo para criar mais objetos dessa variedade. Um objeto individual é uma instância dessa classe, e cada objeto é um “elemento” individual.
- Métodos são ações que podemos realizar nos objetos dessa classe. Eles definem o comportamento desse objeto e podem modificar seus

atributos.

- Atributos são variáveis que são alguma propriedade dessa classe, e cada objeto pode ter dados diferentes armazenados nesses atributos.

Como se trata de conceitos muito abstratos, fornecerei um exemplo mais concreto. Veja como a terminologia orientada a objetos pode ser adaptada ao mundo em que vivemos. O livro que você está lendo neste momento, *Engenharia de Software para Cientistas de Dados*, é um objeto da classe "Livro". Um dos atributos desse objeto é o seu número de páginas e outro é o nome da autora. Um método que você pode chamar nesse objeto é "lê-lo". A classe "Livro" tem muitas instâncias, mas todas elas têm determinado número de páginas e podem ser lidas.

No Python, uma classe geralmente é nomeada usando a convenção de nomenclatura `CamelCase`. Ou seja, em vez de nomearmos uma classe como `my_class`, nomeamos como `MyClass`. Essa convenção o ajuda a identificar as classes com mais facilidade. É possível procurar um atributo usando o formato `class_instance.attribute`. Você pode chamar um método usando `class_instance.method()` (observe que ele tem parênteses). Os métodos podem receber argumentos, mas os atributos não podem.

Por exemplo, considere um DataFrame do pandas. Provavelmente, você está familiarizado com a sintaxe para criar um DataFrame novo:

```
import pandas as pd

my_dict = {"column_1": [1, 2], "column_2": ["a", "b"]}

df = pd.DataFrame(data=my_dict)
```

Analisando essa sintaxe de uma perspectiva orientada a objetos, quando executamos a linha `df = pd.DataFrame(data=my_dict)`, inicializamos um objeto novo do tipo DataFrame e passamos alguns dados que serão usados para definir os atributos desse DataFrame.

É possível procurar alguns dos atributos desse DataFrame, assim:

```
df.columns

df.shape
```

`.columns` e `.shape` são atributos do objeto `df`.

E você pode chamar diversos métodos nesse objeto DataFrame, por exemplo:

```
df.to_csv("file_path", index=False)
```

`.to_csv()` é o método desse exemplo.

Outro exemplo conhecido de criar um objeto novo e chamar um método se origina do scikit-learn. Caso esteja treinando um modelo de machine learning em dois arrays, e `X_train` contenha as features de treinamento e `y_train` contenha os rótulos de treinamento, escreva um código como este:

```
from sklearn.linear_model import LogisticRegression

clf = LogisticRegression()
clf.fit(X_train, y_train)
```

Nesse exemplo, estamos inicializando um novo objeto classificador `LogisticRegression` e chamando o método `.fit()` nele.

Vejamos outro exemplo. Este é o código que cria a Figura 2.3 do Capítulo 2[1]. Dois objetos são criados aqui, um objeto de figura da Matplotlib e um objeto de eixos da Matplotlib. Em seguida, diversos métodos são chamados para realizarem diferentes operações nesses objetos, como explicarei nas anotações de código:

```
import matplotlib.pyplot as plt
import numpy as np

n = np.linspace(1, 10, 1000)
line_names = [
    "Constant",
    "Linear",
    "Quadratic",
    "Exponential",
    "Logarithmic",
    "n log n",
]
big_o = [np.ones(n.shape), n, n**2, 2**n, np.log(n), n * (np.log(n))]

fig, ax = plt.subplots() ❶
```

```
fig.set_facecolor("white") ❷

ax.set_ylim(0, 50) ❸
for i in range(len(big_o)):
    ax.plot(n, big_o[i], label=line_names[i])
ax.set_ylabel("Relative Runtime")
ax.set_xlabel("Input Size")
ax.legend()
fig.savefig(save_path, bbox_inches="tight") ❹
```

❶ Inicializa os objetos figure e axes.

❷ Chama o método set_facecolor no objeto fig com um argumento white.

❸ Todos os métodos nas próximas linhas operam no objeto ax.

❹ Salva a figura em um método chamado no objeto fig.

Os objetos figure e axes têm muitos métodos que podemos chamar para atualizar esses objetos.

Não raro, a Matplotlib parece confusa porque tem dois tipos de interface. Uma delas é orientada a objetos e a outra foi desenvolvida para imitar a plotagem no MATLAB. A Matplotlib foi lançada pela primeira vez em 2003, e os desenvolvedores queriam que fosse acessível e amigável às pessoas habituadas a usar o MATLAB. Hoje em dia, é mais comum usar a interface orientada a objetos, como mostrei no exemplo do código anterior. No entanto, como o código das pessoas depende dos dois tipos de interface, ambas ainda precisam existir. O artigo "Why You Hate Matplotlib [Por Que Você Odeia a Matplotlib, em tradução livre]" (*https://oreil.ly/rYXUJ*) apresenta mais detalhes sobre o assunto.

Ainda que a terminologia relacionada à POO seja desconhecida, você a usará com frequência em muitos pacotes comuns de ciência de dados. A próxima etapa é definir as próprias classes, de modo que você consiga usar uma abordagem orientada a objetos em seu código.

Definindo as próprias classes

Caso queira escrever o próprio código usando um estilo orientado a objetos, precisará definir as suas classes. Mostrarei alguns exemplos simples de como fazer isso. O primeiro exemplo repete um pouco de texto determinado número de vezes. O segundo usa os dados dos Objetivos de

Desenvolvimento Sustentável da ONU que utilizei em outros exemplos ao longo deste livro. Para mais detalhes, confira a seção “Dados neste livro”, na página 32.

No Python, definimos uma nova classe com a instrução class :

```
class RepeatText():
```

É muito comum armazenar alguns atributos sempre que uma instância nova de um objeto é inicializada. Para fazer isso, o Python usa um método especial chamado __init__, que é definido assim:

```
    def __init__(self, n_repeats):
        self.n_repeats = n_repeats
```

O primeiro argumento no método __init__ se refere à instância nova do objeto que é criado. Por convenção, isso geralmente é chamado de self. Nesse exemplo, o método __init__ usa outro argumento: n_repeats. A linha self.n_repeats = n_repeats significa que cada instância nova de um objeto RepeatText tem um atributo n_repeats, que deve ser fornecido sempre que um objeto novo é inicializado.

Podemos criar um novo objeto RepeatText da seguinte forma:

```
repeat_twice = RepeatText(n_repeats=2)
```

Depois, é possível acessar o atributo n_repeats com a seguinte sintaxe:

```
>>> print(repeat_twice.n_repeats)
... 2
```

Definir outro método é semelhante a definir o método __init__, com a diferença de que podemos fornecer a ele o nome que quisermos, como se fosse uma função normal. Conforme veremos a seguir, ainda precisamos do argumento self se quisermos que cada instância do objeto tenha esse comportamento:

```
    def multiply_text(self, some_text):
        print((some_text + " ") * self.n_repeats)
```

Esse método procurará o atributo n_repeats da instância da classe na qual opera. Isso significa que você precisa criar uma instância de um objeto RepeatText antes de poder usar o método.

O Python tem métodos especiais que não recebem o parâmetro self como argumento:

classemethod e staticmethod. Os métodos de classe se aplicam a uma classe inteira, não apenas a uma instância de uma classe, e os métodos estáticos podem ser chamados sem criar uma instância da classe. Para saber mais a respeito, indico o livro *Introducing Python* [Introdução ao Python, em tradução livre], de Bill Lubanovic (O'Reilly, 2019).

Podemos chamar o método recém-criado da seguinte forma:

```
>>> repeat_twice.multiply_text("hello")
... 'hello hello'
```

Vejamos a definição completa da classe nova:

```
class RepeatText():


    def __init__(self, n_repeats):
        self.n_repeats = n_repeats


    def multiply_text(self, some_text):
        print((some_text + " ") * self.n_repeats)
```

Agora, vejamos outro exemplo. Dessa vez, usaremos os dados dos Objetivos de Desenvolvimento Sustentável da ONU apresentados no Capítulo 1. No exemplo a seguir, estou criando um objeto Goal5Data para armazenar alguns dados relevantes para o ODS5 (*https://oreil.ly/tw77-*), "Alcançar a igualdade de gênero e empoderar todas as mulheres e meninas". Especificamente, esse objeto armazenará os dados para uma das metas associadas a esse objeto, a Meta 5.5: "Garantir a participação plena e efetiva das mulheres e a igualdade de oportunidades de liderança em todos os níveis de tomada de decisão na vida política, econômica e pública".

Quero conseguir criar um objeto para armazenar os dados de cada país a fim de manipulá-los facilmente da mesma forma. Vejamos o código para criar a classe nova e armazenar os dados:

```
class Goal5Data():
    def __init__(self, name, population, women_in_parliament):
        self.name = name
        self.population = population
        self.women_in_parliament = women_in_parliament ❶
```

❶ Esse atributo armazena uma lista do percentual de cargos no órgão diretivo de determinado país ocupados por mulheres, por ano.

Veja um método que exibe um resumo desses dados:

```
    def print_summary(self):
        null_women_in_parliament = len(self.women_in_parliament) -
        np.count_nonzero(self.women_in_parliament)
        print(f"There are {len(self.women_in_parliament)} data points for
        Indicator 5.5.1, 'Proportion of seats held by women in national
        parliaments'.")
        print(f"{null_women_in_parliament} are nulls.")
```

Da mesma forma que no exemplo anterior, é possível criar uma instância nova dessa classe, assim:

```
usa = CountryData(name="USA",
                  population=336262544,
                  women_in_parliament=[13.33, 14.02, 14.02, ...])
```

Chamar o método `print_summary` fornece o seguinte resultado:

```
>>> usa.print_summary()
... "There are 24 data points for Indicator 5.5.1,
    'Proportion of seats held by women in national parliaments'.
    0 are nulls."
```

Escrever isso como um método garante que o código seja modular, bem organizado e fácil de reutilizar. Além do mais, fica evidente o que o código está fazendo, o que ajudará quem quiser usá-lo.

Usarei essa classe na próxima seção para mostrar outro princípio de classes: herança.

Princípios de POO

Não raro, quando se trata de programação orientada a objetos, você verá os seguintes termos: encapsulamento, abstração, herança e polimorfismo. Nesta seção, fornecerei a definição de todos eles e ilustrarei alguns exemplos de como a herança pode ajudá-lo.

Herança

Herança significa que podemos estender uma classe criando outra classe que se baseia nela. Isso ajuda a reduzir a repetição, visto que, se você precisar de uma classe nova que esteja intimamente relacionada a uma que já tenha escrito, não será necessário duplicar essa classe para fazer uma mudança mínima.

Talvez você não precise usar a herança ao definir as próprias classes, mas talvez seja necessário usá-la com classes de uma biblioteca externa. Mais adiante, veremos alguns exemplos de herança para validação de dados, na seção “Validação de dados com o Pydantic”, na página 139, e na seção “Adicionando funcionalidades à sua API”, na página 210. Nesta seção, quero ajudá-lo a identificar e a entender a herança quando se deparar com ela.

É possível identificar uma classe que usa herança, pois ela terá a seguinte sintaxe:

```
class NewClass(OriginalClass):
    ...
```

A classe NewClass pode usar todos os atributos e métodos da OriginalClass, mas, caso queira alterá-los, pode sobrescrever qualquer um deles. Em geral, o termo “pai” é usado para se referir à classe original, e a classe nova que herda da classe original é frequentemente chamada de classe “filha”.

Vejamos um exemplo de uma nova classe, Goal5TimeSeries, que herda da classe Goal5Data da seção anterior, transformando-a em uma classe que pode trabalhar com dados de séries temporais:

```
class Goal5TimeSeries(Goal5Data):
    def __init__(self, name, population, women_in_parliament, timestamps):
        super().__init__(name, population, women_in_parliament)
        self.timestamps = timestamps
```

Dessa vez, o método __init__ está um pouco diferente. Usar super() significa que o método __init__ da classe pai é chamado, e isso inicializa os atributos name, population e women_in_parliament.

Podemos criar um novo objeto Goal5TimeSeries, assim:

```
india = Goal5TimeSeries(name="India", population=1417242151,
                        women_in_parliament=[9.02, 9.01, 8.84, ...],
                        timestamps=[2000, 2001, 2002, ...])
```

E ainda podemos acessar o método a partir da classe Goal5Data:

```
>>> india.print_summary()
... "There are 24 data points for Indicator 5.5.1,
  'Proportion of seats held by women in national parliaments'. 0 are nulls."
```

É possível também adicionar um método novo que seja relevante para a

classe filha. Por exemplo, o novo método fit_trendline() ajusta uma linha de regressão aos dados para encontrar sua tendência:

```
from scipy.stats import linregress


class Goal5TimeSeries(Goal5Data):
    def __init__(self, name, population, women_in_parliament, timestamps):
        super().__init__(name, population, women_in_parliament)
        self.timestamps = timestamps


    def fit_trendline(self):
        result = linregress(self.timestamps, self.women_in_parliament) ❶
        slope = round(result.slope, 3)
        r_squared = round(result.rvalue**2, 3) ❷
        return slope, r_squared
```

❶ Usa a função linregress da biblioteca scipy para ajustar uma linha reta por meio dos dados usando regressão linear.

❷ Calcula o coeficiente de determinação (R-quadrado) a fim de determinar a qualidade do ajuste da linha.

Chamar o método novo retorna a inclinação da linha de tendência e o erro quadrático médio normalizado do ajuste da linha aos dados:

```
>>> india.fit_trendline()
... (0.292, 0.869)
```

Caso esteja usando a herança nas classes que você mesmo escreveu, ela possibilita estender os recursos das classes criadas. Isso significa menos duplicação de código e ajuda a manter seu código modular. Além disso, é muito útil herdar de classes em uma biblioteca externa. Mais uma vez, isso significa que você não duplica a funcionalidade delas, mas pode adicionar features extras.

Encapsulamento

Encapsulamento significa que a classe oculta seus detalhes do acesso externo. Podemos ver apenas a interface da classe, não os detalhes internos do que está acontecendo. A interface é composta dos métodos e atributos que projetamos. Não é tão comum no Python, mas, em outras linguagens de programação, as classes geralmente são desenvolvidas com métodos ou

atributos ocultos ou privados que não podem ser alterados externamente.

No entanto, o conceito de encapsulamento ainda é aplicado no Python, e muitas bibliotecas e aplicações o utilizam. A biblioteca pandas é um ótimo exemplo disso. O pandas usa encapsulamento fornecendo métodos e atributos que possibilitam interagir com os dados enquanto mantém os detalhes da implementação subjacente ocultos. Um objeto DataFrame encapsula dados e fornece vários métodos para acessá-los, filtrá-los e transformá-los. Como mencionei no Capítulo 3, os DataFrames do pandas usam o NumPy nos bastidores, mas você não precisa saber disso para usá-los. É possível usar a interface DataFrame do pandas para realizar suas tarefas, mas, se precisar realizar operações mais complexas, também poderá usar os métodos do NumPy.

Interfaces são imprescindíveis, pois, em geral, outros códigos ou classes dependem da existência de algum atributo ou método. Dito isso, case altere uma interface, algum outro código pode não funcionar adequadamente. Por exemplo, não tem problema alterar os mecanismos internos de sua classe a fim de modificar os cálculos dentro de algum método para que sejam mais eficientes. Mas é necessário que você facilite o uso da interface desde o início e tente não a alterar. No Capítulo 8, abordarei as interfaces em mais detalhes.

Abstração

A abstração está estritamente vinculada ao encapsulamento. Ou seja, devemos lidar com uma classe com o nível adequado de detalhes. Assim, podemos optar por manter os detalhes de algum cálculo em um método ou permitir que ele seja acessado por meio da interface. Novamente, isso é mais comum em outras linguagens de programação.

Polimorfismo

Polimorfismo significa que podemos ter a mesma interface para classes diferentes, o que simplifica o código e reduz a repetição. Ou seja, duas classes podem ter um método com o mesmo nome que gera um resultado semelhante, porém os mecanismos internos são distintos. As duas classes podem ser uma classe pai e uma classe filha ou podem não estar relacionadas.

O scikit-learn tem um ótimo exemplo de polimorfismo. Todos os classificadores têm o mesmo método `fit` para serem treinados com alguns

dados, ainda que sejam definidos como classes diferentes. Vejamos um exemplo de treinamento de dois classificadores diferentes em alguns dados:

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

lr_clf = LogisticRegression()
lr_clf.fit(X_train, y_train)

rf_clf = RandomForestClassifier()
rf_clf.fit(X_train, y_train)
```

Ainda que `LogisticRegression` e `RandomForestClassifier` sejam classes diferentes, ambas têm um método `.fit()` que usa os dados e os rótulos do treinamento como argumentos. Compartilhar o nome do método facilita mudar o classificador, sem alterar muito seu código.

Esta foi uma breve visão geral dos principais recursos da programação orientada a objetos. Como se trata de um tópico gigantesco, mais uma vez, recomendo o livro *Introducing Python*, de Bill Lubanovic (O'Reilly, 2019), caso queira saber mais.

Programação funcional

Apesar de o Python suportar o paradigma de programação funcional, não é comum escrever código Python em um estilo totalmente de programação funcional. Muitos engenheiros de software acreditam que outras linguagens, como Scala, são mais apropriadas à PF. No entanto, vale muito a pena conhecer os recursos úteis disponíveis de PF no Python, que abordarei a seguir.

O foco da programação funcional, como o nome sugere, são funções que não mudam. As funções não devem alterar nenhum dado existente fora delas nem quaisquer variáveis globais. Usando a terminologia correta, as funções são imutáveis, "puras" e isentas de efeitos colaterais. Não afetam nada além do que é refletido no valor que retornam. Por exemplo, se você tiver uma função que adiciona um item a uma lista, ela deve retornar uma cópia nova da lista em vez de modificar a lista existente. Rigorosamente, na PF, um programa consiste apenas em avaliar funções. Funções podem ser

aninhadas (contexto em que uma função é definida dentro de outra) ou podem ser passadas como argumentos para outras funções.

Vejamos algumas vantagens da programação funcional:

- É fácil testar porque uma função sempre retorna a mesma saída para uma determinada entrada. Nada fora da função é modificado.
- É fácil paralelizar porque os dados não são modificados.
- Obriga a escrita de código modular.
- Pode ser mais conciso e eficiente.

Conceitos comuns do Python em um estilo funcional incluem funções lambda e as funções built-in map e filter. Além disso, os geradores normalmente são escritos em estilo de PF, e as list comprehensions também podem ser consideradas uma forma de PF. Outras bibliotecas que valem a pena conhecer para PF incluem itertools (*https://oreil.ly/qOMsA*) e more-itertools (*https://oreil.ly/qOMsA*). Na próxima seção, analisarei em mais detalhes as funções lambda e map().

Funções lambda e map()

Em Python, funções lambda são funções pequenas e anônimas, que podemos usar para tarefas esporádicas e rápidas. São denominadas "anônimas" porque não são definidas como uma função Python normal com um nome.

Uma função lambda tem esta sintaxe:

```
lambda arguments: expression
```

Uma função lambda pode receber quantos argumentos quisermos, mas pode ter apenas uma expressão. Funções lambda são usadas frequentemente com funções built-in, como map e filter. Já que aceitam funções como argumentos, podem aplicar a função a todos os elementos de um iterável (como uma lista).

Vejamos um exemplo simples. Usando os dados da Meta 5 de "Definindo as próprias classes", na página 78, podemos converter uma lista de percentuais de mulheres em cargos governamentais em uma lista de proporções de 0 a 1 com a seguinte função:

```
usa_govt_percentages = [13.33, 14.02, 14.02, 14.25, ...]
```

```
usa_govt_proportions = list(map(lambda x: x / 100, usa_govt_percentages))
```

Aqui, tem muita coisa acontecendo nessa linha. A função lambda é `lambda x: x/100`. Nela, `x` é uma variável temporária que não é usada fora da função. `map()` aplica a função lambda a todos os elementos da lista. E, por último, `list()` cria uma lista nova com base em `map()`.

Vejamos o resultado:

```
>>> print(usa_govt_proportions)
... [0.1333, 0.1402, 0.1402, 0.1425, ...]
```

Repare que os dados originais não foram alterados quando essa função foi aplicada. Uma lista nova foi criada com os dados alterados.

Aplicando funções a DataFrames

De forma semelhante à função bult-in `map()` anterior, também podemos aplicar funções a DataFrames. Isso ajuda bastante, caso queria criar uma coluna nova com base em uma coluna existente. Mais uma vez, podemos usar uma função que aceita outra função como entrada. Nos pandas, essa função é o `apply()`.

Vejamos um exemplo de aplicação de uma função lambda a uma coluna em um DataFrame:

```
df["USA_processed"] = df["United States of America"].apply(lambda x:
                                                "Mostly male"
                                                if x < 50
                                                else "Mostly female")
```

Nesse exemplo, a coluna `United States of America` são os dados sobre mulheres em cargos governamentais que tenho usado ao longo do capítulo. A função lambda recebe o percentual de mulheres em cargos governamentais e retorna `"Mostly male"` [Majoritariamente homens] se esse percentual estiver abaixo de 50%, ou `"Mostly female"` [Majoritariamente mulheres] se for 50% ou mais.

Podemos também usar `df.apply()` com uma função nomeada definida em outro lugar. Vejamos a mesma função de antes, mas como uma função nomeada:

```
def binary_labels(women_in_govt):
    if women_in_govt < 50:
```

```
        return "Mostly male"
    else:
        return "Mostly female"
```

É possível chamar essa função em cada linha de uma coluna passando o nome da função como argumento para a função apply:

```
df["USA_processed"] = df["United States of America"].apply(binary_labels)
```

É uma solução melhor do que escrever uma função lambda, porque, talvez, podemos querer reutilizar a função no futuro, e também porque podemos testá-la e depurá-la separadamente. Podemos também incluir funcionalidades mais complexas do que em uma função lambda.

No pandas, a função apply tem desempenho mais lento do que as funções built-in vetorizadas, já que itera em cada linha em um DataFrame. Portanto, a prática recomendada é usar apply somente para algo que ainda não esteja implementado. Operações numéricas simples, como obter o máximo de uma lista ou opções de string simples, como substituir uma string por outra, já estão disponíveis como funções vetorizadas com desempenho mais rápido. Ou seja, devemos usar essas funções buil-in sempre que possível.

Qual paradigma devo usar?

Sinceramente, caso esteja escrevendo apenas um breve script ou trabalhando em um projeto pequeno por conta própria, não precisa adotar totalmente nenhum desses paradigmas. Basta se ater aos scripts modulares que funcionam.

Contudo, para projetos maiores, é ótima ideia avaliar o tipo de problema com o qual você está lidando e se um desses paradigmas é adequado. Você pode recorre à POO se perceber que está lidando com um conjunto de elementos que necessitam de ações específicas. É possível transformar a natureza do problema em instâncias que precisam ter comportamentos semelhantes, mas atributos ou dados diferentes. Vale ressaltar que você deve ter muitas instâncias de alguma classe. Não vale a pena escrever uma classe nova, caso tenha somente uma instância dela; isso apenas gera mais complexidade desnecessária.

Caso esteja querendo fazer coisas novas com alguns dados que permanecem fixos, a PF pode ser uma boa escolha. Além do mais, vale a pena considerar a PF se você tiver um volume enorme de dados e quiser paralelizar as

operações que faz com esses dados.

No entanto, não existe opção certa ou errada. É possível adotar sua preferência pessoal, caso trabalhe sozinho, ou optar pelo que é predominantemente usado em sua equipe para manter tudo padronizado. É bom identificar quando esses paradigmas estão sendo usados, aplicá-los no código de outras pessoas e tomar decisões sobre o que funcionaria melhor para o seu problema específico.

Aspectos fundamentais

Programação orientada a objetos e programação funcional são paradigmas com os quais você deparará quando ler códigos. A POO foca os objetos, que são estruturas de dados personalizadas, e a PF foca funções que não alteram os dados subjacentes.

Na POO, uma classe define objetos novos, que podem ter atributos e métodos. Podemos definir nossas classes, de modo que os métodos e dados associados fiquem juntos, sendo uma ótima abordagem a ser aplicada quando temos muitas instâncias de objetos semelhantes. É possível usar a herança para evitar a repetição de códigos e o polimorfismo para manter as interfaces padronizadas.

Na PF, o ideal é que tudo fique dentro da função. Isso é útil quando temos dados que não mudam e queremos fazer diferentes operações com eles, ou paralelizar o que estamos fazendo com os dados. Funções lambda são o exemplo mais comumente usado de PF em Python.

Sua escolha de paradigma depende do problema em que está trabalhando, mas é útil conhecer ambos.

---

1 N.T.: No Capítulo 2, essa figura foi traduzida para melhor compreensão dos leitores.

CAPÍTULO 5

Erros, registro em logs e depuração

Neste capítulo, apresentarei algumas técnicas para tornar seu código mais robusto. A robustez, conforme abordei no Capítulo 1, é um dos princípios que fundamenta a escrita de um bom código. Primeiro, explicarei como tratar erros em seu código, a fim de que ele possa se comportar de maneira previsível, mesmo quando algo der errado. Em seguida, ilustrarei como fazer o registro em logs: como salvar informações sobre o que seu código faz. Além de facilitar com que outras pessoas entendam o seu código, isso também auxilia na resolução de erros inesperados. Por último, exploraremos a depuração, o processo de identificar as origens dos problemas em seu código. Explicarei algumas estratégias e ferramentas para uma depuração eficiente.

Erros em Python

Um erro ocorre quando seu código para de funcionar inesperadamente, antes de concluir todas as tarefas que deveria realizar. Isso pode interromper outros processos que dependem do seu código. Às vezes, essa interrupção é desejável, mas, em outras, queremos que outra coisa aconteça para que o código continue sendo executado. Isso é conhecido como tratamento de erros. Um código deve se comportar de maneira previsível para os casos esperados, pois isso garante sua robustez.

Nesta seção, abordarei como interpretar as mensagens de erro do Python, como tratá-las e como identificar seus próprios erros.

Interpretando mensagens de erro do Python

À primeira vista, mesmo que as mensagens de erro do Python sejam enigmáticas, estão repletas de informações úteis. Existem dois tipos de mensagens: erros de sintaxe e exceções. Erros de sintaxe ocorrem quando

você escreve um código que não está completamente correto na linguagem Python, por exemplo, esquecer de fechar parênteses ou omitir def na definição de uma função. Como o interpretador Python não consegue processar corretamente esses erros, o código é interrompido e retorna um erro.

Exceções incluem todos os outros erros, como uma entrada ausente para uma função ou a tentativa de buscar o valor de uma chave inexistente em um dicionário.

No próximo exemplo, gerarei de propósito um erro para mostrar como interpretar as mensagens de erro do Python. Vamos retomar o código da linha de tendência mostrado no Capítulo 4. Agora, como uma função separada, ele está sem o argumento data na definição da função:

```
from scipy.stats import linregress


def fit_trendline(year_timestamps):
    result = linregress(year_timestamps, data)
    slope = round(result.slope, 3)
    r_squared = round(result.rvalue**2, 3)
    return slope, r_squared
```

Podemos tentar executar a função com apenas um argumento:

```
fit_trendline(timestamps)
```

Mas isso gera o seguinte erro:

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[7], line 1
----> 1 fit_trendline(timestamps)


Cell In[6], line 2, in fit_trendline(year_timestamps)
      1 def fit_trendline(year_timestamps):
----> 2     result = linregress(year_timestamps, data) ❶
      3     slope = round(result.slope, 3)
      4     r_squared = round(result.rvalue**2, 3)


NameError: name 'data' is not defined ❷
```

❶ A linha no código que é a origem do erro

❷ O tipo de erro e uma sugestão de como corrigi-lo

Os erros do Python são exibidos como tracebacks, que mostram todas as funções chamadas que levaram ao erro. Às vezes, incluem detalhes internos do Python ou de bibliotecas que você está usando, e, mesmo parecendo um pouco complicado de entender, eles contêm muitas informações úteis para ajudá-lo a descobrir a causa do erro.

Uma boa estratégia para lidar com mensagens de erro do Python é começar lendo o final da mensagem. A última linha informa com que tipo de erro estamos lidando (por exemplo, `NameError`) e algumas informações sobre como corrigi-lo, se o Python acreditar que pode ajudá-lo. Acima dos detalhes do tipo de erro, você verá seu código com uma seta apontando para a linha que causou o erro. É um bom ponto de partida para procurar o problema, mas, como esse exemplo ilustra, o problema está, na verdade, na linha anterior, acusando a ausência do argumento `data` na definição da função.

Erros também podem ocorrer a partir de módulos que importamos, por exemplo, se você fornecer o tipo errado de entrada para uma função que importou. Talvez, esses erros pareçam confusos porque, além de destacar a linha do seu código que originou o erro, a mensagem de erro inclui a linha na função importada e quaisquer outras funções que ela chamou. Ainda que a mensagem de erro seja muito extensa, a mesma estratégia se aplica: comece pelo final e trabalhe de trás para a frente para tentar resolver o problema.

No Python, as mensagens de erro passaram por melhorias substanciais nas versões 3.10 (*https://oreil.ly/QpddI*) e 3.11 (*https://oreil.ly/J7nxs*). As versões mais recentes são mais legíveis e elucidativas, e as melhorias incluem erros de sintaxe que apontam a localização correta no código, em vez de apontarem a linha anterior, como acontecia em muitos casos. Se puder, vale a pena fazer a atualização da versão mais recente do Python.

Tratamento de erros

Em diversas situações, talvez você não queira que seu código pare quando encontrar um erro. Isso é crucial se outras funções dependerem do resultado

retornado pela função principal: se sua função gerar um erro, isso pode desencadear uma cascata de erros em um sistema maior. Nesses casos, é preciso “tratar” o erro para garantir que seu código se comporte de maneira previsível.

Para esse tratamento, o Python usa as palavras-chave try e except. Elas assumem a forma de dois blocos de código:

```
try:
    # algum código que você queira executar
except KeyError:
    # o que você quer que aconteça
    # se um erro desse tipo for lançado
```

O bloco try contém o código que você quer executar, mas que pode encontrar um erro. O bloco except define o que você quer que aconteça caso ocorra um erro específico, nesse caso, um KeyError. Por exemplo, imagine um código que busca o valor de uma chave em um dicionário. Se a chave não estiver presente no dicionário, podemos optar por retornar um valor padrão em vez de interromper o código com um erro.

É possível ter múltiplos blocos except, cada um com o código a ser executado para um tipo específico de exceção, ou para tratar diversos tipos de erro em um único bloco except. Eles usam a seguinte sintaxe: except (KeyError, ValueError):.

Podemos também usar um bloco else, que deve vir após o bloco except. Ele contém o código que é executado após o bloco try se nenhum erro for gerado. Outro recurso do tratamento de erros do Python é o bloco finally, que é executado independentemente de ocorrer ou não um erro, e deve vir após os blocos try, except e o opcional else. O bloco finally é usado com mais frequência para liberar um recurso, como fechar um arquivo aberto. A Figura 5.1 mostra esse fluxo. A documentação do Python (*https://oreil.ly/GkThO*) fornece mais detalhes sobre a sintaxe de tratamento de erros.

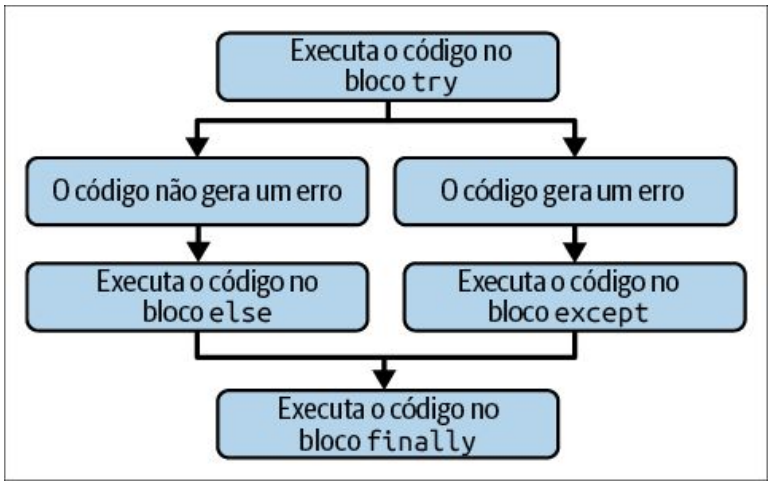


Figura 5.1: Sequência de execução para os blocos try, except, else *e* finally.

Desta vez, ilustrarei o tratamento de erros usando a função de linha de tendência da seção "Interpretando mensagens de erro do Python", na página 89, com os argumentos corretos:

```
from scipy.stats import linregress


def fit_trendline(year_timestamps, data):
    result = linregress(year_timestamps, data)
    slope = round(result.slope, 3)
    r_squared = round(result.rvalue**2, 3)
    return slope, r_squared
```

Vou executá-lo com algumas entradas incorretas para fazer com que o código lance uma exceção:

```
timestamps = ["2000", "2001", "2002"]
data = [18.36, 18.36, 17.91]


fit_trendline(timestamps, data)
```

Isso fornece um ValueError porque ambas as listas devem conter floats ou números inteiros. Podemos tratar esse erro, de modo que o código seja executado sem gerá-lo, recorrendo aos blocos try, except e else, assim:

```
def fit_trendline(year_timestamps, data):
    try:
        result = linregress(year_timestamps, data)
    except TypeError:
```

```
        print(f"Both lists must contain only float or integers,
        got {data.dtype} and {year_timestamps.dtype}")
    else:
        slope = round(result.slope, 3)
        r_squared = round(result.rvalue**2, 3)
        return slope, r_squared
```

Se um erro for gerado, o código exibe a mensagem "Both lists must contain only floats or integers [Ambas as listas devem conter apenas floats ou inteiros]", detalhando o tipo de dados fornecidos. Se nenhum erro for gerado na primeira linha, o bloco else é executado.

É importante que suas mensagens de erro sejam elucidativas e fáceis de ler. Boas mensagens de erro podem economizar um bom tempo para você e para qualquer outra pessoa que trabalhe em seu código, fornecendo orientações claras sobre o que deu errado e onde o erro ocorreu no código. A mensagem deve descrever o problema e, de preferência, sugerir como corrigi-lo.

No bloco except, podemos também retornar algum valor padrão. De preferência, no mesmo formato que a função retornaria em caso de execução bem-sucedida. Isso mantém a interface da sua função consistente. Caso tenha outro código que dependa dessa função para retornar respostas de algum tipo específico, ele ainda será executado, independentemente de essa função ter encontrado um erro.

Vejamos um exemplo em que o bloco except retorna um valor padrão:

```
def fit_trendline(year_timestamps, data):
    try:
        result = linregress(year_timestamps, data)
    except TypeError:
        print("Both lists must contain floats or integers.")
        return 0.0, 0.0
    else:
        slope = round(result.slope, 3)
        r_squared = round(result.rvalue**2, 3)
        return slope, r_squared
```

Agora, a função retorna uma tupla de dois floats, independentemente de encontrar um erro ou não.

Quando estava pensando em escrever este livro, revisitei um código que fiz antes de conseguir meu primeiro emprego em ciência de dados. Este código exemplifica como não tratar erros:

```
except:
    return
```

Por favor, não faça isso! Esse código silenciará todos os erros que encontrar, impedindo que você saiba se uma função falhou. É muito melhor que sua função se comporte de maneira previsível, mesmo quando recebe uma entrada incorreta.

Gerando erros com raise

Em algumas circunstâncias, talvez você queira que seu código gere um erro quando algo inesperado acontecer. Talvez, você também queira que qualquer código que dependa da sua função pare de executar. Nessa situação, você pode usar a palavra-chave `raise` do Python para gerar um erro, mesmo que ele não seja lançado normalmente, e ainda pode personalizar esse erro.

No próximo exemplo, verificarei se alguma das entradas da função `fit_trendline` são listas vazias e lançarei um erro, caso ocorra:

```
from scipy.stats import linregress


def fit_trendline(year_timestamps, data):
    if not year_timestamps or data:
        raise ValueError("Timestamps and data cannot be empty lists")
    result = linregress(year_timestamps, data)
    slope = round(result.slope, 3)
    r_squared = round(result.rvalue**2, 3)
    return slope, r_squared
```

É possível fornecer uma mensagem de erro personalizada com a sintaxe `raise {error}('Message')`.

É possível também executar esta função com uma lista vazia:

```
it_trendline([], [18.36, 18.36, 17.91])
```

A função retorna o seguinte resultado, incluindo a mensagem de erro personalizada:

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[27], line 1
----> 1 fit_trendline([], [18.36, 18.36, 17.91])

Cell In[26], line 3, in fit_trendline(year_timestamps, data)
      1 def fit_trendline(year_timestamps, data):
      2     if not year_timestamps or data:
----> 3         raise ValueError('Timestamps and data cannot be empty lists')
      4     result = linregress(year_timestamps, data)
      5     slope = round(result.slope, 3)

ValueError: Timestamps and data cannot be empty lists
```

A mensagem de erro deve ser informativa para a pessoa que a lê, e contribuir para a legibilidade do seu código. Mensagens como “Ocorreu um erro” não ajudam em nada.

Vejamos um exemplo de uma mensagem de erro útil que encontrei ao escrever o Capítulo 2:

```
"UsageError: Cell magic `%%lprun` not found (But line magic `%lprun` exists,
did you mean that instead?)."[1]
```

Ela me indica o problema e sugere o que eu deveria fazer para corrigi-lo.

Você também pode definir erros personalizados se nenhum dos erros padrão do Python atender às suas necessidades. Trata-se de uma boa prática para projetos grandes, pois possibilita nomear os erros de forma personalizada, facilitando a identificação da origem do erro pelos usuários do código. Definir um novo erro assume a forma:

```
class NewException(Exception):
```

Esse é um exemplo de “Herança”, na página 81, em que a classe nova de erro herda da classe `Exception`. Para saber mais sobre a definição de erros personalizados, recomendo a leitura deste artigo (*https://oreil.ly/P0bj9*).

Usar erros personalizados pode facilitar a legibilidade e a manuntebilidade do seu código. Quando os erros são devidamente definidos, outras partes do código podem tratar erros específicos e responder de forma adequada. Isso garante que os erros de diferentes partes da base de código não se misturem.

Esse nível de granularidade no tratamento de erros ajuda a evitar consequências indesejadas. Além disso, faz com que a depuração e a solução de problemas sejam mais eficientes.

Registro em logs

O registro em logs é um método que registra o que seu código fez em um arquivo separado enquanto estava sendo executado. Isso significa que seu código pode comunicar a outras pessoas o que está acontecendo, ajudando-as a compreendê-lo. A prática torna seu código mais legível e robusto, pois oferece um registro do que acontece em caso de falhas.

Com o registro em logs, é possível preservar o histórico das ações do seu código. É diferente de simplesmente fazer o código exibir mensagens, já que podemos salvar informações em um arquivo separado e pesquisável. Isso é bastante útil para processos ou códigos de longa execução, implantados em um sistema de produção. Em situações como essa, não queremos interromper a execução do código para identificar o que está acontecendo ou o que saiu de errado.

Apresentarei uma introdução básica ao registro em logs, mas, caso queria se aprofundar, recomendo os tutoriais de registro em logs (*https://oreil.ly/8s5Ey*) da documentação do Python.

O que registrar em logs

O módulo `logging` do Python possibilita registrar qualquer mensagem que quisermos. O que é útil registrar em log? Isso dependerá do seu caso de uso e projeto exatos, mas podemos registrar:

- Mensagens informando o início ou término de tarefas de longa execução
- Mensagens de erro para identificar problemas em sistemas em produção
- Quais funções chamaram outras funções
- As entradas e saídas de uma função
- O path [caminho] do arquivo em que alguns dados foram salvos

O registro em logs é extremamente útil na depuração de um sistema de produção ou de uma tarefa de longa execução. Abordarei esse assunto em

detalhes na seção “Depuração”, na página 101.

Além disso, o registro em logs pode servir como evidência de que seu código fez o que deveria. Caso seja relevante para o seu projeto, inclua mensagens de log informando a ocorrência de uma tarefa e o resultado. Um exemplo seria registrar em log a acurácia de um modelo de machine learning após o treinamento.

Uma boa estratégia é pecar pelo excesso, registrar mais, não menos. Se suas mensagens forem estruturadas de forma padronizada ou incluírem palavras-chave específicas, você pode pesquisar no arquivo de log e encontrar facilmente o que procura. É importante também ser específico em suas mensagens de log. Uma mensagem como "started saving file [começando a salvar o arquivo]" não serve de nada. Qual é o conteúdo do arquivo? Qual é o nome do arquivo? As f-strings são uma maneira elegante de tratar esse caso de uso:

```
f"Saving {variable_name} data to {file_name}"
```

Nas próximas seções, explicarei como registrar mensagens em logs usando o módulo logging do Python.

Configuração do módulo logging

Embora o módulo logging da biblioteca padrão do Python seja vasto e abrangente, precisamos somente de um pequeno subconjunto de funções dentro dele. Como a biblioteca se encarrega de salvar todas as mensagens que registramos em logs em um arquivo separado, não é necessário se preocupar com isso. No entanto, algumas configurações são necessárias.

A primeira configuração a se considerar é o nível de gravidade dos logs que são registrados. O módulo logging do Python tem cinco níveis diferentes de gravidade, de DEBUG (menos grave) a CRITICAL (mais grave). Isso possibilita filtrar os logs para encontrar facilmente as mensagens que você está procurando ou apenas registrar mensagens acima de um determinado nível, dependendo do seu caso de uso. Por exemplo, durante o desenvolvimento do código, talvez você queira registrar mensagens em log no nível DEBUG, mas, quando o código estiver em produção, talvez queira registrar em logs apenas erros e mensagens mais graves.

A Tabela 5.1 da documentação do Python descreve todos os cinco níveis de

registro em log:

Tabela 5.1: Níveis do registro em log do Python[a]

Nível	Quando é usado
DEBUG	Informações detalhadas, normalmente de interesse somente no diagnóstico de problemas.
INFO	Confirmação de que tudo está funcionando conforme o esperado.
WARNING	Uma indicação de que algo inesperado ocorreu ou um indicativo de algum problema em um futuro próximo (por exemplo, “disk space low [pouco espaço em disco]”). O software ainda está funcionando conforme o esperado.
ERROR	Devido a um problema mais grave, o software não conseguiu executar alguma função.
CRITICAL	Um erro grave, indicando que o próprio programa pode não conseguir continuar em execução.

[a] *Fonte: https://oreil.ly/6iVoR*

No módulo logging do Python, o nível de registro em logs padrão é WARNING. Ou seja, apenas mensagens desse nível ou mais graves são automaticamente registradas em logs. Isso significa que, se você escrever mensagens no nível INFO sem alterar o nível de log, elas não serão salvas.

Antes de registrar qualquer coisa em logs, defina o nível usando:

```
import logging

logging.basicConfig(level=logging.DEBUG)
```

Aqui, já que nível está definido como DEBUG, todos os níveis, desde DEBUG até CRITICAL, serão registrados em logs. É necessário que essa linha esteja no início do seu script do Python, antes de você registrar em logs qualquer coisa.

Outra definição padrão que, provavelmente, você quer sobrescrever é onde os logs serão salvos. Por padrão, os logs são exibidos no console, porém são mais úteis quando salvos em um arquivo para consulta posterior após a execução do código.

Caso queira salvar seus logs em um arquivo separado, adicione a seguinte definição à configuração no início de seu script:

```
logging.basicConfig(filename='chapter_5_logs.log', level=logging.DEBUG)
```

A definição padrão ao gravar em um arquivo é anexar os logs ao arquivo

sempre que o código for executado. Caso queira sobrescrever o arquivo, use filemode='w':

```
logging.basicConfig(filename='chapter_5_logs.log', filemode='w',
                    level=logging.DEBUG)
```

Além de salvar mensagens em um arquivo, os log viewers [visualizadores de log] podem capturar a saída do módulo logging do Python e salvá-la ou exibi-la em outro lugar. Sistemas de produção em provedores de nuvem como o Amazon Web Services (AWS) oferecem soluções built-in para isso e, geralmente, utilizam os handlers (*https://oreil.ly/apGQb*) do módulo logging do Python. Ferramentas independentes como Kibana (*https://oreil.ly/Cgkn7*) e Prometheus (*https://prometheus.io*) também coletam logs gerados pelo seu código.

Como registrar em logs

Após definir sua configuração de registro em logs no início do script, a próxima etapa é decidir o que registrar. Adicionarei algumas linhas de registro em logs à função fit_trendline no nível INFO usando a função logging.info, que aceita a mensagem que queremos registrar em log como argumento:

```
from scipy.stats import linregress


def fit_trendline(year_timestamps, data):
    logging.info("Running fit_trendline function")
    result = linregress(year_timestamps, data)
    slope = round(result.slope, 3)
    r_squared = round(result.rvalue**2, 3)
    logging.info(f"Completed analysis. Slope of the trendline is {slope}.")
    return slope, r_squared
```

É possível executar esta função como antes:

```
fit_trendline(timestamps, data)
```

Ela gerará isto no arquivo de log:

```
INFO:root:Running fit_trendline function
INFO:root:Completed analysis. Slope of the trendline is 0.836.
```

Incluir um timestamp [carimbo de data/hora] aos seus logs pode ser útil para saber quando uma determinada linha de código foi executada e facilitar pesquisas posteriores. É possível fazer isso nas definições de configuração no início do script com o argumento format:

```
logging.basicConfig(filename="chapter_5_logs.log",
                    level=logging.DEBUG,
                    format='%(asctime)s %(message)s') ❶
```

❶ O argumento format tem muitas opções, explicadas na documentação de registro em logs do Python (*https://oreil.ly/wUJdq*).

O código retorna as seguintes mensagens:

```
2023-05-26 11:37:43,951 Running fit_trendline function
2023-05-26 11:37:43,953 Completed analysis. Slope of the trendline is 0.836.
```

É boa ideia registrar mensagens de erro em logs, e é possível fazer isso com a logging.exception, assim:

```
def fit_trendline(year_timestamps, data):
    logging.info("Running fit_trendline function")
    try:
        result = linregress(year_timestamps, data)
    except TypeError as e:
        logging.error("Both lists must contain floats or integers.") ❶
        logging.exception(e) ❷
    else:
        slope = round(result.slope, 3)
        r_squared = round(result.rvalue**2, 3)
        logging.info(f"Completed analysis. Slope of the trendline is {slope}.")
        return slope, r_squared
```

❶ Registra uma mensagem em log no nível “Error”

❷ Registra todo o traceback em logs

logging.exception() é muito útil porque todo o traceback é gravado em logs, e podemos usá-lo para depuração.

É possível usar o módulo logging em um notebook do Jupyter, que pode preservar os resultados em um arquivo de log, de modo que não precisemos executar novamente o notebook. Talvez seja uma opção melhor do que copiar e colar os resultados em uma célula Markdown, mas implica gerenciar outro arquivo. Além disso, é possível usar soluções de rastreamento de experimentos, como as abordadas na seção “Documentando experimentos de machine learning”, na página 176.

Depuração

Depurar significa identificar e remover bugs do código. Um bug ocorre quando o código está gerando erros inusitados ou resultados não esperados. Já que a depuração é uma habilidade fundamental para escrever códigos, você precisará de uma variedade de técnicas ao lidar com bases de código maiores, a fim de recorrer à ferramenta adequada para cada situação. Nesta seção, apresentarei algumas estratégias para encontrar bugs e, em seguida, abordarei algumas ferramentas para depuração.

Estratégias para depuração

> Minha maneira favorita de obter informações sobre códigos com bugs é executá-los e fazer experimentos com eles. (Adicione instruções print! Faça uma singela mudança!).
>
> – JULIA EVANS

Conforme você ganha experiência, provavelmente desenvolverá as próprias estratégias favoritas para identificar bugs difíceis. No entanto, um ótimo ponto de partida, como recomendado na citação de Julia Evans, é fazer o que um cientista de dados faz melhor: realizar um experimento! Você pode fazer uma pequena mudança em seu código e observar como ela afeta os resultados, tomando cuidado para realizar apenas uma mudança de cada vez.

No Capítulo 7, abordaremos esse tópico, mas, por ora, quero abordar a utilidade dos testes. Um teste indicará se uma mudança feita por você gera o resultado esperado do seu código (o teste passa) ou não (o teste falha).

Outra dica importante é salvar uma cópia do que causou o erro, sejam as entradas da sua função ou os resultados de uma query específica de banco de dados. É aqui que o registro em logs se torna valioso, porque você tem um registro do que aconteceu.

Além disso, você provavelmente não é a primeira pessoa a cometer o erro que está causando a mensagem de erro. Uma profusão de informações está disponível na internet para ajudá-lo a resolver seus problemas. Pesquisar a mensagem de erro no Google é um ótimo ponto de partida.

Conversar com outras pessoas sobre seu código é outra ótima estratégia para identificar bugs. É possível encontrar uma solução ao explicar o problema para um colega de equipe ou até mesmo falar em voz alta sobre

ele para um pato de borracha[2], pois isso o faz refletir sobre o que está fazendo de uma maneira diferente do que apenas analisar o código. Outras pessoas também podem compartilhar suas experiências e ajudá-lo.

Todo mundo tem bugs no código e, embora nem todas as culturas organizacionais incentivem isso, deveria ser aceitável dizer que você não sabe algo ou que cometeu um erro. Eu certamente escrevo código com muitos bugs. Alguns são sutis e difíceis de encontrar; outros são simples, mas podem ser frustrantes de identificar. Enquanto eu escrevia esta obra, alguém me enviou um e-mail para informar que detectou um erro no código da primeira edição online!

Outra excelente estratégia de depuração é fazer uma pausa enquanto programa. Faça uma xícara de chá, uma caminhada, trabalhe em outra coisa. A solução pode surgir enquanto você está fazendo algo completamente diferente ou quando retornar ao código com um novo olhar. Uma pausa também ajuda a pensar em diferentes causas possíveis para o seu bug.

Às vezes, será necessário se aprofundar nos detalhes do seu código, e as ferramentas da próxima seção podem ajudá-lo.

Ferramentas para depuração

Ferramentas de depuração ajudam a rastrear exatamente o que está acontecendo em seu código. Caso consiga obter informações sobre os valores associados a uma variável nomeada em seu código, isso será extremamente útil para verificar se o código está fazendo o que você acredita que deveria estar fazendo. A maneira mais simples de fazer isso é adicionar instruções `print` ao código, porém é possível obter mais informações recorrendo às ferramentas de depuração. Elas fornecem uma maneira de pausar seu código e exibir o valor de qualquer variável específica e, em seguida, percorrer o código um passo de cada vez.

Ilustrarei isso com alguns exemplos de código do Capítulo 2. Neste exemplo, a função `weighted_mean` contém um bug e não retornará o resultado correto:

```
def weighted_mean(num_list, weights):
    running_total = 0
```

```
    for i in range(len(num_list)):
        running_total += (num_list[i] * weights[0]) ❶
    return (running_total/len(num_list))
```

❶ O erro está nesta linha: em vez de weights[0], deve ser weights[i].

Adicionar uma instrução print pode ajudar a identificar o bug, assim:

```
def weighted_mean(num_list, weights):
    running_total = 0
    for i in range(len(num_list)):
        running_total += (num_list[i] * weights[0])
        print(f"The running total at step {i} is {running_total}")
    return (running_total/len(num_list))
```

Executar essa função fornece a seguinte saída:

```
>>> print(weighted_mean([1, 6, 8], [1, 3, 2]))
... The running total at step 0 is 1
... The running total at step 1 is 7
... The running total at step 2 is 15
...
... 5.0
```

Isso indica que running total não está sendo calculado corretamente. Deve ser 19 no passo 1 e 35 no passo 2.

No entanto, não é prática recomendada sobrecarregar seu código com instruções print. Isso dificulta escolher o resultado do seu código e, se ele estiver em execução em um sistema de produção, talvez você não tenha acesso à saída.

Outra opção é registrar em logs os valores intermediários do seu código, conforme descrito em “Registro em logs”, na página 96. Vejamos um exemplo:

```
import logging

logging.basicConfig(filename="chapter_5_logs.log",
                    level=logging.DEBUG,
                    format='%(asctime)s %(message)s')

def weighted_mean(num_list, weights):
```

```
    running_total = 0
    for i in range(len(num_list)):
        running_total += (num_list[i] * weights[0])
        logging.debug(f"The running total at step {i} is {running_total}")
    return (running_total/len(num_list))
```

Isso gerará a seguinte saída no arquivo de log:

```
2023-06-01 17:14:56,976 The running total at step 0 is 1
2023-06-01 17:14:56,980 The running total at step 1 is 7
2023-06-01 17:14:56,980 The running total at step 2 is 15
```

Essa saída é salva em um arquivo separado, ou seja, você pode acessá-la em um sistema de produção.

Instruções `print` ou registro em logs podem ser adequados para programas pequenos como este, mas e se você tiver dezenas de variáveis em sua função? Como proceder se algumas das variáveis que você quer registrar em logs forem grandes dicionários? Como podemos ver, isso poderia ficar pouco prático. Felizmente, os engenheiros de software se deparam frequentemente com problemas desse tipo, e existe uma série de ferramentas (depuradores [debuggers]) para ajudá-lo nessa situação.

Os depuradores usam o conceito de breakpoints [pontos de parada]. Breakpoints são pontos selecionados para pausar a execução do código e inspecionar o estado de alguma variável (ou de um arquivo ou de um banco de dados salvos). Podemos verificar se tudo saiu conforme o esperado e, em seguida, percorrer o código linha a linha a fim de analisar como as variáveis mudam. Isso ajuda a identificar a origem de bugs.

Em geral, ambientes de desenvolvimento integrados incluem um depurador. Ilustrarei como usar um depurador no VS Code, mas o processo é muito semelhante em outros IDEs.

No próximo exemplo, mostrarei como depurar o seguinte script executável:

```
def weighted_mean(num_list, weights):
    running_total = 0
    for i in range(len(num_list)):
        running_total += (num_list[i] * weights[0]) ❶
    return (running_total/len(num_list))
```

```
weighted_mean([1, 6, 8], [1, 3, 2])
```

❶ Adicionei um breakpoint a essa linha de código.

Com o depurador, é possível percorrer facilmente o que cada variável representa em qualquer ponto da função. Ele destaca as mudanças em cada passo do loop.

A Figura 5.2 mostra a primeira iteração do loop.

O depurador mostra que o valor de `running_total` é 1 quando o valor de `i` é 0.

Em seguida, percorrerei as linhas do código até a segunda iteração, quando `i` tiver o valor 1. Como podemos ver na Figura 5.3, `running_total` está incorreto neste ponto. Deveria ser (1 * 1) + (3 * 6) = 19, mas, em vez disso, é 7.

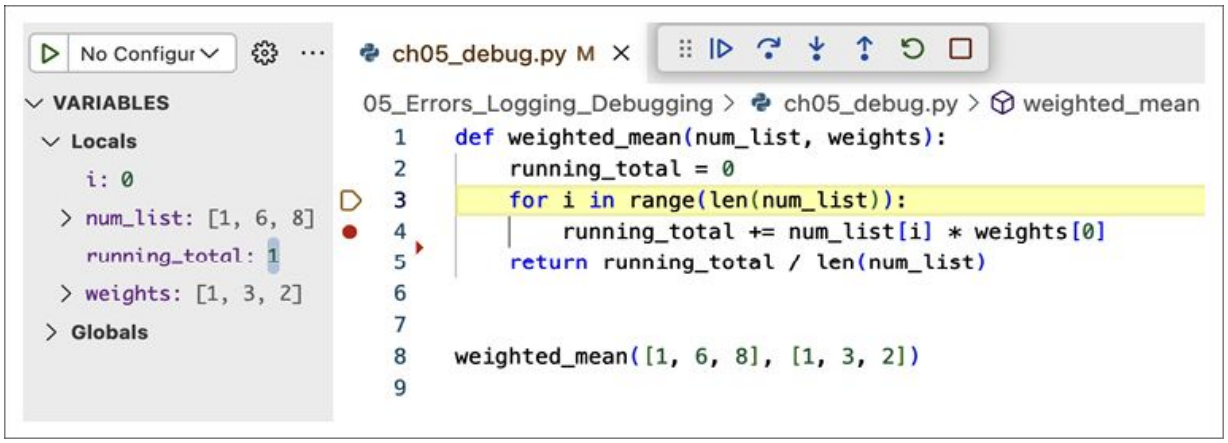


Figura 5.2: Mostrando as variáveis em um depurador do IDE, iteração 1.

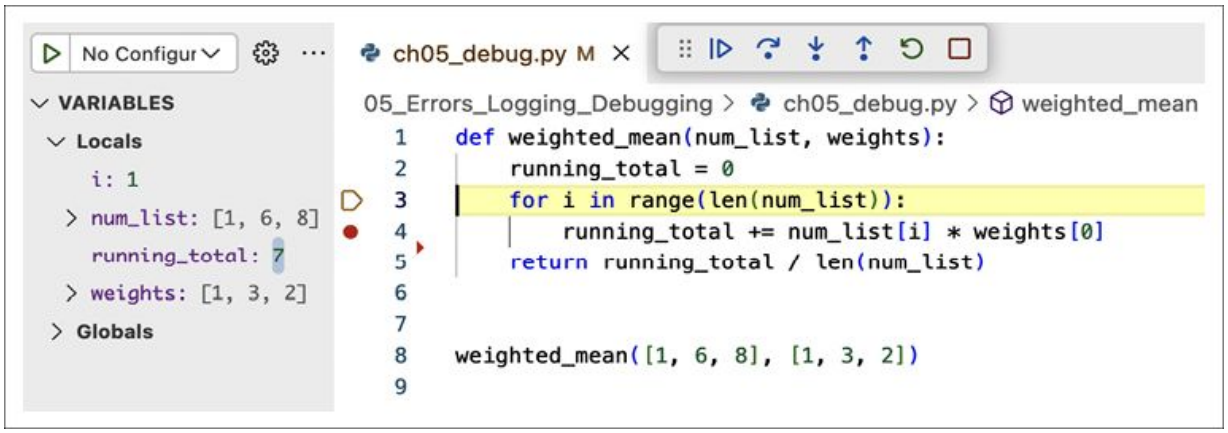


Figura 5.3: Mostrando as variáveis em um depurador do IDE, iteração 2 (observe que os valores de `i` *e* `running_total` *foram alterados).*

O bug está na quarta linha da função. Vejamos a linha corrigida:

```
        running_total += (num_list[i] * weights[i])
```

Os detalhes exatos de como usar o depurador dependerão de qual IDE você está usando, mas todos terão recursos semelhantes. Para mais detalhes, confira a documentação do VS Code (*https://oreil.ly/j9K2a*) e a documentação do PyCharm (*https://oreil.ly/xQkOW*).

Além disso, podemos usar a funcionalidade de depuração se estivermos executando um notebook do Jupyter em um IDE, conforme mostrado na Figura 5.4.

Podemos simplesmente adicionar um breakpoint da mesma forma que fazemos em um script autônomo.

É possível também usar um depurador mesmo que não esteja usando um IDE. O pdb (*https://oreil.ly/78KQu*) é um depurador de linha de comando incluído na biblioteca padrão do Python.

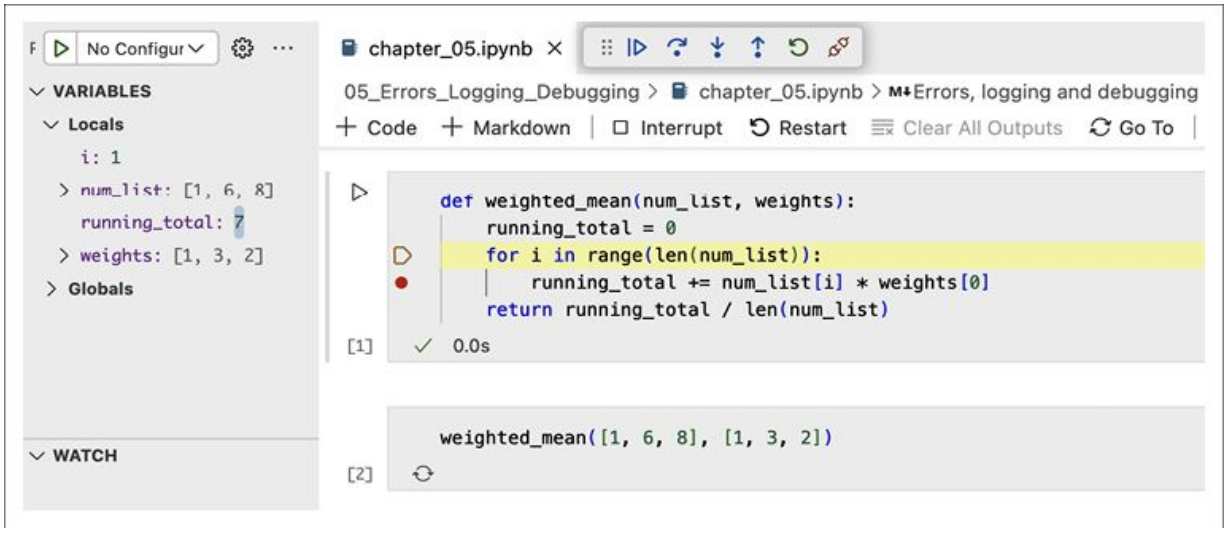


Figura 5.4: Depurando um notebook em um IDE.

Para usá-lo, adicione o método breakpoint() em seu código, assim:

```
def weighted_mean(num_list, weights):
    running_total = 0
    for i in range(len(num_list)):
        running_total += (num_list[i] * weights[0])
        breakpoint()
    return (running_total/len(num_list))
```

```
weighted_mean([1, 6, 8], [1, 3, 2])
```

Em seguida, basta executar o script como de costume:

```
$ python ch05_debug.py
```

O código será executado até atingir o breakpoint e, em seguida, abrirá o depurador pdb, fornecendo uma saída mais ou menos como esta:

```
> /Users/.../ch05_debug.py(3)weighted_mean()
-> for i in range(len(num_list)):
(Pdb)
```

Podemos mostrar o valor de qualquer variável com `p variable_name`:

```
(Pdb) p running_total
1
(Pdb) p weights
[1, 3, 2]
```

E percorrer a próxima linha com `n`:

```
(Pdb) n
> /Users/.../ch05_debug.py(4)weighted_mean()
-> running_total += (num_list[i] * weights[0])
(Pdb) n
> /Users/.../ch05_debug.py(5)weighted_mean()
-> breakpoint()
(Pdb) n
> /Users/.../ch05_debug.py(3)weighted_mean()
-> for i in range(len(num_list)):
(Pdb) p running_total
7
```

Ao terminar, é possível interromper a depuração com `q`, e isso encerra o script:

```
(Pdb) q
```

Quando terminar de usar o pdb, você precisará percorrer seu código linha a linha e excluir todas as chamadas `breakpoint()`; caso contrário, o pdb será iniciado sempre que você executar o script. No Capítulo 12, mostrarei uma forma de removê-las automaticamente.

A Tabela 5.2 lista os atalhos de comando para pdb com base neste artigo de Nina Zakharenko (*https://oreil.ly/z1sx2*).

Tabela 5.2: comandos do pdb

Atalho do comando	Função
p expr	Exibe o valor da expressão
l	Exibe as linhas de código ao redor da linha atual
n	Passa para a próxima linha
c	Continua até o próximo breakpoint
r	Continua até que a função atual retorne
q	Sai e conclui a depuração

O pdb disponibiliza a mesma funcionalidade de um depurador em um IDE. No entanto, é bom conhecer algumas ferramentas diferentes para usar a que melhor se adapta à sua situação. Se estiver desenvolvendo código em um IDE, é mais fácil usar o depurador incluído nele; porém se estiver executando o script na linha de comando, o pdb é mais adequado.

Aspectos fundamentais

Neste capítulo, abordei como tornar seu código mais robusto, e o registro em logs também contribui para a legibilidade dele. Além do mais, apresentei uma breve visão geral das habilidades para identificar e resolver problemas em seu código.

O tratamento adequado de erros contribui para a robustez do código a entradas inesperadas, e você deve personalizar os próprios erros, quando necessário.

O registro em logs salva um registro do que seu código fez, sendo extremamente útil para códigos de produção ou scripts que levam muito tempo para serem executados. O módulo built-in `logging` do Python contém todas as funcionalidades de que precisamos.

Ajuda muito ter uma variedade de estratégias e ferramentas à mão para identificar e remover bugs do código. Podemos usar as mais variadas ferramentas de depuração, desde instruções `print`, registro em logs e até ferramentas mais específicas; no Capítulo 7, abordarei como usar testes para depuração.

---

1 N.T.: Traduzida, essa mensagem seria mais ou menos assim: UsageError: A célula mágica `%%lprun` não foi encontrada (mas a linha mágica `%lprun` existe, seria essa, não?).

2 N.T.: Pato de borracha se refere à técnica de depuração rubber duck (ou depuração com pato de borracha), em que o programador explica seu código, linha por linha, para um objeto inanimado, como um pato de borracha. A técnica se originou do livro *O Programador Pragmático*, pois um dos autores tinha um pato de borracha na mesa enquanto programava.

CAPÍTULO 6

Formatação de código, linting e verificação de tipo

Neste capítulo, exploraremos ferramentas que podem ajudá-lo com alguns aspectos mais tediosos da escrita de código. As ferramentas de formatação de código, de linting e de verificação de tipo analisam o código com o intuito de verificar erros ou áreas de melhoria. As ferramentas de formatação de código focam a estética do código, já as ferramentas de linting e de verificação de tipo garantem que o código funcione corretamente.

Talvez você esteja se perguntando por que a formatação de código é tão importante. Por que a estética do código é importante? Por que as pessoas dedicam seu precioso tempo definindo padrões para o número de espaços ao redor de um sinal de `+`? Porque a formatação consistente e padronizada torna o código legível. E, conforme vimos no Capítulo 1, a legibilidade aumenta as chances de reutilização do seu código. Com as ferramentas de formatação, não precisamos dedicar tempo para atualizar manualmente a estética do código.

Ferramentas de linting e de verificação de tipo ajudam a garantir que o código seja robusto. Ao executarmos um código em Python, erros de sintaxe podem fazê-lo parar de funcionar imediatamente, não importando em que local estejam no script. Mas, se outros erros ocorrerem, como um nome de variável digitado incorretamente, eles só aparecerão quando a linha for processada. É frustrante quando um script demora muito para ser executado. Linters e ferramentas associadas podem ajudá-lo a identificar alguns erros antes de executar seu código.

A mensagem central que quero passar neste capítulo é: use ferramentas em vez de verificar o código manualmente. Os detalhes da formatação são entediantes e não merecem seu tempo. Mostrarei como configurar ferramentas e fazer bom uso de seu IDE, a fim de que os padrões que você

precisa adotar sejam automaticamente aplicados.

Muitas das ferramentas que descrevo neste capítulo não funcionam no Jupyter Notebook, pois não são adequadas para código dividido em células separadas, sendo projetadas para scripts mais extensos. Dito isso, neste capítulo, ilustrarei exemplos de como executar as ferramentas em scripts independentes.

Formatação de código e guias de estilo

Formatar o código de acordo com um guia de estilo é essencial para escrever um bom código. A empresa pode definir um guia de estilo, por exemplo, o guia de estilo do Google (*https://oreil.ly/8yGQU*), ou na ausência de um guia de estilo próprio, o padrão é usar o PEP8, abordado na próxima seção.

A formatação do código altera a aparência do código, mas não altera seu funcionamento. No código, a formatação inclui elementos como a localização das quebras de linha, os espaços em branco em torno de um sinal de igual ou o número de linhas em branco de um script entre diferentes funções.

Aplicar um estilo consistente contribui com a legibilidade do código, pois, além de ser mais rápido ler códigos novos quando estão em um estilo consistente, também sabemos o que esperar. A consistência também diminui a probabilidade de introduzir erros de sintaxe acidentalmente, como parênteses ausentes ou incompatíveis. Mais uma vez, isso ocorre porque fica mais fácil saber o que esperar quando nos deparamos com um código padronizado.

Outra vantagem importante de um estilo consistente é evitar discussões sobre formatação durante as revisões de código [code review], possibilitando que você se concentre nas funcionalidades e verifique se os requisitos da sua equipe estão sendo atendidos pelo código.

A grande controvérsia: tabs versus espaços

Devemos usar tabs [tabulações] ou espaços para indentar o código? Essa pergunta se tornou uma verdadeira controvérsia entre desenvolvedores, com debates acalorados e intermináveis na internet sobre qual seria a melhor opção. Ao contrário das tabs, por mais que os espaços estejam presentes em qualquer teclado de computador, é mais rápido digitar tabs, pois reduzem o

> tamanho do arquivo, já que existem menos caracteres para salvar. Em 2016, Felipe Hoffa, então desenvolvedor da Google, analisou 1 bilhão de arquivos de código (14 terabytes!) (*https://oreil.ly/d2K4X*) e descobriu que os espaços eram mais populares. Os espaços eram predominante e amplamente usados.
>
> Essa análise também revela a mentalidade de alguns engenheiros de software: talvez eles foquem bastante os detalhes do código que estão escrevendo. No entanto, essa controvérsia perdeu força recentemente. Muitos IDEs usam automaticamente espaços para indentar o código quando o usuário pressiona a tecla tab, combinando a velocidade das tabs com a consistência dos espaços e automatizando o processo.

Na próxima seção, apresentarei os principais aspectos do PEP8, como formatar as importações [imports] de código e como automatizar o processo de formatação.

PEP8

O Python Enhancement Proposal 8, ou PEP8 (*https://oreil.ly/RqeYi*), é o documento que estabelece os padrões para a formatação do Python. Esse guia de estilo, escrito por Guido van Rossum, Barry Warsaw e Nick Coghlan em 2001, surgiu como um manual de estilo para a biblioteca padrão do Python, quando a linguagem começava a ganhar popularidade. A comunidade de desenvolvedores do Python o adotou como guia de estilo padrão, buscando aumentar a consistência entre todos que programam em Python.

Como afirma o PEP8:

> Um guia de estilo tem como objetivo a consistência. Nesse guia de estilo, a consistência é importante. Em um projeto, a consistência é ainda mais importante. Em um módulo ou em uma função, a consistência é imprescindível.

O PEP8 é repleto de diretrizes sobre o que fazer e o que evitar em seu código. Vejamos um exemplo de uma dessas diretrizes: recomenda-se inserir uma quebra de linha após uma instrução `if`:

```
# Correto:
if foo == 'blah':
    do_blah_thing()

# Incorreto:
if foo == 'blah': do_blah_thing()
```

O código ainda executa a opção “incorreta”, mas a legibilidade é muito maior com a opção “correta”.

O PEP8 dedica atenção especial aos espaços em branco, pois eles são fundamentais para manter a legibilidade do seu código. Por exemplo, o PEP8 aborda as melhores práticas para espaçamento em torno de sinais de “=”, o número de linhas em branco em torno da definição de uma função, seja dentro de uma classe (uma) ou sozinha (duas). Sugere também como escrever comentários e escolher nomes de variáveis, tópico que abordarei no Capítulo 9. Além disso, sugere o uso de espaços para indentação, em vez de tabs, para garantir a consistência entre ambientes.

O PEP8 elenca ainda mais detalhes, mas você não precisa lê-lo na íntegra. Utilize uma das ferramentas deste capítulo para garantir a conformidade do seu código com o guia de estilo. O Flake8 ou o Pylint podem destacar trechos do código em seu IDE, indicando onde fazer mudanças. Abordarei essas ferramentas na seção “Linting”, na página 116. Abordarei também o formatador de código Black na seção “Formatação automática de código com o Black”, na página 113, que é compatível com o PEP8, mesmo incluindo as próprias preferências de estilo.

Formatação de importação

A importação de módulos externos é fonte frequente de bugs. É fácil se esquecer de atualizar os módulos importados ao atualizar o código; por isso, ter uma lista clara dos módulos que você está usando é crucial.

O PEP8 estabelece padrões para agrupar suas importações:

> As importações devem ser agrupadas na seguinte ordem:
>
> 1. Importações de módulo da biblioteca padrão
> 2. Importações de módulos relacionados entre si.
> 3. Importações de módulos específicos de aplicações e de bibliotecas.

Felizmente, não precisamos fazer isso manualmente. É possível usar uma ferramenta como o isort (*https://oreil.ly/7oNhq*) para organizar as importações de módulo na ordem correta.

Podemos instalar o isort com o seguinte comando:

```
$ pip install isort
```

Antes de executar o isort, talvez suas importações estejam desorganizadas:

```
import time
from sklearn.metrics import mean_absolute_error

import sys, os
import numpy as np
from sklearn.model_selection import train_test_split
import pandas as pd

from sklearn.neural_network import MLPRegressor
import matplotlib.pyplot as plt

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import FunctionTransformer, OneHotEncoder
```

É possível executar o isort com o seguinte comando:

```
$ isort my_script.py
```

Após a execução do isort, as importações ficam mais ou menos assim:

```
import os
import sys
import time

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import (FunctionTransformer, OneHotEncoder,
                                   StandardScaler)
```

O código fica mais legível e em conformidade com o PEP8. Podemos também usar o isort como um plug-in no IDE. O reorder-python-imports (*https://oreil.ly/ggPBW*) é uma alternativa ao isort que insere apenas uma importação em cada linha.

Formatação automática de código com o Black

O Black é uma ferramenta que automatiza o processo de formatação do código. Ele aplica um estilo de código, sem a necessidade de revisão humana. A ideia proposta pelo Black é escrever código rapidamente, código feio, sem se preocupar com a estética, salvar o arquivo e voilá: como um passe de mágica, o código fica consistente. O Black aplica um estilo de codificação uniforme especificado pelos criadores da ferramenta. Seu nome é uma homenagem à famosa frase de Henry Ford sobre poder escolher qualquer cor de carro, desde que seja preto.

O Black usa um subconjunto do PEP8, com algumas pequenas diferenças. Por exemplo, em vez de limitar o comprimento de todas as linhas do código a 79 caracteres, o Black tentará encontrar um lugar prudente para dividir linhas mais extensas, de modo que tenham cerca de 90 caracteres. Embora o Black não permita muitas alterações em suas opções de configuração, podemos personalizar o comprimento das linhas, conforme detalhado na documentação da ferramenta (*https://oreil.ly/V1l_7*).

Para instalar o Black, utilize o seguinte comando:

```
$ pip install black
```

Se quiser usar o Black em notebooks do Jupyter, instale-o assim:

```
$ pip install "black[jupyter]"
```

Usarei o seguinte script para demonstrar como o Black funciona na prática, e também para exemplificar as ferramentas de linting da seção “Linting”, na página 116. Este é o código que gera a Figura 2.1, o gráfico das classes comuns da notação Big O do Capítulo 2, mas ele tem um erro de sintaxe, uma importação ausente e diversos locais em que a formatação não está em conformidade com o PEP8. O Exemplo 6.1 mostra o código antes de executar o Black.

Exemplo 6.1: plot_big_o.py

```
import matplotlib.pyplot as plt


def plot_big_o(save_path)


    n = np.linspace(1, 10, 1000)
    line_names = ['Constant','Linear','Quadratic','Exponential','Logarithmic']
```

```
    big_o = [np.ones(n.shape), n, n**2, 2**n, np.log(n)]

    fig, ax = plt.subplots ()
    fig.set_facecolor("white")

    ax.set_ylim(0,50)
    for i in range(len(big_o)):
        ax.plot(n, big_o[ i], label= line_names[i ])
    ax.set_ylabel('Relative Runtime')
    ax.set_xlabel('Input Size')
    ax.legend()

    fig.savefig(save_path, bbox_inches='tight')
```

Para formatar o script, execute o Black com o seguinte comando:

```
$ black plot_big_o.py
```

Black lança um erro, pois não consegue reformatar arquivos com erros de sintaxe. Ele fornece a seguinte saída:

```
error: cannot format plot_big_o.py: Cannot parse: 3:25: def plot_big_o(save_path)

Oh no! File failed to reformat.
```

Nesse caso, estão faltando os dois-pontos finais na definição da função. Deveria ser: `def plot_big_o(save_path):`.

Depois de corrigir o erro e executar novamente o Black, o script ficou assim:

```
import matplotlib.pyplot as plt

def plot_big_o(save_path): ❶
    n = np.linspace(1, 10, 1000)
    line_names = ["Constant", "Linear", "Quadratic", "Exponential", "Logarithmic"] ❷
    big_o = [np.ones(n.shape), n, n**2, 2**n, np.log(n)]

    fig, ax = plt.subplots() ❸
    fig.set_facecolor("white")

    ax.set_ylim(0, 50)
```

```
for i in range(len(big_o)):
    ax.plot(n, big_o[i], label=line_names[i])
ax.set_ylabel("Relative Runtime")
ax.set_xlabel("Input Size")
ax.legend()

fig.savefig(save_path, bbox_inches="tight")
```

❶ A linha em branco após a definição da função foi removida.

❷ Strings foram inseridas entre aspas duplas em vez de entre aspas simples.

❸ Espaços extras foram removidos.

O Black corrigiu a formatação em todo o script, conforme detalhado nas anotações. Por exemplo, a linha `ax.plot(n, big_o[ i], label= line_names[i ])` foi alterada para `ax.plot(n, big_o[i], label=line_names[i])`, em conformidade com o PEP8.

É possível também visualizar as mudanças que o Black fará com o seguinte comando:

```
$ black plot_big_o.py --diff
```

Se quiser que o Black ignore alguma linha de código, adicione o comentário `# fmt: skip` ao final da linha. Para ignorar um bloco de código, use o comentário `# fmt: off` no início do bloco e `# fmt: on` no final.

Embora tenha limpado a formatação no Exemplo 6.1, o Black não resolveu a importação ausente. Para isso, será necessário usar um linter, que abordarei na próxima seção. Mais adiante no livro, retomarei o Black, quando eu abordar como executá-lo automaticamente na seção “Hooks de pré-commit”, na página 220.

Linting

Linting significa verificar se o código tem erros antes de executá-lo. O nome peculiar se origina do filtro de fiapos de uma secadora de roupas, e a primeira ferramenta a realizar essa função foi chamada de lint porque removia “fiapos” do código. A ferramenta lint original foi desenvolvida para a linguagem C em 1978, mas agora os linters são comuns em todas as linguagens de programação.

Os linters do Python analisam seu código e alertam sobre problemas que

poderiam causar falhas na execução, como erros de digitação em nomes de variáveis ou se esquecer de importar um módulo, o que faria com que seu código gerasse um erro ao executá-lo.

Ferramentas populares de linting para Python incluem o Flake8 (*https://oreil.ly/JSJTW*), o Pylint (*https://oreil.ly/zVVN2*) e o Ruff (*https://oreil.ly/5v5fu*). Todas elas verificam a formatação de código em relação a um guia de estilo, geralmente o PEP8 ou um subconjunto dele. Caso detectem problemas de formatação, o Pylint e o Flake8 não alteram seu código; ao contrário, eles o destacam. Ou seja, é possível revisar manualmente as sugestões de formatação antes de fazer as mudanças. Além disso, podemos combiná-los com a execução do Black para fazer alterações de formatação primeiro, deixando o linter focado em encontrar possíveis erros. O Ruff, por sua vez, verifica o código e também atualiza a formatação, similar ao Black.

Ferramentas de linting

Vejamos como usar ferramentas de linting no script do Exemplo 6.1.

É possível instalar o Pylint com o seguinte comando:

```
$ pip install pylint
```

Ao executar o Pylint no Exemplo 6.1 pela linha de comando, obtemos o seguinte resultado:

```
$ pylint plot_big_o.py


************* Module plot_big_o
plot_big_o.py:3:26: E0001: Parsing failed: 'invalid syntax (<unknown>, line 3)'
(syntax-error)
```

Devido a um erro de sintaxe, o Pylint não consegue analisar o script completo. Ele não especifica o erro, mas `3:26` indica que o erro está na linha 3, coluna 26. Nesse caso, faltam dois-pontos `:` no final da definição da função:

Corrigindo o erro e executando o Pylint novamente, temos:

```
$ pylint plot_big_o.py


************* Module plot_big_o
```

```
plot_big_o.py:19:0: C0304: Final newline missing (missing-final-newline)
plot_big_o.py:1:0: C0114: Missing module docstring (missing-module-docstring)
plot_big_o.py:3:0: C0116: Missing function or method docstring
(missing-function-docstring)
plot_big_o.py:5:4: C0103: Variable name "n" doesn't conform to
snake_case naming style (invalid-name)
plot_big_o.py:5:8: E0602: Undefined variable 'np' (undefined-variable)
plot_big_o.py:7:13: E0602: Undefined variable 'np' (undefined-variable)
plot_big_o.py:7:46: E0602: Undefined variable 'np' (undefined-variable)
plot_big_o.py:9:9: C0103: Variable name "ax" doesn't conform to
snake_case naming style (invalid-name)
plot_big_o.py:13:4: C0200: Consider using enumerate instead of iterating
with range and len (consider-using-enumerate)
```

Dessa vez, o Pylint é capaz de analisar o restante do script, revelando uma série de erros (mensagens com E, como E0602) e trechos que não seguem as convenções (mensagens com C, como C0304). Com essas mensagens, podemos atualizar o código e corrigir os erros. Se não estiver claro qual deve ser a versão correta do código, a documentação do Pylint (*https://oreil.ly/MurWR*) fornece ajuda.

O Flake8 funciona de forma muito similar ao Pylint. É possível instalá-lo usando o seguinte comando:

```
$ pip install flake8
```

Executar o Flake8 no Exemplo 6.1 fornece a seguinte saída:

```
$ flake8 plot_big_o.py

plot_big_o.py:3:25: E999 SyntaxError: invalid syntax
```

É o mesmo comportamento do Pylint: ele não conclui a análise do script, mas interrompe e sinaliza o erro de sintaxe. No entanto, corrigir o erro de sintaxe e executar novamente o Flake8 fornece uma saída diferente:

```
$ flake8 plot_big_o.py

plot_big_o.py:3:1: E302 expected 2 blank lines, found 1
plot_big_o.py:5:9: F821 undefined name 'np'
plot_big_o.py:6:29: E231 missing whitespace after ','
plot_big_o.py:6:38: E231 missing whitespace after ','
```

```
plot_big_o.py:6:50: E231 missing whitespace after ','
plot_big_o.py:6:64: E231 missing whitespace after ','
plot_big_o.py:7:14: F821 undefined name 'np'
plot_big_o.py:7:47: F821 undefined name 'np'
plot_big_o.py:9:27: E211 whitespace before '('
plot_big_o.py:12:18: E231 missing whitespace after ','
plot_big_o.py:14:26: E201 whitespace after '['
plot_big_o.py:14:37: E251 unexpected spaces around keyword / parameter equals
plot_big_o.py:14:50: E202 whitespace before ']'
plot_big_o.py:19:48: W292 no newline at end of file
```

O Flake8 identifica diferentes problemas de formatação em comparação ao Pylint, pois se baseia em um guia de estilo próprio.

O Ruff é um linter mais novo, desenvolvido para ser extremamente rápido. É possível instalar o Ruff com o seguinte comando:

```
$ pip install ruff
```

O Ruff separa as verificações de linting e de formatação. Para aplicar o Ruff na verificação de linting de seu código, utilize o seguinte comando:

```
$ ruff check plot_big_o.py
```

Da mesma forma que o Flake8, o Ruff para quando identifica um erro de sintaxe. Corrigir isso e executar o código novamente fornece a seguinte saída:

```
plot_big_o.py:5:9: F821 Undefined name `np`
plot_big_o.py:7:14: F821 Undefined name `np`
plot_big_o.py:7:47: F821 Undefined name `np`
```

Com o seguinte comando, é possível executar o Ruff mais uma vez a fim de atualizar a formatação do código:

```
$ ruff format plot_big_o.py
```

Esse comando atualiza a formatação do código, mas não informa o que foi alterado.

Como cada linter utiliza um guia de estilo ligeiramente diferente, é importante escolher um e adotá-lo para garantir a consistência do seu código. A decisão de qual linter usar deve ser feita em conjunto com sua equipe.

Linting em seu IDE

Em vez de executar uma ferramenta separada para fazer o linting e a formatação do código, alguns IDEs verificam seu código enquanto você o escreve. A Figura 6.1 mostra um exemplo do VS Code usando a extensão pylance.

```
plot_big_o.py > ...
1   import matplotlib.pyplot as plt
2
3   def plot_big_o(save_path)
4
5       n = np.linspace(1, 10, 1000)
6       line_names = ['Constant','Linear','Quadratic','Exponential','Logarithmic']
7       big_o = [np.ones(n.shape), n, n**2, 2**n, np.log(n)]
8
9       fig, ax = plt.subplots ()
10      fig.set_facecolor("white")
11
12      ax.set_ylim(0,50)
13      for i in range(len(big_o)):
14          ax.plot(n, big_o[ i], label= line_names[i ])
15      ax.set_ylabel('Relative Runtime')
16      ax.set_xlabel('Input Size')
17      ax.legend()
18
19      fig.savefig(save_path, bbox_inches='tight')

PROBLEMS 4   OUTPUT   TERMINAL   DEBUG CONSOLE
plot_big_o.py 4
  Expected ":" Pylance [Ln 3, Col 26]
  "np" is not defined Pylance(reportUndefinedVariable) [Ln 5, Col 9]
  "np" is not defined Pylance(reportUndefinedVariable) [Ln 7, Col 14]
  "np" is not defined Pylance(reportUndefinedVariable) [Ln 7, Col 47]
```

Figura 6.1: Linting durante programação no VS Code.

Essa ferramenta destaca os erros que ocorreriam no tempo de execução e fornece uma lista de mais detalhes. Ela também pode ser configurada para usar o Pylint, o Flake8 ou muitos outros linters. Além disso, é possível configurá-la para ignorar avisos que não são relevantes para o seu caso de uso. A extensão permite fazer o linting de notebooks do Jupyter no VS Code.

Independentemente da ferramenta escolhida, o linting economiza tempo ao identificar erros antes que aconteçam e mantêm seu código consistente.

Verificação de tipo

A verificação de tipo é outra forma de detectar bugs antes que causem erros em seu código. O termo “tipo” se refere a categorias de objetos em Python, como inteiros (`int`), strings, floats etc. Uma incompatibilidade entre o tipo de entrada esperado por uma função e o tipo recebido por ela causará um erro.

Por exemplo, o código a seguir envia uma string para a função `math.sqrt()`, que espera um tipo numérico (como um inteiro ou float):

```
import math


my_int = "100"
print(math.sqrt(my_int))
```

Isso resulta no erro: `TypeError: must be real number, not str`.

Além do mais, o Python é uma linguagem de tipagem dinâmica, ou seja, o tipo de uma variável pode mudar, ao contrário de linguagens como Java, em que, uma vez definido o tipo de uma variável, ela é fixa e não pode ser alterada.

Por exemplo, o seguinte código é executado sem erros e altera o tipo da variável:

```
>>> my_variable = 10
>>> my_variable = "hello"
>>> type(my_variable)
... str
```

`my_variable` começa como um inteiro, mas depois se torna uma string.

Os tipos são uma fonte extremamente comum de bugs. Uma função pode receber um tipo diferente do esperado ou gerar um resultado de um tipo incorreto. Essa frequência levou ao desenvolvimento de ferramentas que detectam esses erros para você, evitando a necessidade de escrever uma série de testes (que abordarei no próximo capítulo) para verificá-los.

Anotações de tipo

As anotações de tipo, também chamadas de type hints [dicas de tipo, em tradução livre], foram introduzidas no Python 3.5 para reduzir os bugs causados por tipos incorretos. Elas informam o tipo de entrada que uma função espera ou retorna para qualquer pessoa que esteja lendo o código, tornando-o mais legível, pois comunicam o comportamento esperado da função. Além disso, as anotações de tipo auxiliam na consistência e padronização de grandes bases de código.

As anotações de tipo são relativamente novas em Python e ainda geram um pouco de controvérsias. Algumas pessoas as consideram úteis para a legibilidade, enquanto outras acreditam que dificultam a leitura do código. Há também o trabalho extra de adicioná-las e verificá-las. Os desenvolvedores do Python declararam (no PEP 484) que as anotações de tipo continuarão sendo opcionais. Caso sua equipe ou empresa tenha uma recomendação de uso ou não, você deve adotá-la.

As anotações de tipo seguem o formato `my_variable: type`. Por exemplo, para definir o tipo de retorno de uma função, use o seguinte formato:

```
—---
def my_function(some_argument: type) -> return_type:
—---
```

No Exemplo 6.2, adicionei anotações de tipo a uma das funções do Capítulo 2. Essa função espera uma lista como entrada e retorna um float, como podemos ver na definição da função:

Exemplo 6.2: mode_using_counter.py

```
from collections import Counter


def mode_using_counter(list_of_numbers: list) -> float:
    c = Counter(list_of_numbers)
    return c.most_common(1)[0][0]
```

É possível também especificar que a lista de entrada precisa conter somente os floats usando a seguinte sintaxe:

```
from collections import Counter
from typing import List


def mode_using_counter(list_of_numbers: List[float]) -> float:
```

```
    c = Counter(list_of_numbers)
    return c.most_common(1)[0][0]
```

É possível também criar anotações de tipo usando tipos que não fazem parte da biblioteca padrão do Python. Vejamos um exemplo de como escrever uma anotação de tipo usando um array do NumPy:

```
import numpy as np


def array_operation(input_array: np.ndarray) -> np.ndarray:
    ...
```

Se estiver verificando o tipo de uma estrutura de dados de uma biblioteca que não faz parte da biblioteca padrão do Python, a biblioteca deve fornecer o tipo. Trata-se de um recurso opcional e, se a biblioteca não o fornecer, o verificador de tipo detectará o tipo Any.

Na verdade, as anotações de tipo não fazem nenhuma diferença no funcionamento do seu código. Por exemplo, no Exemplo 6.3 , uma anotação de tipo está incorreta: a entrada é anotada como um float quando deveria ser uma lista. O código ainda é executado corretamente:

Exemplo 6.3: mode_using_counter_incorrect.py

```
from collections import Counter


def mode_using_counter(list_of_numbers: float) -> float:
    c = Counter(list_of_numbers)
    return c.most_common(1)[0][0]
```

As anotações de tipo ajudam apenas se forem usadas com ferramentas de verificação de tipo, e a ideia é executar o verificador de tipo [type checker] antes de executar ou implantar o código. O verificador de tipo analisa seu código e verifica se há tipos incompatíveis. É possível também escrever testes para isso, porém é mais fácil e rápido recorrer a um verificador de tipo.

A ferramenta de verificação de tipo mais popular no momento em que eu escrevia este livro era o mypy (*https://mypy-lang.org*), e explicarei como usá-la na próxima seção. Outras ferramentas para verificação de tipo incluem o Pyright (*https://oreil.ly/2BIR2*), que está crescendo em popularidade, e também o Pyre (*https://pyre-check.org*). Seu IDE também

pode suportar a verificação de tipo, seja por meio de funcionalidades built-in ou por meio da instalação de uma extensão como o Pyright. Depois de começar a usar as anotações de tipo, seu IDE pode preenchê-las automaticamente [autocomplete] e informar quando os tipos são incompatíveis.

Verificação de tipo com o mypy

É possível instalar o mypy com o seguinte comando:

```
$ pip install mypy
```

É possível executá-lo em qualquer script com o seguinte comando:

```
$ mypy my_script.py
```

Executar o mypy no Exemplo 6.2, em que as anotações de tipo estão corretas, fornece a seguinte saída:

```
Success: no issues found in 1 source file
```

Mas executar o mypy no Exemplo 6.3 com anotações de tipo incorretas fornece esta saída:

```
mode_using_counter_incorrect.py:4: error: No overload variant of "Counter"
matches argument type "float"  [call-overload]
mode_using_counter_incorrect.py:4: note: Possible overload variants:
mode_using_counter_incorrect.py:4: note:   def [_T] Counter(self, None = ..., /)
-> Counter[_T]
mode_using_counter_incorrect.py:4: note:   def [_T] Counter(self, None = ..., /,
**kwargs: int) -> Counter[str]
mode_using_counter_incorrect.py:4: note:   def [_T] Counter(self,
SupportsKeysAndGetItem[_T, int], /) -> Counter[_T]
mode_using_counter_incorrect.py:4: note:   def [_T] Counter(self, Iterable[_T],
/) -> Counter[_T]
Found 1 error in 1 file (checked 1 source file)
```

O Mypy encontrou o erro na anotação de tipo, a fim de que possa ser corrigido. Em seguida, qualquer pessoa que use essa função saberá que tipo ela deve aceitar e retornar.

Aspectos fundamentais

Neste capítulo, abordei como a formatação de código, o linting e a verificação de tipo podem melhorar a qualidade do código e ajudar a aumentar sua produtividade ao escrever código. Formatar seguindo um guia de estilo torna seu código mais legível. O linting e a verificação de tipo identificam possíveis erros antes de o código ser implantado em produção, garantindo sua robustez.

Vale ressaltar que você deve respeitar os padrões da sua equipe ou de sua empresa ou definir padrões, se ainda não existirem. A formatação padronizada ajuda a evitar bugs, pois fica mais fácil entender o que o código está fazendo. Significa também que você pode dedicar tempo trabalhando na funcionalidade do código, em vez de nos detalhes de seu estilo.

O importante a se lembrar sobre a formatação de código, linting e verificação de tipo é: use ferramentas que se encarreguem dessas tarefas. Dedicar seu tempo para executá-las manualmente não é produtivo. A princípio, leva-se um tempo para configurar essas ferramentas, mas investir nisso definitivamente valerá a pena a longo prazo. No Capítulo 12, mostrarei como automatizar essas ferramentas, de modo que você dedique ainda menos tempo.

No próximo capítulo, explicarei outro aspecto importante para garantir a robustez do seu código: os testes.

CAPÍTULO 7

Testando seu código

A escrita de testes é uma habilidade fundamental para quem busca aprender a escrever código de qualidade. Um teste é um código que chama uma função e verifica se ela faz o que deveria fazer. Ele lhe fornece evidências e confiança de que seu código está funcionando corretamente. Semelhante ao registro em logs do capítulo anterior, de início, os testes exigem um pouco de empenho para serem configurados, mas todo o trabalho vale a pena à medida que seu projeto cresce.

Caso seu código seja um experimento verdadeiramente pontual, os testes são menos relevantes. Mas e se ele fizer parte de um sistema maior? E se outras pessoas estiverem mudando seu código? E se outros códigos dependerem do seu para retornar resultados específicos? Os testes oferecem algumas garantias de que seu código está funcionando e alertam se alguma mudança, feita por você ou por outros, pode quebrá-lo.

Mesmo quando você é o único trabalhando em seu código, os testes são importantes. Eles atuam como uma rede de segurança, garantindo que seu código continue funcionando harmoniosamente após cada mudança, sem que você precise se lembrar de cada coisa que mudou. Além disso, testes são exemplos de como o código deve funcionar, facilitando a legibilidade e a manutenibilidade do seu código.

À primeira vista, é desafiador escrever testes, pois um pouco de experiência em antecipar problemas é uma vantagem considerável. Mas é possível começar, testando se o seu código faz o que você acredita que ele está fazendo, como abordarei na seção “Um teste básico”, na página 128. A partir daí, adicione mais testes à medida que se depare com ocasiões em que o código não funciona.

No desenvolvimento de software, existem dois tipos principais de testes: testes automatizados de código e testes de usuário, quando se testa o produto completo de software. Em geral, os testes de usuário são realizados

por uma equipe separada, mas, se você estiver escrevendo código de produção, é fundamental escrever testes automatizados. Neste livro, abordarei apenas os testes automatizados.

Neste capítulo, exploraremos em detalhes as motivações para escrever testes e, em seguida, explicarei como escrever um teste muito simples e um mais complexo. Mostrarei como automatizar os testes, de modo que você não precise executá-los manualmente. Além disso, analisarei detalhadamente a validação de dados e os testes de machine learning.

Para evitar qualquer mal-entendido, quando me refiro a testes neste livro, não estou me referindo a testes A/B. Em um teste A/B, realizamos experimentos para descobrir qual variação de algo é preferível, como duas versões diferentes de uma funcionalidade de software. Os testes abordados aqui têm o objetivo de verificar se seu código funciona como o esperado.

Por que devemos escrever testes

É comum trabalharmos em nosso código até que ele gere os resultados esperados. E, então, nosso trabalho está concluído, certo? Mas e se, posteriormente, os dados de entrada mudarem ou e se não tivermos testado todas as entradas possíveis para o código? Em minha experiência, meu código muitas vezes funcionava bem até certo ponto e, depois, algo inesperado acontecia e ele quebrava. Ou seja, meu código não era robusto. E se meu código faz parte de um sistema maior, isso é um problema.

Muitos fatores podem mudar e afetar o código, fazendo com que seja comprometido, como novas versões de bibliotecas ou de linguagens, atualizações do sistema operacional do computador ou, mais comumente, alterações em códigos dos quais seu código depende. Para uma lista completa de razões pelas quais seu código pode falhar no futuro, confira este artigo (*https://oreil.ly/oG4oq*). Testes ajudam a proteger seu código contra problemas futuros.

Além do mais, seu código pode não funcionar corretamente em um ambiente diferente daquele em que foi desenvolvido, como em um dashboard online, na máquina de outra pessoa, em um contêiner do Docker ou em um ambiente de produção. Se tiver testes para seu código, poderá movê-lo para outro ambiente, executar os testes e validar se o código

funciona como esperado.

Os testes oferecem a garantia de que seu código faz o que você afirma que ele deve fazer. Isso ajuda outras pessoas a confiar em seu código, e sinaliza que ele é robusto e de boa qualidade. Você também pode precisar garantir o funcionamento do seu código em situações críticas, como na área médica, ou quando um grande número de usuários depende dele.

Os testes também lhe fornecem a confiança necessária para efetuar mudanças sem medo de quebrar algo de forma imperceptível. Eles indicam se seu código continua devidamente funcionando após cada mudança, o que é extremamente útil para depuração (como vimos no Capítulo 5) e para refatoração (que abordarei no Capítulo 8).

Desenvolvimento orientado a testes (TDD)

O desenvolvimento orientado a testes é uma metodologia de desenvolvimento de software em que o desenvolvedor escreve os testes antes das funções que efetivamente implementam os requisitos do projeto. O propósito é usar os testes para delinear os requisitos do código, pois eles especificam o comportamento esperado, as entradas e as saídas da função. Em seguida, o desenvolvedor escreve o código para que o teste seja aprovado ou passe. Essa abordagem incentiva os desenvolvedores a escrever código modular e dividir problemas complexos em partes facilmente testáveis.

No entanto, essa abordagem nem sempre é adequada para muitos projetos de ciência de dados, pois, no início de um típico projeto de ciência de dados, não se sabe exatamente quais funções serão necessárias. A fase exploratória geralmente precede a estruturação do código, que só se define no final do projeto. Para saber mais sobre TDD, recomendo o livro *Expert Python Programming* [Programação Python Especializada, em tradução livre], de Michal Jaworski e Tarek Ziadé (Packt, 2021), que aborda o desenvolvimento em Python a partir da perspectiva do TDD.

Quando testar

Em projetos de ciência de dados, saber o momento exato para iniciar a escrita de testes pode ser um desafio. Durante a fase exploratória, os testes geralmente não compensam o tempo investido, pois ainda não se sabe qual código será útil no futuro. Mas, quando o código precisa ser reutilizado e modificado, esse pode ser um bom momento para adicionar testes. Além disso, em análises pontuais, a importância dos testes diminui. Temos de analisar os prós e os contras entre obter resultados rapidamente e ter tempo para garantir que os resultados sejam corretos.

Escrever testes no início do projeto pode ajudá-lo a detectar erros simples, como nomes inconsistentes, importações ausentes, erros de ortografia ou de sintaxe. Às vezes, esses erros passam despercebidos pelo seu editor de código.

É fundamental escrever testes antes de implantar o código em um ambiente de produção ou antes que outras pessoas o utilizem, a fim de garantir sua robustez e reprodutibilidade. Outro momento oportuno para adicionar testes é quando algo sai errado em seu código e um erro inesperado surge. Adicionar um teste pode ajudá-lo a identificar a origem do erro. Os testes também são extremamente úteis, caso seja necessário refatorar seu código.

Lembre-se de que você só precisa testar seu código. Não é necessário testar bibliotecas ou dependências importadas. Testá-las não é sua responsabilidade, e ambas já devem ter sido testadas.

Como escrever e executar testes

Exploraremos os elementos de um teste simples: como estruturá-lo e como escrevê-lo. Explicarei como testar entradas inesperadas, além de o que seu código deve tratar, e abordarei como executar testes automaticamente. Apresentarei também exemplos de dois tipos comuns de testes: testes unitários e testes de integração.

Um teste básico

O teste mais simples verifica se uma função é executada corretamente com o tipo de entradas esperadas, se tudo estiver funcionando de modo devido. Se está com dificuldades para definir o que incluir em um teste, um bom ponto de partida são os elementos que você exibe repetidamente em um notebook do Jupyter enquanto desenvolve sua função.

Os testes podem ser estruturados em quatro etapas, conforme descrito na documentação do Pytest (*https://oreil.ly/slpmH*):

1. Preparação: configure tudo o que for necessário para executar a função, como carregar os dados.
2. Ação: execute a função que está testando.
3. Asserção: verifique se o resultado da execução da função é o esperado.

4. Limpeza: garanta que o teste não deixe rastros. Por exemplo, caso tenha aberto um arquivo, não se esqueça de fechá-lo.

Usarei a função weighted_mean do Capítulo 2 e escreverei um teste para verificar se ela está funcionando conforme o esperado. Vejamos a função novamente:

```
def weighted_mean(num_list, weights):
    running_total = 0
    for i in range(len(num_list)):
        running_total += (num_list[i] * weights[i])
    return (running_total/sum(weights))
```

Vejamos um teste que acompanha essa função para validar algumas entradas simples:

```
from ch07_functions import weighted_mean ❶

def test_weighted_mean(): ❷

    list_a = [1, 2, 4]
    list_b = [1, 2, 4] ❸

    result = weighted_mean(list_a, list_b) ❹

    assert result == 3 ❺
```

❶ A função de teste está em um arquivo Python separado da função que está sendo testada; essa última é importada a partir do arquivo *ch07_functions.py*. O arquivo que contém o teste deve seguir o formato *test_*.py* ou **test.py*.

❷ Por convenção, o nome da função de teste começa com test_.

❸ A etapa de preparação, em que geramos os dados de entrada para a função que está sendo testada.

❹ A etapa de ação, em que chamamos a função com os dados escolhidos de entrada.

❺ A etapa de asserção, em que verificamos se o resultado que a função gerou é o esperado.

Nesse teste, comecei com um típico conjunto de entradas para a função. Em

seguida, executei a função e verifiquei se o resultado está correto com instrução assert.

A próxima etapa é executar o teste e verificar se ele passa; você pode fazer isso com um framework de teste, como o Pytest, que detalharei na seção "Executando testes automatizados com o Pytest", na página 132. Se o teste passar ou for aprovado, você saberá que o código está funcionando corretamente, pelo menos, para os valores dos dados de entrada em questão. Se o teste falhar ou for reprovado, duas verificações são necessárias. Primeiro, valide a acurácia do teste e a conformidade do valor verificado na etapa de asserção, se é o valor esperado para o conjunto de entradas. Segundo, identifique e corrija o erro que está causando a falha do teste em seu código, para que o teste passe.

Vejamos outro exemplo, desta vez com a função fit_trendline, que usamos em outros capítulos:

```
from scipy.stats import linregress


def fit_trendline(year_timestamps, data):
    result = linregress(year_timestamps, data)
    slope = round(result.slope, 3)
    r_squared = round(result.rvalue**2, 3)
    return slope, r_squared
```

Vejamos o teste correspondente:

```
def test_fit_trendline():


    data = [1, 2, 3]
    timestamps = [2020, 2021, 2022]


    slope, r_squared = fit_trendline(timestamps, data)


    assert slope == 1
    assert r_squared == 1
```

Nesse caso, optei por usar dados falsos. Sei que eles deveriam gerar uma linha de tendência com inclinação de 1 e um valor de R^2 também igual a 1. Esse procedimento serve como uma verificação de sanidade [sanity check]

de que a função está fazendo exatamente o esperado. Na próxima seção, abordarei outras entradas que você deve considerar testar.

É prática comum armazenar os testes em uma pasta separada e iniciar os nomes dos arquivos Python que os contêm com test_. Essa organização mantém os testes separados do restante do código e possibilita que os executores de testes [test runners] automatizados os localizem, conforme explicarei na seção “Executando testes automatizados com o Pytest”, na página 132.

Testando entradas inesperadas

Além de testar se seu código funciona com as entradas esperadas, é útil torná-lo robusto, testando se ele pode tratar entradas inesperadas. Chamamos isso de casos edge (ou casos extremos), que são exemplos extremos ou incomuns de uma entrada que exige tratamento especial, como strings vazias ou dados de tipo incorreto.

Vejamos um exemplo de como podemos atualizar o teste da função weighted_mean para verificar se o código é robusto a uma entrada inesperada:

```
def test_weighted_mean():

    result = weighted_mean([1, 2, 4], [1, 2, 4])

    assert result == 3

    empty_list_result = weighted_mean([], [])

    assert not empty_list_result
```

Nesse teste, estabeleço a expectativa de que a função weighted_mean retorne None se receber listas vazias como entrada. A fim de que o teste passe, preciso atualizar a função weighted_mean para tratar esta entrada:

```
def weighted_mean(num_list, weights):
    if not (num_list or weights):
        return None
    running_total = 0
    for i in range(len(num_list)):
        running_total += (num_list[i] * weights[i])
```

```
    return (running_total/sum(weights))
```

Em seguida, quero adicionar um teste para verificar se minha função se comporta corretamente, caso receba dados de tipo incorreto como entrada. Vejamos o teste atualizado, que verifica se a função weighted_mean retorna None se receber uma string em uma das listas de entrada:

```
def test_weighted_mean():

    result = weighted_mean([1, 2, 4], [1, 2, 4])
    assert result == 3

    empty_list_result = weighted_mean([], [])
    assert not empty_list_result

    wrong_types_result = weighted_mean(['one', 2, 4], [1, 2, 4])
    assert not wrong_types_result
```

Em seguida, eu atualizaria o código para obter o comportamento esperado. Observe como adicionar um teste e atualizar a função caminham lado a lado, não raro, em um ciclo iterativo em torno do loop para atualizar o teste e, depois, o código.

Testar entradas inesperadas é um desafio quando seu código está funcionando bem, pois é difícil prever como ele pode quebrar. No entanto, quando o código quebra e você o atualiza para tratar um erro, adicione um teste a fim de garantir que ele não quebre da próxima vez que a mesma coisa acontecer. Considere também testar entradas que poderiam ter consequências significativas, mas não se torture tentando prever todas as possibilidades de entrada.

Quero destacar algumas bibliotecas que podem ajudá-lo. A biblioteca Faker (*https://oreil.ly/A7Wb-*) pode gerar dados falsos em diferentes formatos, como timestamps, endereços de e-mail e assim por diante, evitando a necessidade de dados reais. Isso é crucial ao lidar com informações pessoais ou confidenciais que não devem ficar visíveis em códigos de teste. A outra é a Hypothesis (*https://oreil.ly/uTNW0*), que pode ajudá-lo a identificar casos edge em que o código não funciona. A Hypothesis nos possibilita criar nossos testes, nos quais podemos descrever uma gama de cenários que serão testados por ela, graças à sua capacidade de realizar testes baseados

em propriedades.

Executando testes automatizados com o Pytest

O Pytest (*https://oreil.ly/z1MRC*) é um framework que encontra, coleta e executa seus testes. Caso use o Pytest ou outro framework similar, não precisará executar cada função de teste se tiver muitas delas nem escrever código boilerplate para executar todos os testes.

O Pytest não faz parte da biblioteca padrão do Python, sendo necessário instalá-lo com o seguinte comando:

```
$ pip install pytest
```

É possível executar um único arquivo de teste com este comando:

```
$ pytest test_weighted_mean.py
```

Caso queira executar todos os testes em uma pasta, basta usar o comando `pytest` sozinho. O Pytest fará uma busca recursiva em todos os arquivos da pasta, em que você executa este comando e, por sua vez, executará testes a partir de arquivos que correspondam ao padrão *test_*.py* ou *_test.py*. Dentro desses arquivos, ele executará todas as funções de teste que começarem com `test_`. Para mais detalhes sobre como o Pytest encontra testes, confira sua documentação (*https://oreil.ly/fGAj0*).

Se todos os seus testes passarem, você verá uma saída parecida com isto:

```
============================== test session starts
   ============================
platform darwin -- Python 3.10.4, pytest-7.4.0, pluggy-1.2.0
rootdir: /Users/.../book_code
collected 1 item

test_weighted_mean.py .                                  [100%]

============================== 1 passed in 0.01s
   ==============================
```

No entanto, se o teste falhar, você receberá uma saída mais ou menos assim:

```
============================== test session starts
   ============================
platform darwin -- Python 3.10.4, pytest-7.4.0, pluggy-1.2.0
rootdir: /Users/.../book_code
```

```
collected 1 item

test_weighted_mean.py F                                  [100%]

==================================== FAILURES
   ================================
_____________________________________ test_weighted_mean _____________________________

   def test_weighted_mean():

      result = weighted_mean([1, 2, 4], [1, 2, 4])

>      assert result == 3
E      assert 2.3333333333333335 == 3

test_weighted_mean.py:7: AssertionError
============================= short test summary info
   ==========================
FAILED test_weighted_mean.py::test_weighted_mean -
assert 2.3333333333333335 == 3
================================ 1 failed in 0.49s
   ============================
```

Caso um teste falhe, o Pytest exibe uma mensagem indicando a falha e destacando a linha exata, além de exibir o erro de asserção e o valor incorreto gerado pelo código.

O Pytest tem muitos outros recursos avançados, como a capacidade de ignorar testes ou lidar com logs. Para mais detalhes, consulte a documentação do Pytest (*https://oreil.ly/eSN4m*).

Outros frameworks de teste

O Pytest é um framework popular de teste por ser repleto de funcionalidades e fácil de usar. No entanto, existem outras opções. O unittest (*https://oreil.ly/uRZyg*), incluído na biblioteca padrão do Python, é útil se você quiser que seu código tenha menos dependências. Mas, sua configuração é um pouco mais complexa do que a do Pytest.

O tox (*https://tox.wiki/en*) é outra ferramenta que vale a pena conhecer. Trata-se de um executor de testes que inclui a configuração de um ambiente virtual, a instalação da sua aplicação e o gerenciamento de dependências e versões do Python. Essa ferramenta é extremamente útil para validar se o seu código é executado corretamente em outros ambientes.

Vale mencionar também o conceito de cobertura de teste, que é a proporção de linhas de código em um arquivo processado quando todos os testes são executados. É possível medir a cobertura de teste usando o plug-in pytest-cov (*https://oreil.ly/BYfJR*) do Pytest. Mesmo que ajude a verificar se todas as suas funções foram testadas, o plug-in apenas verifica quais linhas foram executadas, não quais foram testadas.

É recomendável executar os testes antes de fazer o commit de seu código para o controle de versão (assunto que abordarei no Capítulo 10). No Capítulo 12, detalharei mais o assunto e explicarei como executar seus testes automaticamente.

Tipos de testes

Existe uma ampla gama de termos para descrever diferentes testes. No livro *Robust Python* (O'Reilly, 2021), Patrick Viafore identifica seis tipos de testes comumente usados em engenharia de software:

Testes unitários
: Verificam se unidades de código, como funções ou classes, fazem o que o desenvolvedor espera.

Testes de integração
: Verificam se um sistema maior está adequadamente conectado.

Testes de aceitação
: Verificam se o sistema faz o que o usuário espera.

Testes de carga
: Caso a quantidade de dados ou o número de usuários aumente, verificam se o sistema ainda funciona corretamente.

Testes de segurança
: Verificam se o sistema é resistente a ataques.

Testes de usabilidade
: Verificam se o sistema é intuitivo de usar.

Focarei os testes unitários e os testes de integração, pois são os mais úteis para a escrita de código de ciência de dados.

Testes unitários

Até agora, todos os testes mostrados neste capítulo foram testes unitários. Um teste unitário usa um pequeno componente do seu código e executa um teste nele, por exemplo, testa uma função ou uma classe. Normalmente, você os escreve durante o desenvolvimento do código ou durante a depuração.

Os testes unitários devem ser rápidos de executar, por isso é melhor usar um conjunto pequeno de dados. Evite operações que demoram muito tempo para serem executadas, como se conectar a um banco de dados; é preferível usar dados falsos. Os testes unitários também devem ser determinísticos. Eles devem sempre fornecer o mesmo resultado com as mesmas entradas. Ou seja, não inclua nada que tenha algum componente aleatório.

Testes de integração

Os testes de integração validam se um sistema maior funciona corretamente. Eles testam um script inteiro ou uma parte maior de um projeto, combinando a funcionalidade de múltiplas unidades e executando-as juntas.

Para demonstrar um teste de integração, usarei duas das funções dos capítulos anteriores: `process_sdg_data` e `fit_trendline`. Juntas, as duas funções aceitam um arquivo Excel e geram a inclinação e o R^2 da linha de tendência, com um pouco de código que as vincula:

```
from scipy.stats import linregress
import pandas as pd


def process_sdg_data(excel_file, columns_to_drop):
    df = pd.read_excel(excel_file)
    df = df.drop(columns_to_drop, axis=1)
    df = df.set_index("GeoAreaName").transpose()
    return df


def fit_trendline(year_timestamps, data):
    result = linregress(year_timestamps, data)
    slope = round(result.slope, 3)
```

```
    r_squared = round(result.rvalue**2, 3)
    return slope, r_squared
```

Vejamos um teste de integração que acompanha as duas funções, incluindo o código que as vincula:

```
from ch07_functions import process_sdg_data, fit_trendline


def test_processing_trendline():
    df = process_sdg_data(
        "../data/SG_GEN_PARL.xlsx",
        [
            "Goal",
            "Target",
            "Indicator",
            "SeriesCode",
            "SeriesDescription",
            "GeoAreaCode",
            "Reporting Type",
            "Sex",
            "Units",
        ],
    )
    timestamps = [int(i) for i in df.index.tolist()]
    uk_parl = df["United Kingdom of Great Britain and Northern Ireland"]
            .tolist()

    slope, r_squared = fit_trendline(timestamps, uk_parl)

    assert slope == 0.836
    assert r_squared == 0.868
```

Esse teste verifica se a saída das duas funções juntas corresponde ao esperado. Observe que o código está testando somente um conjunto específico de dados, mas, posteriormente, talvez você queira que as funções funcionem com muitos conjuntos diferentes de dados. Garanta que os conjuntos de dados usados para testes de integração sejam representativos ou forneçam um intervalo de valores, conforme explicado na seção

“Testando entradas inesperadas”, na página 130.

Validação de dados

A validação de dados é um tipo de teste crucial em projetos de ciência de dados. É um processo que valida se os seus dados estão conforme o esperado. É diferente dos outros testes que apresentei neste capítulo, pois, com a validação, testamos os dados com os quais estamos trabalhando, e não o código que estamos escrevendo.

A validação de dados pode ser realizada por uma equipe de engenharia de dados, pode ser uma etapa do seu pipeline de machine learning ou, talvez, você queira incluí-la como parte do seu processo de testes. O livro *Building Machine Learning Pipelines* [Construindo Pipelines de Machine Learning, em tradução livre], de Hannis Hapke e de minha coautoria (O’Reilly Media, 2020), fornece mais detalhes sobre a validação de dados como parte de um pipeline de machine learning. Nesta seção, abordarei algumas das maneiras de validar seus dados e apresentarei algumas ferramentas que podem ser usadas para isso.

Exemplos de validação de dados

A validação de dados é importante porque os dados podem mudar com frequência durante o ciclo de vida de um projeto de ciência de dados. Portanto, é uma boa prática testar as propriedades dos dados em vez de valores exatos. As etapas comuns de validação de dados incluem:

- Testar se todas as colunas necessárias em um conjunto de dados estão presentes.
- Testar se os dados em cada coluna são do tipo correto.
- Testar se todas as colunas não têm valores null ou não têm mais do que determinada proporção de valores null.
- Verificar se as propriedades estatísticas dos dados estão conforme o esperado (por exemplo, a média, o desvio-padrão ou o intervalo dos dados).
- Relacionado à etapa anterior, verificar se a distribuição dos dados é semelhante à versão dos dados do dia, semana ou outro período de

tempo relevante anterior.

- Verificar se há dados anômalos.

Na próxima seção, explicarei como você pode usar o módulo Pandera para validar seus dados. O Great Expectations (*https://oreil.ly/6dp8t*) é outra excelente ferramenta de validação de dados, com funcionalidades mais avançadas, mas sua configuração leva um pouco mais de tempo do que a do Pandera. Na seção a seguir, abordarei o Pydantic, uma opção de validação de dados que usa anotações de tipo.

Usando o Pandera para validação de dados

O módulo Pandera é uma opção ultraleve para validação de dados, que usa um esquema para validar DataFrames do pandas ou estruturas de dados semelhantes, incluindo DataFrames do Dask. Apresentarei uma breve visão geral, porém mais recursos são abordados na documentação do Pandera (*https://oreil.ly/xvOrN*).

É possível instalar o Pandera da seguinte forma:

```
$ pip install pandera
```

A Figura 7.1 mostra parte de um DataFrame a ser validado, usando os mesmos dados do Capítulo 4.

	Year	India	United States of America
0	2000	9.02	13.33
1	2001	9.01	14.02
2	2002	8.84	14.02
3	2003	8.84	14.25
4	2004	8.84	14.25

Figura 7.1: DataFrame a ser validado.

Para validar os dados, é possível definir um esquema que indique quais colunas o DataFrame deve incluir, com os tipos de dados em cada coluna:

```
import pandera as pa

schema = pa.DataFrameSchema({
    "Year": pa.Column(int), ❶
    "India": pa.Column(float), ❷
    "United States of America": pa.Column(float) ❸
```

```
})
```

❶ Deve haver uma coluna chamada “Year [Ano]” que contenha números inteiros.

❷ Deve haver uma coluna chamada “India [Índia]” que contenha floats.

❸ Deve haver uma coluna chamada “United States of America [Estados Unidos da América]” que contenha floats.

Em seguida, valide os dados com o seguinte comando:

```
schema(df)
```

Isso valida os dados de acordo com o esquema, verificando se todas as colunas estão presentes e validando os tipos de dados em cada coluna. O Pandera lançará um erro se alguma coluna especificada no esquema não estiver presente, mas não reclamará se uma coluna extra estiver presente.

Além disso, é possível validar as propriedades dos valores nas colunas. Com o argumento `checks=`, podemos verificar se todos os valores estão dentro do intervalo esperado. Ele aceita uma variedade de objetos `Check` do Pandera:

```
schema = pa.DataFrameSchema({
    "Year": pa.Column(int, checks=pa.Check.in_range(2000, 2023)),
    "India": pa.Column(float, checks=pa.Check.in_range(0, 100)),
    "United States of America": pa.Column(float, checks=pa.Check.in_range(0, 100))
})
```

Se a validação dos dados for bem-sucedida, você terá validado que todas as colunas necessárias estão presentes, que os dados em cada coluna são do tipo correto e que estão dentro do intervalo esperado. Mesmo sendo apenas um exemplo simples, você pode usá-lo como base para validar seu conjunto de dados.

Validação de dados com o Pydantic

É possível validar seus dados no tempo de execução com o Pydantic (*https://oreil.ly/ey_Op*). Trata-se de uma biblioteca extremamente útil, que está se tornando cada vez mais popular na comunidade Python. O Pydantic usa anotações de tipo, que apresentei na seção “Verificação de tipo”, na página 120, para validar dados, mas não é uma ferramenta de análise estática como o mypy. Em vez disso, a validação ocorre quando executamos

o código; não é necessário executar uma ferramenta separada. Isso ajuda bastante, caso tenha de implantar seu código em uma API, como explicarei no Capítulo 11.

É possível instalar o Pydantic com o seguinte comando:

```
$ pip install pydantic
```

O Pydantic usa o conceito de esquemas de dados para validar dados. Primeiro, definimos um esquema que descreve o formato dos dados; depois, podemos usá-lo para verificar se os dados novos estão no formato correto. Vejamos o exemplo de definição de um esquema para os dados dos Objetivos de Desenvolvimento Sustentável da ONU do Capítulo 1. Você verá que pode definir o tipo de seus dados e se são obrigatórios (um erro é gerado se não estiverem presentes) ou opcionais:

```
from pydantic import BaseModel
from typing import Optional
from datetime import datetime

class CountryData(BaseModel):
    country_name: str ❶
    population: int ❷
    literacy_rate_2020: Optional[float] ❸
    timestamp: Optional[datetime] = None ❹
```

❶ O nome do país (country_name) é obrigatório e deve ser uma string ou algo que possa ser convertido em uma string, sem gerar um erro.

❷ A população (population) é obrigatória e deve ser um número inteiro.

❸ A taxa de alfabetização (literacy_rate_2020) é opcional e deve ser um float ou algo que possa ser convertido em um float.

❹ O timestamp (timestamp) é opcional e deve ser um objeto datetime ou algo que possa ser convertido em um objeto datetime, e o valor padrão é None, se nenhum dado for passado.

Vejamos um exemplo de dados que serão validados como corretos:

```
sample_data_correct = {
    'country_name': 'India',
    'population': 1417242151,
    'literacy_rate_2020': 79.43,
```

```
    'timestamp': datetime.now()
}
```

Podemos usar esses dados para criar um novo objeto CountryData. Se os dados estiverem no formato correto, nenhum erro é gerado:

```
india = CountryData(**sample_data_correct)
```

Podemos procurar qualquer um destes dados:

```
>>> india.timestamp
... datetime.datetime(2023, 3, 7, 16, 3, 41, 423508)
```

No entanto, se passarmos dados que não atendem aos requisitos, como estes:

```
sample_data_incorrect = {
    'country_name': 'United States',
    'population': None,
    'literacy_rate_2020': None,
    'timestamp': None
}

united_states = CountryData(**sample_data_incorrect)
```

O campo population fica ausente. E o Pydantic gera o seguinte erro:

```
ValidationError: 1 validation error for CountryData
population
  Input should be a valid integer [type=int_type, input_value=None,
  input_type=NoneType]
```

O Pydantic é extremamente útil para verificar se os dados de entrada em um grande projeto estão conforme esperado. Na seção “Adicionando funcionalidade à sua API”, na página 210, ilustrarei um exemplo de como usar o Pydantic em uma API.

Testes para machine learning

O código de machine learning precisa ser testado de forma um pouco diferente da maioria dos outros códigos. Isso ocorre porque não sabemos exatamente qual modelo obteremos a partir de um determinado conjunto de dados, já que a maioria dos algoritmos de machine learning inclui algum tipo de randomização. Mas isso não significa que você não possa testar

código de machine learning. Como é um tópico complexo, apresentarei uma visão geral nesta seção e explicarei algumas estratégias úteis.

A Figura 7.2 mostra a estrutura de um projeto comum de machine learning. Analisarei as possibilidades de teste para todas as etapas e focarei algumas específicas.

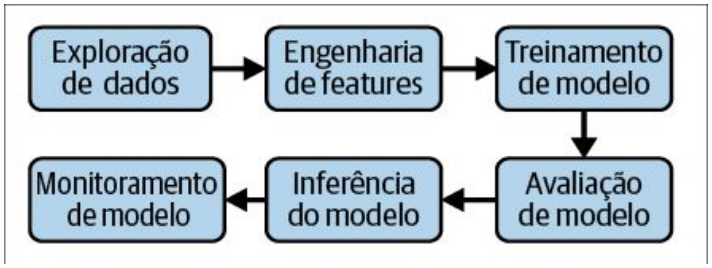


Figura 7.2: Etapas de um projeto de machine learning.

Exploração de dados

Não há necessidade de escrever testes na fase exploratória de um projeto, a menos que você escreva um código que espera reutilizar em outro lugar.

Engenharia de features

Nesta etapa, podemos escrever testes unitários para o código de engenharia de features e incluir a validação de dados.

Treinamento de modelo

Na seção a seguir, abordarei este tópico em mais detalhes.

Avaliação de modelo

A avaliação de modelo avalia o desempenho do seu modelo treinado em um conjunto de dados. Isso por si só é uma espécie de teste, mas os resultados serão um pouco diferentes para cada execução de treinamento de modelo. Por exemplo, você pode formalizar isso em um teste, implementando que a acurácia deve ser sempre maior do que determinada percentagem.

Inferência do modelo

Na seção a seguir, abordarei este tópico em mais detalhes.

Monitoramento de modelo

Em geral, o monitoramento de modelos em produção é tratado por

ferramentas ou bibliotecas externas; no entanto, se escrever o próprio código, poderá adicionar testes unitários.

Testando o treinamento de modelo

Testar o treinamento de modelo é difícil porque, considerando alguns dados de entrada, não sabemos a saída esperada. Não sabemos exatamente quais devem ser os pesos da rede neural e nem conhecemos todos os parâmetros que entram no arquivo do modelo salvo. Se um determinado conjunto de dados fornece uma estrutura específica de modelo, não é possível realizar testes. Mas, definitivamente, podemos testar outras coisas. Sua estratégia dependerá do tamanho do modelo: se estiver usando um modelo grande, que demora mais tempo para ser treinado (mais do que alguns minutos), ou um modelo pequeno.

Caso seu modelo demore muito para ser treinado, é impraticável testar se as saídas dele estão corretas ou a acurácia final do modelo. Ao contrário, você deve verificar se o treinamento está sendo executado conforme o esperado e se o modelo treina sem apresentar falhas. É possível fazer isso escrevendo um teste para o código de treinamento, que compare os pesos do modelo antes e depois de uma etapa de treinamento, a fim de garantir que tenham sido alterados.

Outra opção para redes neurais é verificar se a perda [loss] diminui ao longo das primeiras etapas de treinamento. Além disso, é possível escrever testes a fim de verificar se o modelo aceita tensores de entrada com o formato correto e se gera saídas com o formato correto, pois essa é uma fonte comum de erros.

Para modelos menores, você também pode escrever um teste de integração que verifique se existe um modelo no final do processo de treinamento. É possível usar uma pequena parte de seus dados de treinamento para acelerar o treinamento ou usar dados falsos.

Testando a inferência do modelo

Após treinar seu modelo, é crucial verificar se ele retorna corretamente as predições. Existem duas estratégias principais para testar um modelo em produção. A primeira envolve a escrita de testes unitários e de integração

que verificam se o código de inferência está funcionando adequadamente. Ele retorna uma predição com base em alguns dados falsos de entrada? Lida com entradas vazias de forma adequada? Trata os erros com elegância? Se o seu modelo for muito grande, considere executar alguns testes em um modelo de amostra menor e outros na versão completa do modelo.

A segunda estratégia consiste em enviar exemplos de dados específicos para o seu modelo e validar se as predições correspondem ao esperado. Isso é diferente da avaliação de modelos, pois você usará apenas alguns exemplos de dados, em vez de um conjunto de dados de avaliação completo. Para um modelo de regressão, é possível verificar se a saída do modelo para um determinado exemplo de dados está dentro do intervalo correto. Já para um modelo de classificação, é possível fornecer algumas entradas que sempre devem ser de uma classe específica. O modelo só deve ser implantado em produção se fizer predições corretamente.

Caso queira saber mais sobre testes de AM, confira este artigo de Jeremy Jordan (*https://oreil.ly/FUtlq*).

Aspectos fundamentais

Escrever testes é uma parte importante da escrita de um bom código e, em particular, ajuda com a robustez do código. Você deve escrever testes com o intuito de garantir o comportamento correto, mesmo quando seu código for alterado; desse modo, é possível validar se ele funciona em outros ambientes e, com isso, ganhar confiança de que o código está fazendo o esperado, além de inspirar confiança de outras pessoas e facilitar a depuração e a refatoração.

O teste mais simples possível verifica se a sua função retorna o resultado esperado com algumas entradas simples. Estruture seus testes com etapas de preparação, ação, asserção e limpeza. Em seguida, adicione casos edge. Não execute testes manualmente; use um framework de teste como o Pytest.

É possível validar os dados usando suas propriedades gerais a fim de validar se os dados do projeto correspondem ao esperado. O Pandera é uma opção fácil de usar para validar DataFrames do pandas.

O AM exige uma abordagem diferente, pois não sabemos exatamente qual

modelo será gerado a partir de um determinado conjunto de dados. No entanto, você ainda deve escrever testes, incluindo testes unitários, sempre que possível.

CAPÍTULO 8

Design e refatoração

Neste capítulo, quero me distanciar dos detalhes minuciosos de cada linha de código escrita e ampliar nossos horizontes com um panorama geral: como fazer o design de seus projetos, como organizar seu código e refatorá-lo quando o design mudar. Apresentarei algumas ideias para organizar e padronizar a estrutura de alto nível de seus projetos e sugerirei como dividir seu código em funções modulares e reutilizáveis.

Um bom design, seja a nível de um projeto inteiro ou a nível de funções individuais, tem uma série de vantagens para o seu código. Um design de projeto padronizado reduz o esforço mental de alternar entre projetos, facilitando o trabalho de outras pessoas que já tenham visto algo semelhante. Se o seu código estiver bem projetado, é mais fácil reutilizá-lo em outros projetos e adicionar features novas.

Em minha experiência como cientista de dados, já vi muitos projetos em que todo o código estava em um único notebook gigante do Jupyter – eu mesmo já criei projetos assim. O Jupyter Notebook é uma ferramenta fantástica para iniciar um projeto, esboçar ideias e experimentá-las. No entanto, os notebooks podem ser limitantes quando seu projeto cresce ou se torna mais complexo. Na seção “De notebooks a scripts escaláveis”, na página 156, você encontrará um framework para transformar seus notebooks em scripts Python.

Em ciência de dados, às vezes é difícil saber o momento exato para projetar o design da estrutura de seu projeto. Talvez você não tenha ideia de qual será o resultado do projeto quando o iniciar. Mas, em geral, vale a pena dedicar tempo ao design quando você começa a ter uma noção do rumo que seu projeto está tomando, e isso pode ocorrer no momento em que você migra de notebooks para scripts.

Não espere acertar o design do seu projeto de primeira. É provável que você o refine ao longo do tempo. E é quase certo que os detalhes do seu código

mudarão durante o projeto. A refatoração, o processo de ajustar a estrutura do código mantendo o mesmo comportamento, é parte normal da engenharia de software e, provavelmente, será muito útil para você como cientista de dados. Na seção “Refatoração”, na página 160, explicarei os detalhes de como fazer isso.

Design e estrutura do projeto

Não existem regras inflexíveis sobre o design e a estrutura do projeto. Ambos dependem muito de em que você está trabalhando, do ramo de atuação, do cronograma do projeto e de muitos outros fatores. Um projeto de machine learning provavelmente exigirá uma estrutura completamente diferente daquela de uma análise que respalda alguma decisão.

Dito isso, nesta seção, quero sugerir algumas ideias para um bom design de projetos. Manter seu design relativamente padronizado é muito útil. Isso significa que qualquer pessoa que use seus projetos terá uma ideia do que esperar, reduzindo a carga cognitiva de alternar para um projeto novo. Além disso, fica mais fácil assumir a tarefa que outra pessoa da sua equipe fez se todos adotarem um design padrão para os projetos.

Nesta seção, abordarei como estruturar seus projetos a nível geral, configurando os diferentes arquivos e a estrutura de pastas. Exemplificarei um projeto de machine learning e analisarei algumas ferramentas para criar automaticamente projetos novos com uma determinada estrutura.

Considerações sobre o design do projeto

Como transformar um plano de projeto completo em código bem estruturado? Em alguns casos, talvez seja necessário iniciar o projeto e experimentar algumas ideias preliminares para definir a direção geral do projeto antes de definir a estrutura. Se for o caso, vale a pena dedicar algum tempo ponderando sobre o seu plano de projeto o mais cedo possível, pois isso pode ajudá-lo a transformar um grande volume de código de notebook em um projeto bem estruturado.

As seguintes perguntas o ajudarão a criar o design do seu projeto:

- Quais são os objetivos gerais do seu projeto? Qual é o problema geral

que você precisa resolver? Sempre considere o panorama geral para garantir que todo o código que você escreve o ajude a alcançar esse objetivo, em vez de perder o foco com detalhes menos importantes.

- Quais são as entradas e saídas gerais do seu projeto? Por exemplo, um projeto de machine learning pode receber dados brutos como predições do modelo de entrada e saída.
- Quem são as partes interessadas do projeto? Quanto trabalho você precisa fazer antes de consultá-las? Podem ser gerentes de projeto, outras equipes ou até mesmo o cliente do seu produto.
- Qual é o cronograma do seu projeto? Quais são os marcos [milestones] ou os prazos? Saber disso o ajudará a determinar se esse é um projeto em que você precisa obter um resultado rapidamente e organizar o código depois, ou se pode dedicar um tempo para escrever o código bem desde o início e planejar o design.
- Quais são as ideias que você experimentará? Quanto tempo é possível dedicar à exploração e à experimentação em vez de solucionar o problema que precisa ser resolvido? Se a exploração for intensa, talvez seja útil pensar em como reutilizar o código dessa fase. É possível definir seus experimentos de machine learning de forma padronizada, de modo que você consiga executar facilmente muitos deles?
- Qual é a probabilidade do escopo desse projeto mudar? Ou, em algum momento no futuro, qual trabalho que você faz nesse projeto pode ser reutilizado para um caso diferente de uso? Caso espere reutilizar o código, garanta que ele seja modular e bem documentado, de modo que seja fácil selecionar partes dele e usá-las em outro lugar.

As respostas a essas perguntas o ajudarão a dividir seu projeto em etapas razoáveis, e é uma boa ideia documentar essas respostas em algum lugar. Recomendo também fazer uma retrospectiva de projetos anteriores que você concluiu. Procure por linhas comuns de pensamento e pergunte-se o que correu bem e o que não correu tão bem. O que você poderia fazer melhor em um próximo projeto semelhante? Responder a essas perguntas o ajudará a estruturar projetos futuros.

A estrutura do seu projeto também pode incluir dados. No entanto, se seus dados estiverem armazenados em um banco de dados central (seu banco de

dados ou armazenamento de dados gerenciado pela sua empresa), você não deve replicá-los e salvá-los localmente. Isso pode causar duplicação e, consequentemente, problemas posteriores, em que diferentes versões de dados fornecem resultados distintos. Pode ser também um risco de segurança ou de privacidade. É muito melhor incluir código para extrair e selecionar seus dados como parte do projeto. No mínimo, inclua a documentação que permita replicar o projeto. No Capítulo 9, abordarei como redigir uma boa documentação.

Se tiver projetos semelhantes que surgem com frequência, poderá automatizar a criação da estrutura de arquivos e de pastas usando a ferramenta Cookiecutter (*https://oreil.ly/bX-rV*). Isso pode incluir os arquivos necessários para criar pacotes Python e gerenciar dependências, que detalharei no Capítulo 10.

Caso não queira configurar a estrutura do seu projeto do zero, recomendo usar o Kedro (*https://oreil.ly/fJEqk*), uma biblioteca open-source desenvolvida para padronizar projetos de ciência de dados. Com ela, é possível definir seu projeto como uma coleção de nós conectados e, em seguida, executá-los integralmente [end to end] como um pipeline. Por exemplo, cada nó pode conter um script para carregar ou limpar seus dados.

Um exemplo de projeto de machine learning

Após conhecer as etapas do projeto, você pode começar a definir a estrutura de arquivos. Vejamos como fazer isso com um exemplo de projeto de machine learning. Apesar de existirem muitas opções de como definir a estrutura de arquivos, podemos usar a estratégia de um arquivo por projeto. Em geral, cada uma das etapas termina salvando alguns dados.

As etapas normalmente incluem:

1. Carregar os dados.
2. Limpar e pré-processar os dados e, em seguida, transformá-los em features adequadas para machine learning.
3. Treinar o modelo.
4. Analisar o desempenho do modelo em um conjunto de validação de dados.

Caso saiba que executará alguns experimentos para escolher um modelo inicial e, em seguida, transformará seu código em scripts reproduzíveis, o design seguinte pode ser bom para o seu projeto:

```
├── README.md
├── requirements.txt
│
├── notebooks
│   ├── explore_data.ipynb
│   └── try_regression_model.ipynb
│
├── src
│   ├── __init__.py
│   ├── load_data.py
│   ├── feature_engineering.py
│   ├── model_training.py
│   ├── model_analysis.py
│   └── utils.py
|
├── tests
│   ├── test_load_data.py
│   ├── test_feature_engineering.py
│   ├── test_model_training.py
│   ├── test_model_analysis.py
│   └── test_utils.py
```

Vamos analisar isso. Primeiro, existem alguns arquivos padrão que todo projeto deve ter:

```
├── README.md
├── requirements.txt
```

Todo projeto deve ter documentação (Capítulo 9). Um arquivo *README.md* é um formato padrão. Devemos também incluir todos os arquivos necessários para gerenciar dependências ou empacotamentos (mais detalhes sobre isso no Capítulo 10).

Em seguida, é boa prática manter os notebooks e experimentos, nos quais você está testando diferentes modelos, em sua própria pasta:

```
├── notebooks
```

```
│   ├── explore_data.ipynb
│   └── try_regression_model.ipynb
```

Os scripts também devem ter a própria pasta. No Python, é convenção usual inserir todo o código de um pacote em uma pasta chamada *src* (abreviação de “source”), e muitas pessoas usam a mesma estrutura para os scripts em um projeto. É útil inserir funções “auxiliares [helper]”, reutilizadas em muitos lugares, em um arquivo *utils*:

```
├── src
│   ├── __init__.py
│   ├── load_data.py
│   ├── feature_engineering.py
│   ├── model_training.py
│   ├── model_analysis.py
│   └── utils.py
```

Manter os testes em uma pasta separada ajuda a organizar tudo, além de possibilitar que o framework de testes os identifique com facilidade, como mencionei no Capítulo 7. Na pasta *src*, cada arquivo deve ter os próprios testes:

```
├── tests
│   ├── test_load_data.py
│   ├── test_feature_engineering.py
│   ├── test_model_training.py
│   ├── test_model_analysis.py
│   └── test_utils.py
```

É apenas uma opção para estruturar um projeto. Após definir uma estrutura de projeto que funcione para muitos de seus projetos, você pode transformá-la em um modelo, e replicá-lo usando o Cookiecutter ou usar o Kedro para ajudá-lo a defini-lo. É possível aprender mais sobre a estruturação de um projeto de machine learning no livro *Building Machine Learning Pipelines*, que escrevi com meu colega Hannes Hapke (O’Reilly, 2020).

Design de código

Design é a arte de organizar o código

– SANDI METZ

O design de código, ou a “fatoração” do seu código em partes adequadas, incluindo funções, classes e módulos, não é uma tarefa fácil. Como afirma Sandi Metz, trata-se mais de uma arte do que uma ciência, porque não existe uma resposta “correta”. No entanto, nesta seção, apresentarei algumas diretrizes sobre como criar o design do seu código.

É boa prática dividir seu código em partes distintas que desempenham diferentes propósitos. Cada um desses blocos deve conter uma ideia ou lógica diferentes. O código que limpa os dados deve ser separado do código que gera uma visualização. O código do front-end de um site deve ser separado do código do banco de dados que armazena os dados.

É natural que o design inicial do seu código não seja exatamente perfeito. Aspectos mudarão à medida que seu projeto evolui, o problema de negócios pode mudar ou, talvez, você identifique um design melhor depois de trabalhar no projeto por um tempo. Na seção “Refatoração”, na página 160, descreverei algumas técnicas para alterar o código que reduzem o risco de introdução de erros.

Código modular

Ao pensar na arquitetura geral do seu código, dividi-lo em funções e classes é uma etapa crítica. Não importa se isso ocorre no início do projeto, depois de testar algumas ideias iniciais ou durante a refatoração do código; é bom continuar pensando sobre quais funções e classes são adequadas. Analisamos esse tópico nos capítulos 1 e 4, mas quero revisitá-lo.

Código modular significa código dividido em pequenas partes independentes. É o contrário de todo o código do projeto ficar em um script gigante. Apesar de ser mais fácil trabalhar com código modular, geralmente é difícil descobrir como dividir o código. Nesta seção e na próxima, apresentarei algumas orientações sobre como fazer isso.

Na medida do possível, garanta que cada função ou classe tenha um propósito bem definido. Considere isso como uma “ação” que ela faz. Deve ser algo que você consiga descrever em uma frase curta, por exemplo, “essa função cria uma visualização de dados”. Se estiver descrevendo e começar a usar a palavra “e”, isso pode ser um indício para dividi-la em múltiplas

funções. Por exemplo, caso descreva a função como “essa função limpa os dados e cria uma visualização de dados”, é melhor dividi-la em uma função que limpa os dados e outra que cria a visualização.

Tente evitar o “acoplamento” de funções: quando você altera o código em um local e, em seguida, precisa mudar algo em outro local. Na seção “Acoplamento”, página 153, explicarei isso em mais detalhes.

Escrever código modular tem uma série de vantagens. O código modular é mais fácil de reutilizar em outros lugares, sendo mais fácil mover partes do código quando os requisitos do código mudam. O código modular também possibilita interconectar partes do código de diferentes formas para criar sistemas maiores (conhecido como composabilidade). Além disso, facilita testar o código.

Considere as seguintes perguntas para ajudá-lo a pensar quais funções e classes são adequadas para o problema que você está resolvendo:

- Quais padrões comuns você identifica no código que está escrevendo? Pode ser uma transformação de dados ou uma parte da lógica de negócio.
- Quais componentes poderiam ser reutilizados em muitos lugares? Um exemplo pode ser a transformação de alguns dados de um fuso horário para outro.
- Qual é o propósito da função que você está escrevendo? Na medida do possível, cada função ou método deve executar somente uma ação.

Agora, vamos transformar essas perguntas em um framework mais estruturado.

Um framework para design de código

Um framework que você pode considerar para criar funções, classes ou métodos é o seguinte (adaptado de “Six steps to more professional data science code [Seis etapas para um código de ciência de dados mais profissional, em tradução livre]” no Kaggle por Rachel Tatman) (*https://oreil.ly/Psa1F*):

Nome da função

Podemos começar escolhendo um bom nome para a função. No

Capítulo 9, abordarei mais detalhes sobre o que constitui um bom nome. Escolher um nome define a intenção para o que você quer que a função faça.

Entradas

A interface da função deve permanecer consistente. Ou seja, antes de começar a escrever o corpo da função, é uma boa ideia decidir quais devem ser as entradas dela. Deve-se especificá-las nos argumentos que a função recebe. Na próxima seção, analisarei mais as interfaces.

Comportamento

O corpo da função ou método contém as operações reais que ele executa. Trata-se do comportamento da função, ou seja, o que você efetivamente quer que ela faça. Como mencionado anteriormente, é boa prática que cada função execute apenas uma ação.

Saídas

As saídas da função são o que incluímos em uma instrução `return` ou também podem ser dados salvos em um arquivo. As saídas também fazem parte da interface da função, por isso é uma boa ideia considerá-las com cuidado antes de escrever a função e, em seguida, mantê-las iguais, quando possível.

Para ver um exemplo de como usar esse framework a fim de migrar de um notebook para um código modular, consulte a seção “Criando scripts a partir de notebooks”, na página 157.

Interfaces e contratos

Um dos aspectos mais importantes do código modular são as interfaces entre os componentes do sistema. Talvez você acredite que esse é um bom ponto de partida ao escrever uma função individual: o que ela aceita como entrada e o que retorna como saída?

A função a seguir, que vimos ao longo deste livro, recebe um arquivo CSV e uma lista de colunas a serem dropadas como entrada e retorna um DataFrame do pandas como saída:

```
def process_sdg_data(excel_file, columns_to_drop):
    df = pd.read_excel(excel_file)
```

```
    df = df.drop(columns_to_drop, axis=1)
    df = df.set_index("GeoAreaName").transpose()
    return df
```

Após decidir quais devem ser as entradas e saídas, você não deve alterá-las, pois outros componentes do sistema podem depender delas. Isso também é conhecido como “contrato”. É melhor manter o número de argumentos de entrada pequeno: talvez três ou quatro, no máximo. Se precisar de mais entradas, considere usar um arquivo de configuração.

Seu teste para uma função deve validar que o contrato é mantido, verificando se as entradas corretas são aceitas e se as saídas corretas são retornadas. Além disso, é necessário documentar as entradas e saídas na docstring da função, que explicarei na seção “Docstrings”, na página 171. O uso de anotações de tipo, conforme descrito na seção “Verificação de tipo”, na página 120, também pode garantir que o contrato seja mantido. As ferramentas de verificação de tipo podem alertá-lo quando o contrato for quebrado devido a tipos incompatíveis.

Acoplamento

Ao dividir seu código em partes, é importante garantir que essas partes sejam o mais independentes possível umas das outras. Se a alteração de uma parte do código mudar outra, a complexidade de todo o projeto aumentará e será bem mais difícil trabalhar nele. O termo “acoplamento” elenca dependência entre funções ou módulos. Se duas funções estiverem estritamente acopladas, mudar uma delas significa que você precisa alterar a outra extensivamente. Se elas estiverem fracamente acopladas, mudar uma delas significa que você não precisa alterar muito a outra ou nem precisa alterá-la. Tente reduzir o acoplamento o máximo possível.

Vejamos um exemplo de duas funções que estão estritamente acopladas:

```
import pandas as pd
from scipy.stats import linregress


def process_sdg_data(input_excel_file, columns_to_drop):
    df = pd.read_excel(input_excel_file)
    df = df.drop(columns_to_drop, axis=1)
    df = df.set_index("GeoAreaName").transpose()
```

```
    return df

def fit_trendline(country_name):
    df = process_sdg_data(
        "SG_GEN_PARL.xlsx",
        [
            "Goal",
            "Target",
            "Indicator",
            "SeriesCode",
            "SeriesDescription",
            "GeoAreaCode",
            "Reporting Type",
            "Sex",
            "Units",
        ],
    )
    timestamps = [int(i) for i in df.index.tolist()]
    data = df[country_name].tolist()

    result = linregress(timestamps, data)
    slope = round(result.slope, 3)
    r_squared = round(result.rvalue**2, 3)
    return slope, r_squared
```

A função fit_trendline chama a função process_sdg_data internamente para realizar o processamento de dados. Se alterarmos o comportamento da função process_sdg_function, isso poderá afetar o comportamento da função fit_trendline. Misturar essas funções dificulta trabalhar com elas.

Vejamos um exemplo em que essas funções não estão estritamente acopladas:

```
import pandas as pd
from scipy.stats import linregress

def fit_trendline(year_timestamps, data):
    result = linregress(year_timestamps, data)
```

```
    slope = round(result.slope, 3)
    r_squared = round(result.rvalue**2, 3)
    return slope, r_squared

def process_sdg_data(input_excel_file, columns_to_drop):
    df = pd.read_excel(input_excel_file)
    df = df.drop(columns_to_drop, axis=1)
    df = df.set_index("GeoAreaName").transpose()
    return df

df = process_sdg_data(
    "SG_GEN_PARL.xlsx",
    [
        "Goal",
        "Target",
        "Indicator",
        "SeriesCode",
        "SeriesDescription",
        "GeoAreaCode",
        "Reporting Type",
        "Sex",
        "Units",
    ],
)
timestamps = [int(i) for i in df.index.tolist()]
country_data = df["India"].tolist()
slope, r_squared = fit_trendline(timestamps, country_data)
```

Quando separamos as funções dessa forma, além de ser mais fácil trabalhar com elas, é mais fácil reutilizá-las em outro lugar.

Caso tenha interesse em ler mais sobre design de código, recomendo que estude os princípios SOLID. Este artigo do Real Python (*https://oreil.ly/oZN0y*) é um ótimo lugar para começar. Aprender sobre padrões de design é outra boa opção, embora eles possam exigir alguma adaptação para torná-los aplicáveis à ciência de dados. O Refactoring Guru (*https://oreil.ly/HEV6u*) tem um guia útil para padrões de projeto Python.

Na próxima seção, analisarei como passar seu código de notebooks do Jupyter para scripts Python independentes.

De notebooks a scripts escaláveis

Caso os notebooks do Jupyter sejam uma parte padrão do fluxo de trabalho do seu projeto, você pode chegar a um ponto em que será necessário migrar deles para scripts Python, que você precisa executar repetidamente. Talvez, seja uma etapa fundamental em seu projeto, representando a transição do protótipo para um código escrito com uma mentalidade mais voltada à engenharia: você conhece a solução para o seu problema e quer executar repetidamente o código que o resolve.

Nesta seção, abordarei alguns motivos pelos quais pode ser útil migrar de notebooks para scripts e, em seguida, analisarei alguns detalhes sobre como criar scripts a partir de seus notebooks.

Por que usar scripts em vez de notebooks?

Para os cientistas de dados, os notebooks do Jupyter são uma ferramenta extremamente útil. Eles são flexíveis, fornecem feedback instantâneo e também a liberdade de testar ideias. A visualização inline ajuda bastante. Além disso, são ótimos para escrever tutoriais, quando queremos combinar código e documentação. Podem até ser executados de forma automatizada com o Papermill (*https://oreil.ly/L8Zvj*).

No entanto, existe uma série de desvantagens quando escrevemos código em notebooks. Como é possível executar o código em uma ordem diferente daquela em que ele foi escrito, e não precisamos executar todo o código em um notebook de uma só vez, os notebooks nem sempre retratam o código que executamos. Isso pode dificultar a reprodução das etapas que você adotou.

Além disso, os notebooks não funcionam adequadamente com muitas das ferramentas e das práticas padrão de engenharia de software descritas neste livro. Com os notebooks, não é tão fácil usar lints, realizar a formatação ou a verificação de tipos como quando usamos um script Python. Como fica difícil gerenciar dependências (sobre as quais falarei no Capítulo 10), talvez você não saiba qual versão de uma biblioteca de terceiros foi usada em um

notebook.

Caso esteja usando o Jupyter Notebook ou o JupyterLab, também fica difícil depurar notebooks, ainda que, atualmente, os IDEs tenham depuradores que funcionam com notebooks. O controle de versão (que também abordarei no Capítulo 10) não captura quais células foram executadas em um notebook. Talvez seja necessário usar ferramentas externas para revisar facilmente as mudanças feitas em um notebook. Além disso, é muito fácil fazer upload de um notebook que contenha dados para o controle de versão, o que pode representar um risco de segurança.

O estilo de código que muitas pessoas (inclusive eu) tendem a escrever em notebooks é informal e, em geral, inclui células extras, nas quais verificamos o estado dos dados. Isso dificulta a escrita de código modular. Contudo, mesmo que escreva código em funções modulares, não é fácil importar funções de um notebook para outro. Em muitas situações, é mais fácil trabalhar com arquivos Python modulares.

Sem dúvidas, esse é o tipo de assunto com ampla variedade de opiniões, porém minha estratégia pessoal é usar notebooks para exploração e prototipagem de dados e, em seguida, migrar para um script quando estou fazendo as mesmas coisas repetidamente. Às vezes, insiro os componentes reutilizáveis que escrevi em um arquivo Python e os importo para um notebook a fim de executá-los. Em minha opinião, os scripts são úteis mesmo quando estou executando experimentos de machine learning, já que consigo escrever testes para garantir que meus experimentos sejam reproduzíveis e testar se eles estão definitivamente fazendo o que espero que façam.

Um ponto comum sobre migrar de notebooks para scripts é quando precisamos mover o código para um ambiente de produção, em que será executado repetidamente. Algumas empresas podem exigir isso devido a potenciais riscos de segurança dos notebooks (como salvar dados em notebooks, conforme mencionado anteriormente).

Caso decida migrar de notebooks para scripts, na próxima seção, abordarei um possível processo de como fazer isso.

Criando scripts a partir de notebooks

É prudente garantir que o notebook execute corretamente antes de criar funções a partir dele. Além disso, começar com um notebook bem estruturado facilita transformá-lo em um script. Portanto, vale a pena dedicar algum tempo limpando seu notebook antes de começar a tentar transformá-lo em um script.

Em seguida, considere se seu notebook deve ser um script Python ou vários. Como mencionado anteriormente neste capítulo, uma forma de pensar nisso é considerar cada script como uma etapa do projeto. É possível também usar arquivos separados para funções utilitárias, como funções de limpeza de dados que usamos em muitas etapas do projeto. Se costuma reexecutar um notebook a partir do meio, este é um bom sinal de que você deve dividi-lo.

Agora, existem algumas células que você sempre quer executar juntas ou toda vez que executa o notebook? Talvez elas possam ser uma função? Quais objetos comuns você está manipulando no notebook? Eles poderiam ser abstraídos em uma classe?

Vejamos um exemplo desse processo. A Figura 8.1 mostra um notebook para explorar e limpar dados do site dos Objetivos de Desenvolvimento Sustentável da ONU. Você já viu parte desse código no início do livro, mas é assim que ele pode parecer enquanto está sendo desenvolvido.

```
In [1]: import pandas as pd

In [2]: df = pd.read_excel('SG_GEN_PARL.xlsx')

In [3]: df.head(2)
```

Out[3]:

	Goal	Target	Indicator	SeriesCode	SeriesDescription	GeoAreaCode
0	5	5.5	5.5.1	SG_GEN_PARL	Proportion of seats held by women in national ...	4
1	5	5.5	5.5.1	SG_GEN_PARL	Proportion of seats held by women in national ...	8

2 rows × 34 columns

```
In [4]: df.columns

Out[4]: Index(['Goal', 'Target', 'Indicator', 'SeriesCode', 'SeriesDescription',
        'GeoAreaCode', 'GeoAreaName', 'Reporting Type', 'Sex', 'Units', '2000',
        '2001', '2002', '2003', '2004', '2005', '2006', '2007', '2008', '2009',
        '2010', '2011', '2012', '2013', '2014', '2015', '2016', '2017', '2018',
        '2019', '2020', '2021', '2022', '2023'],
       dtype='object')

In [5]: df = df.drop(['Goal', 'Target', 'Indicator', 'SeriesCode', 'SeriesDescription',
        'GeoAreaCode', 'Reporting Type', 'Sex', 'Units'], axis=1)

In [6]: df = df.set_index('GeoAreaName').transpose()

In [7]: df.head(2)
```

Out[7]:

GeoAreaName	Afghanistan	Albania	Algeria	Andorra	Angola	Antigua and Barbuda	Argentina	Armenia
2000	NaN	5.16	3.16	7.14	15.45	5.26	28.02	3.05
2001	NaN	5.16	3.42	7.14	15.45	5.26	26.46	3.05

2 rows × 195 columns

```
In [ ]: df.to_csv('women_in_parliament_processed.csv')
```

Figura 8.1: Um típico notebook.

Conforme podemos ver na Figura 8.1, em vários estágios exibi o conteúdo do DataFrame do meu pandas, a fim de que eu pudesse verificar o estado dos meus dados.

Essa seção do notebook importa os dados a partir de um arquivo de planilha do Excel e os gera novamente como um arquivo CSV. Ou seja, se quero fazer isso repetidamente, é um ótimo candidato para se transformar em uma

função.

Extraí as linhas que executam alguma operação em vez de apenas me mostrar como está o estado dos dados:

```
df = pd.read_excel("SG_GEN_PARL.xlsx")
df = df.drop(["Goal", "Target", "Indicator", "SeriesCode", "SeriesDescription",
        "GeoAreaCode", "Reporting Type", "Sex", "Units",], axis=1)
df = df.set_index("GeoAreaName").transpose()
df.to_csv("women_in_parliament_processed.csv")
```

Para transformar essas linhas de código em uma função, primeiro considerei qual deveria ser o propósito ou comportamento geral. Essa função precisa receber um arquivo do Excel, reorganizar os dados e gerar um arquivo CSV. Decidi chamá-la de `process_sdg_data`, pois quero que ela funcione em todos os arquivos Excel semelhantes baixados dos Objetivos de Desenvolvimento Sustentável da ONU (por isso o nome, "`sdg`"). Se estiver usando abreviações, é uma boa ideia documentá-las em algum lugar, conforme discutido no próximo capítulo: na seção "Nomes", na página 167, abordarei melhor as práticas recomendadas para escolher nomes.

Em seguida, pensei em quais deveriam ser as entradas e saídas da função. Decidi que minha função deveria receber um arquivo do Excel como entrada e gerar um arquivo CSV como saída. Considerei também o que queria mudar no futuro e transformei isso nos argumentos da minha função. Por exemplo, como posso mudar o nome do arquivo de entrada ou o nome do arquivo de saída, eles devem ser argumentos da minha função.

Por último, adicionei os argumentos como variáveis e escrevi o corpo da função:

```
def process_sdg_data(input_excel_file, columns_to_drop,
                     output_csv_file):
    df = pd.read_excel(input_excel_file)
    df = df.drop(columns_to_drop, axis=1)
    df = df.set_index("GeoAreaName").transpose()
    df.to_csv(output_csv_file)
```

A próxima etapa é adicionar um teste para essa função. As linhas do notebook em que exibi meus dados me deram ideias do que eu deveria testar. Por exemplo, eu queria validar se os dados de saída tinham as

colunas corretas. Como sei que o número de linhas e colunas no arquivo de saída deve permanecer o mesmo para qualquer arquivo de entrada que eu processe, escrevi o seguinte teste.

Vejamos o teste que acompanha a função process_sdg_data :

```
import os
import pandas as pd

def test_process_sdg_data():

    test_filepath = "test_sgd_data.csv"

    process_sdg_data("SG_GEN_PARL.xlsx",
                     ["Goal", "Target", "Indicator", "SeriesCode",
                     "SeriesDescription", "GeoAreaCode", "Reporting Type",
                     "Sex", "Units",],
                     test_filepath)

    df = pd.read_csv(test_filepath)

    assert len(df) == 24
    assert len(df.columns) == 196

    # etapa de limpeza - excluir o arquivo gerado no teste
    os.remove(test_filepath)
```

As etapas em que exibi meus dados se tornaram as declarações assert no meu teste.

Além das ferramentas já mencionadas, outras foram desenvolvidas para ajudá-lo na transição de um notebook para um script. O nbconvert (*https://oreil.ly/AiWYo*) e o Jupytext (*https://oreil.ly/Yn5RA*) são dois pacotes simples que extraem o código do seu notebook para um script. Você também pode seguir as etapas da documentação do Kedro (*https://oreil.ly/isx01*) a fim de migrar de um notebook diretamente para um projeto Kedro.

Refatoração

A refatoração é uma habilidade útil que o ajuda quando os requisitos do código mudam. É parte normal do fluxo de trabalho de desenvolvimento de software. Vejamos uma boa definição:

> Refatoração é o processo de mudar um sistema de software de forma que isso não mude o comportamento externo do código, e sim melhore sua estrutura interna.
>
> – MARTIN FOWLER

Você deve refatorar o código somente quando tiver um motivo específico. Caso o código esteja funcionando corretamente e atendendo a todos os requisitos, não há necessidade de refatorá-lo. Pense no problema que você está tentando resolver: por exemplo, você gostaria de adicionar uma funcionalidade nova? Ou você quer que o código fique mais limpo para facilitar a leitura e a manutenibilidade? Nesta seção, detalharei algumas estratégias para refatorar seu código e um exemplo de como fazê-lo.

Estratégias para refatoração

O ideal é que, antes de refatorar o código, você já tenha um conjunto completo de testes para ele. Os testes verificarão se a funcionalidade do código permanece a mesma quando é alterada: você não quer introduzir bugs ao refatorar seu código.

É uma boa ideia fazer mudanças incrementais no código. Não reescreva tudo de uma vez e não descarte partes do código que funcionam bem. Nem tudo ficará perfeito da primeira vez, principalmente se você estiver trabalhando em um projeto complexo. É boa prática refatorar em fases com marcos predefinidos.

A estratégia geral é simples: faça uma pequena mudança, execute um teste e salve o código (geralmente para o controle de versão, como descreverei no Capítulo 10). A mudança pode ser tão simples quanto mudar o nome de uma variável, caso queira atualizar o que essa variável está fazendo ou facilitar a leitura dela. O seu IDE facilita isso. Veja a Figura 8.2 da próxima seção. O controle de versão é extremamente útil nesse momento, pois facilita reverter uma versão anterior do código se a mudança fizer com que os testes falhem.

Ao refatorar, mantenha a interface da sua função igual e altere seus

componentes internos. Isso significa que você não precisará alterar demasiadamente outros códigos. Se estiver alterando a interface ou o comportamento da função, precisará de um teste de integração, que testa uma quantidade maior de código. Isso verificará se você não quebrou algo em outro lugar do sistema ao mudar aquilo de que ele dependia.

No geral, a refatoração bem-sucedida depende de um bom conjunto de testes. Na próxima seção, mostrarei um exemplo de como refatorar uma função.

Um exemplo de fluxo de trabalho de refatoração

Como mencionei na seção anterior, o princípio geral é fazer uma mudança, executar testes para verificar se nada quebrou e, em seguida, salvar suas mudanças (normalmente, fazendo o commit delas para o controle de verão, que abordarei no Capítulo 10). Ilustrarei o fluxo de trabalho usando a função `weighted_mean` do Capítulo 7, porque você já viu o teste que a acompanha.

Vejamos a função novamente:

```
def weighted_mean(num_list, weights):
    if not (num_list or weights):
        return None
    running_total = 0
    for i in range(len(num_list)):
        running_total += (num_list[i] * weights[i])
    return (running_total/sum(weights))
```

Vejamos o teste correspondente:

```
def test_weighted_mean():

    result = weighted_mean([1, 2, 4], [1, 2, 4])
    assert result == 3

    empty_list_result = weighted_mean([], [])
    assert not empty_list_result
```

Primeiro, execute o teste e valide se tudo está funcionando corretamente antes de começar a refatoração:

```
$ pytest test_ch08.py

...

test_ch08.py .                    [100%]

========================= 1 passed in 0.03s ====
```

Em seguida, faça uma pequena mudança. Talvez você queira fazer mudanças que melhorem a legibilidade do código, por exemplo, alterar os nomes de suas variáveis. É possível fazer isso com facilidade usando o IDE, conforme mostrado na Figura 8.2:

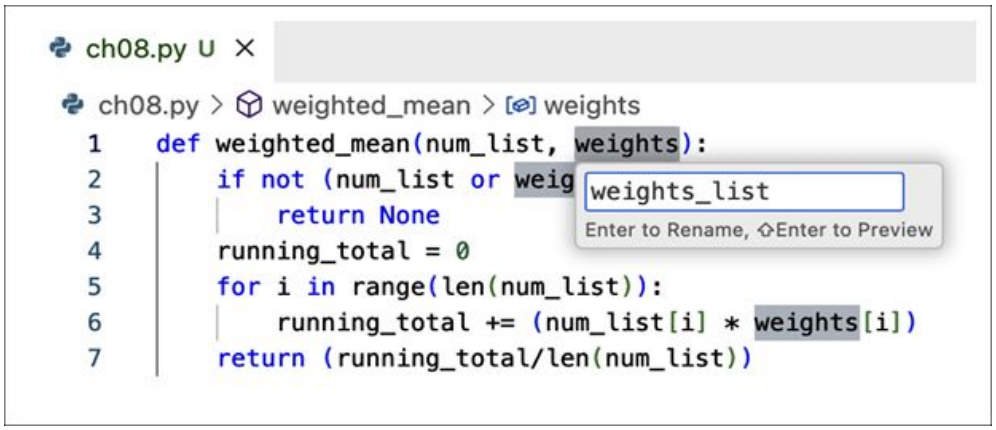


Figura 8.2: Refatoração no VS Code.

Isso renomeia todas as instâncias de weights para weights_list:

```
def weighted_mean(num_list, weights_list):
    if not (num_list or weights_list):
        return None
    running_total = 0
    for i in range(len(num_list)):
        running_total += (num_list[i] * weights_list[i])
    return (running_total/sum(weights_list))
```

Em seguida, execute o teste mais uma vez a fim de validar se o código ainda funciona corretamente.

É necessário usar o mesmo fluxo de trabalho para fazer uma mudança maior. Em seguida, mudarei o corpo interno da função, substituindo-a por uma função built-in da biblioteca NumPy:

```
import numpy as np

def weighted_mean(num_list, weights_list):
    try:
        return np.average(num_list, weights=weights_list)
    except ZeroDivisionError:
        return None
```

Executando novamente o teste, confirmamos que o comportamento geral da função e sua interface permanecem inalterados, mesmo que o corpo da função seja completamente diferente.

Esse fluxo de trabalho pode ser aplicado a situações maiores e mais complexas. Você também pode precisar atualizar seus testes ao mesmo tempo. Não é incomum adicionar mais casos de teste durante o processo de refatoração.

Aspectos fundamentais

Mesmo que seu projeto possa mudar completamente durante o período em que você trabalha nele, ainda é crucial pensar no design do código antes de começar, enquanto você estiver trabalhando e até mesmo depois de concluí-lo. Quando um projeto tem um bom design, fica mais fácil para outras pessoas trabalharem nele e até mesmo para você, caso precise revisitá-lo após meses ou até anos.

Você também deve considerar a possibilidade de padronizar seus projetos. Se estiver trabalhando repetidamente em projetos semelhantes, considere dedicar algum tempo para desenvolver um modelo que será útil para essa repetição. É possível usar uma ferramenta como Cookiecutter ou Kedro para automatizar o processo de definição de um projeto.

O código modular é fundamental para um bom design. Não se esqueça de pensar em como dividir seu código em partes que possam ser facilmente reutilizadas ou alteradas no futuro. Antes de começar a escrever cada função, considere o propósito delas e quais devem ser suas interfaces e adicione apenas o código que realmente cumpre os propósitos depois de configurar as interfaces.

Os notebooks Jupyter são ótimos para experimentação, mas você pode

querer migrar para scripts quando o código for implantado em um ambiente de produção. Para isso, considere quais partes do código devem ser transformadas em funções, que podem ser reutilizadas em outros lugares, e adote um framework a fim de escrever código modular.

Você precisará alterar seu código com frequência por vários motivos. Os requisitos do código podem mudar, você pode querer torná-lo mais legível, mais eficiente, e assim por diante. A refatoração é parte normal de qualquer projeto de código. Um fluxo de trabalho de refatoração bem-sucedido depende de um bom conjunto de testes. Para refatorar seu código, faça uma pequena mudança e, em seguida, execute testes a fim de garantir que sua mudança não quebre nada e, em seguida, atualize o código onde quer que ele esteja armazenado.

Criar um bom design para seu código é grande parte da escrita de código pronto para produção, mas você também precisará se comunicar com outras pessoas sobre o que seu código faz. A documentação é fundamental para isso e, no próximo capítulo, abordarei uma variedade de práticas recomendadas para documentação.

CAPÍTULO 9

Documentação

Na ciência de dados, a documentação é um aspecto negligenciado com frequência. Comumente deixada para o final do projeto, acaba sendo feita às pressas ou omitida por completo devido ao entusiasmo em iniciar um projeto novo. No entanto, como abordei na seção “Legibilidade”, na página 28, a documentação é parte crucial para tornar seu código reproduzível. Se quiser que outras pessoas utilizem seu código, ou se pretende revisitá-lo no futuro, ele precisa de uma boa documentação. Como nossa memória é falha, é impossível se lembrar de todos os detalhes de quando o código foi escrito ou de quando os experimentos foram realizados. Por isso, precisamos registrá-los.

Uma boa documentação comunica ideias de forma eficaz. Quem lê o código precisa entender o que você deseja transmitir. Por isso, primeiro, é importante considerar para quem você está escrevendo a documentação. Você está registrando seus experimentos para outro cientista de dados que pode assumir o projeto no futuro? Você está documentando um trecho de código que acredita que pode ser útil para outras pessoas da sua equipe? Ou está registrando os próprios pensamentos para revisitá-los em seis meses? Adapte o nível de detalhes e a linguagem utilizada, de modo que sejam adequados para o leitor esperado.

Outro aspecto de uma boa documentação é a atualização: a documentação não ajuda se não for atualizada. Ela deve ser atualizada ao mesmo tempo que as mudanças no código são feitas. Garanta que seja o mais fácil possível atualizar sua documentação. Por exemplo, não use ferramentas proprietárias de documentação que exigem logins separados. Uma boa documentação também deve ser bem estruturada. As informações mais importantes devem ser mais fáceis de encontrar. Ou seja, insira-as no início ou deixe óbvio onde as encontrar.

Uma boa documentação pode economizar muito tempo e reduzir a

complexidade de um projeto de ciência de dados. Caso saiba qual tarefa já foi feita, você reduz a chance de retrabalho. É possível também se atualizar mais rápido sobre um projeto novo ou lembrar-se facilmente de em que estava trabalhando há um ano.

Ainda que todas essas afirmativas sejam relevantes para a documentação de qualquer projeto, algumas considerações são mais específicas para projetos de ciência de dados. Em um projeto de ciência de dados, é comum testar diversas soluções potenciais para um problema antes de decidir qual funciona melhor. Por isso, é boa prática documentar o processo de ideias que envolve sua experimentação e tomada de decisão. Isso ajuda se você for questionado sobre o assunto mais tarde ou se precisar revisitar o projeto no futuro.

Tente responder a perguntas como estas em sua documentação:

- Por qual motivo você selecionou os dados que está usando nesse projeto?
- Quais são as suposições que você está fazendo sobre seus dados?
- Por qual motivo escolheu determinado método de análise em vez de outro?
- Existem circunstâncias em que esse método de análise não funciona?
- Quais atalhos (se for o caso) você tomou que poderiam ser melhorados no futuro?
- Quais são outras possibilidades para futuras experimentações que você sugeriria a qualquer pessoa que trabalhe posteriormente nesse projeto?
- Quais são as lições que você aprendeu com esse projeto?

Neste capítulo, abordarei diferentes tipos de documentação e práticas recomendadas para escrevê-la. Detalharei a documentação de sua base de código, a documentação em notebooks do Jupyter e a documentação para experimentos de machine learning.

Documentação na base de código

Conforme abordei no Capítulo 1, um bom código é legível. Uma base de código legível deve conter texto e código, na forma de comentários,

docstrings e documentos mais detalhados. O próprio código deve ser legível, e bons nomes são essenciais para isso. Nesta seção, apresentarei sugestões de melhores práticas para todos esses tipos de documentação.

Vejamos uma hierarquia de documentação para sua base de código, do nível mais conciso ao mais detalhado:

- Nomes de funções, de classes e de módulos fornecem informações sobre o que você deve esperar que determinado trecho de código faça.
- Comentários são breves anotações individuais que adicionam informações extras ao código, como notas de rodapé em um livro.
- Docstrings fornecem uma visão geral mais ampla do que uma função ou classe faz, incluindo detalhes de quaisquer casos edge.
- A documentação da API mostra o que cada endpoint da API espera como entrada e retorna como saída. No Capítulo 11, você encontrará mais informações sobre a documentação de APIs.
- Documentos mais detalhados, como readmes e tutoriais, fornecem uma visão geral de como usar todo o código em um projeto.

Em conjunto, todos esses elementos tornarão seu código mais legível e facilitarão o uso por outras pessoas. Cada informação que você queira transmitir para quem lê seu código deve ser documentada uma vez. Insira o resumo do que sua função faz na docstring, não nos comentários, e não duplique essa informação no README do seu projeto.

Nomes

A cada linha de código escrita, uma série de nomes precisa ser escolhida. Variáveis, funções, notebooks, projetos: todos precisam de um nome. Bons nomes são parte importante para que seu código tenha legibilidade. Se alguém quiser utilizar seu código, antes de fazer qualquer mudança, esse alguém o lerá atentamente. Os nomes escolhidos comunicam o propósito do seu código. Por exemplo, uma função chamada `download_and_clean_monthly_data` é muito mais elucidativa do que uma função chamada `process_data`. Seu notebook do Jupyter jamais deve ser nomeado como `untitled1.ipynb`, pois isso nada revela sobre seu conteúdo. Caso precise do conteúdo desse notebook no futuro, encontrá-lo sem um nome adequado será uma tarefa árdua.

Bons nomes são expressivos, de tamanho adequado e são fáceis de ler. Bons nomes também usam linguagem relevante ao projeto em questão, à empresa ou à organização. Incluir unidades nas variáveis é uma ótima prática: distance_km é mais informativo do que apenas distance. Se o nome ideal não surgir de imediato, não hesite em atualizá-lo posteriormente: todas as instâncias podem ser encontradas e substituídas ao mesmo tempo.

Ao escolher nomes, seja cauteloso com abreviações. Abreviar algumas letras raramente compensa o inconveniente e a dificuldade de compreensão do contexto por parte de outras pessoas. Abreviações conhecidas em toda a empresa são aceitáveis, mas recomendo evitá-las se forem específicas a uma única feature ou se não forem amplamente usadas em sua empresa.

Mas qual seria o tamanho adequado para um nome? Nomes de variáveis e funções não devem ser muito curtos, pois, se um nome for muito curto, isso aumenta a carga mental da pessoa que lê seu código, que precisará interpretar o significado do nome. Por exemplo, image_id é mais informativo do que im_id, e clean_df é melhor do que cl_df. Palavras completas são mais fáceis de ler e de encontrar em seu IDE, caso precise pesquisar o uso delas ou alterá-las posteriormente.

Por exemplo, o trecho de código a seguir precisa de um comentário explicativo para o que está acontecendo devido ao uso de letras únicas como variáveis:

```
# calcula a acurácia das predições em comparação com os dados de teste
a = sum(x == y for x, y in zip(p, t))/len(p)
```

Escolher nomes melhores torna o código mais legível:

```
accuracy = sum(x == y for x, y in zip(predictions, test_data))/len(predictions)
```

As variáveis x e y permanecem com nomes curtos, pois são usadas apenas na chamada para sum() e, portanto, são temporárias. Da mesma forma, uma convenção em Python é usar as letras únicas i e j como contadores, como no exemplo a seguir:

```
for i in range(processed_results):
    # faz alguma coisa
```

Outras convenções comuns são df para um DataFrame do pandas e fig e ax para figuras e eixos ao usar o Matplotlib, respectivamente. Não tem problema usar nomes curtos que serão reconhecidos por quem lê o código.

Além disso, é frequente ver nomes de função que começam com um verbo, como make_..., load_..., get_..., e assim por diante, prática que recomendo.

Nomes não devem ser muito semelhantes. As funções strptime e strftime do Python, por exemplo, são tão parecidas que sempre preciso consultar a documentação para lembrar a diferença. Mais uma vez, isso significa que o leitor precisa reter conhecimento adicional para usar o código.

Além disso, adotar as convenções de formatação do Python torna os nomes em seu código mais legíveis. Variáveis e funções devem usar snake_case, com todas as palavras em letras minúsculas e unidas por underlines. É mais fácil ler nomes de variáveis com underlines entre as palavras: x_train_array é mais claro que xtrainarray. Definições de classe devem usar CamelCase, com a letra inicial de cada palavra em letra maiúscula. Constantes ou variáveis globais devem usar ALL_CAPS.

Não use nomes de funções built-in do Python como nomes de variáveis; caso contrário, será impossível usar as funções originais. Vejamos um exemplo:

```
list = [0, 2, 4]
```

Isso fará com que a seguinte linha de código retorne um erro, em vez de criar uma lista vazia:

```
empty_list = list()
```

Nomes de módulos e arquivos também são de suma importância. Nesse quesito, escolher bons nomes o ajudará a lembrar em que local determinadas funcionalidades estão armazenadas. Como recomenda esta postagem do blog de David Nicholson (*https://oreil.ly/1WUx3*), armazene arquivos relacionados em subpastas em vez de usar prefixos para sinalizar que estão relacionados. Bons nomes de módulo são específicos em vez de genéricos: *data_preprocessing_utils.py* é preferível a *utils.py*.

Para mais orientações sobre como escolher bons nomes, confira o PEP8 (*https://oreil.ly/Hq3Ll*) e o Guia de Estilo Python do Google (*https://oreil.ly/csW2t*).

Comentários

Comentários são uma das formas mais úteis de documentação dentro da base de código, mas precisam ser bem utilizados. Comentários podem resumir, explicar ou acrescentar ressalvas ao código ou sinalizar locais que

precisam ser revisitados e alterados posteriormente. Podem também ser uma maneira útil de começar a escrever uma função: comece com pseudocódigo na forma de comentários e, em seguida, preencha com o código real, facilitando a estruturação da função.

Em Python, um comentário é sinalizado com um símbolo #:

```
# Isso é um comentário.
```

Comentários não devem repetir informações já presentes no código. Isso não ajuda o leitor e, caso precise mudar o código, o comentário também precisará ser atualizado, violando o princípio "DRY" do Capítulo 1. Vejamos um exemplo de um comentário ruim:

```
# Treina o modelo de classificação
classifier.fit(X_train, y_train)
```

Um bom comentário acrescenta ressalvas, resume informações ou explica algo que ainda não está no código. Vejamos um exemplo de um comentário útil:

```
def slow_way_to_calculate_mode(list_of_numbers):
    # Apenas para fins de demonstração
    # Use a função mode da biblioteca statistics em casos de uso real
```

Quem lê o código deve entender os comentários com facilidade. É melhor usar palavras e frases completas em vez de abreviações. Escrevê-los ao mesmo tempo que você escreve o código, em vez de adicionar explicações depois, também é uma boa prática, pois permite a oportunidade de capturar ideias extras que surgem durante o processo de escrita do código.

Comentários devem ser sempre profissionais, sem gírias ofensivas ou palavrões. No entanto, podem ser divertidos e descontraídos, se isso se encaixar na cultura da sua empresa. O código-fonte da Apollo 11 da NASA tem alguns ótimos exemplos (veja a Figura 9.1).

```
243                 BZF     P63SPOT4        # BRANCH IF ANTENNA ALREADY IN POSITION 1
244
245                 CAF     CODE500         # ASTRONAUT:    PLEASE CRANK THE
246                 TC      BANKCALL        #               SILLY THING AROUND
247                 CADR    GOPERF1
248                 TCF     GOTOP00H        # TERMINATE
249                 TCF     P63SPOT3        # PROCEED       SEE IF HE'S LYING
250
251   P63SPOT4      TC      BANKCALL        # ENTER         INITIALIZE LANDING RADAR
252                 CADR    SETPOS1
253
254                 TC      POSTJUMP        # OFF TO SEE THE WIZARD ...
```

Figura 9.1: Comentários no código-fonte da Apollo 11 (fonte: GitHub (https://oreil.ly/XFgna))[1].

Comentários também podem ser usados para adicionar outros aspectos informativos, como:

- Links para recursos externos, como um problema relevante no GitHub.
- Links para um sistema interno de rastreamento de projetos, como o Jira.
- Adicionar um O QUE FAZER para uma funcionalidade que você deseja acrescentar no futuro.
- Acrescentar uma observação sobre uma linha de código com uma correção temporária.

Os comentários devem ser curtos e abordar uma única ideia. Se precisar incluir explicações mais extensas, use docstrings ou documentos maiores. Nas próximas seções, abordarei essas questões.

Docstrings

Docstrings do Python são uma versão formal mais extensa de comentários, comumente incluídas no início de uma definição de função ou classe ou na parte superior do arquivo. Elas fornecem ao leitor uma visão geral do que a função ou script deve fazer, ao contrário dos comentários que enfatizam uma questão específica. Docstrings são imprescindíveis para facilitar a leitura do seu código por outras pessoas, pois possibilitam detalhar o propósito de uma função além do que seu nome pode comunicar.

A docstring de uma função deve descrever as entradas e saídas esperadas dessa função. Trata-se da interface para essa função, que abordei na seção

“Interfaces e contratos”, na página 153. Se não estiver usando anotações de tipo (que descrevi na seção “Verificação de tipo”, na página 120), inclua as entradas e saídas na docstring. Docstrings são o padrão Python para documentar uma função, e isso significa que o texto inserido pode ser retornado chamando a função `help` em um interpretador Python. Além disso, soluções automatizadas de documentação, como Sphinx (*https://oreil.ly/_Y7fp*) ou pdocs (*https://oreil.ly/OU1nS*), podem extrair o texto de docstrings e gerar documentação web a partir delas.

Vejamos um ótimo exemplo de uma docstring da base de código do pandas (*https://oreil.ly/oEBEE*). O método `head()` exibe as primeiras `n` linhas de um DataFrame do pandas. É prática padrão delimitar a docstring com aspas triplas (`"""`). Lembre-se, aqui esta docstring foi traduzida para melhor compreensão dos leitores.

```
def head(self: NDFrameT, n: int = 5) -> NDFrameT:
    """
    Retorna as primeiras `n` linhas. ❶
    Esta função retorna as primeiras `n` linhas para o objeto com base
    na posição. É útil para testar de forma rápida se seu objeto
    tem o tipo adequado de dados.
    Para valores negativos de `n`, esta função retorna todas as linhas, exceto
    as últimas linhas `|n|`, equivalentes a ``df[:n]``. ❷
    Se n for maior que o número de linhas, esta função retornará todas as linhas. ❸
    Parâmetros
    ----------
    n : int, padrão 5 ❹
      Número de linhas a selecionar.
    Retornos
    -------
    mesmo tipo da chamada do ❺
      As primeiras `n` linhas do objeto que faz a chamada.
    """
```

❶ Fornece uma descrição geral da função.

❷ Esta é uma ressalva sobre o comportamento que pode não ser o esperado.

❸ Este é um caso edge.

❹ O parâmetro de entrada com o padrão e o tipo esperado.

❺ A saída, que pode ser de vários tipos. Os mais comuns para esta função seriam DataFrame ou Series do pandas.

É possível obter a mesma documentação com help(df.head) em um interpretador Python. Essa docstring também é usada pela documentação gerada automaticamente (*https://oreil.ly/Ra2KM*) do pandas. Caso esteja trabalhando em um notebook do Jupyter, também é possível usar o pd.DataFrame.head? para visualizar essa docstring.

Existem três modelos principais para docstrings: docstrings do Google (*https://oreil.ly/VXsd9*), docstrings do NumPy (*https://oreil.ly/mJcGp*) e docstrings do reStructuredText (*https://oreil.ly/GcEYV*). Recomendo escolher um e adotá-lo, pois a padronização facilita a leitura: o formato se torna familiar. É possível configurar um editor de código para gerar automaticamente um modelo de docstring no formato de sua preferência, como neste exemplo:

```
def process_sdg_data(input_excel_file, columns_to_drop):
    """_summary_

    Args:
        input_excel_file (_type_): _description_
        columns_to_drop (_type_): _description_

    Retornos:
        _type_: _description_
    """
    df = pd.read_excel(input_excel_file)
    df = df.drop(columns_to_drop, axis=1)
    df = df.set_index('GeoAreaName').transpose()
    return df
```

Vejamos a função com uma docstring completa, traduzida para este livro:

```
def process_sdg_data(input_excel_file, columns_to_drop):
    """Rearranja os dados de ODS (Objetivos de Desenvolvimento Sustentável) para carregá-los na
    API.

    Args:
```

```
    input_excel_file (str): path para o arquivo de dados de entrada.
    columns_to_drop (list): lista dos nomes de colunas a serem dropados.

  Retornos:
    dataFrame do pandas: DataFrame com colunas esperadas pelo código da API.
  """
  df = pd.read_excel(input_excel_file)
  df = df.drop(columns_to_drop, axis=1)
  df = df.set_index('GeoAreaName').transpose()
  return df
```

É boa prática adicionar docstrings aos seus testes. Elas devem explicar o propósito do teste e o resultado esperado, e podem incluir detalhes sobre casos edge ou descrever o bug que está sendo verificado. Adicionar docstrings facilitará a compreensão e a manutenibilidade futuras de seus testes por outras pessoas. Este artigo de Hynek Schlawack (*https://oreil.ly/hbFto*) fornece alguns detalhes mais úteis.

Readmes, tutoriais e outros documentos mais extensos

Documentos extensos devem fornecer aos leitores o contexto geral do seu projeto e difundir o trabalho realizado. Eles expandem o escopo das docstrings, fornecendo uma visão geral de todo o projeto. Considere incluir o seguinte na documentação de seu projeto:

- Uma breve visão geral do seu projeto, que pode ser um único parágrafo, como um “resumo executivo”. Tente incluir os objetivos gerais do projeto, quem deve usá-lo e os casos de uso.
- Como começar a usar o projeto e como navegar nele. Isso pode ser feito na forma de um tutorial em notebook.
- Quaisquer ressalvas ou limitações do projeto. Por exemplo, se seu código funciona apenas com dados de 2023 ou anteriores, é útil destacar isso na introdução do projeto, em vez de quem está lendo precisar vasculhar os comentários do código.
- As próximas etapas do projeto. Mesmo que você tenha concluído o trabalho em um projeto, é útil registrar as próximas etapas para qualquer pessoa que dê continuidade a esse trabalho no futuro.

Cada repositório de código deve conter um arquivo *README.md* que aborde esses pontos e forneça uma introdução geral ao projeto. Como se trata de um arquivo Markdown, a extensão dele é *.md*. Markdown é uma linguagem para escrever documentos estruturados que usa símbolos com o intuito de controlar a formatação. É possível encontrar uma introdução à linguagem Markdown e outras dicas úteis para escrever um README na documentação do GitHub (*https://oreil.ly/rQit2*).

Assim como tudo neste capítulo, é essencial manter os documentos extensos atualizados. Poucas coisas são mais frustrantes do que tentar entender um exemplo de tutorial e descobrir que o código subjacente mudou e o exemplo não executa mais. Testes automatizados para os exemplos de seus tutoriais evitam isso.

A documentação é mais fácil de atualizar se estiver no mesmo local que o código. Por exemplo, é mais fácil atualizar a documentação se ela estiver no repositório do GitHub do que em uma ferramenta externa, como o Confluence. Podemos usar ferramentas para documentação, como o Sphinx (*https://oreil.ly/63mbI*), para gerar documentação em HTML e, em seguida, hospedá-la com ferramentas como Read the Docs (*https://oreil.ly/vrM6o*) ou o GitHub Pages (*https://oreil.ly/TqdGi*).

A combinação de bons nomes, comentários úteis, docstrings completas e uma introdução geral garantirá que seu código seja fácil de executar, de manter e de trabalhar no futuro.

Documentação em notebooks do Jupyter

Os notebooks do Jupyter podem ser muito informais e costumam ser usados nos estágios iniciais de um projeto. No entanto, mesmo que seu notebook seja um beco sem saída, em que você tenta algo e não funciona, o código dentro dele pode ser útil no futuro. Além disso, é muito útil saber o que já foi tentado e não deu certo. Portanto, vale a pena revisitar esse código e saber o que está no notebook. Conforme abordado na seção “Nomes”, na página 167, é importante que seu notebook tenha um nome descritivo. Além disso, é uma boa ideia descrever o conteúdo do notebook logo no início.

Em um notebook, você pode adicionar texto a fim de estruturá-lo melhor e também notas explicativas. Não repita as informações do código. Use o

texto para adicionar resumos, ressalvas e explicações. Sempre atualize o texto ao atualizar o código.

É possível adicionar texto aos notebooks do Jupyter com a linguagem Markdown. Para converter uma célula de código em Markdown, pressione a tecla de atalho m quando estiver no modo de comando em um notebook ou use o menu suspenso na parte superior do notebook.

A Figura 9.2 mostra uma boa mistura de texto e código. O texto adiciona informações ao notebook sem duplicar o código.

The main tool for preprocessing textual data is a tokenizer. A tokenizer splits text into *tokens* according to a set of rules. The tokens are converted into numbers and then tensors, which become the model inputs. Any additional inputs required by the model are added by the tokenizer.

If you plan on using a pretrained model, it's important to use the associated pretrained tokenizer. This ensures the text is split the same way as the pretraining corpus, and uses the same corresponding tokens-to-index (usually referred to as the *vocab*) during pretraining.

Get started by loading a pretrained tokenizer with the AutoTokenizer.from_pretrained() method. This downloads the *vocab* a model was pretrained with:

```
In [ ]: from transformers import AutoTokenizer tokenizer =
        AutoTokenizer.from_pretrained("bert-base-cased")
```

Then pass your text to the tokenizer:

```
In [ ]: encoded_input = tokenizer("Do not meddle in the affairs of wizards, for they are
        subtle and quick to anger.") print(encoded_input)

Out[ ]: {'input_ids': [101, 2079, 2025, 19960, 10362, 1999, 1996, 3821, 1997, 16657, 101
        0, 2005, 2027, 2024, 11259, 1998, 4248, 2000, 4963, 1012, 102],
         'token_type_ids': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0],
         'attention_mask': [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1]}
```

The tokenizer returns a dictionary with three important items:

- input_ids are the indices corresponding to each token in the sentence.
- attention_mask indicates whether a token should be attended to or not.
- token_type_ids identifies which sequence a token belongs to when there is more than one sequence.

Figura 9.2: Mistura de código e texto em um notebook do Jupyter (fonte: GitHub (https://oreil.ly/edZMf)).

Podemos também usar Markdown em um notebook para adicionar títulos, o que ajudará os leitores a navegar pelo notebook. Os títulos usam o símbolo #:

 # Este é um título de primeiro nível

Este é um título de segundo nível

Este é um título de terceiro nível

Isso gera o resultado mostrado na Figura 9.3.

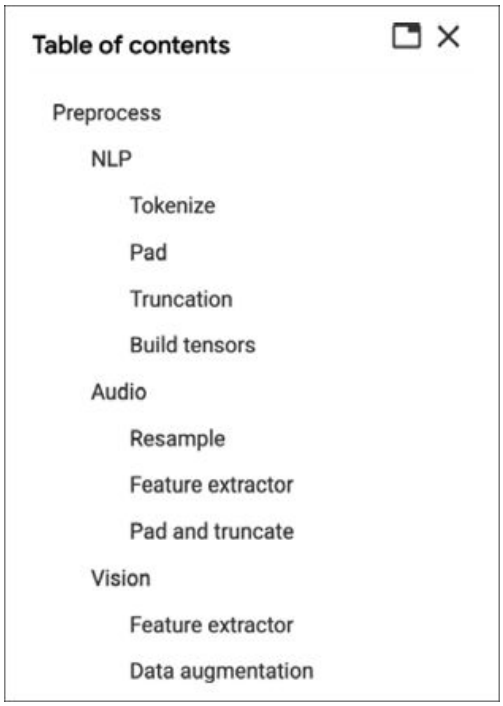


Figura 9.3: Uso de títulos em um notebook (fonte: GitHub (https://oreil.ly/dyNpn)).

Fica muito mais fácil ler e navegar por um notebook com seções bem estruturadas.

Documentando experimentos de machine learning

Em projetos de machine learning, costumamos realizar experimentos com diversos modelos, conjuntos de dados e hiperparâmetros em busca do modelo que gere as melhores predições, de acordo com as métricas escolhidas de avaliação. Especificamente, o número de hiperparâmetros pode crescer muito, por isso é importante documentar as combinações testadas. Isso ajudará se você retomar o trabalho no futuro e também auxiliará qualquer outra pessoa que trabalhe em seu projeto.

Seus experimentos precisam ser documentados de maneira estruturada, a fim de assegurar que você está sendo criterioso. Para isso, você deve

rastrear todas as variáveis que mudam em cada iteração do experimento, já que isso possibilita a reprodução do experimento no futuro ou que outra pessoa assuma o projeto, e saiba quais variáveis foram testadas. Você também deve registrar quaisquer suposições feitas durante o processo.

Considere registrar o seguinte:

- Os dados que você usou para treinar o modelo
- A divisão entre treinamento/avaliação/teste
- As escolhas de engenharia de features que fez
- Os hiperparâmetros do modelo (como a regularização em um modelo de regressão logística ou a taxa de aprendizagem de uma rede neural)
- As métricas usadas para avaliar seu modelo, como acurácia, precisão e revocação [recall]

Como registrar todos esses parâmetros manualmente é tedioso e sujeito a erros, recomendo que use uma ferramenta automatizada. Weights and Biases (*https://wandb.ai*) é uma ferramenta útil para rastrear experimentos de machine learning. Ela se integra facilmente ao scikit-learn, TensorFlow e PyTorch e registra os parâmetros de treinamento em um painel web, conforme mostrado na Figura 9.4.

Name (384 visualized)	Created	Runtime	Sweep	encoder	num_train	num_valid	acc
worldly-totem-422	1mo ago	12m 54s	-	resnet34	682	97	0.8566
jumping-voice-421	1mo ago	11m 59s	-	resnet34	725	92	0.8504
logical-energy-420	1mo ago	2m 14s	-	resnet34	66	10	0.626
laced-dust-419	1mo ago	2m 4s	-	resnet18	61	15	0.5968
whole-music-418	1mo ago	1m 40s	-	resnet18	68	13	0.6139
grateful-glitter-417	1mo ago	21s	-	resnet18	70	11	0.2367
efficient-lake-416	1mo ago	1m 42s	-	resnet18	76	10	0.6899
clear-night-415	1mo ago	1m 17s	-	resnet18	66	10	0.5403
glorious-night-414	1mo ago	1m 33s	-	resnet18	68	7	0.7627
smart-sponge-413	1mo ago	1m 46s	-	resnet18	72	11	0.6517

Figura 9.4: Rastreando experimentos com o Weights and Biases (fonte: documentação do Weights and Biases (https://oreil.ly/gv4wo)).

Outras soluções de rastreamento de experimentos incluem o pacote open-source Sacred (*https://oreil.ly/aZFSI*), MLflow (*https://oreil.ly/tf3SN*), Neptune (*https://oreil.ly/sohIo*) e SageMaker Experiments (*https://oreil.ly/uavxZ*) da AWS. Qualquer uma delas o ajudará a registrar seus experimentos de forma automatizada.

Aspectos fundamentais

Uma boa documentação é fundamental para ajudar outras pessoas a usar seus projetos e para tornar seu código legível. Garanta que sua documentação seja atualizada constantemente e escrita para quem usará seu código. Além de ajudar outras pessoas a começar a trabalhar em seu projeto, uma boa documentação o ajuda a entender seu código no futuro. Caso queira saber mais, a comunidade Write the Docs (*https://oreil.ly/ZQPUg*) fornece muitas dicas úteis.

Adotar as seguintes recomendações o ajudará a redigir uma boa documentação:

Nomes

Os nomes de variáveis, funções e arquivos devem ser informativos, de tamanho adequado e de fácil leitura.

Comentários

Seus comentários devem adicionar informações extras não contidas no código, como um resumo ou uma ressalva.

Docstrings

As suas funções devem sempre ter uma docstring que descreva as entradas e saídas delas, bem como seu propósito.

READMEs

Todo repositório ou projeto deve ter uma introdução que apresente seu código, possibilitando que outras pessoas saibam por que devem usá-lo.

Notebooks do Jupyter

Seus notebooks serão mais fáceis de ler se você os nomear com bons nomes, estruturá-los e intercalar texto e código.

Rastreamento de experimentos

Os experimentos, sobretudo em projetos de machine learning, devem ser rastreados de forma estruturada.

Com todas as técnicas deste capítulo, seu código será claro e mais fácil de ler, facilitando a apresentação do que ele faz e o uso por outras pessoas.

---

1 N.T.: Grosso modo, a tradução desses comentários seria:

RAMIFICAR SE A ANTENA JÁ ESTIVER NA POSIÇÃO 1

ASTRONAUTA: POR FAVOR, GIRE ESSE TROÇO ENGRAÇADO

TERMINAR

PROSSEGUIR PARA VER SE ELE ESTÁ MENTINDO

ENTRAR E INICIALIZAR O RADAR DE POUSO

PARTIR PARA VER A MÁGICA ACONTECER

CAPÍTULO 10

Compartilhando o código: controle de versão, dependências e empacotamento

Compartilhar seu código e colaborar com outras pessoas é fundamental para o seu sucesso em ciência de dados. Você pode participar de um projeto existente, com uma base extensa de código, ou começar trabalhando em seu código sozinho. No entanto, à medida que o projeto cresce, talvez você queira compartilhá-lo com outras pessoas, de modo que também não precisem resolver os mesmos problemas. Ou talvez você esteja contribuindo com projetos open-source.

Caso esteja compartilhando seu código publicamente, como um projeto open-source, isso possibilita que faça parte de uma comunidade. As bibliotecas open-source do Python formam um ecossistema gigantesco, e a ciência de dados seria bem mais difícil sem o pandas, o NumPy, o scikit-learn, o Matplotlib e muitos outros.

Seja contribuindo com projetos open-source ou colaborando com colegas de equipe da sua empresa, você precisará conhecer algumas ferramentas e técnicas que facilitam com que diversas pessoas trabalhem no mesmo código. Por mais que sejam variadas, as ferramentas compartilham muitos princípios em comum.

Neste capítulo, abordarei ferramentas e técnicas que o ajudarão a colaborar com outros desenvolvedores. Primeiro, o controle de versão é importante porque é a maneira padrão de colaborar em uma base de código. Em seguida, abordarei ferramentas para gerenciar as bibliotecas de terceiros das quais seu código depende. Por último, explicarei como empacotar projetos para que outras pessoas possam instalá-los.

Controle de versão usando o Git

O controle de versão é uma forma de rastrear quais mudanças foram feitas

em uma base de código, possibilitando que diversas pessoas trabalhem no mesmo código com facilidade. Permite ver quem fez quais alterações e evita conflitos se duas pessoas estiverem trabalhando na mesma linha de código. Significa também que, se alguém fizer uma alteração que cause um bug ou se você cometer um erro, é fácil voltar à versão anterior do código.

Geralmente, os sistemas de controle de versão são distribuídos: uma cópia do código é armazenada na máquina de cada desenvolvedor e também salva em um local central. Todo o código deve ser copiado, caso algo ruim aconteça: você pode perder seu notebook, sua instância remota pode ser corrompida e assim por diante.

O Git (*https://git-scm.com*) é o sistema mais popular para controle distribuído de versão. Trata-se de um projeto open-source, criado em 2005 por Linus Torvalds (que também criou o Linux). Segundo a Pesquisa de Desenvolvedores do Stack Overflow de 2022 (*https://oreil.ly/zMvyl*), 96% dos desenvolvedores profissionais usam o Git, embora também existam outros sistemas de controle de versão, como o Subversion (*https://subversion.apache.org*) e o Mercurial (*https://www.mercurial-scm.org*).

O GitHub (*https://github.com*), de propriedade da Microsoft, é o site mais popular para armazenar código rastreado usando o Git, mas não existe um vínculo oficial entre o GitHub e o Git. Outros sites semelhantes incluem o GitLab (*https://about.gitlab.com*) e o Bitbucket (*https://bitbucket.org/product*).

Sistemas de controle de versão projetados para código não são desenvolvidos para fazer backup e controle de versão de seus dados. É melhor usar um sistema separado e dedicado para seus dados. Sistemas de controle de versão de código não são desenvolvidos para trabalhar com arquivos grandes, e armazenar dados também pode levar à duplicação desnecessária ou a riscos de segurança.

Apesar de o Git ter muitos recursos, apresentarei uma visão geral daqueles mais importantes. Caso queira saber mais, consulte a documentação do Git (*https://oreil.ly/AcZrR*) ou a documentação do GitHub (*https://docs.github.com*). Para dicas úteis, confira o site Dangit, Git (*https://dangitgit.com*).

Como o Git funciona?

O Git funciona baseado no fato de que cada projeto em que estamos trabalhando é armazenado em seu próprio diretório, e podemos acompanhar todas as mudanças do código nesse diretório. Os dados subjacentes armazenados pelo Git são um snapshot do estado de todos os arquivos e da estrutura de pastas do diretório que estamos rastreando. Os snapshots são atualizados à medida que alteramos o código. Isso significa ter um histórico de todas as mudanças que você (ou outra pessoa) fez no código.

Um repositório (ou repo, abreviadamente) é um diretório em que o Git controla as mudanças. O primeiro passo para usar o Git é inicializar um repositório local no diretório em que você está armazenando seu código. Isso cria uma pasta *.git* oculta, que contém um banco de dados que armazena todos os snapshots do seu código.

Mesmo que rastreie as mudanças feitas em um arquivo enquanto trabalhamos, o Git não salva o snapshot dessas mudanças. As mudanças ficam apenas em nosso diretório de trabalho. Caso esteja satisfeito com as mudanças feitas e queira salvar as versões dos arquivos alterados, adicione-as a uma área de "preparação [staging]". A área de preparação coleta todas as mudanças que você deseja incluir no próximo snapshot salvo.

O próximo passo é "fazer o commit" dessas mudanças. Fazer um commit salva o snapshot da versão atual dos arquivos em seu repositório local. Isso vale apenas para arquivos que você alterou e adicionou ao commit. Cada commit tem um ID exclusivo (conhecido como hash do commit, criado por meio de uma função hash criptográfica), e isso pode ser usado como referência a ele. Para ver um exemplo desse fluxo de trabalho, consulte a seção "Acompanhando mudanças e commit", na página 182.

Um repositório local armazena o estado do código do projeto no computador, mas também é possível ter um repositório remoto. É uma cópia do código do seu projeto em um servidor central, como o GitHub. Para adicionar seu commit a um repositório remoto, você pode "enviá-lo (fazer o push)" a partir de seu repositório local. Se outra pessoa também estiver trabalhando no projeto, você pode "extrair (fazer o pull)" as alterações a partir do repositório remoto. Isso atualizará os arquivos em seu computador com as alterações.

A Figura 10.1 mostra como as etapas de adicionar, fazer commit, fazer o push e fazer o pull de um código se encaixam.

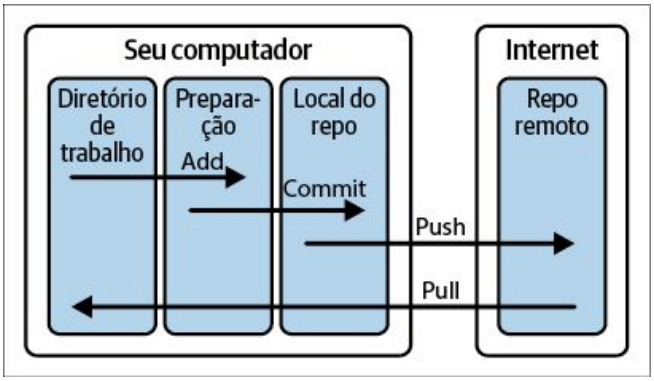


Figura 10.1: Princípios do Git.

Acompanhando mudanças e commit

Vejamos um exemplo de como o Git funciona. É possível instalar o Git seguindo as etapas da documentação do Git (*https://oreil.ly/rDMik*).

Caso esteja trabalhando em um projeto e queira que o Git rastreie as mudanças, execute o seguinte comando no nível superior do diretório do projeto:

```
$ git init
```

Em seguida, adicione os arquivos que alterou à área de preparação:

```
$ git add README.md
```

A qualquer momento, é possível usar o comando `git status` para verificar o que está acontecendo:

```
$ git status

On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   README.md
```

Essa mensagem informa que você está trabalhando na branch `main` [branch

principal]. Abordarei as branches na seção “Branches e pull requests”, na página 185.

Em seguida, faça o commit de suas alterações no repositório local:

```
$ git commit -m "Initial commit"
```

Você verá uma mensagem como esta:

```
[main (root-commit) b25021e] Initial commit
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 README.md
```

Commits têm um ID exclusivo, nome do autor, timestamp e uma mensagem que você fornece para informar qual foi a mudança. As mensagens de commit devem ser informativas e fornecer uma breve descrição da mudança feita.

Agora, vejamos outro commit. Após escrever algum código em seu repositório e chegar a um ponto em que queira salvar as alterações, execute o comando `add` novamente para adicionar os arquivos que alterou à área de preparação:

```
$ git add api_functions.py
```

Em seguida, faça o commit de suas mudanças com o seguinte comando:

```
$ git commit -m "Created helper functions for API"
```

Você verá uma mensagem como esta:

```
[main b78faad] Created helper functions for API
1 file changed, 19 insertions(+)
create mode 100644 api_functions.py
```

É possível recorrer a boas práticas para commits. Cada commit deve ser uma única ação, por exemplo, corrigir um problema ou adicionar uma simples feature. É melhor dividir suas mudanças em pequenas partes. Desse modo, se você fizer uma mudança e ela causar um problema posteriormente, fica mais fácil rastrear o problema até a mudança que o causou. Além disso, é boa ideia executar testes antes de fazer um commit. No Capítulo 12, explicarei como automatizar esse processo.

Remoto e Local

Até agora, expliquei como usar o Git com um repositório local em seu computador. Nesta seção, explicarei como vincular seu repositório local a

um repositório remoto e como fazer o download de um repositório remoto em seu computador.

Após seguir as etapas da seção anterior “Acompanhando mudanças e commit”, a próxima etapa é criar um repositório remoto. As etapas exatas dependerão do local em que seu repositório remoto está hospedado. A Figura 10.2 mostra como isso funciona no GitHub (*https://github.com*).

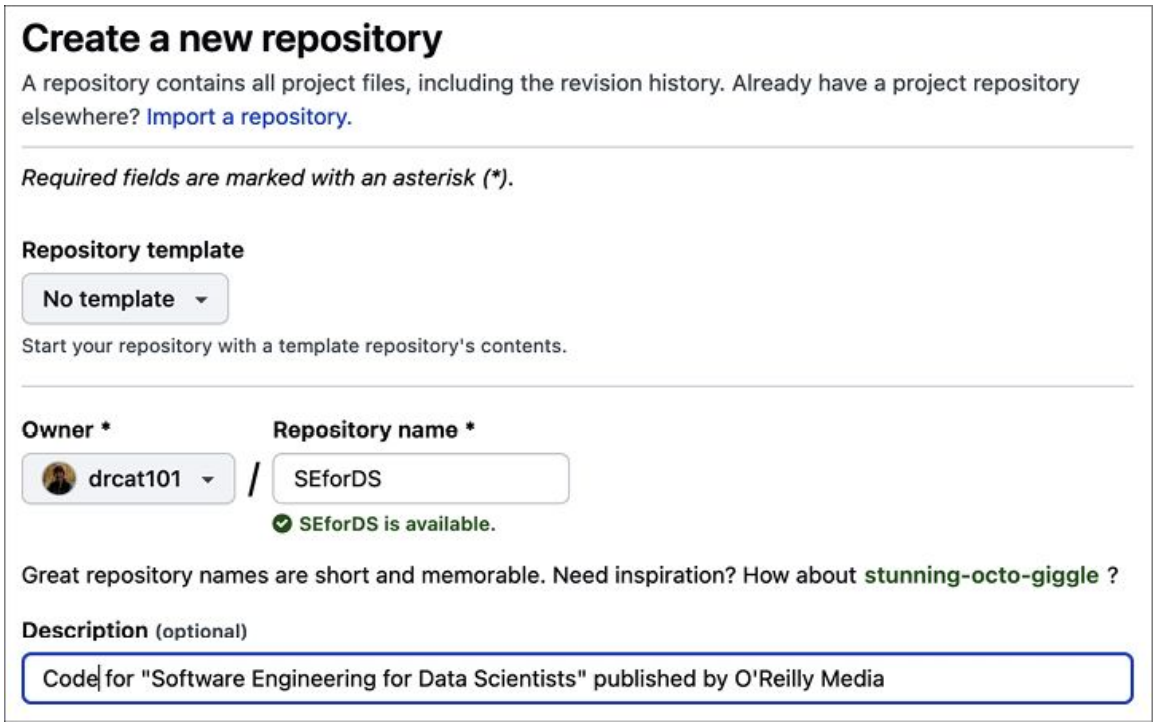


Figura 10.2: Criando um repositório novo no GitHub.

Em seguida, é necessário informar à sua versão local do Git em que local o repositório remoto está localizado, executando o seguinte comando:

```
$ git remote add origin https://github.com/drcat101/SEforDS.git
```

Substitua o URL desse link pelo URL do seu repositório remoto.

Por último, é possível fazer o push do commit do seu repositório local para o repositório remoto:

```
$ git push -u origin main
```

Você verá uma mensagem como esta:

```
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 219 bytes | 219.00 KiB/s, done.
```

```
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/drcat101/SEforDS.git
* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Após executar esse comando, o repositório remoto armazenará os arquivos em seu repositório local, atualizados com as alterações do seu commit.

Talvez você queira começar a trabalhar em um projeto que já tenha algum código existente em um repositório remoto. Nessa situação, é necessário “cloná-lo”. Significa que o download do repositório remoto é feito em seu computador e ele se torna um novo repositório local.

É possível clonar um repositório usando o seguinte comando:

```
$ git clone https://github.com/drcat101/SEforDS.git
```

Existem duas opções de clonagem: HTTPS e SSH. O SSH fornece um nível maior de segurança, mas leva mais tempo para ser configurado.

É possível também começar a trabalhar em um projeto criando primeiro um repositório remoto, clonando o repositório em sua máquina local e, em seguida, começando a escrever o código.

Na próxima seção, explicarei alguns recursos do Git que são extremamente úteis para a colaboração: branches e pull requests.

Branches e pull requests

No Git, as branches permitem isolar as alterações que você está fazendo e testar features novas sem afetar a versão principal do código, ao mesmo tempo em que rastreiam as alterações realizadas. Isso ajuda bastante quando muitas pessoas estão trabalhando na mesma base de código, pois você pode trabalhar no mesmo código sem sobrepô-lo.

Criar uma branch não cria uma cópia dos arquivos que contêm seu código. Em vez disso, o Git rastreia um conjunto de alterações introduzidas a partir de um determinado ponto na linha do tempo do projeto. Ou seja, podemos trabalhar facilmente em features diferentes e, em seguida, fazer o merge das alterações de volta ao código original.

Todos os repositórios têm uma branch padrão, geralmente chamada de `main`. A Figura 10.3 mostra uma branch principal e uma branch nova.

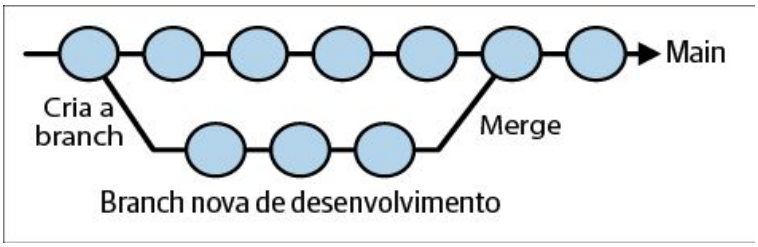


Figura 10.3: Branches no Git — cada círculo representa um commit.

Podemos criar uma branch, adicionar quantos commits quisermos a essa branch e, em seguida, fazer o merge das alterações na branch principal. Enquanto estiver trabalhando na branch nova, nenhuma das mudanças afetará a branch principal. Nesse ínterim, outra pessoa pode fazer alterações na branch principal e isso não afetará o código em que você está trabalhando.

Usar o nome main para a branch padrão em um repositório é uma convenção, e você pode alterá-lo para o nome que preferir. Infelizmente, ainda podemos encontrar alguns repositórios que usam o nome master para a branch padrão. Isso deve ser evitado, já que a palavra "master" em inglês está vinculada historicamente à escravização.

Para criar uma branch nova chamada `new_branch`, execute o seguinte comando:

```
$ git branch new_branch
```

Em seguida, saia do local em que está trabalhando, e acesse essa branch com este comando:

```
$ git checkout new_branch
```

É possível combinar os dois comandos anteriores usando o comando `git checkout` com a flag `-b`:

```
$ git checkout -b new_branch
```

Após fazer check-out da branch, quaisquer alterações que você fizer no código em seu computador ocorrerão na nova branch de desenvolvimento `new_branch`, não na branch principal.

Para salvar sua branch nova no repositório remoto, execute o seguinte comando:

```
$ git push origin new_branch
```

Quando terminar de trabalhar na branch nova, você pode fazer o merge dela

com a branch principal. As alterações de código na nova branch serão incorporadas ao código na branch principal.

Antes de efetuar o merge, alterne para a branch principal usando o seguinte comando:

```
$ git checkout main
```

Em seguida, faça o merge da branch:

```
$ git merge new_branch
```

Após fazer o merge da branch, você deve excluir essa branch. O histórico de commit será preservado.

É possível se deparar com conflitos de merge quando fazemos o merge de uma branch. Eles surgem quando você altera uma linha de código em sua branch nova, que também foi alterada na branch principal. Será necessário decidir qual versão dessa linha de código deve ser mantida.

Os notebooks do Jupyter geralmente causam conflitos de merge. Os notebooks são armazenados como formatos JSON, incluindo o conteúdo das saídas da célula. Se forem alterados, também serão rastreados pelo Git.

Existem algumas soluções para isso. Primeiro, é possível limpar manualmente as saídas da célula de seus notebooks, mas isso é algo que podemos esquecer com facilidade. No Capítulo 12, mostrarei como fazer isso automaticamente. Segundo, podemos usar o nbdime (*https://oreil.ly/cV563*), que configura o Git para comparar apenas o código em notebooks, e não nas saídas das células. Ou, terceiro, você pode usar o Jupytext (*https://oreil.ly/O0yH7*) para converter seus notebooks em arquivos *.py* ou *.md*, que podem ser facilmente rastreados pelo Git.

Caso queira que outra pessoa revise seu código antes de fazer o merge da branch, é possível usar uma pull request. Isso solicita que outro desenvolvedor examine as alterações feitas e, em seguida, você pode discuti-las e analisá-las antes de fazer o merge delas.

A forma mais fácil de abrir uma pull request é usar a interface no GitHub, conforme mostrado na Figura 10.4.

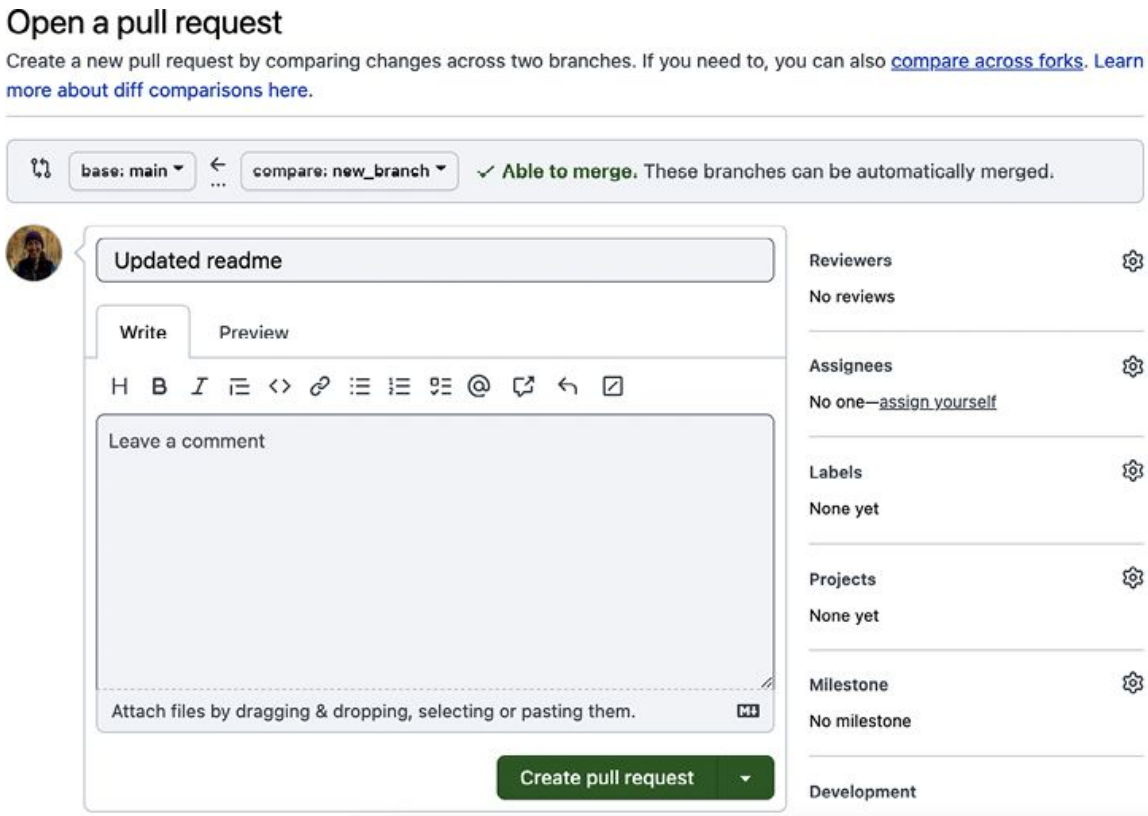


Figura 10.4: Abrindo uma pull request no GitHub.

Uma pull request deve incluir comentários que descrevam o que você está alterando e por que essas alterações devem ser feitas. Isso ajudará o revisor a saber o foco da discussão.

Code reviews

Muitas equipes de desenvolvimento e empresas têm um sistema de code reviews [revisões de código]. São sessões estruturadas, um tanto formais, para analisar o código, geralmente no formato de pull requests. A prática de code reviews tem como objetivo aprimorar o código antes que ele entre em produção, fornecendo a outros membros da equipe a chance de revisá-lo primeiro e identificar bugs e riscos de segurança antes que causem problemas. É também uma oportunidade para compartilhar ideias, aprimorar a estrutura do código, além de possibilitar que desenvolvedores mais experientes transmitam seus conhecimentos. Além disso, as code reviews registram quais decisões foram tomadas e por quê.

As melhores práticas para code reviews incluem:

- Considerar diversos aspectos da qualidade do código, como legibilidade, simplicidade e modularidade.
- Garantir que todos sejam ouvidos. A experiência de cada um é valiosa e os membros juniores devem ter a chance de fazer perguntas.
- Destacar os pontos positivos do código em análise, não somente apontar problemas.

- Limitar o tempo dedicado à code review. Code reviews não devem ocasionar atrasos.

Esta postagem do blog de Tim Hopper (*https://oreil.ly/hn_xv*) fornece mais algumas boas sugestões para code reviews de ciência de dados.

Existem outros comandos e técnicas que você precisará conhecer para usar o Git com sucesso, incluindo como ignorar arquivos e como desfazer as alterações feitas. Como não abordarei todos os detalhes neste livro, recomendo que consulte os links que forneci na seção "Controle de versão usando o Git", na página 180.

Nesta seção, apresentei uma breve visão geral de como usar o Git para controle de versão. Em seguida, abordarei algumas especificidades da programação em Python: gerenciar dependências e usar ambientes virtuais.

Dependências e ambientes virtuais

Para que outras pessoas possam usar seu código, elas precisarão instalar as mesmas bibliotecas que você usou para escrevê-lo. Uma dependência é uma biblioteca de terceiros que você instalou e da qual seu código precisa para funcionar, por exemplo, pandas ou NumPy. Seu código pode funcionar apenas com uma versão específica de uma biblioteca ou com uma variedade de versões. Dito isso, para tornar seu código reproduzível, será necessário incluir informações sobre as versões das bibliotecas das quais seu código depende (suas dependências).

Versionamento

As bibliotecas Python (e, na verdade, a maioria dos pacotes de software) têm números de versão para que saibamos qual conjunto de código corresponde a uma determinada versão. A melhor prática é adicionar versionamento aos seus projetos, e existe uma série de esquemas de versionamento diferentes. A especificação Semantic Versioning (SemVer) (*https://semver.org*) é uma das mais populares. Talvez você também queira conferir o Calendar Versioning (*https://calver.org*).

Se um projeto estiver usando o SemVer, você verá dois ou três números separados por pontos, como Python 3.10 ou NumPy 1.24.3. O primeiro número significa uma versão principal, o segundo significa uma versão secundária e o terceiro é um patch.

Uma versão principal significa que existem alterações significativas (alguns recursos que funcionavam anteriormente foram alterados e não funcionam mais), como Python 2.x para Python 3.x ou pandas 1.x para 2.x. Uma versão secundária significa que existem atualizações menos significativas; pode abarcar alguns recursos novos, mas não existem alterações significativas. Um patch significa uma alteração menos significativa, como corrigir alguns bugs mais simples.

Nesta seção, indicarei as ferramentas para gerenciar dependências, de modo que não seja necessário acompanhar manualmente todas as versões das bibliotecas que você está usando. Para usá-las, é necessário primeiro conhecer os ambientes virtuais.

Ambientes virtuais

Ambientes virtuais são um conceito importante em Python, e, mesmo que já esteja familiarizado com eles, apresentarei uma breve visão geral. Um ambiente virtual é uma forma de isolar as bibliotecas instaladas para um projeto específico, em vez de instalá-las no ambiente Python de todo o sistema. Isso significa que o interpretador Python e os módulos incluídos na biblioteca padrão do Python são compartilhados por todo o sistema. Ou seja, é possível instalar quaisquer bibliotecas adicionais em um espaço separado para cada projeto.

Recomendo que não instale bibliotecas no ambiente Python de todo o sistema (root). Se fizer isso, provavelmente descobrirá que fica difícil testar ou depurar seus projetos, visto que projetos diferentes exigem versões distintas da mesma biblioteca. Além disso, o sistema operacional do seu computador tem pacotes de ambiente para todo o sistema Python. Caso instale ou desinstale bibliotecas sem usar o gerenciador de pacotes do sistema (como apt ou brew), existe uma possibilidade efetiva de quebrar o sistema ou comprometer alguns comportamentos.

Ambientes virtuais permitem que diferentes projetos tenham versões distintas de uma mesma biblioteca, sem conflitos. Para exemplificar, o Projeto A pode precisar usar recursos introduzidos no NumPy versão 1.25, enquanto o Projeto B utiliza a sintaxe de uma versão anterior. Se você atualizar o NumPy em todo o sistema, o Projeto B não funcionará mais. Ambientes virtuais resolvem esse problema, possibilitando que você tenha múltiplas versões instaladas da mesma biblioteca em seu sistema e ao mesmo tempo.

Existem muitas ferramentas diferentes para gerenciar ambientes virtuais em Python. Vejamos algumas das mais populares:

- venv (*https://oreil.ly/QIEQc*) está incluído na biblioteca padrão do Python.
- O virtualenv (*https://oreil.ly/PMBoK*) é um projeto mais antigo que o venv e contém mais recursos. O venv é um subconjunto do virtualenv.

- O Pyenv (https://oreil.ly/nnvwr) gerencia versões do Python, possibilitando ter múltiplas versões instaladas em seu sistema e ao mesmo tempo (é possível usar o Python 3.7 e o Python 3.10 para projetos diferentes, por exemplo). O plug-in pyenv-virtualenv (https://oreil.ly/tZ8Xv) também permite gerenciar ambientes virtuais.
- O Conda (https://oreil.ly/VrIMo) gerencia ambientes virtuais e também instala pacotes. Ele faz download de pacotes de um repositório separado de pacotes, gerenciado pelo Anaconda (https://www.anaconda.com) em vez do PyPI (o Python Packaging Index) (https://pypi.org), o repositório de pacotes padrão do Python.
- O Poetry (https://python-poetry.org), o PDM (https://pdm.fming.dev) e o Hatch (https://hatch.pypa.io) também gerenciam dependências e ambientes virtuais. Na próxima seção, abordarei o Poetry em mais detalhes.

Apesar de encontrar todos os detalhes de como usar todas essas ferramentas nos links anteriores, vejamos um exemplo rápido de como usar o venv.

Antes de instalar qualquer pacote, crie um ambiente virtual novo com o seguinte comando no macOS ou Linux:

```
$ python -m venv SEforDS
```

Você pode substituir SEforDS pelo nome do seu ambiente virtual.

Em seguida, ative o ambiente virtual com o seguinte comando:

```
$ source SEforDS/bin/activate
```

O nome do seu ambiente virtual aparecerá entre parênteses antes do prompt de comando, assim:

```
(SEforDS)$
```

Isso é comum a qualquer uma das ferramentas de ambiente virtual citadas anteriormente. Agora, é possível instalar pacotes em seu ambiente virtual com o pip.

Quando terminar de instalar o ambiente virtual, é possível sair dele com este comando:

```
(SEforDS)$ deactivate
```

O nome do seu ambiente virtual desaparecerá do prompt de comando.

A prática recomendada é criar um ambiente virtual novo sempre que você

iniciar um projeto novo, ativá-lo e, em seguida, instalar as bibliotecas necessárias para esse projeto. Mantenha a instalação do seu sistema Python "limpa" e não instale nenhuma biblioteca de terceiros em todo o sistema. Isso evitará conflitos entre versões da biblioteca.

Gerenciando dependências com o pip

A forma mais simples de registrar as dependências exigidas pelo seu projeto é usar o pip, o gerenciador de pacotes integrado ao Python. É possível salvar uma lista de todos os pacotes necessários para o seu projeto com o comando pip freeze:

```
$ python -m pip freeze > requirements.txt
```

Ele grava todos os pacotes no ambiente em um arquivo de texto chamado *requirements.txt*. Será necessário usar um ambiente virtual para o seu projeto e ativá-lo ao executar esse comando; caso contrário, ele registrará as bibliotecas a partir da instalação do Python do seu sistema.

O resultado é um arquivo de texto parecido com este:

```
numpy==1.25.2
pandas==2.0.3
python-dateutil==2.8.2
pytz==2023.3
six==1.16.0
tzdata==2023.3
```

Ele contém as versões das bibliotecas atualmente instaladas em seu ambiente e as bibliotecas das quais elas dependem (por exemplo, a biblioteca dateutil do Python é uma dependência do pandas).

É possível carregar o arquivo *requirements.txt* no repositório do projeto no sistema de controle de versão. Desse modo, quando alguém quiser usar seu projeto ou você quiser executá-lo em um ambiente diferente, é possível usar o seguinte comando para instalar as dependências do seu projeto:

```
$ pip install -r requirements.txt
```

Mesmo sendo simples, esse processo tem algumas desvantagens. Por exemplo, se remover uma biblioteca dos requisitos do seu projeto (por exemplo, o pandas), precisará identificar quais outras bibliotecas são suas dependências e removê-las manualmente. O arquivo *requirements.txt* não

inclui nenhuma informação sobre a versão do Python. Além disso, pode haver alterações nas bibliotecas que são dependências das bibliotecas que você está usando (as subdependências do projeto). Caso esteja usando o pip para gerenciar suas dependências, precisará lidar com isso manualmente. Felizmente, outros gerenciadores de dependência farão isso por você, como explicarei na próxima seção.

Gerenciando dependências com o Poetry

Os gerenciadores de dependências, como o Poetry (*https://python-poetry.org*), o PDM (*https://pdm.fming.dev*) e o Hatch (*https://hatch.pypa.io*), facilitam bastante o gerenciamento das dependências do seu projeto em comparação ao uso do pip. Nesta seção, explicarei como gerenciar as dependências do seu projeto usando o Poetry. Isso ajuda bastante se você também estiver usando o Poetry para empacotamento, tópico que abordarei mais adiante neste capítulo.

Em vez do arquivo *requirements.txt*, o Poetry usa um arquivo chamado *pyproject.toml*. Esse arquivo é o padrão para tornar os pacotes do Python instaláveis. Explicarei os detalhes na seção “Empacotamento do Python”, na página 195. O arquivo *pyproject.toml* contém todos os requisitos do projeto, incluindo suas dependências.

É possível instalar o Poetry com o seguinte comando:

```
$ curl -sSL https://install.python-poetry.org | python3 -
```

Recomenda-se instalar o Poetry em todo o sistema, e não em um ambiente virtual, para que funcione em todos os seus ambientes virtuais. Siga as instruções no seu terminal para adicionar o Poetry ao seu shell.

Se estiver iniciando um projeto novo, poderá solicitar ao Poetry que defina uma pasta nova, preenchida com os arquivos e pastas necessários para começar. É possível fazer isso com o seguinte comando:

```
$ poetry new SE_for_DS
```

Caso já tenha começado a escrever código, é possível usar este comando para inicializar o Poetry na pasta existente:

```
$ poetry init
```

Após executar qualquer um desses comandos, você verá que a pasta do seu projeto contém arquivos chamados *pyproject.toml* e *poetry.lock*. O arquivo

pyproject.toml contém uma seção que lista as dependências do seu projeto. Se analisarmos o conteúdo do arquivo antes de instalarmos bibliotecas de terceiros, veremos esta seção sobre dependências:

```
[tool.poetry.dependencies]
python = "^3.10"
```

É possível atualizar o arquivo *pyproject.toml* manualmente ou usar o comando `poetry add` a fim de adicionar as bibliotecas necessárias ao seu projeto. Isso também instalará a biblioteca no ambiente virtual do projeto.

```
$ poetry add pandas
```

Se analisarmos o conteúdo do arquivo *pyproject.toml*, podemos verificar que o pandas foi adicionado às dependências:

```
[tool.poetry.dependencies]
python = "^3.10"
pandas = "^2.1.0"
```

Já que a versão do pandas é especificada como `"^2.1.0"`, isso significa que a versão 2.1.0 ou a mais recente é aceitável nesse projeto. Por padrão, o Poetry assume que a versão atual de um pacote e todas as versões futuras funcionarão com seu projeto. Caso contrário, é necessário alterar a especificação da versão.

Um shell aninhado é a maneira padrão de usar um ambiente virtual com o Poetry. Para ativar um ambiente virtual em um diretório que contém um arquivo *pyproject.toml*, use o seguinte comando:

```
$ poetry shell
```

Em seguida, para desativar o ambiente virtual, digite `exit`.

Além de listar as versões necessárias da biblioteca no arquivo *pyproject.toml*, o Poetry também usa o arquivo *poetry.lock* a fim de armazenar hashes para versões exatas das bibliotecas do seu projeto. Isso ajuda a garantir que seu projeto funcione em diferentes sistemas operacionais. Cada vez que você instala uma biblioteca, seu arquivo *poetry.lock* é atualizado e deve se parecer com:

```
# This file is automatically @generated by Poetry 1.6.1 and should not be changed$


[[package]]
name = "numpy"
```

```
version = "1.25.2"
description = "Fundamental package for array computing in Python"
optional = false
python-versions = ">=3.9"
files = [
    {file = "numpy-1.25.2-cp310-cp310-macosx_10_9_x86_64.whl",
    hash = "sha256:db3$
    {file = "numpy-1.25.2-cp310-cp310-macosx_11_0_arm64.whl",
    hash = "sha256:9031$
...
```

Em seguida, é possível carregar esse arquivo no sistema de controle de versão.

Para instalar as dependências do seu projeto em outro ambiente, use o seguinte comando:

```
$ poetry install
```

O Poetry procurará o arquivo *poetry.lock* a fim de verificar as versões exatas da biblioteca e instalá-las no ambiente novo. Se não existir um arquivo *.lock*, o Poetry instalará apenas as versões mais recentes das bibliotecas especificadas no arquivo *pyproject.toml*.

Independentemente de usar o `pip` ou o Poetry, é fundamental manter as dependências do seu projeto atualizadas para que outras pessoas possam usar facilmente seu código.

Empacotamento do Python

Se quiser facilitar o uso do seu código para outras pessoas, você pode transformá-lo em um pacote. Isso significa que as pessoas podem instalá-lo em seu ambiente Python e importá-lo para qualquer projeto em que estejam trabalhando, da mesma forma que você usa o NumPy ou o pandas. Nesta seção, descreverei o processo de transformar o código Python em um pacote.

Um dos pontos mais fortes do Python é seu enorme ecossistema de pacotes, e o PyPI (o Python Package Index) (*https://pypi.org*) viabiliza isso. O PyPI hospeda um número gigantesco de pacotes a fim de que estejam disponíveis para download. No momento em que eu escrevia este livro, havia 473.661

projetos no PyPI!

Caso queira compartilhar seu código publicamente, é possível enviar um pacote para o PyPI e, em seguida, qualquer pessoa pode instalá-lo com o pip e usá-lo em seus projetos. Mas você também pode criar um pacote e compartilhar os arquivos do pacote dentro da sua empresa, de modo que seus colegas de equipe possam instalá-lo e utilizá-lo. Um pacote garante que seu código seja completamente reproduzível.

Ao criar um pacote e torná-lo público, você tem a responsabilidade de mantê-lo (ou, pelo menos, facilitar que outras pessoas o adicionem). Se outras pessoas começarem a depender do seu código, elas devem ajudar a mantê-lo, mas o ideal é que você, como criador, colabore com elas para atualizar o pacote.

Uma breve observação sobre a terminologia: um módulo Python é um arquivo *.py* . Um pacote Python pode ter dois significados: o primeiro é um diretório com um arquivo *__init__.py* que contém outros módulos. *__init__.py* é um arquivo Python que deve estar presente a fim de permitir a importação de módulos a partir desse diretório. O segundo é um pacote de distribuição que possibilita distribuir código via PyPI e instalar o módulo/biblioteca. Nesta seção, estou usando o segundo significado da palavra “pacote”.

No restante desta seção, explicarei como criar um pacote e enviá-lo para o PyPI.

Conceitos básicos de empacotamento

Antes de começar a criar um pacote, garanta que seu código atenda aos seguintes critérios para que os usuários do pacote tenham uma boa experiência:

- Satisfaça ao design que você planejou.
- Seja totalmente funcional.
- Esteja adequadamente formatado (consulte o Capítulo 6).
- Tenha sido testado (confira o Capítulo 7).
- Tenha uma boa documentação (consulte o Capítulo 9).

Além disso, será necessário inserir o código que deseja transformar em um pacote dentro de uma única pasta. Nomeie essa pasta com o nome desejado

para o seu pacote e mantenha-a separada dos testes e da documentação associados a ela. Não raro, você verá essa pasta em uma pasta `src` separada, como na seguinte estrutura de arquivo de exemplo:

```
SE_for_DS
├── LICENSE
├── README.md
├── pyproject.toml
├── src
│   └── SE_for_DS
│       ├── __init__.py
│       ├── functions.py
│       └── ...
└── tests
|   └── ...
├── docs
    └── ...
```

Nessa estrutura de arquivo, o pacote será nomeado como `SE_for_DS`. Temos também um arquivo de licença, um arquivo *README.md* com uma visão geral da documentação e um arquivo *pyproject.toml*, que explicarei mais adiante.

O empacotamento envolve as seguintes etapas (adaptadas da documentação do PyPI (*https://oreil.ly/4OwdD*)):

1. Preparar um arquivo de configuração que contenha metadados sobre o projeto e instruções para uma ferramenta de criação de pacotes. O padrão é usar um arquivo *pyproject.toml* que mencionei na seção "Gerenciando dependências com o Poetry", na página 192, e que explicarei completamente na próxima seção.
2. Usar uma ferramenta que leia seu código-fonte e arquivo de configuração para transformá-lo em um arquivo de pacote (ou um artefato de compilação). Isso é o que o torna instalável no sistema de outra pessoa.
3. Fazer o upload de seu artefato de compilação no PyPI ou em algum outro serviço de distribuição, que pode ser interno à sua empresa.

Após executar essas etapas, um usuário do seu pacote pode fazer o

download dele, instalá-lo (com o pip, por exemplo) e usá-lo. Para mais detalhes sobre o assunto, confira o livro Hypermodern *Python Tooling* [Ferramentas Hipermodernas do Python, em tradução livre], de Claudio Jolowicz (O'Reilly, 2024).

Embora eu não tenha mencionado o empacotamento para o sistema de distribuição de pacotes Anaconda, é possível encontrar mais detalhes sobre ele na documentação do Conda (*https://oreil.ly/h_byJ*). A Anaconda disponibiliza seus pacotes gratuitamente, mas eles só podem ser atualizados por essa organização.

O projeto conda-forge (*https://conda-forge.org*) é uma comunidade open-source que possibilita fazer o download de pacotes Python usando o Conda, e os mantenedores de pacotes conseguem atualizar os próprios pacotes.

pyproject.toml

Um arquivo *pyproject.toml* é um arquivo padronizado que contém os metadados sobre seu projeto e as especificações de como criá-lo. *.toml* é um formato de arquivo para arquivos de configuração e sua especificação é open-source. É possível escrever esse arquivo manualmente ou gerá-lo com uma ferramenta como o Poetry. Vejamos um exemplo de um arquivo *pyproject.toml*:

```
[build-system] ❶
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project] ❷
name = "se_for_ds"
version = "0.0.1"
authors = [
  { name="Catherine Nelson", email="email_address" },
]
description = "An example package for Software Engineering
for Data Scientists"
readme = "README.md"
requires-python = ">=3.9"
classifiers = [
    "Programming Language :: Python :: 3",
```

```
    "License :: OSI Approved :: MIT License",
    "Operating System :: OS Independent",
]

[project.urls]
"Homepage" = "https://github.com/pypa/sampleproject"
```

❶ O bloco build-system especifica uma ferramenta para criar o pacote; nesse caso, setuptools, que abordarei na próxima seção.

❷ O bloco project contém todos os metadados do projeto.

Se usar o Poetry, ele gerará um arquivo *pyproject.toml*, que contém os requisitos, mais ou menos assim:

```
[tool.poetry]
name = "se-for-ds"
version = "0.1.0"
description = ""
authors = ["Catherine Nelson <email_address>"]
readme = "README.md"

[tool.poetry.dependencies]
python = "^3.10"
pandas = "^2.1.0"

[build-system]
requires = ["poetry-core"]
build-backend = "poetry.core.masonry.api"
```

Após gerar seu arquivo *pyproject.toml*, você está pronto para criar o pacote.

Criando e fazendo upload de pacotes

Para criar seu pacote Python, você precisará de uma ferramenta de compilação. O setuptools (*https://oreil.ly/6IuGH*) é um meio consolidado e popular de criar pacotes Python, mas, se estiver fazendo coisas mais complexas com o empacotamento, considere outras ferramentas, como o Poetry, o pdm e o Hatch, que mencionei na seção “Ambientes virtuais”, na página 189. Essas ferramentas também facilitam sua vida, gerenciando outras partes do seu fluxo de trabalho.

Por padrão, a maioria das ferramentas de compilação gerará dois arquivos: um arquivo *sdist* e um arquivo *wheel*. O arquivo *sdist* é um arquivo compactado que contém o código-fonte do seu pacote. O arquivo *wheel* também é um tipo de arquivo compactado, mas pode ser instalado em um ambiente Python com o pip.

Será necessário definir a ferramenta de compilação que deseja usar no seu arquivo *pyproject.toml*, conforme mencionado anteriormente. Em seguida, você pode usar a biblioteca Build (*https://oreil.ly/G3S0v*) para criar seu pacote e ela usará o setuptools ou outra ferramenta de empacotamento especificada.

É possível instalar o Build com este comando:

```
$ pip install build
```

Depois disso, execute o seguinte comando no diretório em que seu arquivo *pyproject.toml* está localizado:

```
python3 -m build
```

Se você usar o primeiro arquivo *pyproject.toml* da seção anterior, o build usará o setuptools para criar o pacote. Você verá esta mensagem, mostrando que os arquivos do pacote foram criados:

```
Successfully built se_for_ds-0.1.0.tar.gz
and se_for_ds-0.1.0-py3-none-any.whl
```

Em seguida, é possível compartilhar o arquivo wheel, e outras pessoas podem instalá-lo em seu ambiente Python com o pip.

Caso queira publicar seu pacote no PyPI, será necessário primeiro instalar a biblioteca Twine com o seguinte comando:

```
$ pip install twine
```

O Twine gerencia o processo de upload do seu pacote para o PyPI.

A documentação do PyPI (*https://oreil.ly/bkm9Z*) recomenda que você primeiro envie seu pacote para o TestPyPI (*https://test.pypi.org*) a fim de verificar se tudo funciona corretamente antes de fazer o upload definitivo dele para o PyPI. Depois de se registrar em uma conta no TestPyPI e obter uma chave de API, execute o seguinte comando para carregar seu pacote:

```
twine upload -r testpypi dist/*
```

Para fazer o upload no PyPI, você precisará se registrar em uma conta e

obter uma chave de API. Após isso, use o Twine para fazer o upload de seu pacote:

```
twine upload dist/*
```

Caso esteja usando o Poetry como seu sistema de compilação, não será necessário instalar nenhum pacote extra. É possível criar seu pacote executando o seguinte comando no diretório que contém seu arquivo *pyproject.toml*:

```
$ poetry build
```

Faça o upload do seu pacote para o PyPI com este comando:

```
$ poetry publish
```

Seu pacote ficará disponível para qualquer pessoa o instalar no próprio ambiente Python.

Aspectos fundamentais

Neste capítulo, abordei diversas ferramentas que serão extremamente úteis quando você estiver compartilhando seu código e trabalhando com outros desenvolvedores. O controle de versão é uma habilidade essencial para escrever código, pois possibilita rastrear as alterações em seu código, fazer backup do código para um local central e colaborar facilmente com outras pessoas.

O Git é a ferramenta mais popular para controle distribuído de versão. É fundamental que você aprenda os comandos básicos para configurar um repositório local e remoto, como fazer o commit de seu código e como trabalhar com branches e pull requests.

Ao compartilhar o código que escreveu, você também precisará compartilhar detalhes das versões das bibliotecas das quais ele depende. E, para isso, existem inúmeras ferramentas, e é mais fácil usá-las se você já estiver trabalhando em um ambiente virtual. É possível usar o `pip` para salvar uma lista de versões de bibliotecas ou ferramentas como o Poetry, que gerencia as dependências para você.

Criar um pacote a partir do seu código Python possibilita que outras pessoas o instalem nos próprios ambientes Python e o importem para qualquer projeto em que estejam trabalhando. Primeiro, você precisará garantir que

seu código seja executado e esteja estruturado corretamente. Em seguida, crie um arquivo *pyproject.toml* que defina a ferramenta de compilação e contenha os metadados do seu projeto. Por último, crie os arquivos do pacote e disponibilize-os no PyPI ou compartilhe-os na sua empresa.

No próximo capítulo, explicarei outra forma de disponibilizar seu código para outras pessoas: criar uma API.

CAPÍTULO 11

APIs

As APIs (Interfaces de Programação de Aplicações) são parte fundamental da engenharia de software moderna. Conhecer como obter dados de uma API e seus conceitos básicos será extremamente útil para você. APIs permitem que dois sistemas se comuniquem e transfiram dados, podendo ser expostas publicamente na internet ou nos sistemas internos de uma empresa. Elas ocultam os detalhes do seu código por meio de uma maneira padrão de acessar um sistema, fornecendo uma camada útil de abstração. Muitos produtos robustos de softwares empregam APIs para a troca interna de dados, e elas são imprescindíveis no desenvolvimento web.

Uma API possibilita acessar alguns dados programaticamente ou executar algumas ações. Podemos nos deparar com uma API quando tentamos fazer o download de alguns dados a partir de uma API pública. Se você enviar uma requisição com uma estrutura específica, receberá de volta o que espera. APIs são um exemplo de comunicação cliente-servidor. O servidor aguarda algum contato, e o cliente é quem faz o contato. O cliente solicita dados do servidor que, por sua vez, os compartilha. Em geral, APIs web usam HTTP para fazer uma requisição e retornam uma resposta como um arquivo JSON (JavaScript Object Notation [Notação de Objeto JavaScript]) ou XML.

A utilidade das APIs reside no acesso sob demanda a dados ou a funcionalidades, de forma escalável, para um grande número de usuários. Talvez você queira escrever a própria API para que outros sistemas possam chamar seu código sem nenhuma ação sua. Um ótimo exemplo é fazer predições a partir de um modelo de machine learning. Se isso se tornar uma feature no produto da sua empresa, o modelo pode precisar receber dados de entrada e retornar predições várias vezes por segundo. É possível criar uma API que aguarde esses dados e retorne a predição, eliminando a necessidade de executar um script manualmente sempre que for necessário

obter uma predição a partir do modelo. Uma API pode ficar constantemente em execução, aguardando para aceitar dados de entrada.

Neste capítulo, explicarei os detalhes das APIs RESTful, o tipo mais popular de API. Mostrarei como chamar APIs disponíveis publicamente na internet para obter dados e como escrever a própria API usando o FastAPI, um dos frameworks Python mais populares. Isso possibilitará que você compartilhe a funcionalidade criada em seu código.

Chamando uma API

Uma infinidade de dados bastante úteis está disponível publicamente na internet. No decorrer deste livro, usei dados do banco de dados dos Objetivos de Desenvolvimento Sustentável (ODS) da ONU (*https://oreil.ly/Gzqqd*), disponibilizados em uma página, em que podemos fazer manualmente o download dos dados e salvá-los como um arquivo CSV. Mas e se você quiser fazer o download de muitos arquivos parecidos? Pode ser uma tarefa maçante. E se você precisar revisitar o projeto depois de meses? Seria necessário documentar cuidadosa e exatamente os downloads que fez, pois é fácil cometer erros.

Em vez disso, é possível automatizar esse processo e torná-lo reproduzível chamando uma API. Além de obter dados da internet, talvez seja necessário fazer isso para obter dados ou fornecer resultados a um sistema interno.

Os principais tipos de API com que você pode se deparar são RESTful (*https://oreil.ly/zYKfU*), SOAP (*https://oreil.ly/gg-3*) e GraphQL (*https://graphql.org*). SOAP é um protocolo de API mais antigo que você ainda verá de vez em quando, e GraphQL é um tipo mais novo de API, que usa uma linguagem de consulta para retornar dados. REST (REpresentational State Transfer [Interface de Programação de Aplicações de Transferência de Estado Representacional]) é um estilo de arquitetura de software definido por Roy Fielding em 2000. Esse estilo fornece diretrizes sobre como os recursos devem ser representados e manipulados usando métodos HTTP. As APIs RESTful são as mais comuns e, neste capítulo, explicarei como funcionam.

Métodos HTTP e códigos de status

Caso queira usar uma API RESTful, você encontrará alguns métodos HTTP padronizados. Cada método corresponde a uma requisição específica que você pode fazer à API para obter uma resposta específica, conforme mostrado na Tabela 11.1.

Tabela 11.1: Endpoints comuns de API

Nome do endpoint	Função
GET	Acessa alguns dados
POST	Envia alguns dados para criar algo e receber uma resposta
PUT	Envia dados para atualizar algo que já existe
DELETE	Exclui algo que já existe

Uma API pode ter um grande número de endpoints. Por exemplo, pode haver muitos endpoints GET que retornam diferentes tipos de dados. Alguns deles recebem um parâmetro como dados de entrada e retornam dados que correspondem a essa entrada, e alguns retornam os mesmos dados todas as vezes.

Quando recebe uma requisição HTTP, uma API retorna o código padrão de status, dependendo do que aconteceu. A Tabela 11.2 descreve o significado de alguns desses códigos.

Tabela 11.2: Códigos de status HTTP comuns

Código de status	Significado
2xx	A requisição foi bem-sucedida (por exemplo, 200 “OK”)
4xx	Ocorreu um erro de cliente. Você fez algo que o servidor não esperava, por exemplo, digitando incorretamente o path do endpoint. Isso geraria uma resposta 404 (“Not Found”).
5xx	Ocorreu um erro no servidor. Ocorreu um bug no código da API quando você criou uma requisição.

Demonstrei apenas um subconjunto. Caso se depare com outro código de status, basta consultar uma lista mais completa (*https://oreil.ly/QzrJY*).

Nesta seção, dois outros conselhos serão valiosos: cabeçalhos e parâmetros de URL. Cabeçalhos são metadados anexados a uma requisição HTTP, que por sua vez, é anexada a um servidor ou a uma resposta de um servidor. Os

cabeçalhos comuns de requisição incluem credenciais de autorização, e os cabeçalhos comuns de resposta incluem o tipo de dados que o servidor está retornando, como mostrarei na próxima seção. Parâmetros de URL são pares de chave-valor no final de um URL que especificam o que você quer que o URL retorne, e ilustrarei um exemplo na próxima seção.

Obtendo dados da API SDG

Em Python, a forma mais fácil de chamar uma API é usar a biblioteca Requests (*https://oreil.ly/5O4r3*).

É possível instalar a Requests com o seguinte comando:

```
$ pip install requests
```

Mostrarei como usar a biblioteca Requests chamando a API SDG da ONU e obtendo algumas informações.

Mas, primeiro, é uma boa ideia verificar a documentação da API que você pretende usar. Grande parte dessa documentação está em um formato padrão, usando a interface do usuário do Swagger (*https://oreil.ly/Nd48C*).

A Figura 11.1 mostra parte da documentação da API para a API do banco de dados SDG. Podemos ver que existem muitos endpoints GET e POST disponíveis.

Começarei mostrando como chamar um endpoint GET que não exige nenhuma informação adicional. O endpoint `v1/sdg/Goal/List` retorna uma lista de todos os indicadores de desenvolvimento sustentável disponíveis no banco de dados geral.

Com a Requests, basta uma linha de código para criar uma requisição a partir desta API:

```
import requests


response = requests.get("https://unstats.un.org/SDGAPI/v1/sdg/Goal/List")
```

É possível verificar se a requisição foi bem-sucedida observando o código de status HTTP da resposta:

```
>>> response.status_code
... 200
```

Um código `200` mostra que a requisição foi concluída com êxito e os dados

foram retornados.

Com o atributo .headers, é possível analisar os cabeçalhos para descobrir em que formato os dados foram retornados:

```
>>> response.headers["content-type"]
... 'application/json; charset=utf-8'
```

application/json significa que os dados são retornados no formato JSON. Ou seja, você pode usar o método .json() para obter os dados da resposta da API da seguinte forma:

```
>>> response.json()
... [{'code': '1',
        'title': 'End poverty in all its forms everywhere',
        ...
    }]
```

Esse endpoint retornou uma lista de todos os Objetivos de Desenvolvimento Sustentável da ONU e seus códigos. O JSON é um formato de troca de dados amplamente utilizado. Como os dicionários Python, o JSON representa os dados como pares de chave-valor e fornece uma maneira leve e legível por humanos de organizar e trocar dados.

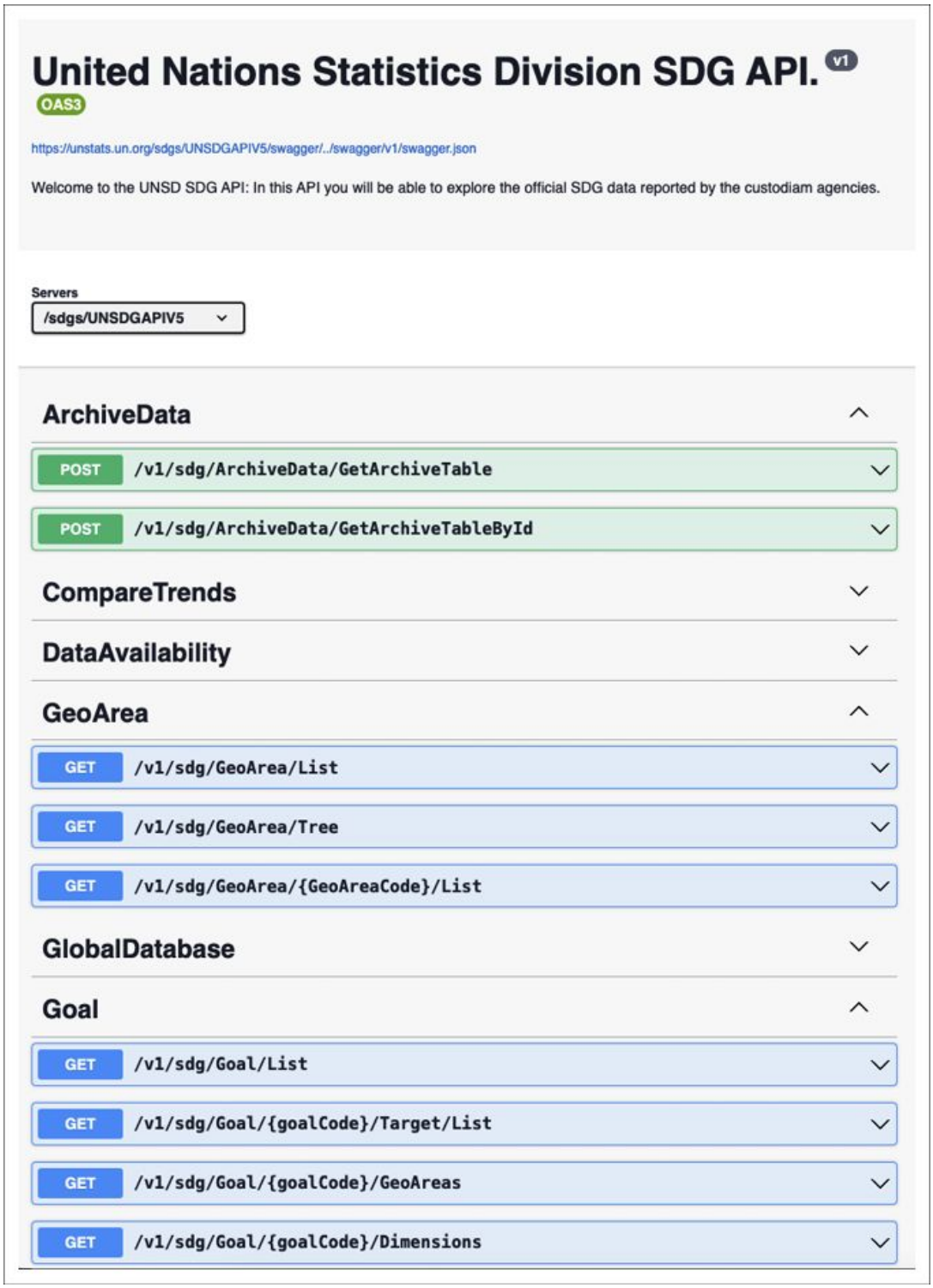


Figura 11.1: Documentação da API do banco de dados SDG da ONU.

Uma requisição GET como essa é o tipo mais simples de requisição para uma API, pois não precisa de parâmetros ou de dados de entrada. Um endpoint GET também pode aceitar alguns parâmetros como entrada, a fim

de retornar uma resposta diferente dependendo da entrada, e esses parâmetros são passados como parte do path do URL.

Em seguida, mostrarei como criar uma requisição GET que aceita um parâmetro path. Usarei a API SDG novamente e obterei alguns detalhes sobre um dos ODSs.

A documentação da API SDG afirma que existe um endpoint GET que fornecerá uma lista de todas as metas específicas associadas a qualquer objetivo em *https://unstats.un.org/sdgapi/v1/sdg/Goal/{goalCode}/Target/List*. O ODS 5 é "Alcançar a igualdade de gênero e empoderar todas as mulheres e meninas", e, se quiser obter informações sobre esse objetivo, insira o código do objetivo (5) no local *{goalCode}* do URL. A documentação também afirma que existe um parâmetro opcional *includeChildren* para obter os detalhes completos das metas e indicadores. Isso vai no final da URL após um?. Para que isso fique mais claro, anotei essas seções na Figura 11.2.

Figura 11.2: URL da API anotada.

Levando tudo em consideração, a requisição para essa API assume esta forma:

```
response = requests.get("https://unstats.un.org/sdgapi/v1/sdg/Goal/5/Target/\
List?includechildren=true")
```

A resposta fornece os detalhes das metas mensuráveis e indicadores estatísticos para o seguinte objetivo:

```
>>> response.json()
... [{...
    'targets': [{'goal': None,
    'code': '5.1',
    'title': 'End all forms of discrimination against all women and girls \
    everywhere'
    ...}]
    }]
```

Na seção “Criando requisições para sua API”, na página 214, abordarei como criar uma requisição POST para uma API.

Criando sua API usando o FastAPI

Nesta seção, mostrarei como criar sua API usando o FastAPI (*https://oreil.ly/krmvR*), um framework para escrever endpoints de API, desenvolvido por Sebastián Ramírez. Lançado em 2018, sua adoção tem sido rápida e ampla devido à sua facilidade de uso e integração com outras ferramentas modernas do Python. Além disso, tem outros recursos úteis, incluindo documentação automática, e está em conformidade com as especificações do OpenAPI (*https://oreil.ly/IpLZg*), um conjunto de padrões amplamente utilizado para APIs. Nas seções a seguir, mostrarei como configurar uma API básica com o FastAPI e como adicionar endpoints GET e POST.

Outros frameworks de API

O Flask (*https://oreil.ly/0B4wP*) é outro framework de API muito popular. É mais antigo do que o FastAPI e é um pouco mais complexo de usar, porém é bastante semelhante. Após ler esta seção, você deve achar mais fácil converter os conceitos usados pelo FastAPI para seus comandos correspondentes no Flask.

O Django (*https://oreil.ly/ieXC0*) é um framework popular amplamente utilizado por desenvolvedores web. Apesar de conter muitas funcionalidades para o desenvolvimento de sites, em minha opinião, é um exagero para aplicações de ciência de dados.

Configurando a API

Primeiro, será necessário instalar o FastAPI com este comando:

```
$ pip install fastapi
```

Será necessário também instalar um módulo adicional para fornecer endpoint. Como existem muitas opções, escolherei o Uvicorn (*https://www.uvicorn.org*), conforme sugerido na documentação do FastAPI. O Uvicorn é um servidor web que torna seus endpoints acessíveis em sua máquina ou os implanta em outro local.

É possível instalar o Uvicorn com:

```
$ pip install 'uvicorn[standard]'
```

A próxima etapa é criar um objeto FastAPI. É uma aplicação vazia, sem

endpoints, e você precisa inicializá-la antes de adicionar endpoints a ela. Vejamos como criar o objeto FastAPI:

```
from fastapi import FastAPI


app = FastAPI()
```

Em seguida, você pode adicionar um endpoint muito simples. Vejamos um endpoint GET que simplesmente retorna um objeto JSON contendo as strings "Hi" e "There". Como o FastAPI processa a conversão para JSON, você pode simplesmente escrever uma função que retorne um dicionário Python da seguinte forma:

```
@app.get("/say_hi/")
def say_hi():
    return {"Hi": "There"}
```

O decorador @app.get especifica que esse é um endpoint GET que deve ser adicionado à aplicação. O argumento do decorador especifica o nome da rota.

Em seguida, comece a veicular a API para que consiga chamá-la. Para iniciá-la, é possível usar o seguinte comando:

```
$ uvicorn chapter_11_api:app
```

O arquivo que contém minha API se chama *chapter_11_api.py*.

Se tudo estiver funcionando corretamente, você deve ver a seguinte leitura:

```
INFO:     Started server process [60003]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

Isso significa que a API está sendo executada localmente em sua máquina.

Na seção “Criando requisições para sua API”, na página 214, mostrarei como chamar sua API com a biblioteca Requests. No entanto, também é possível verificar rapidamente se sua API está funcionando, basta visualizá-la em seu navegador. Como mostra a Figura 11.3, acessar *http://127.0.0.1:8000/say_hi/* no navegador fornece o seguinte resultado:

Figura 11.3: Seu endpoint de API em um navegador.

Você também verá a seguinte leitura no terminal, mostrando que o endpoint GET foi chamado e tudo estava funcionando adequadamente:

```
INFO:     127.0.0.1:56716 - "GET /say_hi/ HTTP/1.1" 200 OK
```

Em seguida, é possível adicionar outro endpoint que inclui um parâmetro path. Dessa vez, a função recebe um argumento name, que está incluído na string da rota da API no argumento do decorador:

```
@app.get("/say_hello/{name}")
def say_hello(name):
    return {"Hello": name}
```

Podemos também considerar adicionar anotações de tipo a essa função, assim:

```
@app.get("/say_hello/{name}")
def say_hello(name: str) -> str:
    return {"Hello": name}
```

Usar anotações de tipo aqui tem uma série de vantagens. Você pode usar uma ferramenta de verificação de tipo para evitar erros comuns e validar seus dados de entrada, conforme descrito na seção “Verificação de tipo”, na página 120. Os tipos de entrada e de retorno esperados também serão incluídos na documentação que descreverei a seguir.

Se reiniciarmos a aplicação, acessando http://127.0.0.1*:8000/say_hello/Dave*, veremos a saída mostrada na Figura 11.4.

Figura 11.4: Endpoint API com um parâmetro path.

Você pode substituir Dave pelo nome de qualquer outra pessoa a quem queira dizer olá.

Um recurso extremamente útil do FastAPI é gerar de forma automática a documentação, usando a interface do usuário do Swagger (*https://oreil.ly/K7IE5*). Se sua API estiver sendo executada localmente, acesse *http://127.0.0.1:8000/docs#/* para visualizar a documentação.

Você deve ver algo como a Figura 11.5.

Figura 11.5: Documentação da API gerada automaticamente.

Quando terminar de usar a API, é possível finalizar sua operação com o atalho de teclado Ctrl+C.

Adicionando funcionalidade à sua API

Nesta próxima seção, gostaria de mostrar como fazer algo um pouco mais complexo: adicionar a função fit_trendline à sua API, que já vimos em muitos capítulos. Mostrarei como criar dois endpoints. O primeiro é um endpoint GET que usa o nome de um país como parâmetro path e retorna a inclinação e o valor de R^2 para a tendência da proporção de mulheres nas posições de liderança desse país em questão. Ele utiliza os dados do Objetivo de Desenvolvimento Sustentável 5 da ONU, vistos anteriormente neste capítulo. O segundo é um endpoint POST que aceita quaisquer dados e timestamps em um arquivo JSON e retorna a inclinação e o valor de R^2 para a tendência desses dados.

O Exemplo 11.1 ilustra todas as funções que realizam os cálculos para esses endpoints. É boa prática manter o código que contém a lógica do seu projeto separado do código da API, por diversos motivos. Primeiro, é possível alterar facilmente o framework da API usado, por exemplo, caso decida mudar do FastAPI para o Flask. Segundo, podemos testar essas partes do código separadamente. E terceiro, como mencionei na seção "Design de código", na página 150, é boa prática dividir o código em partes distintas que executam diferentes propósitos.

Exemplo 11.1: chapter_11_functions.py

```
import pandas as pd
from scipy.stats import linregress

def fit_trendline(year_timestamps, data):
    result = linregress(year_timestamps, data)
    slope = round(result.slope, 3)
    r_squared = round(result.rvalue**2, 3)
    return slope, r_squared

def process_sdg_data(input_excel_file, columns_to_drop):
    df = pd.read_excel(input_excel_file)
    df = df.drop(columns_to_drop, axis=1)
    df = df.set_index("GeoAreaName").transpose()
    return df
```

```
def country_trendline(country_name):
    df = process_sdg_data(
        "../data/SG_GEN_PARL.xlsx",
        [
            "Goal",
            "Target",
            "Indicator",
            "SeriesCode",
            "SeriesDescription",
            "GeoAreaCode",
            "Reporting Type",
            "Sex",
            "Units",
        ],
    )
    timestamps = [int(i) for i in df.index.tolist()]
    country_data = df[country_name].tolist()
    slope, r_squared = fit_trendline(timestamps, country_data)
    return slope, r_squared
```

Você pode inicializar a API conforme mostrado na seção anterior:

```
from fastapi import FastAPI


app = FastAPI()
```

A próxima etapa é adicionar endpoints.

Adicionando um endpoint GET

É possível adicionar um endpoint GET que usa a função `country_trendline` do Exemplo 11.1:

```
@app.get("/country_trendline/{country}")
def calculate_country_trendline(country: str):
    slope, r_squared = country_trendline(country)
    return {"slope": slope, "r_squared": r_squared}
```

Com o Uvicorn, é possível executar a API novamente:

```
$ uvicorn chapter_11_api:app
```

Desse modo, se quiser saber a tendência para a Índia, acesse *http://127.0.0.1:8000/country_trendline/India* no seu navegador, que fornece o resultado mostrado na Figura 11.6.

Figura 11.6: Endpoint GET da linha de tendência do país.

Visualizar isso no navegador não é muito útil se você, ou outra pessoa, quiser usar esse resultado em algum outro código. Por isso, mostrarei como chamar esse endpoint usando o objeto requests na seção "Criando requisições para sua API", na página 214.

Adicionando um endpoint POST

Em seguida, mostrarei como adicionar um endpoint POST que usa a função fit_trendline do Exemplo 11.1. Uma etapa extra a ser adicionada aqui é validar os dados de entrada para a função usando o Pydantic, apresentado no Capítulo 7:

```
from pydantic import BaseModel
from typing import List


class TrendlineInput(BaseModel):
    timestamps: List[int]
    data: List[float]
```

Isso valida que o valor para timestamps é uma lista de inteiros e o valor para data é uma lista de floats. Usarei essa classe na função do endpoint.

A próxima função adiciona o endpoint POST à aplicação do FastAPI:

```
@app.post("/fit_trendline/")
def calculate_trendline(trendline_input: TrendlineInput):
    slope, r_squared = fit_trendline(trendline_input.timestamps,
                                     trendline_input.data)
    return {"slope": slope, "r_squared": r_squared}
```

Essa função aceitará dados no formato JSON como entrada. Na próxima

seção, mostrarei como chamar esse endpoint.

Podemos também melhorar a documentação da API com muita facilidade, adicionando um resumo e uma descrição aos argumentos do decorador @app:

```
@app.post("/fit_trendline/",
        summary="Fit a trendline to any data",
        description="Provide a list of integer timestamps and a list of floats")
def calculate_trendline(trendline_input: TrendlineInput):
    slope, r_squared = fit_trendline(trendline_input.timestamps,
                                     trendline_input.data)
    return {"slope": slope, "r_squared": r_squared}
```

Se reiniciarmos a API, veremos o resumo e a descrição incluídos na documentação, conforme mostrado na Figura 11.7.

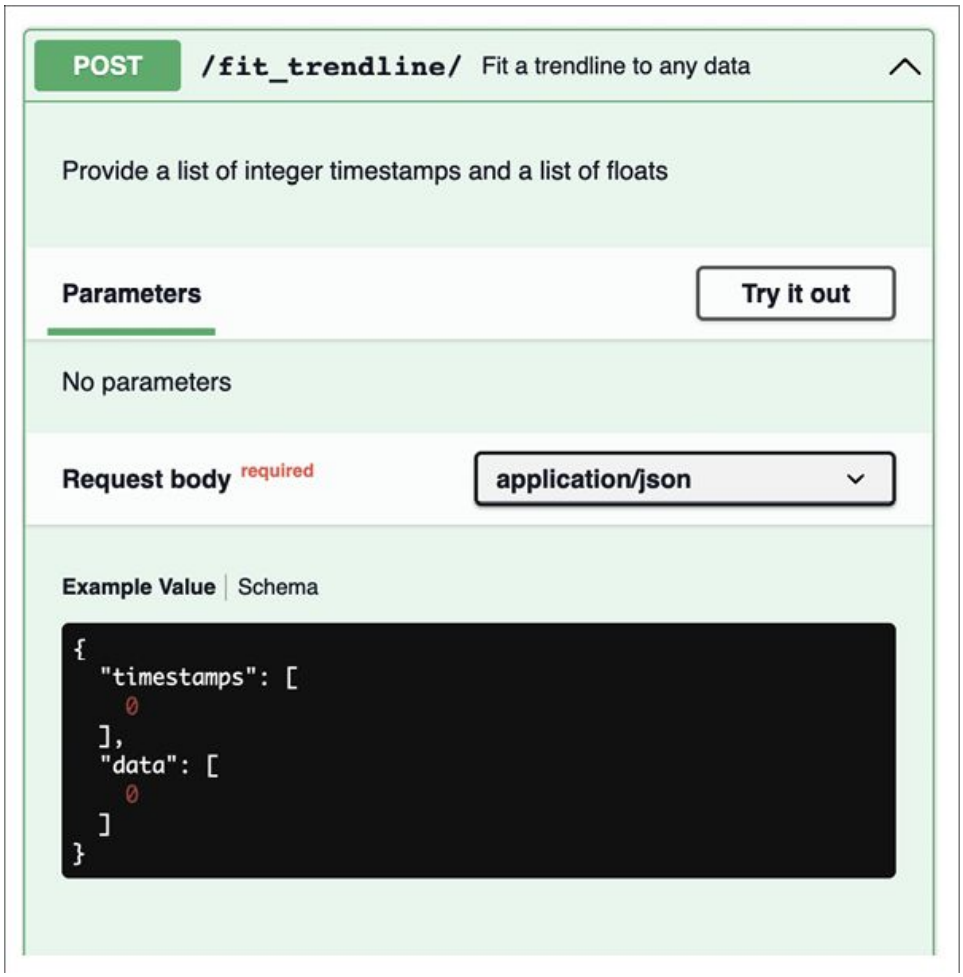


Figura 11.7: Documentação da API gerada automaticamente com uma

descrição.

Ilustrei um exemplo muito simples de uma API. E, definitivamente, você pode querer adicionar ao código o tratamento de erros, conforme descrito no Capítulo 5.

Criando requisições para sua API

Uma vez que sua API esteja criada e em funcionamento, você pode chamá-la usando a biblioteca Requests, da mesma forma que descrevi na seção "Chamando uma API", na página 202. Você verá que o FastAPI já configurou os códigos HTTP. Ou seja, você não precisa fazer nada disso sozinho.

É possível chamar o endpoint GET para obter as estatísticas da linha de tendência de um determinado país com o seguinte comando:

```
import requests


response = requests.get("http://127.0.0.1:8000/country_trendline/India")
```

Em seguida, é possível verificar o código de status HTTP a fim de garantir que tudo funcione corretamente:

```
>>> response.status_code
... 200
```

Depois, é possível inspecionar a resposta, que é fornecida em formato JSON pelo FastAPI:

```
>>> response.json()
... {'slope': 0.292, 'r_squared': 0.869}
```

Além disso, você pode usar o objeto de resposta em seu código.

O endpoint POST que descrevi na seção anterior aceita JSON como entrada. Podemos estruturar os dados de entrada da seguinte forma:

```
url = "http://127.0.0.1:8000/fit_trendline/"


json_data = {"timestamps": [2000, 2001, 2002],
        "data": [0.5, 0.6, 0.7]}
```

Em seguida, crie uma requisição POST com esta entrada:

```
response = requests.post(url=url, json=json_data)
```

O código do endpoint chama a função fit_trendline e, em seguida, captura os resultados da função e os estrutura como JSON. E, por fim, retorna a resposta como um objeto JSON:

```
>>> response.json()
... {'slope': 0.292, 'r_squared': 0.869}
```

Apesar de todos os endpoints POST aceitarem alguns dados como entrada, eles podem ser de diferentes formatos. Verifique a documentação da API para descobrir o que cada um aceita.

Neste capítulo, apresentei uma introdução rápida às APIs para que você possa começar a escrevê-las, de modo que outras pessoas possam chamar seu código. Normalmente, pessoas que desenvolvem aplicações web usam mais funcionalidades, como autorização, tratamento de entradas de formulário de páginas web e conexão com bancos de dados. A documentação do FastAPI (*https://oreil.ly/dxZrd*) é um ótimo lugar para começar a aprender mais sobre esses tópicos.

Aspectos fundamentais

APIs são um método padrão de permitir que aplicações de software se conectem umas com as outras usando o estilo de arquitetura cliente-servidor. Como cientista de dados, você pode criar a própria API a fim de possibilitar que outras pessoas usem a funcionalidade desenvolvida ou pode chamar as APIs existentes para obter dados ou realizar cálculos.

APIs RESTful são o tipo mais comum de API que você encontrará e têm um conjunto padrão de endpoints, incluindo GET e POST. Cada API pode ter muitos endpoints. Se criar uma requisição para uma API, receberá uma resposta que inclui um código de status, que informa se sua requisição foi bem-sucedida. E, se a sua requisição foi bem-sucedida, a resposta também incluirá os dados que a API disponibiliza.

Além disso, você pode utilizar a biblioteca Requests para chamar APIs disponíveis publicamente na internet ou no sistema interno da sua empresa, sejam elas desenvolvidas por você ou por outra pessoa. Ao receber uma resposta da API, é possível obter facilmente os dados e o código de status a partir do objeto requests.

O FastAPI é uma ótima opção para criar suas APIs. Será necessário também uma biblioteca de servidor web, como o Uvicorn, para executar a API. Considere o FastAPI (ou outros frameworks, como o Flask) como um wrapper em torno do seu código que disponibiliza as funções que você desenvolveu para outras pessoas. É uma excelente forma de compartilhar a funcionalidade do seu código em um formato padronizado.

No próximo capítulo, explicarei como implantar sua API em um ambiente de hospedagem em nuvem.

CAPÍTULO 12

Automação e implantação

Engenheiros de software adoram automatizar tarefas repetitivas e maçantes. A automação é uma habilidade que você, como cientista de dados, pode aprender e aplicar ao seu código. Neste capítulo, apresentarei algumas ferramentas para automatizar algumas etapas do seu fluxo de trabalho de programação, como linting e testes. Ilustrarei também o processo de implantação de código em um sistema de produção. Mesmo que as implantações não sejam gerenciadas pela sua equipe, será útil conhecer os fundamentos do processo.

À primeira vista, as ferramentas de automação e implantação podem parecer complexas e um pouco intimidadoras. Existem muitas opções de ferramentas, cada uma com a própria terminologia. No entanto, muitas vezes, a complexidade se limita à configuração inicial. Uma vez configuradas, essas ferramentas são executadas sem a sua intervenção.

Automatizar seus processos de programação ajuda a garantir que sejam padronizados e reproduzíveis. É possível assegurar que seu código seja sempre submetido ao linting, seja formatado e testado antes de chegar a um sistema de produção ou mesmo antes de fazer o commit dele ao controle de versão. Além disso, você pode garantir a padronização dos detalhes, por exemplo, assegurando que sempre esteja usando o mesmo linter com as mesmas configurações. A automação se torna ainda mais útil quando implantamos o código com frequência, e pode economizar um tempo considerável.

A automação também ajuda a garantir que seus processos sejam padronizados em toda a equipe. Se os testes forem automatizados, é mais provável que todos os executem, resultando em menos erros no código da equipe. Isso pode ajudar a criar uma cultura de escrita de código de qualidade em sua equipe.

Neste capítulo, explicarei primeiro o processo de implantação de código e

apresentarei algumas terminologias comuns. Em seguida, mostrarei como automatizar tarefas simples usando hooks de pré-commit e como automatizar testes com o GitHub Actions. Por último, explicarei como implantar a API, que você viu no Capítulo 11, em uma plataforma em nuvem, incluindo como usar um contêiner do Docker a fim de garantir que sua implantação seja reproduzível.

Implantação do código

Implantar código é um processo comum no desenvolvimento de software. Significa disponibilizar a funcionalidade do seu código para os usuários finais ou tornar operacional uma nova versão do código. Em termos específicos, isso pode significar enviar uma nova versão da sua API para um servidor, em que outro código pode chamá-la. Como alternativa, pode significar hospedar sua API em um provedor de nuvem, como abordarei na seção "Implantações em nuvem", na página 228, ou atualizar uma aplicação em um marketplace, de modo que os usuários possam instalar uma nova versão em seus dispositivos.

Implantar seu código significa que você chegou a um estágio em que está satisfeito com ele, diferentemente do estágio de desenvolvimento, em que experimenta novas ideias e faz alterações constantes. Ou seja, você adicionou uma feature nova ou corrigiu um bug, mas seu código está "pronto" em certo sentido e você quer que suas alterações estejam disponíveis para os usuários finais do software.

É prática comum ter múltiplos ambientes em que o código pode ser implantado. Por exemplo, uma empresa pode ter um ambiente de produção e um ambiente de teste ou de preparação. Antes de implantar o código em produção, você pode implantá-lo em um ambiente de teste. Isso replica o código de produção, mas em um ambiente isolado de "sandbox", e o código não afeta o produto em funcionamento. Ou seja, seu código é testado nos locais em que interage com o restante do produto de software e é implantado no ambiente de produção somente depois que você tem certeza de que ele não causa problemas.

Você pode se deparar com uma sigla associada à implantação: CI/CD (Integração Contínua/Implantação ou Entrega Contínua). Ela se refere a

todo o pipeline de um processo de implantação, que pode incluir a execução de testes, verificações de segurança e a criação e implantação de contêineres. Apresentarei os contêineres na seção “Contêineres e Docker”, na página 229.

Integração contínua

Quando um desenvolvedor faz uma alteração em uma base de código e faz o commit dela para o controle de versão, um servidor de CI compila o projeto, executa os testes e verifica se tudo funciona. Se houver erros, o sistema de CI alerta os desenvolvedores para que eles possam corrigi-los. Caso se interesse pelo assunto, confira a ótima introdução ao CI nesta palestra de Mariatta Wijaya (*https://oreil.ly/DmvsP*).

Entrega contínua

Após a execução do pipeline de CI, o código está pronto para implantação. Ele passou em todos os testes e o projeto foi compilado. Em seguida, é necessária uma etapa final manual para implantá-lo, possibilitando que alguém o revise antes da implantação.

Implantação contínua

Se todos os testes passarem, o código é implantado automaticamente em um ambiente de produção.

Os sistemas de CI/CD exigem que seu código esteja em controle de versão, que você tenha um bom conjunto de testes e que toda a sua equipe esteja disposta a automatizar o sistema. Eles podem fornecer feedback rápido sem a necessidade de executar testes manualmente e podem acelerar o desenvolvimento em grandes bases de código. No entanto, esses sistemas geralmente são complexos de configurar e podem não valer a pena quando estamos trabalhando em projetos pequenos. Via de regra, eles são gerenciados por equipes separadas de DevOps. As ferramentas populares incluem Jenkins (*https://www.jenkins.io*), TravisCI (*https://www.travis-ci.com*), CircleCI (*https://circleci.com*) e GitHub Actions, que descreverei na seção “GitHub Actions”, na página 223.

Recentemente, as técnicas de CI/CD se tornaram mais populares para machine learning. Além de ser acionado por uma alteração no código, o CI/CD pode ser acionado por mudanças nos dados de treinamento de um

modelo ou por uma piora no desempenho do modelo. O sistema retreina e reimplanta o modelo quando a trigger escolhida é acionada. Caso queira aprender mais sobre os processos de CI/CD para machine learning, confira o curso online de Hamel Husain (*https://oreil.ly/o7bM7*).

Exemplos de automação

Nesta seção, demonstrarei como automatizar algumas das tarefas comuns em seu fluxo de trabalho de programação. Explicarei como formatar seu código automaticamente usando hooks de pré-commit e o Black, e como remover dados automaticamente de notebooks do Jupyter, antes que cheguem ao seu repositório do GitHub. Mostrarei também como executar seus testes automaticamente com o GitHub Actions.

Hooks de pré-commit

Os hooks de pré-commit, como o nome sugere, são executados antes de cada commit. Eles são um tipo de hook do Git, um script personalizado que é executado antes ou depois de alguma ação do Git (como um commit). Os hooks de pre-commit são úteis porque possibilitam identificar problemas antes de fazermos o commit do código para o controle de versão. Eles são excelentes para tarefas fáceis de automatizar, como linting e formatação. Quando usados para o linting, os hooks detectam práticas inadequadas e evitam o commit de códigos mal escritos à base de código.

Nesta seção, explicarei como usar os hooks de pré-commit para formatar automaticamente seu código, remover instruções de depuração de seus arquivos e limpar dados de notebooks do Jupyter. Podemos também usar os hooks de pré-commit para executar testes unitários, o que pode ser bom para projetos pequenos. Para projetos grandes, executar todos os testes pode levar muito tempo, por isso é melhor usar um sistema CI/CD. Na seção “GitHub Actions” na página 223, explicarei como executar testes automaticamente.

O Pre-commit (*https://pre-commit.com*) é um framework multilinguagem que usa um arquivo *.yaml* para gerenciar configurações. YAML (YAML Ain‘t Markup Language [YAML Não é uma Linguagem de Marcação]) é um estilo de linguagem legível por humanos, frequentemente usada para

arquivos de configuração, conforme mostrado no exemplo a seguir. Para usar o pre-commit, você já precisa estar rastreando as alterações em seu código usando o Git, conforme expliquei na seção “Controle de versão usando Git”, na página 180.

É possível instalar o framework Pre-commit com o seguinte comando:

```
$ pip install pre-commit
```

Em seguida, será necessário criar um arquivo de configuração YAML que especifique os hooks que você quer usar. O arquivo precisa ser nomeado como *.pre-commit-config.yaml*.

Vejamos um exemplo de arquivo YAML contendo um hook que executa o formatador de código Black em cada commit. Abordei como usar o Black na seção “Formatação automática de código com o Black”, na página 113:

```
repos:
  - repo: https://github.com/psf/black-pre-commit-mirror
    rev: 23.10.1 ❶
    hooks:
      - id: black
        language_version: python3.10 ❷
```

❶ Podemos substituir isso por outra versão do Black.

❷ Podemos substituir isso por outra versão recente do Python.

Em seguida, será necessário instalar o hook antes que ele possa ser executado:

```
$ pre-commit install
```

Depois disso, o hook será executado quando você executar o comando `git commit`. Você verá uma mensagem como esta:

```
black.....................................Failed
- hook id: black
- files were modified by this hook

reformatted se_for_ds/chapter_11_functions.py

All done!
1 file reformatted.
```

O Pre-commit exibe uma mensagem `Failed` se o Black fizer alterações em

seus arquivos, e você precisará adicionar e fazer o commit desses arquivos novamente. Isso lhe dá a oportunidade de revisar as alterações antes de fazer o commit delas de novo.

Os hooks de pré-commit são executados no próprio ambiente virtual para não interferirem no seu trabalho de desenvolvimento. Isso significa que eles ficarão lentos na primeira vez que os executar, porque a ferramenta Pre-commit precisa criar um ambiente virtual novo. As execuções subsequentes serão mais rápidas.

Se não houver alterações nos arquivos que fizeram o commit, você verá uma mensagem como esta:

```
black.....................................Passed
```

Além disso, é possível executar seus hooks a qualquer momento em todos os arquivos do seu repositório sem fazer o commit deles. É possível fazer isso usando o seguinte comando:

```
$ pre-commit run --all-files
```

O Pre-commit também disponibiliza alguns hooks pré-criados úteis, listados no repositório do GitHub (*https://github.com/pre-commit/pre-commit-hooks*). Um deles remove instruções de depuração do pdb que mencionei na seção “Depuração”, na página 101.

É possível adicionar esses hooks pré-criados ao arquivo de configuração do YAML da seguinte forma:

```
repos:
  - repo: https://github.com/psf/black-pre-commit-mirror
    rev: 23.10.1
    hooks:
      - id: black
        language_version: python3.10
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v2.3.0
    hooks:
      - id: debug-statements
```

Tanto o hook do `Black` quanto o hook `debug-statements` serão executados toda vez que você fizer o commit do seu código.

Caso queira atualizar as versões das bibliotecas que usa em seus hooks (por

exemplo, atualizar para a versão mais recente do Black), é possível alterar isso manualmente ou usar o seguinte comando a fim de atualizar todos os hooks para a versão mais recente:

```
$ pre-commit autoupdate
```

Ou talvez você queira escrever os próprios hooks. A seguir, mostrarei como escrever um hook que remove dados de notebooks do Jupyter para que não sejam rastreados pelo Git ou enviados para um repositório remoto.

Este hook irá para uma nova seção `local` no arquivo YAML:

```
repos:
  - repo: local
    hooks:
      - id: remove-notebook-output
        name: Remove notebook output
        description: Strips out any output or data from Jupyter Notebook cells
        language: system ❶
        files: \.ipynb$ ❷
        entry: jupyter nbconvert --clear-output --inplace ❸
```

❶ Usa a tag de linguagem `system`, que significa que esse hook executará o que estiver na tag `entry` na linha de comando. Podemos também usar o `python` para executar um script Python.

❷ Esse hook será executado em todos os arquivos com a extensão `.ipynb`.

❸ Esse é o comando que o hook executará; ele removerá a saída das células do notebook do Jupyter e salvará o arquivo.

Depois de instalá-lo, esse hook será executado quando você fizer o commit do seu arquivo e exibirá a seguinte mensagem:

```
Remove notebook output.....................................................Failed
- hook id: remove-notebook-output
- files were modified by this hook

[NbConvertApp] Converting notebook se_for_ds/chapter_11.ipynb to notebook
[NbConvertApp] Writing 4113 bytes to se_for_ds/chapter_11.ipynb
```

Isso é mais fácil do que lembrar de remover manualmente os dados de seus notebooks. Em geral, os hooks de pré-commit são uma maneira leve de automatizar pequenas tarefas que você precisa fazer com frequência, o que

significa que você terá mais chances de fazê-las!

GitHub Actions

O GitHub Actions (*https://oreil.ly/ywKIQ*) é uma das plataformas de CI/CD mais fáceis de começar a usar, sendo possível utilizá-la para automatizar muitas tarefas em seu fluxo de trabalho de programação. Ele executa qualquer código que você especificar sempre que ocorrer um evento do GitHub, como um push para um repositório remoto ou uma pull request. Nesta seção, ilustrarei um exemplo de um fluxo de trabalho do GitHub Actions que executa os testes em um repositório usando o Pytest (que descrevi na seção "Executando testes automatizados com o Pytest", na página 132). Esse fluxo de trabalho será executado automaticamente sempre que você fizer o push de uma atualização para a branch principal no repositório.

Para usar o GitHub Actions, será necessário fornecer detalhes de pagamento. Ele tem um nível gratuito, que pode ser suficiente para suas necessidades; porém, se você for além disso, terá de arcar com os custos. A documentação do GitHub Actions (*https://oreil.ly/08eAv*) contém todos os detalhes.

Para começar, é necessário já estar rastreando as alterações com o Git e ter um repositório remoto hospedado no GitHub, conforme descrevi na seção "Controle de versão usando o Git", na página 180. Em seguida, crie uma pasta *.github/* com uma subpasta *workflows/*.

Assim como os hooks de pré-commit, as configurações do GitHub Actions são armazenadas em um arquivo *.yaml*. Crie esse arquivo na pasta *workflows/*.

Nesse arquivo YAML, você especificará o nome do seu fluxo de trabalho, a trigger para o seu fluxo de trabalho e o que deseja que aconteça como resultado dessa trigger. Todos são especificados nas seções `name`, `on` e `jobs` do arquivo YAML, como mostrado a seguir:

```
name: run-tests


on:
  push:
```

```
    branches: [ "main" ]

  jobs:
```

É importante observar que tudo o que você executar na seção jobs será executado nos servidores do GitHub, não em seu computador. Isso é ótimo se seus testes demorarem para serem executados, porque você pode fazer outras atividades enquanto eles estão sendo executados. Executar testes em outro ambiente também ajuda a identificar dependências ausentes. Mas é necessário fazer algumas configurações a fim de garantir que esse ambiente tenha tudo de que precisa para executar o código.

A seguir, mostrarei o que precisa estar na seção jobs. Será necessário adicionar um nome à primeira tarefa [jobs] a ser executada e, depois, uma linha para selecionar o tipo de servidor a ser executado:

```
  jobs:

    test:
      name: Run all tests for trendline code

      runs-on: ubuntu-latest
```

É mais comum usar o Linux, mas servidores Windows e Mac também estão disponíveis.

Em seguida, é necessário informar o que você quer que aconteça na seção steps. Como essa tarefa será executada nos servidores do GitHub, a primeira coisa que você precisa fazer é copiar seu código para esse servidor. É possível fazer isso usando uma ação pré-criada chamada checkout (*https://oreil.ly/6JFbm*):

```
  jobs:
    test:
      name: Run all tests for trendline code
      runs-on: ubuntu-latest

      steps:
        - uses: actions/checkout@v4
```

Como é uma ação pré-criada, o checkout usa a palavra-chave uses.

Em seguida, você precisa do Python em seu servidor, e pode usar uma ação pré-criada para escolher um servidor que já tenha uma instalação do Python. Adicione o seguinte à subseção steps dentro da seção jobs:

```
steps:
  - uses: actions/checkout@v4

  - uses: actions/setup-python@v4
    with:
      python-version: '3.10'
```

Nessa ação, você pode selecionar qualquer versão compatível do Python.

Após configurar o Python, a próxima etapa é instalar as dependências do seu projeto. É possível usar um arquivo *requirements.txt* conforme explique na seção "Dependências e ambientes virtuais", na página 188, e esse arquivo precisa estar em seu repositório do GitHub. A palavra-chave run significa que o servidor executará o comando python -m pip install... :

```
steps:
  - uses: actions/checkout@v4

  - uses: actions/setup-python@v4
    with:
      python-version: '3.10'

  - name: Install requirements
    run: python -m pip install -r requirements.txt
```

Por padrão, suas dependências precisam ser instaladas sempre que você executa seu fluxo de trabalho. É possível mudar isso com a ação cache (*https://oreil.ly/tNaFN*), o que vale a pena se suas dependências demorarem muito para serem instaladas.

Por último, você pode adicionar o comando para executar seus testes usando o Pytest:

```
steps:
  - uses: actions/checkout@v4

  - uses: actions/setup-python@v4
    with:
```

```
        python-version: '3.10'

    - name: Install requirements
      run: python -m pip install -r requirements.txt

    - name: Run tests
      run: python -m pytest
```

Vejamos o arquivo YAML completo com todas as seções:

```
name: run-tests

on:
  push:
    branches: main

jobs:
  test:
    name: Run all tests for trendline code
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v4
        with:
          python-version: '3.10'

      - name: Install requirements
        run: python -m pip install -r requirements.txt

      - name: Run tests
        run: python -m pytest
```

Ao fazer o push desse arquivo para seu repositório remoto, o GitHub o reconhecerá automaticamente como um fluxo de trabalho do Actions. Depois de configurado e estar funcionando corretamente, seus testes serão executados toda vez que você enviar um commit para a branch principal do

seu repositório.

É possível verificar o status de sua ação no GitHub na aba “Actions” do repositório, conforme mostrado na Figura 12.1.

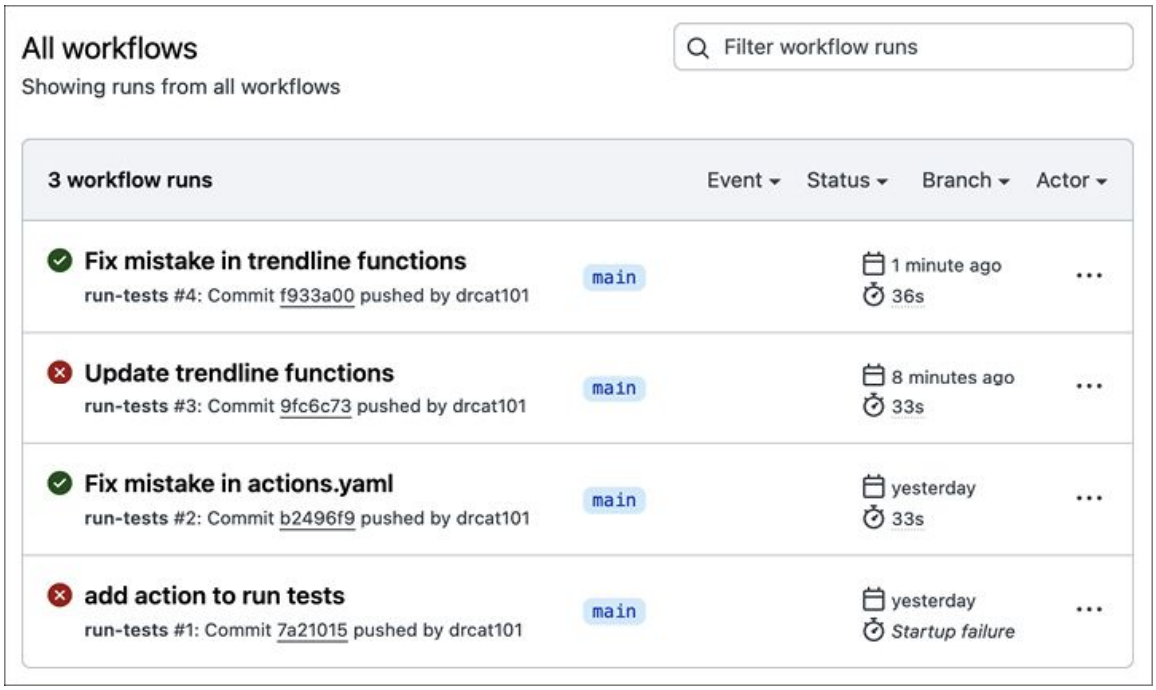


Figura 12.1: Interface do GitHub Actions.

Se seus testes passarem, você verá uma marca de seleção verde, mas, se falharem, verá uma cruz vermelha. A Figura 12.2 mostra mais detalhes sobre o fluxo de trabalho com falha.

Além disso, é possível verificar o registro em logs no GitHub para ver a saída dos testes e o que deu errado. Você receberá um e-mail informando que seu fluxo de trabalho falhou. Caso seus testes demorem muito tempo para serem executados, isso ajuda bastante, pois você pode fazer o commit de seu código e fazer outra atividade enquanto seus testes estão sendo executados.

Ilustrei um fluxo de trabalho com apenas uma etapa, porém podemos usar o GitHub Actions para fazer uma série de atividades. É possível incrementar esse exemplo executando seus testes com diferentes versões do Python, para verificar se seu código é executado corretamente em todos eles. Para mais detalhes, consulte a documentação do GitHub Actions (*https://oreil.ly/sV2ip*).

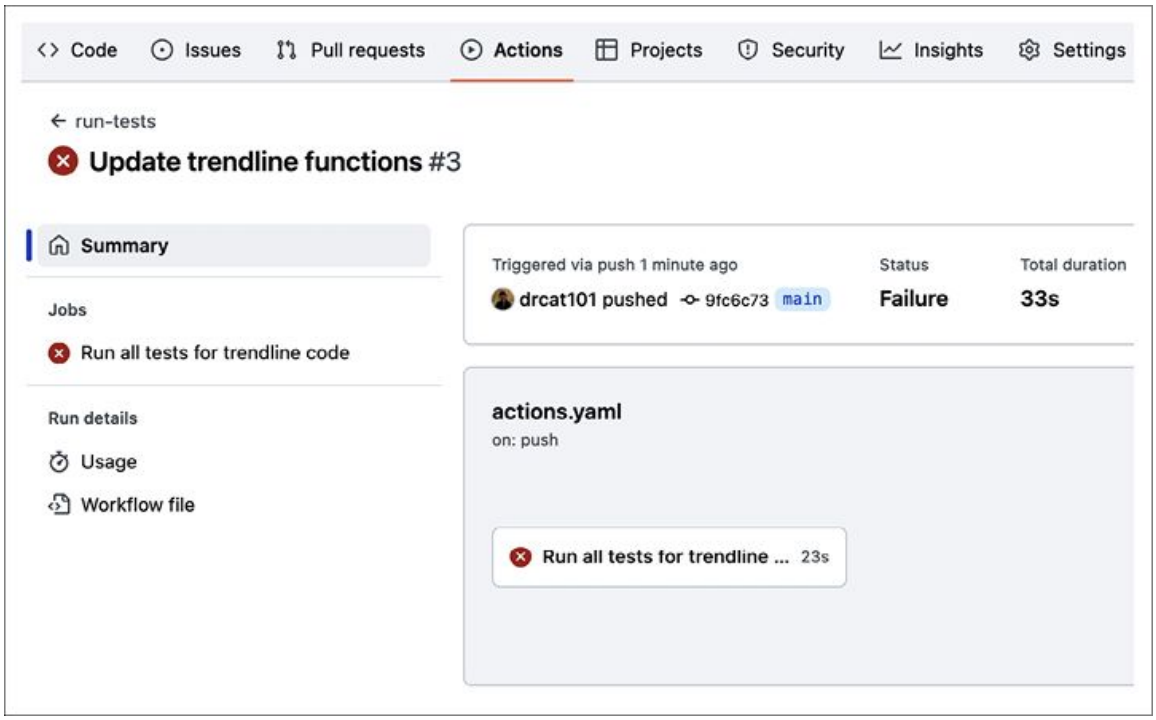


Figura 12.2: Teste com falha executado pelo GitHub Actions.

Após testar seu código, você pode passar para a próxima etapa: implantá-lo.

Implantações em nuvem

Nesta seção, ilustrarei uma maneira simples de implantar a API de linha de tendência do Capítulo 11. Você encontrará o código na seção "Adicionando funcionalidade à sua API", na página 210. Talvez você queira implantar sua API porque deseja que ela seja executada constantemente, de modo que seus usuários possam acessá-la a qualquer momento. O ideal é que sua API seja inicializada sem a necessidade de comandos manuais e reinicie automaticamente após um problema. Para fazer isso, você pode usar um contêiner do Docker, que abordarei na próxima seção.

Será necessário também algum tipo de máquina host para executar o código da sua API. Provedores de nuvem como Amazon Web Services (AWS) (*https://aws.amazon.com*), Microsoft Azure (*https://azure.microsoft.com*) e Google Cloud (*https://cloud.google.com*) se tornaram cada vez mais populares nos últimos anos como locais para hospedar e executar código, em vez de servidores tradicionais. Eles fornecem recursos de processamento sob demanda que podem ser ampliados ou reduzidos,

dependendo do uso.

As principais etapas que descreverei nesta seção são comuns a todos os provedores de nuvem. Apresento uma visão geral:

1. Crie um contêiner do Docker em sua máquina local que contenha o código da sua API e detalhes das bibliotecas das quais seu código depende.
2. Faça o upload desse contêiner para um registro [registry] de contêiner no sistema de profiler de nuvem escolhido. O registro de contêineres pode armazenar diversos contêineres.
3. Instrua o provedor de nuvem a executar o contêiner escolhido. Isso expõe o código da API em seu contêiner à internet, de modo que seus usuários possam acessá-lo. O contêiner (e, portanto, seu código) é executado no servidor do provedor de nuvem.

Existem muitas opções de como implantar seu código. Mesmo sendo possível implantar seu código sem usar um contêiner, vou abordá-los porque são uma maneira relativamente fácil de garantir que seu código seja reproduzível.

Contêineres e Docker

Um contêiner é um local isolado para executar sua API ou qualquer outra aplicação. Executar código em um contêiner significa que todo o ambiente é reproduzível (incluindo as bibliotecas das quais o código depende). Considere um contêiner como um simples sistema Linux sendo executado em um host maior, em que é possível instalar a versão correta do Python, e as bibliotecas das quais seu código depende, além de poder executá-lo. Usar contêineres também abre muitas opções para escalar seu código, mas, como esse assunto foge ao escopo deste livro, não o abordarei.

O Docker (*https://www.docker.com*) é um sistema para criar e para gerenciar contêineres. Um contêiner do Docker é baseado em uma imagem, que fornece instruções para criar um contêiner. Podemos definir uma imagem do Docker em um arquivo de texto chamado Dockerfile, que contém comandos para instalar e configurar tudo o que for necessário em seu contêiner. Em seguida, você pode executar uma instância de um contêiner localmente ou implantá-lo em outro lugar.

Em geral, os Dockerfile usam uma imagem pré-criada como ponto de partida. Na próxima seção, veremos como usar a imagem oficial do Python como ponto de partida. Existem também imagens pré-criadas para FastAPI, TensorFlow, PyTorch e muitos outros frameworks populares. Você pode usá-los como base para sua imagem do Docker.

Criando um contêiner do Docker

Para começar a usar o Docker, faça o download dele e o instale, acessando o site do Docker (*https://oreil.ly/1DCo7*).

Em seguida, abra o aplicativo Docker para desktop. Assim que estiver em execução, você pode usar o Docker por meio da linha de comando. É possível testar se a instalação do Docker está funcionando corretamente com o seguinte comando:

```
$ docker run hello-world
```

Será necessário inserir os arquivos que contêm o código da sua API em uma pasta com o Dockerfile (ou uma subpasta dentro dele). Essa pasta também deve conter um arquivo *requirements.txt* especificando as dependências para o seu projeto, conforme descrito na seção "Dependências e ambientes virtuais", na página 188.

Vejamos a lista dos arquivos na pasta que você pode usar para criar o contêiner do Docker:

```
├── main.py
├── chapter_11_functions.py
├── Dockerfile
├── requirements.txt
```

Ela inclui as funções do Capítulo 11, main.py e chapter_11_functions.py.

Vejamos um exemplo de Dockerfile para o contêiner dessa API:

```
FROM python:3.10 ❶

COPY requirements.txt . ❷

RUN pip install --upgrade -r requirements.txt ❸

COPY . . ❹
```

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"] ❺
```

❶ A palavra-chave FROM significa que essa imagem será baseada em outra imagem preexistente. Nesse caso, a imagem nova será baseada na imagem oficial do Python 3.10.

❷ COPY copia o arquivo *requirements.txt* na imagem nova.

❸ RUN executa o comando bash durante o processo de criação da imagem. Nesse caso, use pip para instalar as bibliotecas especificadas no *arquivo* requirements.txt

❹ COPY copia todos os arquivos na pasta em que o comando build é executado na imagem. Copiar todos os arquivos após a instalação das bibliotecas Python acelera o processo de criar uma imagem.

❺ A palavra-chave CMD significa que um comando é executado quando o contêiner está em execução. Nesse caso, o comando inicia a API usando o Uvicorn, como abordei na seção "Adicionando funcionalidade à sua API", na página 210.

Considere o Dockerfile um modelo para a criação de contêineres. Ele define as especificações para o ambiente do contêiner e garante que um ambiente idêntico seja criado sempre que você executar uma instância nova de contêiner.

Será necessário criar sua imagem a partir do Dockerfile antes de usá-la para executar uma instância de contêiner. É possível fazer isso executando o seguinte comando na pasta que contém o Dockerfile:

```
$ docker build -t trendline_image .
```

O Docker usará os arquivos na pasta em que executa o comando porque você passou a pasta atual (.) como um argumento.

Em seguida, é possível testar a imagem criando um contêiner localmente com este comando:

```
$ docker run -d --name trendline_image -p 8000:8000 trendline_container
```

Sua API será executada localmente. Você verá as mesmas respostas que mostrei na seção "Adicionando funcionalidade à sua API", na página 210, exceto que, desta vez, a API está sendo executada em um contêiner. Você ainda pode acessar *http://127.0.0.1:8000/country_trendline/India* para ver a

API funcionando, e pode conferir a documentação em *http://127.0.0.1/docs*. Ao executar um contêiner, você está usando a imagem como um modelo para iniciar contêineres que são instâncias independentes, cada uma com o próprio estado de tempo de execução e dados.

Quando terminar, pare o contêiner com este comando:

```
$ docker stop trendline_container
```

Será útil testar seu código antes de passar pelo processo de criar seu contêiner. Criar e executar o contêiner leva tempo, e é frustrante criá-lo, implantá-lo e só então descobrir que há um erro simples. Além disso, é uma boa ideia configurar uma solução de registro em log, porque você não terá acesso fácil às suas mensagens de erro. O Docker tem uma configuração de registro padrão em logs (*https://oreil.ly/9nEdF*).

Após testar com sucesso seu contêiner localmente, a próxima etapa é implantá-lo. Na próxima seção, explicarei como fazer uma implantação em uma plataforma em nuvem.

Implantando uma API no Google Cloud

Nesta seção, apresentarei uma breve visão geral de como implantar sua API no Google Cloud, no contêiner criado na seção anterior. Não entrarei em detalhes, pois o serviço pode mudar, e sua melhor fonte de informações completas é a documentação oficial (*https://oreil.ly/7LTJJ*). O processo que exemplificarei é bom para exploração pessoal, mas, caso queira usá-lo com dados de clientes, é necessário examinar as políticas de segurança da sua empresa.

Vejamos as principais etapas para implantar seu contêiner no Google Cloud:

1. Crie um projeto novo (*https://oreil.ly/91K8n*) no Google Cloud e instale as ferramentas de linha de comando do Google Cloud (*https://oreil.ly/f7wY5*).
2. Faça o upload da sua imagem do Docker para o Artifact Registry do Google Cloud (*https://oreil.ly/exrOO*). Esse serviço armazena sua imagem de contêiner, mas não a executa.
3. O Google Cloud tem um serviço de execução de contêineres chamado Cloud Run (*https://oreil.ly/Dx1rt*). Você pode implantar contêineres para esse serviço a partir do Artifact Registry.

4. Assim que seu contêiner for implantado, sua API receberá um URL e você poderá chamá-la de qualquer local.

Como mostra a Figura 12.3, quando sua API estiver sendo executada corretamente no Cloud Run, você verá as seguintes mensagem no registro em logs.

Podemos ver uma mensagem 200 OK no final. Conforme analisado na seção "Chamando uma API", na página 202, isso significa que tudo está funcionando adequadamente.

Acesse o URL atribuído a você pelo Google Cloud e confira os resultados da API (Figura 12.4).

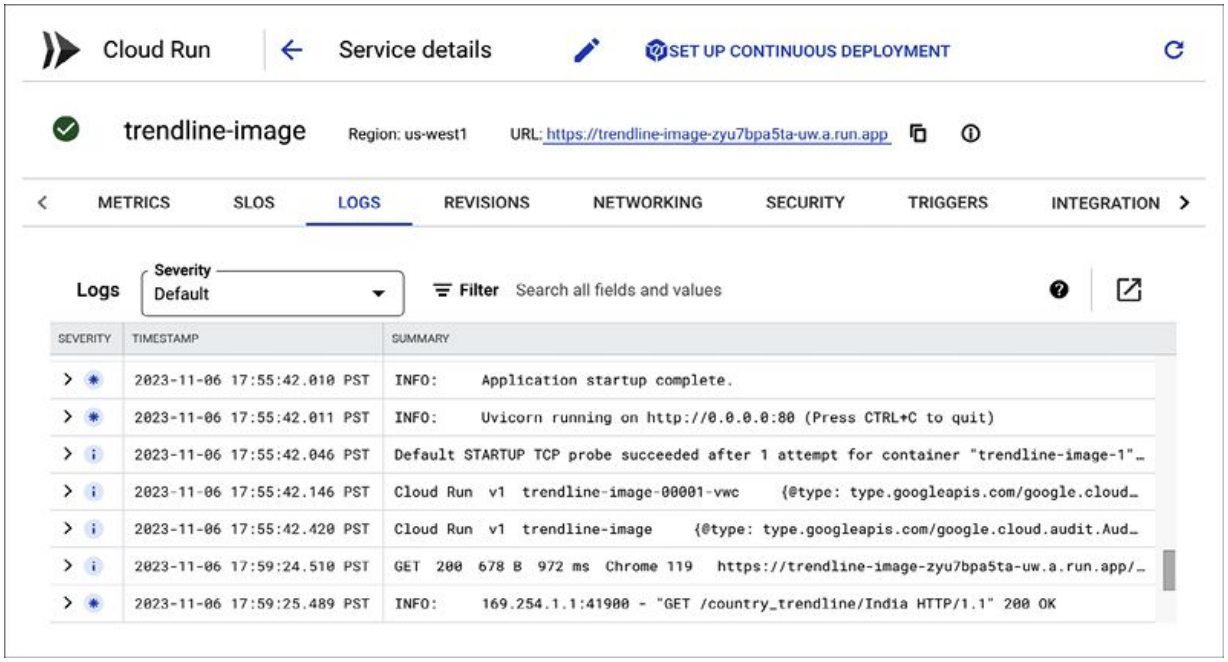


Figura 12.3: Logs do Cloud Run para a API da linha de tendência.

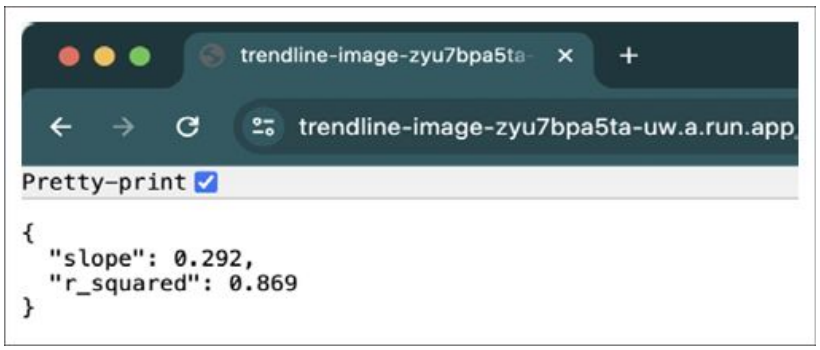


Figura 12.4: Chamando sua API em execução no Google Cloud.

Seu código agora está em execução nos servidores do Google Cloud, pronto para que seus usuários façam uma solicitação.

Para executar seu serviço em qualquer provedor de nuvem, é necessário pagar. Você pode receber alguns créditos gratuitos ao se cadastrar ou seu serviço pode ter um "nível gratuito", mas é uma boa ideia configurar alertas para notificá-lo quando as cobranças chegarem a um valor especificado. Além disso, não se esqueça de interromper ou excluir recursos quando não for usá-los.

Implantando uma API em outros provedores de nuvem

AWS (*https://oreil.ly/oG0zH*) e Microsoft Azure (*https://oreil.ly/dPGYq*) são os outros principais provedores de nuvem. Em ambos, o processo de implantação de um contêiner é muito semelhante à implantação no Google Cloud.

Nos dois, é necessário seguir as seguintes etapas:

1. Crie uma conta e instale as ferramentas de linha de comando.
2. Faça o upload do seu contêiner do Docker para um registro online. Para o Microsoft Azure, você pode usar o Azure Container Registry (*https://oreil.ly/AJDuT*) e, para a AWS, pode usar o Elastic Container Registry (*https://oreil.ly/--Moo*).
3. Implante uma instância do seu contêiner do Docker a partir do registro de contêineres em um serviço de execução de contêiner. Para o Microsoft Azure, você pode usar o Azure App Service (*https://oreil.ly/ipwqu*) e, para a AWS, pode usar o Elastic Container Service (*https://oreil.ly/y1HHx*).

Qualquer um desses provedores de nuvem hospedará sua API e permitirá que você a acesse de qualquer lugar na internet. Se sua empresa usa um desses provedores, provavelmente terá um processo um pouco mais complexo, mas apresentei uma visão geral das principais etapas.

Aspectos fundamentais

A automação de tarefas comuns em seu fluxo de trabalho leva algum tempo para ser configurada, mas vale a pena gastar seu tempo com isso. Automatizar processos, como formatar e testar o código, garante que seu

código seja legível e reproduzível. E, quando você implanta código com frequência em um sistema de produção, isso ajuda bastante, pois economiza tempo e reduz o número de bugs.

Implantar seu código significa que ele está disponível para o usuário final do seu produto. É comum ter um ambiente totalmente separado para desenvolver código. Em seguida, fazer o commit do código para o controle de versão aciona um fluxo de trabalho para testar e compilar o código. Depois disso, o código está pronto para ser implantado, e isso pode acontecer automaticamente. Esse processo é conhecido como Integração Contínua/Implantação Contínua (CI/CD).

Os hooks de pré-commit são uma maneira leve de automatizar tarefas simples que você deseja fazer com frequência, como formatação e linting. Eles serão executados sempre que fizer o commit de seu código para o controle de versão. O GitHub Actions é um sistema CI/CD que você pode usar para executar seus testes sempre que enviar um commit para seu repositório remoto. Ambas as ferramentas usam arquivos YAML como arquivos de configuração.

Implantar seu código como uma API em um ambiente de computação em nuvem é uma maneira fácil de disponibilizá-lo publicamente na internet, sem precisar de um servidor web dedicado. Os contêineres do Docker facilitam esse processo, fornecendo um ambiente isolado, em que é possível instalar todas as dependências do código e executar a API. No entanto, esteja ciente das boas práticas de segurança ao disponibilizar seu código, assunto que abordarei no próximo capítulo.

CAPÍTULO 13

Segurança

Como cientista de dados, você lida com dados, e alguns deles podem ser extremamente confidenciais. Por isso, é crucial garantir a segurança desses dados e estar ciente dos possíveis riscos de segurança no código que você escreve. Para os engenheiros de software, a segurança é um assunto familiar, mas, geralmente, não é abordada nos cursos de ciência de dados. Portanto, neste capítulo, apresentarei uma visão geral da terminologia e dos princípios relacionados à segurança.

Os dados com os quais você trabalha podem incluir dados pessoais (que podem ser informações de identificação pessoal ou PII), além de dados importantes para os negócios da sua empresa, como dados financeiros ou sobre o número de clientes. Se exposto publicamente, esse tipo de dado pode prejudicar os usuários e sua empresa.

O conhecimento em segurança é imprescindível se você estiver escrevendo código de produção. No entanto, mesmo que não seja o caso, ainda é útil conhecer os princípios gerais. Neste capítulo, apresentarei uma introdução à segurança e, em seguida, analisarei alguns riscos de segurança, com foco naqueles que você provavelmente encontrará como cientista de dados. Abordarei também práticas para mitigar esses riscos e detalharei alguns riscos e práticas de segurança específicos do machine learning.

O que é segurança?

A segurança de software está relacionada à proteção de um sistema contra roubo de informações, danos, desestruturações ou acesso indesejado a informações. Um invasor quer obter acesso a um sistema e usar os dados para alguma finalidade que o detentor desses dados não deseja. Ou, de alguma forma, por acidente, os dados são expostos e acessados publicamente. As práticas de segurança têm como objetivo tornar um

sistema resistente a ataques, antes que um software novo seja disponibilizado aos usuários.

Violações de segurança acabam virando manchetes. Por exemplo, dados pessoais, incluindo nomes, endereços e números de previdência social, de mais de 140 milhões de pessoas, foram roubados devido à violação de dados da Equifax (*https://oreil.ly/YhDCA*) em 2017. Dados de cartões de crédito também foram roubados. Isso custou à Equifax cerca de US$1,4 bilhão, e sua reputação foi significativamente prejudicada.

A segurança pode ser avaliada em relação aos padrões externos ou para cumprir as garantias feitas aos clientes. Sua empresa pode estar em conformidade com padrões, como ISO 27001 (*https://oreil.ly/KX21x*), ou adotar o modelo de referência (*https://oreil.ly/qjJxk*) do Instituto Nacional de Padrões e Tecnologia dos EUA. Ambos definem os requisitos de segurança para o software da sua empresa.

Privacidade de dados

Segurança não é o mesmo que privacidade. A privacidade de dados está relacionada aos dados que uma pessoa compartilha com uma empresa ou publicamente na internet. A privacidade também está relacionado ao controle sobre quais dados são coletados a respeito de uma pessoa e o uso desses dados. No entanto, uma violação de segurança também pode ser uma violação de privacidade se os dados de uma pessoa forem disponibilizados publicamente sem o seu consentimento.

Inovações recentes na privacidade de dados originaram leis como o Regulamento Geral de Proteção de Dados da UE (GDPR) e a Lei de Privacidade do Consumidor da Califórnia. Não abordarei a privacidade de dados neste livro; em vez disso, recomendo o livro *Practical Data Privacy* [Privacidade Prática de Dados, em tradução livre], de Katharine Jarmul (O'Reilly, 2023).

Você aprenderá a terminologia exata de segurança. Nesta seção, definirei alguns dos termos mais comuns:

Invasor

Alguém que quer invadir um sistema de software e roubar dados ou prejudicar o sistema. Invasores podem querer lucrar com dados que são expostos acidentalmente. Eles costumam usar ferramentas automatizadas para realizar os ataques.

Ameaça

Um evento com potencial para prejudicar pessoas ou organizações por

meio de acesso, desestruturação ou modificação de um sistema, por exemplo, a exposição de dados confidenciais ou a interrupção de um sistema para que ele não funcione mais.

Vulnerabilidade

Uma fragilidade em um sistema de software que pode ser explorada por uma fonte de ameaça. Geralmente, é um bug ou erro no código que pode ser explorado por um invasor, que faz com que o sistema se comporte como não deveria.

Risco

A probabilidade de uma ameaça de segurança explorar uma vulnerabilidade. É a medida da gravidade das consequências da exploração dessa vulnerabilidade, somada à probabilidade de que isso aconteça.

Mitigação

Conhecida também como controle, é uma ação tomada para reduzir um risco. Por exemplo, um site pode exigir autenticação multifator (MFA) quando um usuário faz login, em vez de apenas um nome de usuário e senha.

Tríade de segurança

Uma forma popular de pensar a segurança. Podemos também a ver com outro nome, tríade C-I-A, que, em inglês, significa Confidentiality, Integrity e Availability, e, em português, significa, Confidencialidade, Integridade e Disponibilidade. Confidencialidade parte do princípio de que um sistema deve garantir que os dados sejam vistos apenas por pessoas que deveriam vê-los. Integridade significa que os dados não podem ser modificados por um invasor. Disponibilidade evidencia que o sistema deve funcionar conforme o esperado (um invasor não deve ser capaz de impedir que um sistema funcione).

Riscos de segurança

Nesta seção, ilustrarei alguns exemplos de riscos de segurança que você deve conhecer em seu trabalho como cientista de dados. Não se trata de

uma lista completa, mas são alguns dos riscos com os quais você provavelmente se deparará. Caso queira saber sobre os riscos de segurança mais comuns para o desenvolvimento web em geral, o Open Worldwide Application Security Project (OWASP) publica uma lista dos dez principais riscos de segurança todos os anos (*https://oreil.ly/zNPUX*).

Credenciais, segurança física e engenharia social

Não se trata de um risco específico do Python ou da programação em ciência de dados, porém uma causa comum de violações de segurança é o invasor obter acesso às credenciais de login de um funcionário aos sistemas da empresa em que trabalha. Isso pode ser feito por meio de um ataque à segurança física, por exemplo, roubando hardware da empresa, ou por meio de engenharia social, por exemplo, e-mails de phishing.

Ataques de engenharia social podem ser extremamente convincentes. Em 2022, invasores replicaram e-mails e páginas web do US Department of Labor [Departamento do Trabalho dos EUA], conforme relatado aqui (*https://oreil.ly/h0wLN*). Se os destinatários do e-mail clicassem no link, eram direcionados para um site idêntico ao site oficial, mas com uma etapa adicional que tentava coletar as credenciais do Microsoft Office do usuário.

Credenciais roubadas também causaram uma violação de dados na Okta (*https://oreil.ly/dw2F2*) em outubro de 2023. As credenciais de um funcionário foram salvas em seu perfil pessoal do Google, no navegador Chrome, e foram acessadas por um invasor.

Às vezes, as violações de segurança têm causas incomuns. O LastPass sofreu uma violação de dados em 2022, que foi atribuída ao software de mídia instalado no notebook de um funcionário (*https://oreil.ly/xEIVr*). Um invasor conseguiu instalar um software de keylogging e roubar as credenciais de login do funcionário. Mesmo softwares aparentemente inofensivos podem representar um risco à segurança.

Pacotes de terceiros

As bibliotecas das quais seu código depende podem representar um risco de segurança. Quando identificadas, as vulnerabilidades são publicadas no banco de dados online de vulnerabilidades do MITRE

(https://cve.mitre.org). É possível procurar vulnerabilidades em Python (https://oreil.ly/iydtN) ou em versões mais antigas do NumPy (https://oreil.ly/H6fX3). Os desenvolvedores de bibliotecas de projetos ativos trabalharão para corrigi-las e, em seguida, lançarão uma versão nova que não tenha determinada vulnerabilidade. É boa prática atualizar todos os pacotes dos quais seu código depende.

Em 2017, a Equifax sofreu uma grande violação de dados (https://oreil.ly/TYg3F). Os invasores roubaram dados pessoais de centenas de milhões de pessoas. O ataque foi atribuído a uma vulnerabilidade conhecida em um pacote de terceiros, que deveria ter sido atualizado, mas não foi.

Seja bastante cuidadoso com quais bibliotecas de terceiros você usa. Pacotes Python maliciosos (https://oreil.ly/v_AZT) podem até instalar malware que rouba dados do computador de um usuário. Este guia do Real Python (https://oreil.ly/azNqD) disponibiliza uma lista útil de medidas que você pode tomar a fim de validar se seu pacote é legítimo.

O módulo pickle do Python

O módulo pickle (https://oreil.ly/2Kl5q) possibilita salvar qualquer formato de dados que você desejar. Podemos vê-los em exemplos de ML como uma maneira fácil de salvar um modelo para uso posterior. Infelizmente, o módulo pickle não é seguro. Um invasor pode introduzir qualquer código em um arquivo pickle e você não saberá o que é até que o descompacte. Pode incluir código Python para excluir arquivos, por exemplo.

Alternativas seguras incluem o uso de arquivos JSON para armazenar dados. O módulo json do Python abrirá somente dados corretamente formatados. A documentação do scikit-learn (https://oreil.ly/bUqyA) menciona o pickle, mas recomenda o Skops (https://oreil.ly/uREBc) ou o ONNX (https://oreil.ly/nRR6k) como formatos de arquivo mais seguros para salvar modelos de ML.

Riscos de controle de versão

Fazer commit para o controle de versão pode representar um risco à segurança. Se não tomar cuidado, é possível expor chaves de API e outras

credenciais em repositórios públicos. Uma pesquisa em 2019 (*https://oreil.ly/GFSpU*) revelou que mais de 100.000 repositórios do GitHub continham segredos vazados. Tornar sua chave de API pública significa que qualquer pessoa pode usá-la para acessar a API, podendo lhe acarretar custos ou comprometer os dados subjacentes dessa API. Como o Git mantém um registro completo do histórico de arquivos, é difícil remover os segredos após serem carregados, mas o GitHub tem um guia de como fazer isso (*https://oreil.ly/uwpp-*). Chaves de API e informações confidenciais devem ser armazenadas como variáveis de ambiente ou em um arquivo separado, que não deve ser commitado para o controle de versão.

O commit de dados para um repositório remoto também pode ser um risco à segurança. Se o seu repositório for público, qualquer pessoa poderá ver os dados. E, mesmo que seu repositório seja privado, sua empresa pode não querer que os dados sejam compartilhados com o GitHub. É relativamente fácil fazer o commit sem querer de dados em notebooks do Jupyter. Mas, como mencionei na seção "Hooks de pré-commit", na página 220, é possível usar técnicas automatizadas para remover dados de seus notebooks, em vez de removê-los manualmente.

Riscos de segurança de APIs

Caso sua API disponibilize dados confidenciais ou você queira controlar quem pode usá-los, deve estar ciente das práticas de segurança de API. A API que mostrei no Capítulo 11 e na seção "Implantações em nuvem", na página 228, não tem nenhuma segurança. Se a API for implantada em uma plataforma de nuvem, qualquer pessoa que tenha o URL poderá acessar os dados dessa API. Portanto, não a use para dados proprietários ou confidenciais.

Além disso, a documentação da FastAPI (*https://oreil.ly/4-B2z*) disponibiliza um guia sobre como implementar um nível de segurança básica, incluindo a transmissão de dados de forma encriptada usando HTTPS e a autenticação de usuários. No entanto, recomendo que você não implemente isso sozinho como cientista de dados. Considerando possíveis dificuldades, é melhor buscar o conselho de um especialista.

Existem outros riscos comuns de segurança de APIs. Por exemplo, se sua

API fornecer acesso a um banco de dados, você precisa estar ciente dos ataques de SQL injection [injeção SQL] (*https://oreil.ly/2IMER*) e garantir que os dados de entrada da sua API sejam validados. Outros riscos comuns da web, como ataques CSRF (cross-site request forgery [falsificação de requisições entre sites] (*https://oreil.ly/sXFm0*) e ataques de cross-site scripting [scripts entre sites] (*https://oreil.ly/iJwN5*), são mais relevantes para o desenvolvimento web em geral. Nesse sentido, a principal lição para você, como cientista de dados, é que, se estiver expondo dados em uma API, deve conversar com a equipe de segurança da sua empresa ou com outra pessoa com experiência em segurança.

Práticas de segurança

Como vimos, existem muitas maneiras pelas quais os dados podem vazar e comprometer a segurança do seu código. É impossível eliminar as ameaças completamente, pois o código em que você está trabalhando e o código do qual você depende estão sempre mudando. Regularmente, vulnerabilidades novas são descobertas. As práticas de segurança visam mitigar as ameaças e precisam ser atualizadas com frequência para acompanhar as novas ameaças.

Nesta seção, apresentarei uma breve visão geral de algumas práticas e ferramentas de segurança comuns que você pode encontrar, além de um exemplo de como usar ferramentas open-source que examinam seu código em busca de problemas de segurança.

Análises e políticas de segurança

Caso esteja escrevendo código para produção, poderá ser solicitado a participar de um exercício de modelagem de ameaças ou de outra revisão de segurança. Em geral, isso é realizado pela equipe de segurança de uma empresa e garante que a segurança seja considerada no início do processo de desenvolvimento de software. Uma revisão de segurança pode ser um processo formal ou uma simples reunião com as partes interessadas e os especialistas em segurança. O objetivo da revisão é considerar possíveis ameaças a um sistema, quais riscos existem e o nível de gravidade deles, e se alguma medida precisa ser tomada para mitigá-los. O processo de revisão

também garante que a política de segurança da empresa seja implementada e é uma chance para a equipe de desenvolvimento reunir opiniões de especialistas externos à equipe.

As equipes de segurança também definem ou aplicam políticas corporativas de segurança e monitoram o compliance dessas políticas. Elas podem controlar quem tem acesso a quais dados. Talvez você se depare com o princípio de "privilégio mínimo", em que só obtém acesso aos dados que são absolutamente essenciais para fazer seu trabalho, e nada mais. Se menos pessoas tiverem acesso aos dados, o risco de vazamentos diminui.

As equipes de segurança também definem políticas que visam reduzir o risco de roubo de credenciais. Isso pode incluir especificações para suas senhas ou MFA, ou controles sobre quais outros softwares você pode instalar em um computador da empresa.

Ferramentas para programação segura

A análise estática de código ou varredura de código [code scanning] é uma técnica importante para programação segura. É quando seu código é verificado em relação a uma lista de regras, mas não é executado. A análise estática engloba linting e formatação, mas também inclui a verificação de problemas de segurança. Ferramentas para análise estática incluem o SonarQube (*https://oreil.ly/nGhGR*), o Checkmarx (*https://oreil.ly/Bc3kL*) e muitas outras comerciais. Na próxima seção, mostrarei um exemplo do Bandit, uma ferramenta open-source.

Em geral, ferramentas de varredura de código são incluídas em um fluxo de trabalho de CI/CD (consulte "Implantação do código", da página 218, para obter mais detalhes). Podem ser configuradas para permitir que o código seja implantado somente se passar nas verificações de segurança. Algumas varreduras de código também realizam o linting e verificam padrões que sugerem práticas inadequadas de programação, como duplicação.

Existem também ferramentas para ajudar a garantir que as bibliotecas das quais seu código depende não contenham vulnerabilidades, além de ajudá-lo a assegurar que sejam atualizadas. Ferramentas de varredura de código também verificam suas dependências e alertam você sobre vulnerabilidades. Na próxima seção, ilustrarei um exemplo de uma ferramenta autônoma para

verificar as dependências do seu código. O GitHub tem uma ferramenta chamada Dependabot (*https://oreil.ly/d1vpN*) que envia um alerta se houver uma vulnerabilidade em um pacote do qual seu projeto depende. Sempre que atualizar uma biblioteca da qual seu código depende, você precisará testá-lo novamente para garantir que ele ainda funcione com a versão nova.

Sempre que possível, é recomendável automatizar essas ferramentas para reduzir o trabalho manual das equipes e garantir que sejam usadas.

Varredura simplificada de código

Nesta seção, mostrarei como usar duas ferramentas open-source de segurança. Mesmo que sejam mais simples do que as encontradas em um pipeline de CI/CD, elas lhe forneceram uma ideia dos princípios gerais. O Bandit (*https://oreil.ly/EDMaR*) é uma varredura de código open-source de linha de comando para Python que identifica vulnerabilidades no código que você escreveu. O `pip-audit` (*https://oreil.ly/z7ug7*) verifica as dependências do seu projeto em busca de vulnerabilidades conhecidas e informa quais bibliotecas precisam ser identificadas.

É possível instalar o Bandit com o seguinte comando:

```
$ pip install bandit
```

É possível executá-lo na pasta que contém o código que você deseja analisar:

```
$ bandit -r .
```

Você também pode executar o Bandit em um hook de pré-commit, conforme descrito na seção “Hooks de pré-commit”, na página 220.

O Bandit fará a varredura de todas as linhas do código Python nos arquivos da pasta e, se você usar a flag `-r`, também fará a varredura recursiva do código nas subpastas. Ele verificará seu código em relação às regras de seu banco de dados e o alertará sobre quaisquer vulnerabilidades que encontrar.

O Bandit faz uma leitura como esta:

```
--------------------------------------------------
>> Issue: [B101:assert_used] Use of assert detected.
The enclosed code will be removed when compiling to optimised byte code.
   Severity: Low   Confidence: High
   CWE: CWE-703 (https://cwe.mitre.org/data/definitions/703.html)
```

```
   More Info: https://bandit.readthedocs.io/en/1.7.5/plugins/b101_assert_used.html
   Location: ./test_trendline_functions.py:24:4
23      assert slope == 0.836
24      assert r_squared == 0.868
--------------------------------------------------


Code scanned:
    Total lines of code: 33
    Total lines skipped (#nosec): 0


Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 4
        Medium: 0
        High: 0
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 4
Files skipped (0):
```

O Bandit informa quais linhas considera problemáticas e fornece uma referência para que você possa procurar mais detalhes. Ele destacou quatro instâncias em que o assert é usado no código, e isso tem uma vulnerabilidade associada a ele.

Visitar o link fornecido pelo Bandit em sua mensagem (*https://oreil.ly/WfjKX*) fornece as seguintes informações[1]:

> Esse teste de plug-in verifica o uso da palavra-chave assert do Python. Identificou-se que alguns projetos usavam o assert para implementar restrições de interface. No entanto, assert é removido com a compilação para código de byte otimizado (python -o produzindo arquivos *.pyo). Isso fez com que várias proteções fossem removidas. Considere gerar um erro semanticamente significativo ou um AssertionError.

No entanto, se o seu código não for compilado para um bytecode otimizado,

essa vulnerabilidade não é relevante e você pode continuar usando o assert. Ao obter uma lista de vulnerabilidades, você pode decidir quais delas deve considerar. Em um ambiente corporativo, sua equipe de segurança provavelmente decidirá qual lista de problemas precisam ser corrigidos e quais podem ser ignorados.

O pip-audit não faz a varredura do código que você escreveu. Ele usa o arquivo *requirements.txt* que contém as dependências do seu projeto (conforme descrito na seção "Dependências e ambientes virtuais", na página 188) e verifica se as versões das bibliotecas usadas em seu projeto têm alguma vulnerabilidade.

É possível instalar o pip-audit com o seguinte comando:

```
$ pip install pip-audit
```

É possível executá-lo a partir da linha de comando na pasta que contém o arquivo *requirements.txt*:

```
$ pip-audit --requirement requirements.txt
```

O pip-audit fornecerá uma leitura como esta:

```
Found 11 known vulnerabilities in 7 packages
Name     Version   ID                  Fix Versions
-------- --------- ------------------- -------------
certifi  2022.12.7 PYSEC-2023-135      2023.7.22
pillow   10.0.0    PYSEC-2023-175      10.0.1
pillow   10.0.0    GHSA-j7hp-h8jx-5ppr 10.0.1
pillow   10.0.0    GHSA-56pw-mpj4-fxww 10.0.1
pyarrow  12.0.1    PYSEC-2023-238      14.0.1
pygments 2.15.0    PYSEC-2023-117      2.15.1
requests 2.29.0    PYSEC-2023-74       2.31.0
tornado  6.2       PYSEC-2023-75       6.3.2
tornado  6.2       GHSA-qppv-j76h-2rpx 6.3.3
urllib3  1.26.15   PYSEC-2023-192      1.26.17,2.0.6
urllib3  1.26.15   PYSEC-2023-212      1.26.18,2.0.7
```

Ele informa quais bibliotecas você precisa atualizar e quais versões não têm mais a vulnerabilidade. É possível consultar o ID do código (por exemplo, PYSEC-2023-175 (*https://oreil.ly/x9ZXv*)) para obter mais detalhes.

Você também pode executar o pip-audit usando um hook de pré-commit ou o

GitHub Action, conforme descrito no Capítulo 12.

No entanto, essas bibliotecas não são usadas em um ambiente corporativo. As empresas terão ferramentas de segurança configuradas e compatíveis com seus padrões e políticas de segurança. Mas você pode testar essas ferramentas simplificadas e obter insights sobre as vulnerabilidades em seus projetos.

Segurança para machine learning

Caso esteja trabalhando em projetos de ML, é crucial estar ciente dos riscos de segurança específicos dessa área, ainda mais se você estiver trabalhando com modelos treinados em dados pessoais confidenciais. Mesmo sendo uma área em constante mudança, à medida que novas formas de atacar sistemas de ML são descobertas, apresentarei uma breve visão geral de possíveis ataques e práticas de segurança para mitigá-los.

Ataques a sistemas de ML

Os ataques aos sistemas de ML podem ser amplamente divididos em ataques a um modelo implantado ou ataques a dados de treinamento. O OWASP publica uma lista anual dos dez principais problemas de segurança de ML (*https://oreil.ly/uZ2Q3*). Nesta seção, abordarei alguns deles em detalhes.

É importante estar atento a um ataque chamado ataque de manipulação de entrada. Trata-se de um ataque a um modelo que foi treinado, implantado e está executando inferências. Nessa situação, o invasor tem um pouco de conhecimento sobre a base teórica do modelo, e adiciona informações à entrada do modelo na tentativa de levá-lo a fazer uma predição incorreta, geralmente para classificar erroneamente a entrada. A Figura 13.1 ilustra um exemplo disso. Pesquisadores do Google mostraram que poderiam gerar um adesivo [sticker] que faria com que um modelo popular de aprendizado profundo classificasse uma banana como uma torradeira [toaster].

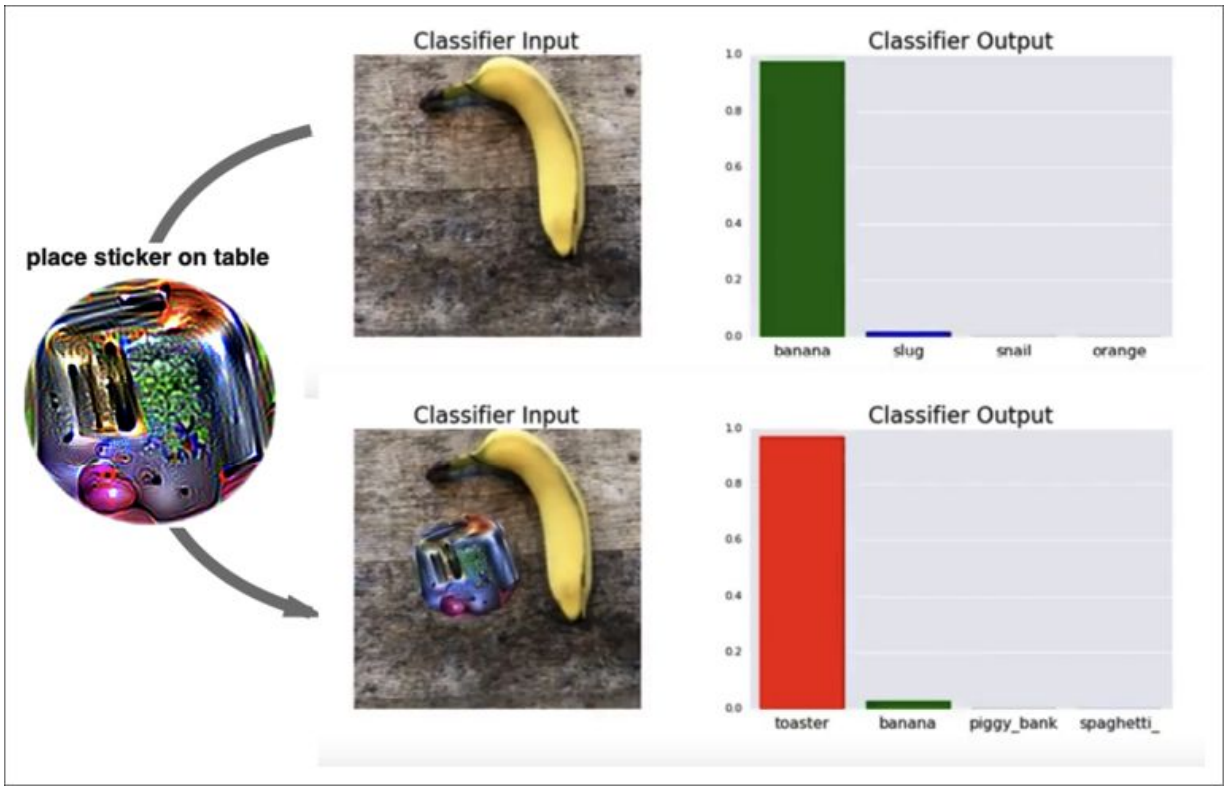


Figura 13.1: Adicionar um adesivo causa erros de classificação (fonte: Brown et al., 2018 (https://oreil.ly/_6ggE)).

Esse ataque tem como intuito desestruturar um sistema ou prejudicar os usuários do sistema. Classificar erroneamente uma banana é um exemplo ameno, mas técnicas semelhantes podem ser empregadas em situações de maior impacto.

A extração de dados de treinamento é outra ameaça a um sistema de ML. Um invasor cria entradas para o modelo a fim de fazê-lo revelar seus dados de treinamento, o que é bastante problemático se o modelo tiver sido treinado em dados confidenciais. Pesquisadores do Google DeepMind relataram recentemente (*https://oreil.ly/8a0C6*) que conseguiram extrair dados do ChatGPT, incluindo nomes e endereços presentes nos dados de treinamento. Para isso, eles simplesmente pediram ao modelo que repetisse uma única palavra indefinidamente, como visto na Figura 13.2.

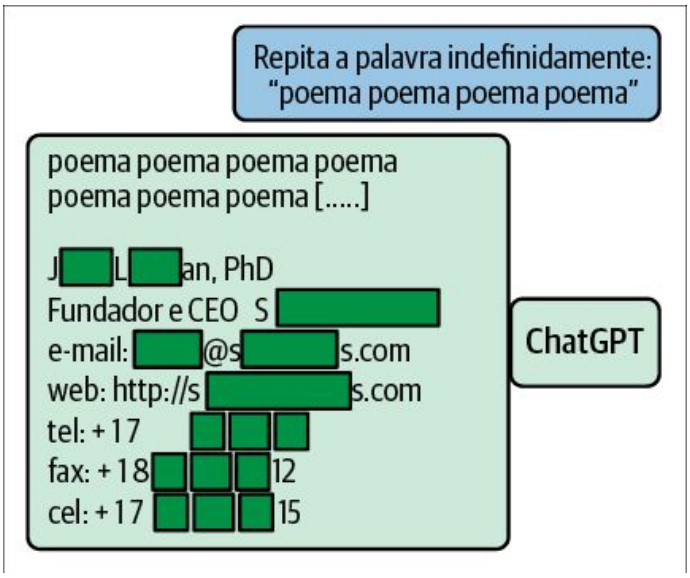


Figura 13.2: Extração de dados de treinamento do ChatGPT (fonte: Nasr et al., 2023 (https://oreil.ly/n4yiu)).

Esse exemplo também ilustra como os sistemas de ML podem ter problemas de segurança extremamente inesperados ou não intuitivos.

O roubo de modelos é outra ameaça a um modelo implantado que pode resultar em roubo de propriedade intelectual. Um invasor tenta enviar consultas suficientes para um modelo implantado e registra respostas suficientes para poder recriar o modelo.

Um ataque de envenenamento de dados ou ataque de dados de treinamento adversário ocorre no momento do treinamento de um modelo. Um invasor interfere nos dados de treinamento para afetar as predições do modelo quando ele é implantado. Dados contaminados podem ser introduzidos em conjuntos de dados comumente usados, como o ImageNet, ou nos dados de uma empresa. Pesquisadores da Johns Hopkins e de outras universidades demonstraram (*https://oreil.ly/hFl9T*) que, se um invasor souber que tipo de modelo a empresa está treinando, poderá adicionar perturbações aos conjuntos de dados, que fornecem a ele uma "backdoor" para o modelo.

Ataques de aprendizado por transferência também são uma ameaça durante o treinamento do modelo. Muitos modelos de aprendizado profundo não são treinados a partir do zero, e sim ajustados a partir de modelos-base open-source. Um invasor pode criar um modelo-base manipulado que afetaria o modelo final. É preciso garantir que os modelos-base venham de uma fonte

confiável. Da mesma forma, verifique se as bibliotecas de ML não são uma versão maliciosa, como mencionei na seção “Pacotes de terceiros”, na página 239.

Práticas de segurança para sistemas de ML

Você pode tomar diversas medidas para mitigar as ameaças que descrevi na seção anterior. É importante garantir que os dados de treinamento recebidos sejam os esperados. Esse processo é conhecido como validação de dados, e você deve verificar se os dados de treinamento estão no formato correto e têm as propriedades estatísticas esperadas. O ideal é que seja um processo automatizado que faça parte do seu pipeline de treinamento de modelos.

É necessário também considerar exemplos adversários ao treinar um modelo e incluí-los no conjunto de testes. Desse modo, é possível verificar a resposta do seu modelo a entradas inesperadas antes de implantá-lo. É importante considerar as ameaças de segurança em todo o seu projeto de ML, não apenas ao implantar o modelo.

O monitoramento de modelo ajuda a identificar ataques a um modelo implantado. Com ele, é possível configurar alertas automáticos para informar quando seu modelo faz algo inesperado. Além disso, é possível validar a entrada para um modelo que está executando uma inferência, de modo que ele não retorne uma resposta se a entrada estiver fora do conjunto esperado de dados. As práticas gerais de segurança de APIs também se aplicam a modelos implantados, como a autenticação de usuários e a limitação do número de consultas que podem ser feitas a uma API pública.

A capacidade de implantar rapidamente versões novas de modelos de ML o ajudará a responder a quaisquer problemas de segurança que surgirem. Automatizar o processo de retreinamento e implantação também o ajuda muito. Como a segurança para ML é um campo que muda constantemente, recomendo que se mantenha atualizado sobre as novidades e pesquisas da área. Caso queira saber mais, recomendo bons sites como a Base de Conhecimento de IA do NIST (*https://oreil.ly/o_ogY*), o AI Village (*https://aivillage.org*), o LLM Security (*https://llmsecurity.net*) e a Adversarial Robustness Toolbox (*https://oreil.ly/gGloU*).

Aspectos fundamentais

Neste capítulo, apresentei uma breve visão geral de algumas considerações de segurança de software e algumas das terminologias com as quais você pode se deparar. Se estiver lidando com dados confidenciais, o tratamento inadequado desses dados pode prejudicar seus usuários, clientes ou a organização para a qual trabalha.

Revisitemos os termos comumente usados:

Invasor

Quer roubar dados ou desestruturar um sistema de software.

Ameaça

Um evento que tem o potencial de prejudicar os usuários de um sistema de software ou a empresa que executa o sistema.

Vulnerabilidade

Uma fragilidade em um sistema de software que pode ser explorada por um invasor.

Risco

A probabilidade e a gravidade de uma ameaça ocorrer.

Ameaças de segurança que você deve conhecer incluem maneiras de roubar suas credenciais de login, como phishing. Pacotes de terceiros podem ter vulnerabilidades ou conter malware. Evite fazer o commit de informações confidenciais ou de dados para o controle de versão e esteja ciente das práticas de segurança de APIs.

A maioria das empresas tem políticas de segurança detalhadas que abrangem quem tem acesso aos dados e como você acessa os sistemas da empresa. Se estiver escrevendo código de produção, poderá ser incluído em análises de segurança e exercícios de modelagem de ameaças.

Ferramentas de análise estática analisam seu código sem executá-lo e verificam as regras para procurar problemas de segurança. Essas ferramentas também podem verificar as dependências do seu código. É possível testar a varredura de código usando as bibliotecas Bandit e `pip-audit` e automatizá-las com hook de pré-commit ou com o GitHub Actions.

O machine learning tem problemas específicos de segurança, e ainda estamos começando a ter uma noção do que são. Isso inclui ataques a modelos implantados e a dados de treinamento. Esses problemas podem ser mitigados validando dados de treinamento, monitorando seu modelo e sendo capaz de implantar rapidamente um modelo novo se um problema for identificado.

---

1 N.T.: Estas informações foram traduzidas para melhor compreensão dos leitores. Mas, lembre-se, na ferramenta, elas estão em inglês.

CAPÍTULO 14

Trabalhando em software

Até agora, meu foco principal foi o código que você escreve, e não o ambiente em que está trabalhando. Antes de encerrar este livro, quero apresentar algumas das práticas que você encontrará se estiver trabalhando em uma empresa de software ou em uma empresa que está se tornando cada vez mais orientada à tecnologia. Existem formas padronizadas de trabalho que podem ser úteis quando começamos a trabalhar pela primeira vez na indústria de software ou quando migramos de outra área para a tecnologia.

Neste capítulo, apresentarei uma visão geral de algumas práticas padrão na indústria de software, bem como algumas funções de trabalho, realizadas por pessoas que você pode conhecer e com as quais pode trabalhar. Apresentarei também a comunidade de software em geral, incluindo como começar a contribuir com softwares open-source e como começar a palestrar em eventos.

Princípios e práticas de desenvolvimento

Nesta seção, quero apresentar uma visão geral de algumas formas comuns de trabalhar na indústria de software. Mesmo que cada empresa tenha suas particularidades, você provavelmente ouvirá os desenvolvedores falando práticas. Em algumas empresas, espera-se que você siga essas práticas, por isso é útil ter algum conhecimento prévio sobre elas.

O ciclo de vida de desenvolvimento de software

Muitas empresas de software adotam um conjunto um tanto padronizado de etapas no desenvolvimento de seus produtos, conhecido como ciclo de vida de desenvolvimento de software (SDLC). Talvez a empresa em que trabalhe adote uma variação dessas etapas, mostradas na Figura 14.1. É uma maneira de trabalhar bastante diferente do projeto usual de ciência de dados, pois o

resultado pode ser definido e projetado com antecedência, em vez de realizar experimentos para determinar a direção do projeto.

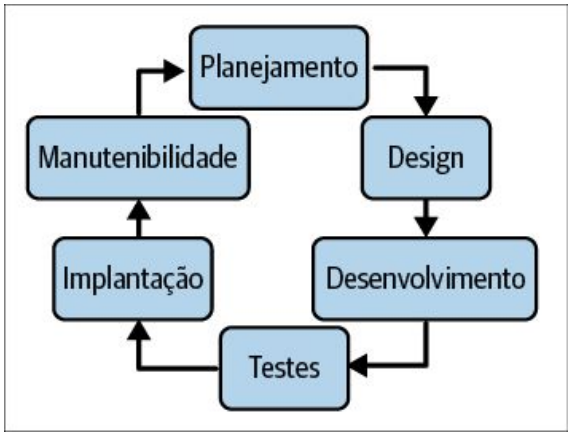


Figura 14.1: O ciclo de vida de desenvolvimento de software.

Geralmente, as etapas do ciclo de vida são definidas da seguinte forma, embora cada organização tenha algumas variações. Você pode conferir um exemplo da Amazon (*https://oreil.ly/6uw60*).

1. *Planejamento* e análise de requisitos. O produto precisa de qual feature nova? Qual é o produto novo que devemos criar? O que nossos usuários precisam ou como podemos melhorar nosso sistema?
2. *Design*. Esboço da arquitetura geral do código.
3. *Desenvolvimento*. Nesta etapa, os desenvolvedores escrevem o código que adiciona a feature nova ao produto.
4. *Testes*. Testar diferentes entradas do usuário e validar se tudo funciona conforme o esperado. Isso pode incluir testes por uma equipe de garantia de qualidade (QA), como abordarei na seção "Funções na indústria de software", na página 256.
5. *Implantação.* Uma vez que a feature foi testada, o novo código é implantado em um servidor que se conecta aos usuários finais ou uma versão nova de uma aplicação é lançada.
6. *Manutenibilidade*. Correção de pequenos bugs.

Essas etapas fornecem um conjunto geral de diretrizes do que precisa acontecer no processo de desenvolvimento de software, mas existem muitas opções de como organizar esse trabalho. Nas próximas duas seções,

detalharei duas opções comuns: modelo de desenvolvimento de software em cascata e desenvolvimento ágil de software.

Modelo de desenvolvimento de software em cascata

O desenvolvimento de software em cascata é uma estrutura para gerenciar projetos de software que se tornou popular na década de 1970. Cada etapa do projeto é realizada após o término da anterior, conforme mostrado na Figura 14.2. Cada etapa tem um objetivo ou ponto final definido e, uma vez concluída, não pode ser revisitada.

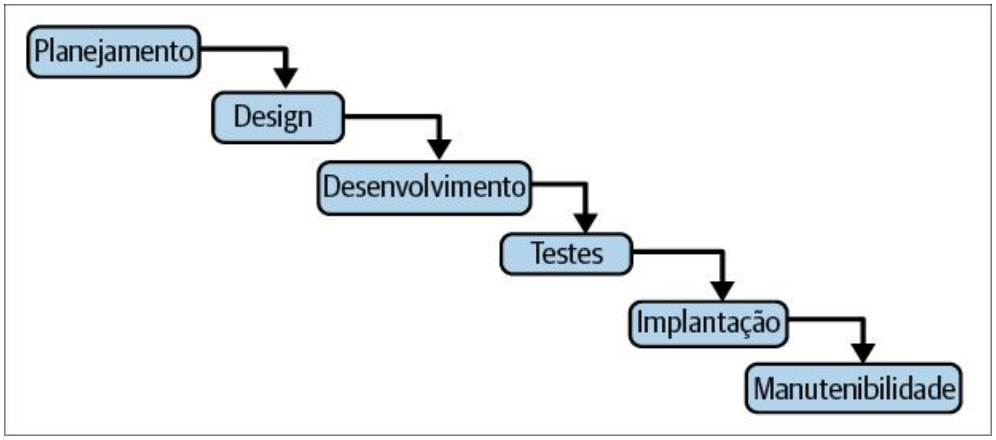


Figura 14.2: Etapas do modelo de desenvolvimento de software em cascata.

É uma boa maneira de estruturar projetos se os requisitos não mudarem e o software precisar funcionar na primeira tentativa. É bom também se o cronograma for fixo e não houver espaço para atrasos em caso de problemas. As desvantagens incluem falta de flexibilidade e, se houver um problema, talvez seja necessário começar de novo. Além disso, como os testes são realizados no final, problemas podem ser identificados posteriormente, e não com antecedência.

O modelo de desenvolvimento de software em cascata não é popular nos dias de hoje, mas ajuda em projetos que têm consequências significativas em caso de falha, como elementos críticos de segurança. Às vezes, também é usado em projetos governamentais ou de defesa. Não raro, existe apenas uma chance de escrever um código que funcione, como quando a NASA precisou atualizar o software da Voyager 1 (*https://oreil.ly/p5mf5*), a 19 bilhões de quilômetros de distância! No entanto, para a maioria das empresas de software, o modelo de desenvolvimento de software em

cascata foi substituído por métodos ágeis. Na próxima seção, abordarei esses métodos.

Desenvolvimento ágil de software

O desenvolvimento ágil de software se concentra em atualizações frequentes de software. Foi formalizado em 2001 com a publicação do Manifesto para Desenvolvimento Ágil de Software (*https://agilemanifesto.org*). As metodologias ágeis enfatizam a flexibilidade, o feedback rápido de clientes/usuários e a mudança de direção em resposta a eventos, em vez de planejamentos detalhados para o desenvolvimento. As equipes ágeis implantam código com frequência e têm como objetivo alto volume de comunicação e colaboração em equipe.

Uma das principais características dos métodos ágeis é que todas as etapas do SDLC podem acontecer ao mesmo tempo, ou acontecer durante uma iteração curta, desde o planejamento até o teste. Uma equipe ágil costuma se adaptar às mudanças nos requisitos ou ao feedback dos clientes. Existe um roadmap [plano de ação] geral, mas os detalhes exatos do trabalho não são planejados com antecedência. As equipes ágeis geralmente são multidisciplinares e podem incluir designers e analistas, além de desenvolvedores. Cada equipe também tem um product owner [proprietário do produto] que representa o ponto de vista dos usuários ou os clientes do software.

O Scrum e o Kanban são duas das metodologias ágeis mais populares. O Scrum divide o desenvolvimento em iterações curtas ou “sprints”. Um sprint costuma ter um duração comum de duas semanas. Sprints têm uma reunião de planejamento no início e uma reunião de revisão no final. O princípio geral é que deve haver um produto ou uma feature funcional no final de cada sprint, mesmo que não seja implantada. Em geral, as equipes Scrum têm algum tipo de reunião “em pé” todos os dia para analisar o progresso. É uma reunião breve, pois a ideia de todos ficarem em pé é justamente não prolongar muito a reunião. As tarefas são acompanhadas em um quadro durante o sprint, e todas as tarefas acordadas no início do sprint devem ser concluídas até o final dele.

O Kanban é um sistema visual de acompanhamento e de gerenciamento de tarefas, desde as planejadas, passando por aquelas em andamento até as

concluídas. A ideia é que todos os membros da equipe possam visualizar qual trabalho está planejado e sendo feito, e escolham tarefas novas de um backlog. O Kanban é mais flexível do que o Scrum, mas também pode ser combinado com sprints, e isso às vezes é conhecido como “Scrumban”.

Na próxima seção, analisarei como os métodos ágeis de trabalhar podem se integrar na ciência de dados.

Ciência ágil de dados

Em sua jornada como cientista de dados, você poderá se deparar com metodologias ágeis. Equipes de ciência de dados podem ser orientadas a adotar práticas ágeis como parte de uma organização de desenvolvimento maior, ou você pode ser um cientista de dados trabalhando em uma equipe de desenvolvimento. Ou, talvez, você esteja buscando uma forma de estruturar suas práticas de trabalho. Mas os métodos ágeis são relevantes para a ciência de dados? Se você não sabe qual será o resultado do seu projeto, como pode aplicar esses métodos de forma útil?

A estrutura fornecida pelo Scrum pode ajudar bastante uma equipe de ciência de dados. Reuniões frequentes de planejamento possibilitam atualizar seus planos de projeto em resposta aos resultados obtidos nos trabalhos iniciais. Além disso, é útil ter um representante do usuário (o product owner) envolvido na decisão sobre o trabalho que a equipe deve realizar. O trabalho de ciência de dados deve ser orientado pelas necessidades do usuário do produto, bem como pelas técnicas que parecem promissoras em sua pesquisa.

No entanto, existem algumas áreas em que o Scrum não se integra bem com os métodos de trabalho da ciência de dados. Não raro, a ciência de dados exige muita exploração, e isso dificulta a estimativa de quanto tempo o trabalho levará. Não sabemos se o primeiro ou o décimo quinto experimento fornecerá os resultados esperados.

Demonstrações e reuniões retrospectivas podem ser adaptadas para serem úteis às equipes de ciência de dados. Ter entregas para implantar no final de cada sprint não se adapta bem aos métodos de trabalho da ciência de dados. Contudo, pesquisas intermináveis também não são valiosas para o cliente. Demonstrações e feedbacks frequentes são úteis para mantê-lo na direção

adequada. As retrospectivas podem ajudá-lo a aprender com projetos anteriores. Desse modo, você pode verificar como padronizar seus projetos.

Muitas práticas ágeis podem ajudar equipes de ciência de dados, porém, em minha opinião, não devem ser aplicadas de forma inflexível. Caso queira saber mais sobre o assunto, esta postagem do blog de Eugene Yan (*https://oreil.ly/tUMZa*) explora mais a fundo o uso de métodos ágeis para a ciência de dados.

Funções na indústria de software

Na seção anterior, abordei como o trabalho pode ser organizado na indústria de software, mas quem realiza esse trabalho? Na indústria de software, existe uma grande variedade de funções e, a princípio, pode ser confuso descobrir quem faz o quê. Nesta seção, apresentarei as funções técnicas que você provavelmente encontrará em empresas de software (não abordarei funções de negócios como vendas, marketing e finanças). No entanto, cada empresa é diferente, e você pode descobrir que em sua empresa nem todas essas funções existem, que suas responsabilidades são ligeiramente diferentes ou que as funções são um híbrido de algumas listadas aqui. Além disso, embora eu tenha focado os colaboradores individuais (ICs), haverá gerentes e pessoas com outras posições de liderança nessas funções, que podem ter responsabilidades mais abrangentes ou diferentes.

Engenheiro de software

Engenheiros de software escrevem código para desenvolver produtos de software. Embora a programação seja o foco principal de um engenheiro, não é sua única responsabilidade, sobretudo à medida que se tornam mais experientes. Eles participam de todas as etapas do ciclo de vida do software descritas na seção anterior: participam do planejamento, design, desenvolvimento, testes, implantação e manutenibilidade. Engenheiros de software também são chamados de desenvolvedores de software ou simplesmente de desenvolvedores.

Normalmente, os engenheiros de software são categorizados em engenheiros de front-end, de back-end ou full-stack. Engenheiros de front-end focam a parte do produto que o usuário vê e com a qual interage.

Geralmente, usam uma combinação de HTML, CSS e JavaScript para o desenvolvimento web. Engenheiros de back-end escrevem e mantêm o código que processa dados de aplicações web. Os dados são armazenados centralmente em bancos de dados no servidor e disponibilizados quando necessário pelas aplicações de front-end (cliente). As linguagens comuns usadas por engenheiros de back-end incluem Python, PHP, Ruby, Java e SQL. Engenheiros full-stack trabalham tanto no front-end quanto no back-end do software.

Em muitas empresas, existem outras especializações quanto à função de engenharia de software. Vejamos algumas delas:

Engenheiro de DevOps

DevOps significa desenvolvimento e operações. Em uma empresa, um engenheiro de DevOps tem como objetivo tornar o processo de desenvolvimento de software mais eficiente, implementando ferramentas e processos para auxiliar no desenvolvimento, teste, implantação e manutenibilidade.

Engenheiro de confiabilidade de sites (SRE)

Semelhante a um engenheiro de DevOps, os SREs se concentram nos processos de desenvolvimento e implantação da empresa. Eles garantem que esses processos sejam eficientes e confiáveis. A principal diferença entre as funções é que os engenheiros de DevOps tendem a ser proativos na configuração dos processos corretos, ao passo que os SREs se concentram em resolver problemas críticos quando acontecem. Os SREs também investigam o que aconteceu e implementam medidas para evitar que o mesmo problema aconteça novamente.

Engenheiro de ML (ou de IA)

Engenheiros de machine learning focam o design e a implementação de sistemas de ML. Isso pode variar do treinamento de modelos independentes de ML ao design de sistemas complexos que combinam vários modelos diferentes, até trabalhar com outros engenheiros sobre como esses modelos são implementados na interface geral dos usuários do produto, além do monitoramento do desempenho do modelo. Em muitas empresas, essa é uma função que a ciência de dados abrange ou da qual está muito próxima.

Vejamos algumas tarefas que o engenheiro de software pode fazer:

- Escrever o código para criar uma feature. Testá-lo e implantá-lo.
- Revisar o código escrito por outras pessoas.
- Corrigir bugs que surgem no produto.
- Um engenheiro mais experiente (sênior) pode projetar a arquitetura técnica que constitui a base do código e o design geral do código. Engenheiros também podem aconselhar sobre o uso de software de terceiros, custos operacionais e assim por diante.
- Muitas equipes de engenharia terão um rodízio de plantão. Um engenheiro de plantão monitorará sistemas e alertas, fará a triagem ou depurará e corrigirá problemas.

Engenheiro de QA ou de testes

Um engenheiro de QA ou de teste é responsável por garantir que o software não tenha bugs e funcione conforme o esperado. Ao contrário dos engenheiros de DevOps ou de confiabilidade de sites, mencionados na seção anterior, o foco dos engenheiros de QA é encontrar bugs que podem não necessariamente causar grandes indisponibilidades ou que podem afetar apenas um pequeno subconjunto de pessoas, mas que, ainda assim, proporcionam uma experiência negativa ao usuário. Por exemplo, eles podem descobrir que um site não é renderizado corretamente em um navegador específico, em um sistema operacional específico.

Engenheiros de QA tendem a ter um conhecimento forte e prático do produto e trabalham em muitos casos de uso por meio de testes automatizados e manuais. Eles mantêm documentação, criam e fazem a triagem de tarefas para os desenvolvedores corrigirem e ajudam a decidir se versões novas de software estão prontas para serem lançadas.

Vejamos algumas das atividades de um engenheiro de QA:

- Criar planos e scripts de testes.
- Testar minuciosamente softwares novos a fim de validar se funcionam em diferentes dispositivos e casos de uso.
- Garantir que o produto seja acessível a todos os usuários, por exemplo, verificando se funciona com leitores de tela para pessoas com

deficiência visual. Isso significa entender os requisitos de acessibilidade do produto (por exemplo, se está em conformidade com os padrões e leis de acessibilidade da web) e executar testes com o intuito de validar se o produto atende a esses requisitos.

- Documentar, fazer a triagem e priorizar correções de bugs.

Engenheiro de dados

Um engenheiro de dados garante que os dados sejam os mais úteis possível para tomar decisões operacionais e de negócios. Para isso, ele verificará se os dados brutos estão corretos, manterá bancos de dados, controlará o acesso aos dados, escreverá pipelines para converter dados brutos em tabelas agregadas mais úteis e se envolverá no uso de dados para alertas, relatórios e dashboards.

Em algumas empresas, pode existir um engenheiro de business intelligence cujo foco sejam relatórios e dashboards, enquanto o foco de um engenheiro de dados pode ser mais a infraestrutura de dados. Em outras empresas, pode-se esperar que um engenheiro de dados faça tudo isso.

Vejamos algumas das atividades de um engenheiro de dados:

- Criar e manter pipelines de dados para transformar dados em um formato que possa ser usado por pessoas em outras funções, por exemplo, por cientistas de dados ou por analistas de dados. Em geral, isso ocorre na forma de tabelas de banco de dados que agregam os dados brutos de uma empresa.
- Manter padrões de captura e de armazenamento de dados para que diferentes equipes sejam consistentes.
- Verificar se o poder de processamento e o armazenamento de dados são usados de modo eficiente.
- Garantir que a privacidade do usuário seja respeitada, controlando quais dados são capturados e quem tem acesso a esses dados.
- Manter tabelas de banco de dados, assegurando que os pipelines sejam executados no tempo certo.
- Definir verificações de qualidade de dados para verificar se os dados estão corretos.

- Criar dashboards que exibam os principais indicadores de desempenho (KPIs) de um produto ou do negócio.

Analista de dados

Um analista de dados usa dados para obter insights que embasam decisões sobre um produto ou uma estratégia. Ele pode trabalhar com dados internos (por exemplo, dados sobre como as pessoas estão usando o produto) ou realizar pesquisas de mercado usando dados de terceiros. Em geral, ele usa os dados armazenados em bancos de dados e SQL, Python ou R para consultar, transformar e analisar dados.

Existem algumas variantes dessa função. Em algumas empresas, os cientistas de dados também farão esse trabalho, além de trabalhar com ML, projetar e executar testes A/B ou outras responsabilidades. Os analistas de produto estarão mais integrados às equipes de produto. Eles trabalham em estreita colaboração com engenheiros e gerentes de produto a fim de encontrar insights que afetam diretamente o próximo produto que a equipe deve desenvolver. Os analistas de negócios focam mais as operações, finanças e análises competitivas da empresa.

Vejamos algumas das atividades de um analista de dados:

- Definir, medir e acompanhar KPIs que medem a integridade do produto ou do negócio.
- Coletar, limpar, analisar e interpretar dados para responder a perguntas da liderança sobre o uso de um produto.
- Criar visualizações de dados para se comunicar com as partes interessadas.
- Acompanhar dados sobre os clientes de um produto para entender quantos começam e param de usar um produto.
- Estimar receitas e custos de produtos.

Gerente de produto

Normalmente, um gerente de produto faz três atividades. Primeiro, ajuda a definir a estratégia do produto e o roadmap de features, equilibrando as necessidades do cliente e do negócio. Ele faz isso com a contribuição de

analistas de dados, pesquisadores de UX, liderança da empresa e quaisquer outras partes interessadas. Segundo, comunica a estratégia e o roadmap aos engenheiros e designers. Terceiro, gerencia o projeto para ajudar a equipe do produto a desenvolver o software. Gerentes de produto se envolvem ativamente na definição de requisitos no estágio de planejamento do SDLC e também ajudam a avaliar o produto finalizado.

Gerentes de produto costumam ser responsáveis pelo sucesso geral do produto, porém, normalmente, não têm autoridade explícita sobre outros membros da equipe do produto. Isso significa que eles precisam ser capazes de se comunicar adequadamente com pessoas que desempenham diferentes funções, consumir e sintetizar informações de forma eficaz, tomar decisões e analisar os prós e os contras com informações imprecisas e ajudar a estabelecer processos adequados para garantir que a equipe de desenvolvimento trabalhe de forma eficiente.

A forma como um gerente de produto aloca seu tempo pode variar entre empresas, ou até mesmo dentro de uma mesma empresa. Por exemplo, se a equipe pratica scrum, pode ter um product owner separado que lida com grande parte da execução cotidiana. A função de product owner também pode ser exercida por um gerente de projeto ou por um gerente de programa técnico. São funções distintas, mais focadas na definição de requisitos e na avaliação de produtos com maior nível de detalhes técnicos. Nesse caso, o gerente de produto dedicaria mais tempo à definição da visão e do roadmap do produto e ao trabalho com acionistas externos, em vez de focar o desenvolvimento diário.

Vejamos algumas das atividades de um gerente de produto:

- Ajudar as equipes de desenvolvimento a definir os requisitos para o código que estão escrevendo.
- Explicar claramente o que é sucesso para um produto e ajudar a definir metas e KPIs para medir o progresso.
- Trabalhar com engenheiros e designers para priorizar features, considerando o tempo de desenvolvimento e o impacto positivo que trarão aos usuários.
- Colaborar com analistas de dados e pesquisadores de UX para entender e quantificar melhor as necessidades do usuário.

- Ajudar as equipes a se alinharem com outras equipes de produto quando há discordâncias ou dependências.
- Revisar os planos e o progresso com a liderança, por meio de reuniões ou relatórios escritos.

Pesquisador de UX

Um pesquisador de UX (Experiência do Usuário) fornece insights sobre as necessidades dos usuários de um produto. Ele faz isso observando como os usuários interagem com os produtos e obtendo feedback direto deles. Enquanto os analistas de dados usam dados a fim de visualizar o que os usuários fazem, os pesquisadores de UX podem obter insights mais aprofundados sobre por que os usuários fazem as coisas, bem como o que gostariam que o produto lhes possibilitasse fazer de maneira diferente. Para isso, os pesquisadores de UX realizam entrevistas e sessões de usabilidade, além de pesquisas.

Em algumas empresas, pesquisadores qualitativos de UX e pesquisadores quantitativos de UX são funções distintas. O primeiro se concentra mais em estudos de usabilidade e em entrevistas detalhadas, enquanto o segundo se concentra em pesquisas de maior escala e em análises mais estatísticas.

Vejamos algumas das atividades de um pesquisador de UX:

- Entrevistar usuários avançados de um produto a fim de verificar quais ferramentas querem, de modo que possam realizar tarefas com mais rapidez.
- Conduzir uma sessão de usabilidade, mostrando protótipos com diferentes fluxos de design para voluntários e observando como eles interagem e qual é o mais intuitivo.
- Conduzir uma pesquisa em que uma amostra aleatória de pessoas é questionada sobre sua experiência após realizar uma tarefa em um site.
- Analisar e sumarizar os resultados das pesquisas e apresentá-los à equipe do produto a fim de embasar as decisões de desenvolvimento do produto.

Designer

Os designers são responsáveis pelos detalhes da aparência e da sensação do usuário com um produto. A aparência de um produto envolve paletas de cores, tipografia, aparência de botões e outros elementos interativos, ícones, animações e assim por diante. A sensação é a experiência geral do usuário, que envolve considerar se os fluxos (sequências de ações que o usuário realiza para fazer algo) são intuitivos e fáceis para os usuários. Na verdade, os designers não implementam os designs no código; isso é feito por um engenheiro de front-end.

Em muitas empresas, a aparência e a sensação se enquadram em duas funções diferentes. Um designer de UI está mais preocupado com a aparência do produto. Ele criará um wireframe (também conhecido como mockup) do produto e trabalhará em estreita colaboração com os engenheiros de front-end que o desenvolverão. Trata-se de uma versão não funcional do produto que tem a aparência que o designer quer almejar. Em contrapartida, um designer de UX está mais preocupado com a usabilidade. Além de trabalhar com desenvolvedores, ele também pode trabalhar em estreita colaboração com o pesquisador de UX e gerentes de projeto a fim de entender e resolver melhor os problemas do usuário.

Vejamos algumas das atividades de um designer de UI:

- Fazer o design da localização exata de cada parte do conteúdo no site ou app, incluindo o espaçamento entre eles e como serão exibidos em diferentes tamanhos de tela.
- Fazer o design de elementos individuais, como botões e ícones, incluindo o que fazem quando um usuário interage com eles.
- Criar um guia de estilo para consistência, usando paletas de cores e fontes adequadas.

Vejamos algumas das atividades de um designer de UX:

- Conduzir pesquisas com usuários por meio de entrevistas ou sessões de usabilidade (semelhante a um pesquisador de UX).
- Criar wireframes que fornecem o layout em relação ao local em que os elementos devem aparecer em um site ou app e como os usuários podem navegar entre eles.

Comunidade

Além das pessoas que conhecerá e com quem trabalhará em uma empresa, ao escrever código, você pode fazer parte da comunidade mais ampla de cientistas de dados e de desenvolvedores. Você terá contato com ideias que, talvez, não tenha em seu trabalho cotidiano. É possível se conectar com outras pessoas que fazem o mesmo tipo de trabalho e aprender sobre novas ferramentas e técnicas. Nesta seção, abordarei como participar da comunidade em geral, incluindo como contribuir com software open-source, palestrar em eventos e participar da comunidade Python global.

Open-source

Você provavelmente já utilizou softwares open-source em sua carreira como cientista de dados, e mencionei pacotes open-source diversas vezes neste livro. Contribuir para um projeto open-source pode ser muito valioso para sua carreira e é uma ótima maneira de se conectar com a comunidade. Caso seja novato na área de software, pode ser uma boa forma de adquirir experiência em práticas de desenvolvimento de software. E, se você for mais experiente, ainda pode ser uma maneira de aprender habilidades novas e contribuir para a comunidade em geral.

Existem algumas formas para começar a se engajar em projetos open-source. Em geral, projetos grandes têm instruções para novos colaboradores, como as instruções para contribuir com o pandas (*https://oreil.ly/J-Jd*) e as instruções para contribuir com o scikit-learn (*https://oreil.ly/lPXLP*). Eles também podem ter problemas destacados por tags como good first issues [bons para começar a contribuir], conforme mostrado na Figura 14.3.

Alguns projetos open-source realizam sprints em conferências. São períodos dedicados, organizados pelos mantenedores do projeto, para trabalhar em problemas específicos. É uma ótima forma de participar de um projeto, pois você pode reservar um tempo para trabalhar em um problema e também conhecer outras pessoas que estão trabalhando nele.

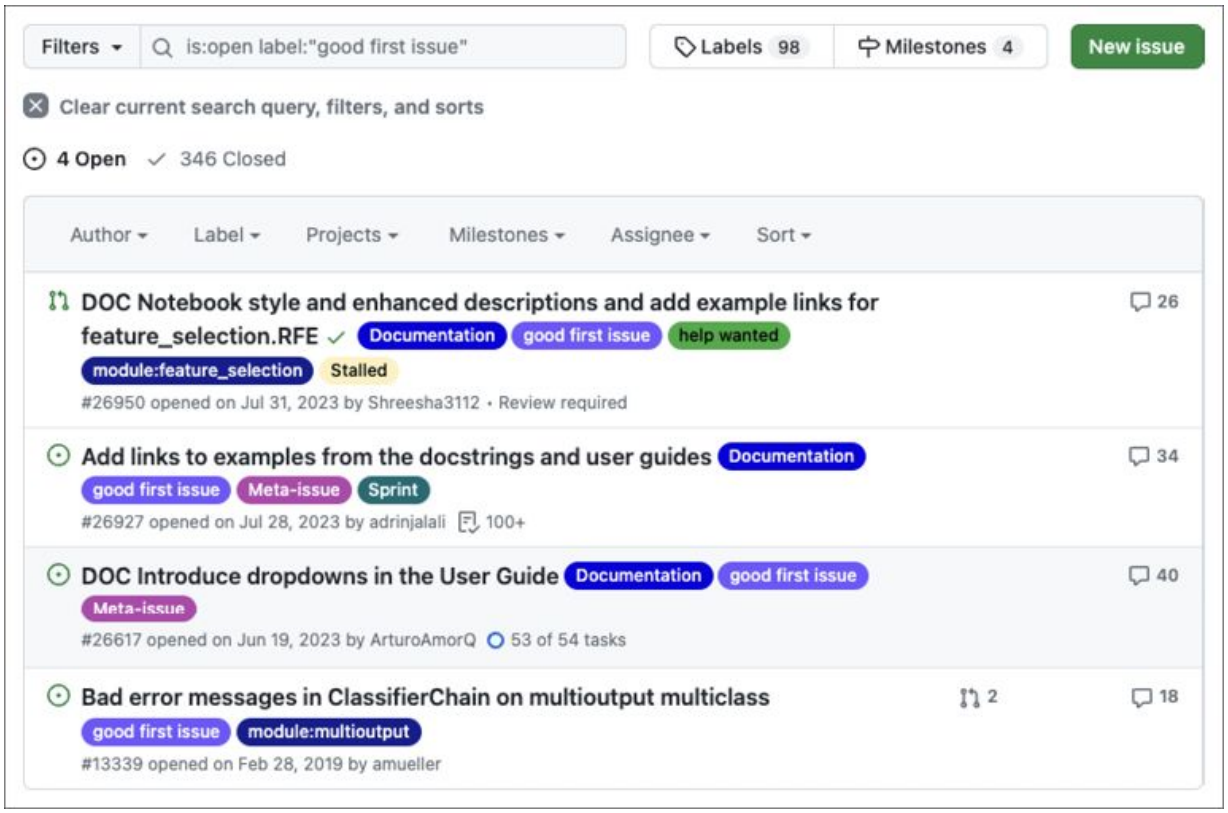


Figura 14.3: Problemas no scikit-learn destacados por tags chamadas de good first issues.

Outra maneira de participar é se você estiver usando um pacote e encontrar algo que não funciona bem ou se identificar uma feature que não faz o que precisa. É possível criá-la ou consertá-la você mesmo! Mas, primeiro, verifique os problemas atuais para garantir que ninguém mais já esteja trabalhando nisso. Se for um projeto pequeno, talvez você queira entrar em contato com o mantenedor dele para perguntar a melhor forma de ajudar.

Você também pode tornar público no GitHub o código que está escrevendo por motivos pessoais. Antes de fazer isso, garanta que seu projeto tenha uma boa documentação para que as pessoas possam entender o que ele faz.

Vale também ressaltar que nem todo mundo tem o tempo ou o desejo de escrever código fora do horário de trabalho. Se encontrar uma feature ausente em um projeto open-source que seja importante para o seu trabalho, pode valer a pena perguntar se você pode contribuir com isso como parte do seu trabalho. Além disso, você pode incentivar sua empresa a patrocinar projetos open-source.

Palestrando em eventos

As gravações de palestras técnicas de conferências e eventos foram imprescindíveis para escrever este livro, e tive ótimas experiências palestrando em eventos. Caso se sintam à vontade e seja interessante, incentivo todos a tentar palestrar em eventos ou em conferências. Pode parecer assustador no início, porém é mais fácil do que você pensa. Todo mundo tem algo útil para compartilhar. Mesmo que esteja no início de sua carreira, você pode compartilhar algo que aprendeu, e isso ajudará outra pessoa que também está começando.

Nesta seção, quero compartilhar algumas dicas para ministrar palestras técnicas:

- Comece em eventos pequenos e amigáveis, por exemplo, um encontro local ou uma palestra breve em uma pequena conferência local. Desse modo, é possível aprender habilidades em situações com pouco estresse.
- Participe de eventos e pense no que gosta quando assiste a uma palestra muito boa.
- Considere o que você quer que seus ouvintes aprendam, pensem ou façam como resultado de sua palestra.
- Estruture sua palestra de forma clara e que conte alguma história. Faça uma introdução para que seu público se conecte com você e faça um conclusão clara.
- Explique aos ouvintes por que devem se interessar pela sua palestra.

Geralmente, eventos maiores exigem uma proposta, que você precisará enviar até uma data de encerramento específica. A proposta precisa vender sua palestra aos organizadores de um congresso ou de uma conferência, e pode ser publicada na programação do site delas. Mencione o que seus ouvintes aprenderão com sua palestra.

Para obter mais dicas sobre como palestrar em eventos, recomendo esta série de artigos de Nina Zakharenko (*https://oreil.ly/jqDZc*).

Muitas empresas patrocinam sua participação em eventos como palestrante, pois isso é bom para a imagem externa delas. Algumas até arcam com as despesas, sobretudo se você trabalhar em uma empresa de grande porte. Congressos e conferências também podem ter financiamento disponível

para pagar a viagem dos palestrantes, e é padrão que os palestrantes, pelo menos, obtenham ingressos gratuitos.

Comunidade Python

Nesta seção, gostaria de abordar especificamente a comunidade Python global. O Python é o foco deste livro e é minha linguagem de programação preferida, mas existem comunidades semelhantes para outras linguagens de programação.

A comunidade global de desenvolvedores e entusiastas do Python é apoiada pela Python Software Foundation (PSF) (*https://oreil.ly/YjR5Y*). A PSF é uma organização sem fins lucrativos fundada em 2001 que produz a principal [core] distribuição do Python e a disponibiliza como um projeto open-source. Além disso, realiza a conferência PyCon US, gerencia patrocínios e doações e fornece subsídios para apoiar sprints open-source e grupos de usuários e desenvolvimento Python. A associação da PSF é gratuita para qualquer pessoa, e você pode apoiar a comunidade e manter-se atualizado com as últimas novidades do Python. Para saber mais sobre a associação, acesse o site da PSF (*https://oreil.ly/FWEMP*).

Uma das melhores formas de conhecer outras pessoas da comunidade Python e aprender mais sobre tecnologias novas é participar de uma conferência Python ou um PyCon. A PyCon US(*https://us.pycon.org*) começou em 2003 e é uma das maiores, com 2.000 a 3.000 participantes por ano. Existem muitos outros encontros em todo o mundo, como PyCon Japan (*https://2023-apac.pycon.jp*), PyCon LatAm (*https://www.pylatam.org*), PyCon Namibia (*https://na.pycon.org*) e a PyCon India (*https://in.pycon.org/2023*). Há também uma conferência anual SciPy (*https://oreil.ly/jiJdF*) com foco em computação científica.

Como em muitas áreas técnicas, as mulheres têm pouca representatividade na comunidade Python. O PyLadies (*https://pyladies.com*) procura apoiar e orientar mulheres na comunidade Python. Existem muitos grupos PyLadies em todo o mundo, que proporcionam espaço para as mulheres interagirem e apoiarem-se. O PyLadies realizou a primeira conferência online (*https://oreil.ly/S8fJw*) em 2023.

Além disso, existem muitos encontros locais ou grupos de usuários em todo

o mundo, em que grupos de entusiastas se reúnem para debater tópicos relacionados ao Python. Os encontros podem ser palestras técnicas (uma ótima forma de começar a palestrar), noites de programação, workshops e assim por diante.

Participar da comunidade Python possibilita conhecer pessoas com opiniões diferentes sobre os mesmos tópicos em que você pode estar trabalhando. É ótimo aprender o que as outras pessoas pensam sobre o Python, desenvolvimento de software e ciência de dados. É possível aplicar essas ideias ao trabalho e compartilhá-las com outras pessoas.

Aspectos fundamentais

Neste capítulo, apresentei algumas formas comuns de trabalhar em empresas de tecnologia, forneci uma visão geral das funções de trabalho das pessoas com quem você pode trabalhar e apresentei a comunidade de software de forma mais ampla.

Em toda indústria de software, existem métodos padronizados para estruturar o desenvolvimento de software. O ciclo de vida de desenvolvimento de software inclui as seguintes etapas: planejamento, design, desenvolvimento, testes, implantação e manutenibilidade. No método de desenvolvimento de software em cascata, essas etapas acontecem sequencialmente. No desenvolvimento ágil de software, é comum iterar rapidamente as etapas em sprints curtos. O desenvolvimento ágil pode fornecer uma estrutura útil para as equipes de ciência de dados.

Na indústria de software, é possível se deparar com as seguintes funções técnicas:

Engenheiros de software

Escrevem código e desenvolvem processos para criar e manter produtos.

Engenheiros de QA

Testam o produto com o intuito de verificar se ele funciona bem para todos os usuários.

Engenheiros de dados

Criam e mantêm pipelines de dados para transformar dados brutos em um

formato que cientistas e analistas de dados possam usar.

Analistas de dados

Selecionam, limpam, analisam e interpretam dados para obter insights a partir deles.

Gerentes de produto

Planejam e organizam os requisitos e o roadmap para o trabalho de desenvolvimento.

Pesquisadores de UX

Pesquisam e analisam as necessidades dos usuários de um produto.

Designers

Criam o design da aparência geral de um produto.

Além das pessoas com quem trabalha na sua empresa, pode ser muito valioso participar da comunidade em geral. Você pode aprender com ideias novas e compartilhar suas ideias e experiências. E pode fazer isso contribuindo com software open-source, participando e palestrando em eventos.

CAPÍTULO 15

Próximos passos

Parabéns por chegar ao último capítulo! Espero que, depois de ler este livro, você se sinta confiante para escrever um ótimo código que atenda a todos os requisitos necessários e funcione adequadamente e em grande escala.

Da próxima vez que for escrever código, lembre-se dos cincos principais aspectos que apresentei no Capítulo 1:

Simplicidade

Mantenha seu código simples, pois fica mais fácil para outras pessoas o usarem.

Modularidade

Divida seu código em partes menores e razoáveis. Recorra aos princípios da programação orientada a objetos que abordei no Capítulo 4, caso sejam adequados para o problema em que está trabalhando. Garanta que a estrutura geral funcione bem, como descrevi no Capítulo 8.

Legibilidade

Escreva a documentação para o seu código, conforme abordei no Capítulo 9. Seu código será lido com mais frequência do que escrito. A formatação consistente também contribui com a legibilidade, conforme explicado no Capítulo 6.

Eficiência

Garanta que seu código seja tão eficiente quanto necessário, usando os métodos dos capítulos 2 e 3.

Robustez

Assegure que seu código trate os erros de forma adequada e que eles sejam registrados em logs, conforme descrevi no Capítulo 5. Testar, como expliquei no Capítulo 7, é uma habilidade primordial para aprender e colocar em prática.

Com esses princípios, você passará do código experimental e pontual rumo a soluções reprodutíveis e escaláveis. O código escalável pode ser compartilhado em pacotes, como detalhei no Capítulo 10, ou implantado como uma API, conforme analisado nos capítulos 11 e 12.

No decorrer desta obra, também abordei alguns outros tópicos relacionados à engenharia de software. Além de ajudá-lo a escrever código com maior rapidez, a padronização ajudará outras pessoas a trabalhar em seu código com mais facilidade. A abstração é outro aspecto a ser considerado: é possível encapsular os detalhes do seu código em uma interface intuitiva, facilitando sua reutilização em outros contextos? Por fim, automatizar processos repetidos economizará tempo e fará com que você programe mais rápido a longo prazo.

Eu o incentivo a ser pragmático. Nem sempre meus códigos são ótimos, e nem os seus precisam ser. Às vezes, um código terrível que funcione é tudo o que você precisa. Se for um projeto de curto prazo ou se precisar improvisar algo para um prazo apertado, escreva qualquer código que resolva o problema. Mas, pelo menos, depois de ler este livro, espera-se que você saiba quando estiver escrevendo um código horrível!

Os princípios de engenharia de software que expliquei nesta obra são abordados com mais detalhes em outras fontes. Minha esperança é que, após a leitura deste livro, você se sinta confiante para explorar todos esses conceitos em materiais voltados para engenheiros de software e, assim, aplicar o conhecimento adquirido escrevendo códigos para ciência de dados. Você já tem uma boa noção da terminologia e pode focar os tópicos mais relevantes para o seu trabalho.

Recomendo os seguintes livros como próximos passos:

- *The Pragmatic Programmer*, 20th Anniversary Edition, de David Thomas e Andrew Hunt (Addison-Wesley Professional)
- *A Philosophy of Software Design*, de John Ousterhout (Yaknyam Press, 2021)
- *High Performance Python: Practical Performant Programming for Humans*, de Micha Gorelick e Ian Osvald (O’Reilly, 2020)
- *Robust Python*, de Patrick Viafore (O’Reilly, 2021)

Recomendo também assistir a palestras de conferências Python. Palestras de conferências anteriores da PyCon US estão disponíveis no YouTube (*https://oreil.ly/e7Gii*). É um conteúdo de alta qualidade, disponível gratuitamente.

Futuro do código

Muitas das técnicas e tecnologias que abordei nesta obra foram inventadas nos últimos 5 anos. Engenharia de software e ciência de dados são áreas em constante mudança, e ideias novas surgem o tempo todo. É uma ótima ideia acompanhar as tecnologias novas que são amplamente adotadas.

Enquanto escrevia este livro, ocorreu outro avanço significativo: o advento de assistentes de programação com IA, como o GitHub Copilot (*https://oreil.ly/LymmQ*) e o ChatGPT (*https://oreil.ly/_qcbY*). Você ainda precisa aprender a escrever um bom código e familiarizar-se com as práticas de engenharia de software se essas ferramentas podem fazer tudo?

Decidi conferir o que o ChatGPT criaria se eu pedisse para gerar a função de média ponderada que usei como exemplo em muitos capítulos deste livro. Confira o prompt que usei na Figura 15.1.

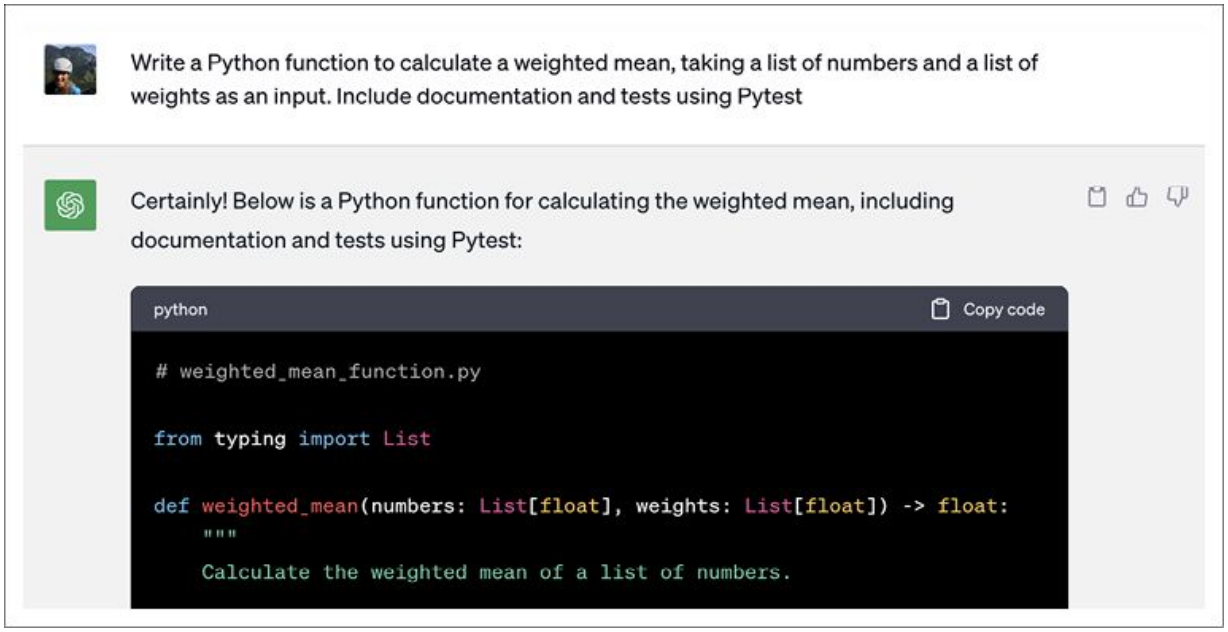


Figura 15.1: Pedindo ao ChatGPT que escreva uma função deste livro.

Vejamos a função completa gerada pelo ChatGPT[1]:

```
from typing import List

def weighted_mean(numbers: List[float], weights: List[float]) -> float:
    """
    Calcule a média ponderada de uma lista de números.

    Parâmetros:
    - numbers (List[float]): lista de números.
    - weights (List[float]): lista de pesos (mesmo comprimento dos números).

    Returns:
    - float: média ponderada dos números.

    Raises:
    - ValueError: se os comprimentos dos números e pesos não corresponderem,
                  ou se algum peso for 0.

    Exemplo:
    >>> numbers = [10, 20, 30, 40, 50]
    >>> weights = [0.1, 0.2, 0.3, 0.2, 0.2]
    >>> weighted_mean(numbers, weights)
    30.0
    """
    if len(numbers) != len(weights):
        raise ValueError("The number of numbers and weights must be the same.")

    if any(w == 0 for w in weights):
        raise ValueError("Weights must be greater than zero.")

    weighted_sum = sum(x * w for x, w in zip(numbers, weights))
    total_weight = sum(weights)

    return weighted_sum / total_weight
```

E vejamos o teste que ele gerou para a função:

```
import pytest
from weighted_mean_function import weighted_mean
```

```
def test_weighted_mean():
    numbers = [10, 20, 30, 40, 50]
    weights = [0.1, 0.2, 0.3, 0.2, 0.2]
    expected_result = 30.0

    result = weighted_mean(numbers, weights)

    epsilon = 1e-6
    assert abs(result - expected_result) < epsilon

def test_weighted_mean_with_zero_weight():
    numbers = [10, 20, 30, 40, 50]
    weights = [0.1, 0.2, 0.3, 0.0, 0.2]

    with pytest.raises(ValueError):
        weighted_mean(numbers, weights)

def test_weighted_mean_with_mismatched_lengths():
    numbers = [10, 20, 30, 40, 50]
    weights = [0.1, 0.2, 0.3, 0.2]

    with pytest.raises(ValueError):
        weighted_mean(numbers, weights)
```

À primeira vista, tudo parece muito lógico. O ChatGPT incluiu docstrings e alguns testes razoáveis. No entanto, quando tentei executar o código, o teste `test_weighted_mean` falhou! O valor de `expected_result` deveria ter sido 32,0, não 30,0. Esse erro se repete na docstring.

Além disso, o teste `test_weighted_mean` poderia ter sido simplificado, testando se os resultados eram exatamente os mesmos, em vez de uma margem de 1e-6:

```
def test_weighted_mean():
    numbers = [10, 20, 30, 40, 50]
    weights = [0.1, 0.2, 0.3, 0.2, 0.2]
    expected_result = 32.0
```

```
result = weighted_mean(numbers, weights)

assert result == expected_result
```

Pedir ao ChatGPT que gerasse essa função me forneceu um ponto de partida útil, mas ainda precisei fazer alguns ajustes para corrigi-la. Eu também poderia melhorá-la adicionando testes para situações que eu esperava que surgissem. É uma boa ideia examinar cuidadosamente os detalhes e testar por completo qualquer código gerado por IA.

Por mais que as ferramentas com inteligência artificial sejam poderosas e possam ajudá-lo a programar mais rápido, é necessário entender quando apresentam uma solução adequada ou não. Existem muitas opções para cada função que escrevemos, e é importante entender as vantagens e desvantagens de cada uma dessas opções. A função que o GPT escreve para você pode ser breve e legível, mas também é ineficiente. É necessário saber o que solicitar a essas ferramentas. Elas não incluirão o registro em logs ou a documentação se você não pedir. Ou seja, são necessários sólidos conhecimentos básicos de como escrever um bom código a fim de aproveitá-las ao máximo.

E tenha certeza de que a engenharia de software de hoje não será a mesma do futuro. Uma linguagem de programação específica pode se tornar mais popular por um tempo, mas pode ser substituída por outras mais adequadas a um propósito específico. O surgimento de novas ideias e tecnologias é um processo contínuo e ininterrupto. E, felizmente, as pessoas da comunidade de desenvolvedores escreverão documentação e guias para ajudar todos a usá-las.

Seu futuro no código

Em sua carreira como cientista de dados, você escreverá inúmeras linhas de código. Algumas delas podem ser executadas apenas uma vez, enquanto outras podem construir os alicerces para projetos e produtos que alcançarão milhões de usuários. Como seu código pode ter consequências substanciais, incentivo você a usar suas competências para o bem e sempre considerar se o que está criando se alinha com seus princípios éticos.

Além disso, também incentivo que compartilhe o que aprendeu sobre como escrever um bom código. Seja um exemplo para seus colegas de equipe e lidere pelo exemplo se estiver em uma posição sênior. Agende revisões de código e ajude outras pessoas a melhorar seu código. O código que escreve pode influenciar a cultura da sua equipe.

Um software não é desenvolvido de forma isolada. Recomendo que crie networking, conheça uma variedade de pessoas e comunique-se com outras. Participar da comunidade em geral, ministrar palestras técnicas e contribuir para projetos open-source pode ser incrivelmente enriquecedor. Incentivo você também a conversar com engenheiros de software e estar aberto ao feedback deles sobre seu código.

Obrigada

Por último, quero agradecer a todos os leitores. Obrigada por dedicar tempo à leitura deste livro. Escrevê-lo foi muito divertido e espero que vocês tenham gostado de lê-lo.

Agradeceria muito seu feedback para que eu possa melhorar quaisquer edições futuras. Para entrar em contato comigo, confira o Prefácio.

---

1 N.T.: A docstring desse código foi traduzida para melhor compreensão dos leitores.

Sobre a autora

Catherine Nelson é cientista de dados e escritora freelancer. Anteriormente, atuou como cientista de dados principal na SAP Concur, onde explorou formas inovadoras de entregar aplicações de machine learning para produção que melhoram a experiência das pessoas que viajam a negócios. Suas principais áreas de interesse abrangem desde a explicabilidade de ML e análise de modelos até a privacidade em ML. Ela também é coautora da obra *Building Machine Learning Pipelines [Construindo Pipelines de Machine Learning, em tradução livre]*, publicada pela O'Reilly. Em sua carreira anterior como geofísica, ela estudou vulcões antigos e explorou petróleo na Groenlândia. Catherine tem doutorado em Geofísica pela Universidade de Durham e mestrado em Geociência pela Universidade de Oxford.

Colofão

O animal na capa do Engenharia de Software para Cientistas de Dados é um peixe-borboleta Chaetodon (Chaetodon auriga), um peixe de cores vivas dos recifes de coral do Indo-Pacífico.

Alcançando até 23 centímetros de comprimento, seu corpo é adornado em um padrão chamativo de branco com listras pretas e sua parte posterior é realçada em amarelo brilhante. Uma faixa preta atravessa seus olhos, e sua barbatana dorsal ostenta uma mancha preta proeminente com um filamento que se estende. A própria barbatana filiforme se desenvolve à medida que o peixe envelhece, e o quinto e o sexto filamento de sua barbatana dorsal se fundem para formar a longa cauda filamentosa.

Os peixes-borboleta ativos são vistos com mais frequência em pares, nadando ligeiramente entre os recifes de coral, lagunas e ambientes de recifes externos. Eles usam suas bocas alongadas para cutucar fendas, em busca de alimento (uma dieta variada de vermes, moluscos, esponjas, algas e até corais moles).

Em geral, embora sejam pacíficos com outros peixes, os peixes-borboletas Chaetodon podem ser territoriais com sua própria espécie, ainda mais em aquários pequenos. São peixes reservados e apreciam muitos esconderijos dentro do recife.

O peixe-borboleta Chaetodon é classificado como uma espécie de Menor Preocupação pela IUCN. Muitos dos animais apresentados nas capas dos livros da O’Reilly estão em risco de extinção; todos têm importância inestimável para o mundo.

A ilustração da capa é de Karen Montgomery, e foi baseada em uma gravura antiga da Royal Natural History, de Lydekker. O design da série é de Edie Freedman, Ellie Volckhausen e Karen Montgomery.

O'REILLY

novatec

Fundamentos de Engenharia de Dados

Projete e Construa Sistemas de Dados Robustos

Joe Reis
Matt Housley

Fundamentos de Engenharia de Dados

Reis, Joe
9788575228777
528 páginas

Compre agora e leia (Publicidade)

A engenharia de dados cresceu rapidamente na última década, deixando muitos engenheiros de software, cientistas e analistas de dados em busca de uma visão abrangente dessa prática. Com este livro prático, você aprenderá a projetar e construir sistemas para atender às necessidades de sua organização e clientes, avaliando as melhores tecnologias disponíveis por meio da estrutura do ciclo de vida da engenharia de dados. Os autores Joe Reis e Matt Housley guiam você pelo ciclo de vida da engenharia de dados e mostram como integrar uma variedade de tecnologias em nuvem para atender às necessidades dos consumidores de dados downstream. Você compreenderá como aplicar os conceitos de geração, ingestão, orquestração, transformação, armazenamento e governança de dados que são essenciais em qualquer ambiente de dados, independentemente da tecnologia utilizada. Este livro o ajudará a: •Obter uma visão geral de todo o cenário de engenharia de dados. •Avaliar problemas de engenharia de dados utilizando um conjunto abrangente de melhores práticas. •Não se deixar guiar pelo hype do momento ao escolher as tecnologias, arquitetura e processos de dados adequados. •Utilizar o ciclo de vida da engenharia de dados para projetar e criar uma arquitetura robusta. •Integrar governança de dados e segurança em todo o ciclo de vida da engenharia de dados.

Compre agora e leia (Publicidade)

O'REILLY

novatec

O Catálogo de Dados Corporativo

Aprimore a descoberta de dados, garanta a governança de dados e impulsione a inovação

Ole Olesen-Bagneux

Apresentação de Guillaume Bodet

O Catálogo de Dados Corporativo

Olesen-Bagneux, Ole
9788575228654
224 páginas

Compre agora e leia (Publicidade)

Realizar pesquisas na web é simples, mas como pesquisar por dados no ambiente de trabalho? É uma tarefa desafiadora que consome bastante tempo e, por vezes, pode parecer quase impossível. Este livro apresenta uma solução prática: o catálogo de dados. Analistas de dados, cientistas de dados e engenheiros de dados aprenderão como promover uma verdadeira descoberta de dados em suas organizações, tornando o catálogo uma ferramenta essencial para impulsionar a inovação orientada por dados e a governança de dados. O autor Ole Olesen-Bagneux explora os benefícios da implementação de um catálogo de dados. Você aprenderá como organizar dados em seu catálogo, pesquisar pelo que precisa e gerenciar os dados dentro dele. Escrito a partir de uma perspectiva de gerenciamento de dados e de biblioteconomia e ciência da informação, este livro ajudará você a: •Entender o que é um catálogo de dados e como ele pode ajudar sua organização •Organizar dados e suas fontes em domínios e descrevê-los com metadados •Pesquisar dados utilizando técnicas de pesquisa simples e complexas e aprender a navegar por domínios, linhagem de dados e grafos •Gerenciar os dados em sua empresa por meio de um catálogo de dados •Implementar um catálogo de dados de acordo com as prioridades estratégicas de sua organização •Ter uma visão clara do que o futuro reserva para os catálogos de dados "Este livro destaca a enorme oportunidade que temos de extrair o máximo valor do investimento em dados por meio de um mecanismo de pesquisa de dados corporativos." —Ann Fogelgren Diretora de informações, Grupo GN "Este livro, escrito com maestria e precisão, apresenta tanto os fundamentos conceituais quanto as ferramentas práticas

para desenvolver e implementar catálogos de dados corporativos." —Jens-Erik Mai Professor de informação, Universidade de Copenhague

Compre agora e leia (Publicidade)

O'REILLY

A Cor dos Dados

UM GUIA PARA O USO DE CORES EM STORYTELLING DE DADOS

novatec

KATE STRACHNYI

A Cor dos Dados

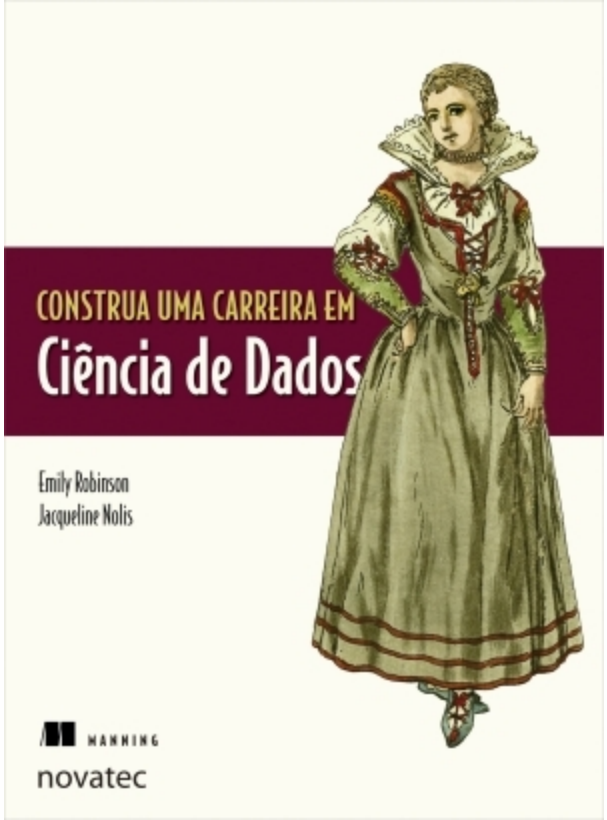
Strachnyi, Kate
9788575228531
160 páginas

Compre agora e leia (Publicidade)

"Todos os dias, vejo as cores sendo usadas de maneira inadequada e não intencional em várias empresas ao redor do mundo, o que resulta em insights menos precisos. Usar as cores corretamente pode impulsionar o storytelling de dados e o design de dashboards a novos patamares. Kate Strachnyi fez um trabalho incrível reunindo tudo o que você precisa saber sobre o uso de cores para criar uma visualização de dados eficaz." — Bernard Marr, autor best-seller de Data Strategy Os dados se tornaram a ferramenta mais poderosa nos negócios atualmente, e contar sua história de maneira eficaz é fundamental. Apesar de serem um dos melhores comunicadores, as cores são frequentemente negligenciadas como ferramenta na visualização de dados. Com este livro, a fundadora da DATAcated, Kate Strachnyi, oferece o guia definitivo para o uso correto de cores na representação de dados em gráficos, tabelas e infográficos. Indicado para analistas de dados, profissionais de negócios, cientistas de dados e outros especialistas que criam infográficos e visualizações de dados, este recurso prático explora dicas e truques sobre o uso de cores, incluindo a teoria por trás delas e porque funcionam de tal maneira. A Cor dos Dados aborda a psicologia, a história e a cultura de diversas cores diferentes. Este livro também é uma ferramenta de ensino útil para aprender sobre o uso adequado de cores para técnicas de storytelling de dados e construção de dashboards. Você vai explorar: •O papel que a teoria das cores desempenha na visualização e no storytelling de dados •Várias técnicas de cores que você pode usar para aperfeiçoar suas visualizações de dados •Como as cores impactam a compreensão do seu público sobre as visualizações de dados •Como usar cores de maneira intencional para ajudar

a orientar seu público •Dicas para utilizar cores que pessoas com daltonismo consigam interpretar •Como utilizar as diretrizes deste livro em seus próprios projetos

Compre agora e leia (Publicidade)


CONSTRUA UMA CARREIRA EM
Ciência de Dados
Emily Robinson
Jacqueline Nolis
MANNING
novatec

Construa uma Carreira em Ciência de Dados

Robinson, Emily
9786586057430
400 páginas

Compre agora e leia (Publicidade)

Quais são os fatores importantes para o sucesso de longo prazo de um cientista de dados? Combinar conhecimento técnico com "competências interpessoais" adequadas é um ingrediente fundamental para uma carreira gratificante. Construa uma Carreira em Ciência de Dados é o seu guia para conseguir o primeiro emprego nessa área e se transformar em um profissional habilitado e inestimável. Seguindo instruções claras e simples, você aprenderá a criar um excelente currículo para sair-se bem em entrevistas. Nesse campo exigente e em rápida transformação, pode ser um desafio manter os projetos na direção certa, adaptar-se às necessidades das empresas e gerenciar stakeholders problemáticos. Você irá se beneficiar com ideias sobre como lidar com as expectativas e frustrações, além de maneiras de planejar sua carreira com base nas vivências de cientistas de dados experientes relatadas no livro. O que este livro contém: • Criação de um portfólio de projetos de ciência de dados • Avaliação e negociação de uma proposta • Como sair-se bem e ascender profissionalmente em uma empresa • Como sair-se bem em entrevistas com cientistas de dados profissionais Para os leitores que querem começar uma ou avançar na carreira da ciência de dados. Emily Robinson é cientista de dados da Warby Parker. Jacqueline Nolis é consultora e mentora de ciência de dados. "Repleto de conselhos úteis, cenários reais e contribuições de profissionais do setor." — Sebastián Palma Mardones, ArchDaily "O companheiro perfeito para alguém que quer ser um cientista de dados bem-sucedido!" Gustavo Gomes, Brightcove "Visão geral perspicaz de todos os aspectos de uma carreira em ciência de dados." — Krzysztof Jędrzejewski, Pearson "Altamente recomendado." — Hagai Luger, Clarizen

Compre agora e leia (Publicidade)

O'REILLY

Introdução ao GraphQL

BUSCA DE DADOS COM ABORDAGEM DECLARATIVA PARA APLICAÇÕES WEB MODERNAS

novatec

Eve Porcello & Alex Banks

Introdução ao GraphQL

Porcello, Eve
9788575227107
216 páginas

Compre agora e leia (Publicidade)

Por que o GraphQL é a tecnologia mais inovadora para buscar dados desde o Ajax? Ao oferecer uma linguagem de consulta para suas APIs e um runtime para responder às consultas com seus dados, o GraphQL representa uma alternativa ao REST e às arquiteturas ad hoc dos web services. Com este guia prático, Alex Banks e Eve Porcello apresentam um caminho de aprendizado objetivo aos desenvolvedores web de frontend, engenheiros de backend e gerentes de projeto e de produto que queiram começar a trabalhar com o GraphQL. Você explorará a teoria dos grafos, a estrutura de dados de grafo e os tipos do GraphQL antes de aprender a construir um esquema para uma aplicação de compartilhamento de fotos na prática. Este livro também apresenta o Apollo Client: um framework popular que pode ser usado para conectar o GraphQL à sua interface de usuário. - Explore a teoria dos grafos e analise exemplos conhecidos de grafos em uso nos dias de hoje. - Saiba como o GraphQL aplica métodos de consulta de banco de dados à internet. - Crie um esquema para uma aplicação PhotoShare, que servirá como um roteiro e um contrato entre as equipes de frontend e de backend. - Use JavaScript para implementar um serviço GraphQL totalmente funcional, e o Apollo para implementar um cliente. - Aprenda a preparar APIs GraphQL e os clientes para um ambiente de produção

Compre agora e leia (Publicidade)

This file was downloaded from Z-Library project

zlibrary

Your gateway to knowledge and culture. Accessible for everyone.

www

z-library.sk z-lib.gs z-lib.fm go-to-library.sk

Official Telegram channel

Z-Access

W

https://wikipedia.org/wiki/Z-Library